

Magic State Distillation via Codes over Binary Extension Fields

Anqi Gong^{1,3}, Christopher A. Pattison², Patrick Rall³, and Adam Wills^{4,3}

¹ETH Zürich

²UC Berkeley

³IBM Quantum

⁴MIT

August 11, 2026

Abstract

Fault-tolerant quantum computation architectures are frequently bottlenecked by the overhead of producing high-fidelity magic states. In this work, we use algebraic geometric techniques to construct codes over binary extension fields $\mathbb{F}_{2^s}$, thus discovering new protocols for the distillation of qubit magic states, where our focus is on the regime of practical qubit-based quantum computing architectures. To do this, we show that multi-qubit gates of interest such as CS, CCZ, and $\text{TOF\#} = \text{CCZ}_{123}\text{CCZ}_{345}$, can be packaged into simple gates over the larger fields, and we derive simple algebraic conditions in the extension fields allowing the distillation of these gates. Because they are derived from Galois qudits, the corresponding qubits codes naturally handle the correlated errors present on such multi-qubit states. Moreover, the protocols we discover are extremely compact; for example, we show that 4 CS states can be distilled to 1 CS state at distance 2, using only 4 logical qubits.

For a case study, we consider the distillation of CS and CCZ states from injected T and CS states. When optimized for magic state production per unit time, or logical spacetime volume, we find that our protocols outperform the state-of-the-art in almost every situation, both at input error rates 10^{-3} (direct injection), and 10^{-6} (allowing some cultivation pre-injection).

Contents

1	Introduction	2
1.1	Overview	2
1.2	Relation to Prior Work	4
1.3	Results	4
1.3.1	Catalogue of Protocols	4
1.3.2	Spacetime Footprints for Distillation Tasks	4
1.4	Open Questions	8
1.5	Organization	9
2	Preliminaries	10
2.1	Diagonal Gates in the Clifford hierarchy	10
2.2	Introduction to rings, finite fields and Reed-Muller codes	11
2.3	Affine and projective space	13
3	Galois Qudits over $\text{GF}(2^s)$	13
3.1	Galois Qudits and their Implementation in Qubits	13
3.2	Expansion of gates	16
3.2.1	Qudit-CCZ gate	16
3.2.2	Control- S gate in $\text{GF}(4)$	18
3.2.3	The norm gate	18
3.2.4	Classification of the diagonal Galois qudit Clifford hierarchy with real phases	18
3.2.5	U_7 gate	22

4	Distillation: transversality conditions and protocol constructions	24
4.1	Generalized triorthogonality on qubit codes and TOF# protocols	24
4.2	Qudit CCZ gate protocols	25
4.3	U_7 gate protocols	26
4.4	Norm gate protocols	30
4.5	Qubit CS protocols from $\mathbb{F}_4$ -linear codes	32
5	Protocols from Algebraic Geometry Codes	37
5.1	Introduction to algebraic curves	37
5.1.1	Genus of a smooth plane curve	37
5.1.2	Curves over finite fields with many points	38
5.2	Introduction to one-point codes	39
5.3	Qudit distillation protocols from punctured one-point codes	43
5.4	Distillation without puncturing	47
5.5	Protocols based on trace codes	48
	Acknowledgments	50
	Appendix A Orthogonality condition for the CS gate	56
A.1	Method 1: Embedded code	56
A.2	Method 2: Direct calculation	57
A.3	Concatenated $[[4, 1, 2]]_4$ and decreasing monomial code	60
A.4	Optimality of asymptotic overhead two in $d=2$ CS-to-CS distillation	62
	Appendix B Algebraic curves: definitions and applications	64
B.1	Advanced preliminaries / Formal Algebraic Geometry Code definitions	64
B.2	Calculation of metric	67
B.2.1	Klein quartic codes	67
B.2.2	Elliptic curve codes	69
B.2.3	One-point codes	69
B.3	Multiplication in binary extension fields	70
	Appendix C Building Distillation Circuits	72
C.1	Distillation protocols and Pauli-based computation (PBC)	72
C.1.1	Galois qudit gates as measurements	73
C.1.2	Realization of Distillation Protocols on Fewer Qudits	75
	Appendix D Time, Space, and Error Calculations	77
D.1	Time and Space Calculations	77
D.2	Error Calculations	79
D.3	Tables of Schemes	83

1 Introduction

1.1 Overview

Gate teleportation is a common way to achieve a universal quantum gateset. Most commonly, the teleported gate is in the third level of the Clifford hierarchy (termed a “third-level gate”) where the input resource state is known as a *magic state*. The usual recipe for the preparation of magic states is to first prepare encoded noisy magic states and *distill* them to high-fidelity magic states, which requires only Clifford operations and measurements. In the standard procedure for magic state distillation [BK05; BH12], a quantum error-detecting stabilizer code encoding k qubits into n qubits with a transversal third-level gate can be used to construct a magic state distillation protocol by 1) preparing a codestate, 2) applying the noisy transversal third-level gate U using input noisy magic states, 3) measuring the syndrome of the code, and 4) outputting the unencoded state if the syndrome is trivial. If the action

of the third-level gate is Clifford-equivalent to encoded $V^{\otimes k}$ for some third-level gate V , then this protocol turns n noisy magic states for U into k less-noisy magic states for V .

In this paper, we construct magic state distillation protocols leveraging algebraic geometry codes over binary extension fields, with a focus on using novel theory to construct practical protocols. A binary extension field is a finite field of size 2^s , denoted $\mathbb{F}_{2^s}$. A CSS code over $\mathbb{F}_{2^s}$ is also a CSS code over $\mathbb{F}_2$ [Wil26]. However, by working directly with the extension field, one is often able to construct qubit codes with desirable properties that would have been very difficult to construct by working directly over qubits. This intuition has been leveraged for magic state distillation before [WHY25; GG25a; Ngu25; NP25], although this is the first time these techniques have been considered in a truly practical setting. The main messages of this paper are as follows:

1. Multi-qubit gates of interest, including CS and CCZ, may be neatly packaged into gates for single Galois qudits.¹ It was previously unknown that such widely-considered multi-qubit gates admitted such neat algebraic descriptions in the Galois qudit picture. With these descriptions, we can prove conditions for Galois qudit codes to admit these gates transversally, thus allowing their distillation.
2. Moving to codes over binary extension fields offers richer algebraic structures, allowing for better code parameters than are possible than by working with the binary field directly, including in the small-size regime of practicality, ultimately resulting in record-breaking protocols.
3. Working with Galois qudit codes allows us to naturally handle the multi-qubit errors present on the corresponding multi-qubit states. This is the same intuition that has led to the wide-spread use of codes over larger alphabets to handle burst errors in classical communication and storage.
4. Constructing codes with novel algebraic techniques often leads to the possibility of distilling exotic magic states. If such an exotic magic state is called for in a particular algorithm, it will often be more efficient to use a specialized magic state distillation protocol for that state, rather than using generic schemes that distill states such as $|T\rangle$, and synthesizing the gate.

Let us give some examples of the above points in our work. For point 1, in Section 4.5, we show that the well-known two-qubit CS gate can be written into a natural form as a single-qudit gate over $\mathbb{F}_4$, and that its transversality can be achieved using a simple three-orthogonality condition. Using the Reed-Solomon code over $\mathbb{F}_4$, we use this to construct a protocol that distills 4 $|\text{CS}\rangle$ states into 1 $|\text{CS}\rangle$ state at distance 2, that can be run on 4 logical qubits. Moving to something slightly larger, by using the $\mathbb{F}_8$ Klein quartic algebraic curve, we show that 24 $|\text{CCZ}\rangle$ states can be distilled to 4 $|\text{CCZ}\rangle$ states at distance 3. For some context here, prior to this work, we are not aware of any way to perform magic state distillation with rate $1/6$ at distance 3, let alone at such small sizes (outside of very pathological and/or large-qudit magic states). There are several examples of such breakthroughs in our catalogue.

Note that, in the two examples mentioned above, the notion of *distance* is the correct one for this situation — that is — the distance is the minimum number of input magic states that must have errors to cause an undetectable logical failure. Because multi-qubit states like $|\text{CS}\rangle$ and $|\text{CCZ}\rangle$ can have correlated errors, this is the correct notion of distance, rather than the usual qubit distance of a code; this is the content of point 3 above. For an example of a more exotic distillation protocol as mentioned in point 4 above, we will show using the $\mathbb{F}_4$ Hermitian algebraic curve that 8 $|\text{CS}\rangle$ states may be distilled into the 5-qubit magic state for the TOF# gate, which is $\text{CCZ}_{123}\text{CCZ}_{345}$. This gate is Clifford-equivalent to a shared-controlled-SWAP, which is used in fermionic Hamiltonian simulation [Bab+18], and other algorithmic subroutines [Chi+03]. We show our full catalogue of new protocols in Section 1.3.1. Following that, in Section 1.3.2, we show that our novel protocols turn out to be exceed the state-of-the-art in a very broad sense, when considering natural distillation tasks, as well as sizes, states, and error rates relevant to realistic quantum computation architectures.

¹Galois qudits are the quantum systems that encode finite extension fields like $\mathbb{F}_{2^s}$. Such a Galois qudit is equivalent to a set of s qubits [Wil26]. Throughout this work, when we refer to a qudit, it is left implicit that we simply mean a Galois qudit, that is, a set of qubits. Moreover, whenever we refer to a quantum code for qudits, we mean a CSS code over $\mathbb{F}_{2^s}$.

1.2 Relation to Prior Work

As discussed above, quantum codes over Galois qudits [Wil26], meaning quantum codes over binary extension fields, were considered recently, to construct asymptotic families of protocols with constant distillation rate for qubits [WHY25; GG25a; Ngu25], following a long line of work improving the asymptotic rate of distillation [BK05; MEK12; Jon13b; Haa+17; HH18b; KT18]. Codes over binary extension fields were subsequently used to construct quantum error-correcting codes with desirable properties for qubits [NP25; GL25; He+25c; He+25b; GG25b; Wil+26]. Finite-size magic state distillation protocols were constructed in the recent work [Cer+25] using codes over binary extension fields, although the resulting protocols are for qudits, not qubits, and the sizes considered are larger than would allow for immediate practical implementation. Nevertheless, our works on the distillation of qudit CCZ gates in binary extension fields — see Section 5 — may be viewed as complementary to theirs.

Further recent works have constructed, or surveyed, magic state distillation protocols in several parameter regimes [NH22; SGJ26; Jac+26], although our protocols lie outside the purview of all of these. A further, related group of works [JMB25; LPX26; Men+26; TB26] constructs transversal non-Clifford gates in the finite length regime, where here the focus is on qLDPC codes and implementation at the *physical* level. Complementarily, our protocols have stronger parameters, and generally distill more directly-useful gates, although are intended for use at the *logical* level. One might consider if our techniques, allowing the construction of distillation protocols taking in exotic multi-qubit gates at the logical level, can be used in tandem with those works, which generate such exotic magic states from physical-level magic.

1.3 Results

1.3.1 Catalogue of Protocols

We begin by presenting a catalogue of our new protocols in Table 1.

1.3.2 Spacetime Footprints for Distillation Tasks

In this section, we discuss how we benchmark the performance of our schemes on distillation tasks of interest, at sizes and error rates of interest, against metrics of interest. The resulting Pareto frontiers are shown in Figures 1 and 2, where we will find that the optimal schemes use our protocols, in the overwhelming majority of situations. We are going to consider various distillation tasks, and examine which concatenation of protocols, both known and novel to this paper, performs best. The distillation tasks we consider are encoded in the following choices:

- **Injected state.** This is the state at the start of the concatenated tower of distillation protocols. We focus on either injected $|T\rangle$ states or $|CS\rangle$ states;
- **Input error rate.** We consider input error rate $p = 10^{-3}$, modeling a situation of direct injection, or $p = 10^{-6}$, modeling a situation of some small amount of cultivation [GSJ24] before injection, as is becoming popular² [Gid25; Lee+25; Xu+25b; Xu+26; Web+26; Cai+26]. It is worth noting that the cultivation of $|CS\rangle$ states has not been studied directly, as far as we know, but is entirely reasonable.
- **Outputted state.** We consider the distillation of either $|CS\rangle$ states, or $|CCZ\rangle$ states, of a given target error rate, as these are valuable states for the purposes of real logical algorithms.

²While cultivation on its own can reach error rates lower than 10^{-6} , doing so can be costly in terms of physical spacetime volume. To reach extremely low logical error rates, or to save physical spacetime volume, a desirable middle ground is to cultivate a small amount to error rates of (roughly) 10^{-6} , before performing distillation. It is true, however, that if one is willing to pay the physical spacetime volume to cultivate down to very low error rates, or if one is not aiming for extremely low logical error rates, only one round of a simple distance-2 protocol [CH17; SGJ26] is likely necessary post-cultivation.

Input	Output	Distance	Method
$(2k+2)$ CS	k CS	$d=2$	$\mathbb{F}_4$ Reed-Solomon
8 CS	1 TOF#	$d=2$	$\mathbb{F}_4$ Hermitian code
$(6k+2)$ CCZ	$2k$ CCZ	$d=2$	$\mathbb{F}_8$ Reed-Solomon
2^s $C^{s-1}Z$	2 $C^{s-1}Z$	$d=2$	$\mathbb{F}_{2^s}$ Reed-Solomon
8 CCS	1 CCS	$d=2$	$\mathbb{F}_8$ Reed-Solomon
16 CS	1 TOF#	$d=3$	$\mathbb{F}_4$ affine monomial code
9 CCZ	1 CCZ	$d=3$	$\mathbb{F}_8$ projective Reed-Solomon
16 CCZ	2 CCZ	$d=3$	$\mathbb{F}_8$ Reed-Solomon
21 CCZ	3 CCZ	$d=3$	$\mathbb{F}_8$ projective monomial code
(2^s+1) $C^{s-1}Z$	1 $C^{s-1}Z$	$d=3$	$\mathbb{F}_{2^s}$ projective Reed-Solomon
12 CS	1 CS	$d=3$	$\mathbb{F}_4$ projective monomial code
16 CS	2 CS	$d=3$	$\mathbb{F}_4$ affine monomial code
21 CS	4 CS	$d=3$	$\mathbb{F}_4$ projective monomial code
24 CCZ	4 CCZ	$d=3$	$\mathbb{F}_8$ Klein quartic
64 T	2 TOF#	$d=4$	$\mathbb{F}_2$ Reed-Muller
64 T	1 $\mathbb{F}_4$ -CCZ	$d=4$	$\mathbb{F}_{16}$ Hermitian trace code
64 T	2 CCZ	$d=4$	$\mathbb{F}_{16}$ Hermitian trace code
64 CCZ	8 CCZ	$d=4$	$\mathbb{F}_8$ affine monomial code

Table 1: Catalogue of new protocols in this paper. For each new protocol, we write the input, the output, the distance, and the method used to obtain the protocol. Each method is hyperlinked to the construction in the paper. Note that a protocol distilling 64 T states to 2 CCZ states at distance 4 is already known [HH18a], although our novel protocol has a lower leading-order probability of any error in the factory: $2720p^4$ versus $2944p^4$. In the table, we emphasize that all states are regular qubit magic states. TOF# is the 5-qubit magic state $\text{CCZ}_{123}\text{CCZ}_{345}$. Similarly, $\mathbb{F}_{2^s}$ -CCZ is a $3s$ -qubit magic state formed from a certain product of CCZ gates; see Section 3.2.1. There are further novel protocols taking $\mathbb{F}_{2^s}$ -CCZ as input and output; see Sections 4.2 and 5.

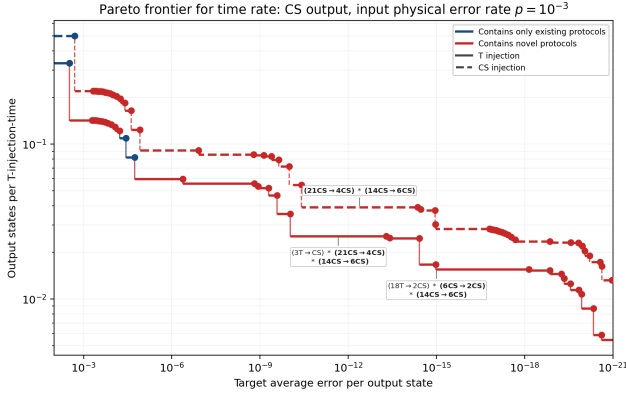
For each of the 8 distillation tasks encoded in the above 3 choices, we aim to construct the best distillation protocols according to some metric, considering concatenated chains of both our novel protocols, and the existing protocols. We may make different choices for the success metric, as follows:

- **Success metric:** Either *time rate*: average retained number of output magic states of the target fidelity per unit time, or *logical volumetric rate* (or simply *volume rate*): average retained number of output magic states of the target fidelity per unit logical spacetime footprint. Our calculations of time and logical spacetime volume will be elucidated below, and explained in detail in App. D.

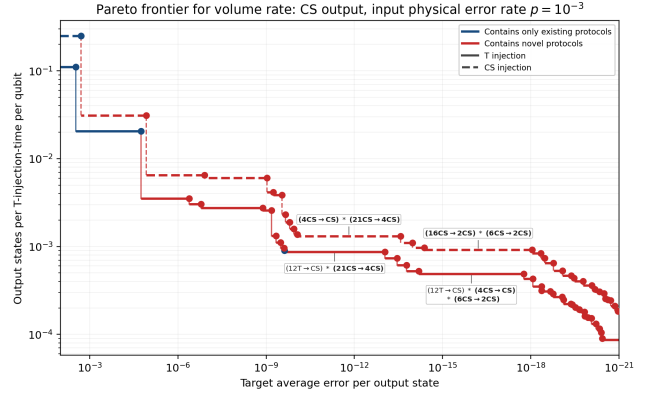
Ultimately, we will find that our novel protocols will feature extensively in the optimal frontier of protocols, for almost every choice of injected state, input error rate, outputted state, and success metric.

We assume that magic state distillation protocols take place on logical qubits. Note that, when setting up magic state distillation protocols in an architecture, several design choices may be made, and we find that these different choices compound significantly when considering concatenated schemes. However, we aim to make choices that capture many of the popular architectures for realistic fault-tolerant quantum computation.

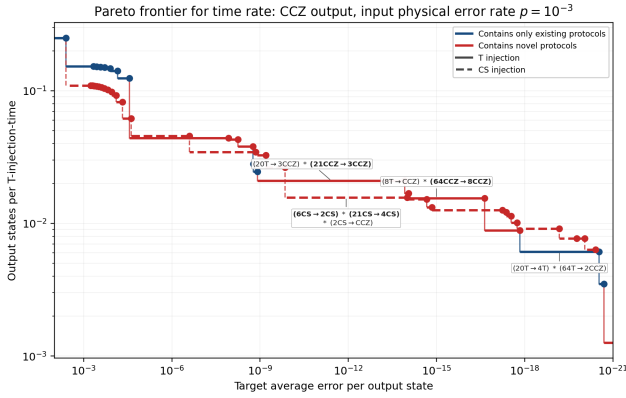
To this end, we consider the implementation of magic state distillation protocols in an abstract Pauli-based computation (PBC) model as detailed in [Lit19b]. This approach allows us to define a relatively architecture-agnostic cost model for implementation of magic state distilleries in logical qubits, be they in surface codes [Lit19a], bivariate bicycle codes [Yod+25], homological product codes [Xu+25a], or general qLDPC architectures [He+25a]. We assume magic state distillation protocols are executed on logical qubits, using a gate set of multi-qubit Pauli measurements between logical qubits and optionally undistilled $|T\rangle$ states. Techniques for protocol implementation using PBC [Lit19b] readily generalize [Bev+20] to non- $|T\rangle$ multi-qubit input and output states using diagonal gate injection.



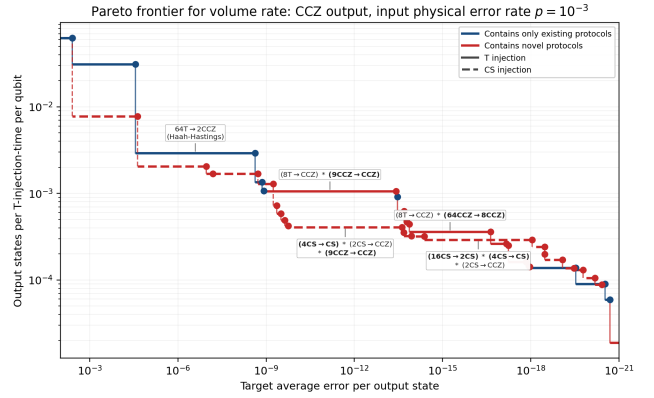
(a) CS distillation: optimal for time rate



(b) CS distillation: optimal for volumetric rate



(c) CCZ distillation: optimal for time rate



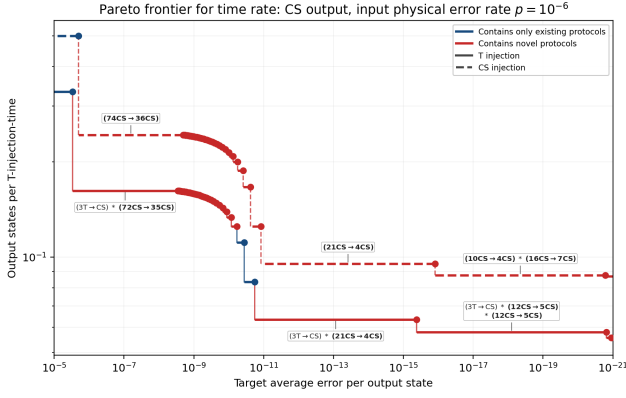
(d) CCZ distillation: optimal for volumetric rate

Figure 1: Pareto frontiers for physical input error $p = 10^{-3}$. Solid lines use $|T\rangle$ state injection and dashed lines use $|CS\rangle$ state injection. Red lines indicate that our novel protocols are used somewhere in the competitive scheme; blue lines indicate that they are not. Our novel protocols are bolded in labels. Statistics for some of the competitive protocols are given in Appendix D.3.

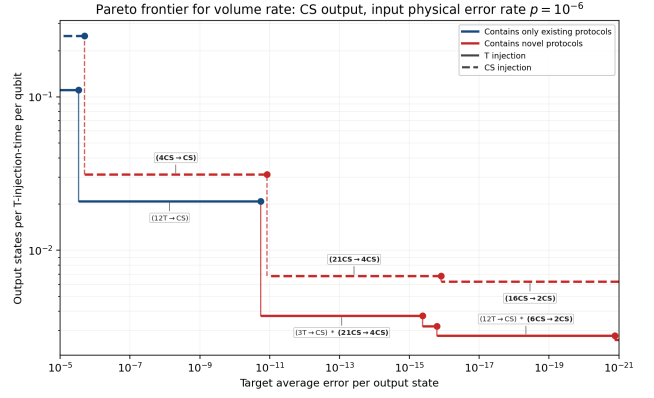
In Appendix C, we extend these techniques to the Galois qudit codes considered in this paper.

In large quantum computing architectures targeting extremely low error rates, magic state factories are expected to dominate the overall cost of the computation [Lee+21]. This motivates dedicating a large number of logical qubits in the computer to magic state production, meaning output states per unit spacetime volume is the natural cost metric. However, in early fault-tolerant quantum architectures, logical error rates are less stringent (e.g., [Gid25]), but space constraints are more severe. In this setting, architectures [Yod+25] consider allocating a single magic state factory instance within a module. Then, footprint becomes more of a measure of feasibility (does the factory fit within a single module, yes or no?), and the relevant measure of cost is time. Our analysis considers both situations and can therefore be used to evaluate both large-scale and small-scale FTQC settings. To these ends, we elucidate the calculation of our cost metrics; full explanations are deferred to App. D.

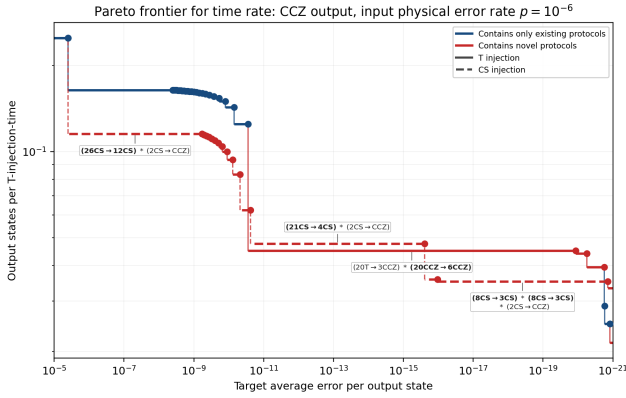
Time: We consider a simplified timing model for which a unit of time is one $|T\rangle$ state injection. We imagine that $|T\rangle$ states are always consumed sequentially: an implementation choice. Our model neglects additional overhead from Clifford synthesis and routing deliberately, since these concerns are architecture-specific. For the concatenated distillation protocols involving $|CS\rangle$ state injections, one state injection is deemed to take two units of time. The time metric we will calculate will be a rate: the average number of retained output magic states of the target fidelity per unit time. Ignoring the possibility of postselection, this is simply the number of magic states produced by a concatenated protocol divided by the number of inputs (divided by two in the case of $|CS\rangle$ injection). We account for the overhead from repeating the protocol due to postselection in our analysis, although the corresponding effect on the time rate is generally



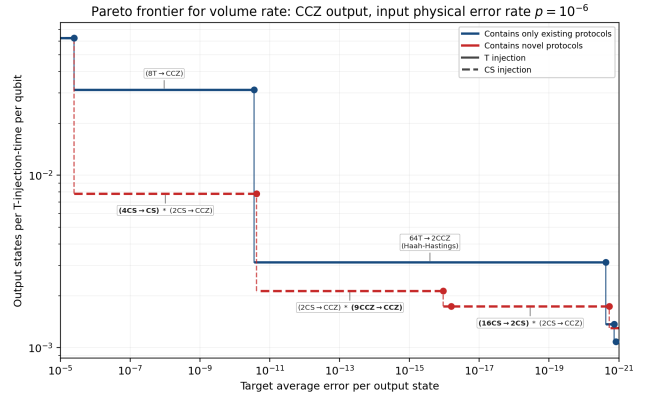
(a) CS distillation: optimal for time rate



(b) CS distillation: optimal for volumetric rate



(c) CCZ distillation: optimal for time rate



(d) CCZ distillation: optimal for volumetric rate

Figure 2: Pareto frontiers for physical input error $p = 10^{-6}$ (modeling some small amount of cultivation pre-injection). Solid curves use $|T\rangle$ state injection and dashed curves use $|CS\rangle$ state injection. Red lines indicate that our novel protocols are used somewhere in the competitive scheme; blue lines indicate that they are not. Our novel protocols are bolded. Statistics for some of the competitive protocols are given in Appendix D.3.

small.

Spatial Footprint: In a small, early fault-tolerant quantum computer, or in a modular architecture with a limited number of qubits per module, it is preferable to implement protocols on a small number of logical qubits. Typically, this is the number of rows in the binary matrix describing the protocol [Lit19a].³ For our $\mathbb{F}_4$ and $\mathbb{F}_8$ -Galois qudit codes describing CS-to-CS distillation and CCZ-to-CCZ distillation, respectively, we show in App. D that Litinski’s ideas [Lit19a] carry over naturally, and that the protocols may be run on a number of logical Galois qudits equal to the number of rows in the $\mathbb{F}_4$ or $\mathbb{F}_8$ matrix. The corresponding logical qubit footprint is then twice, or three times this, respectively. We further consider a compression technique [Xu+26], which in certain special cases allows the protocol to be run in even smaller space, although this applies to a minority of the protocols considered.

Spacetime Volume: Simply defined as the product of time and compressed footprint, volumetric considerations are natural when scaling production across many parallel factory instances. The metric calculated will again be a rate: the average number of retained output magic states of the target fidelity per unit spacetime volume.

We add a final remark on the error rate calculations.

³The number of rows is equal to the number of qubit X logicals and X stabilizer generators describing the corresponding code.

Output error rate: Since our protocols generally give more than one magic state output, the natural error rate to calculate is average error rate per-output. This differs from the more commonly quoted error rate of a distillation protocol, which is the probability of any error. However, note that because a failing protocol typically has highly correlated errors, the average error rate per-output is usually much higher than the probability of any error divided by the number of outputs. We go into much more detail on these calculations in App. D.

Now turning to our results, we define a concatenated protocol to be competitive if, for some distillation task, for some metric, for some target output error rate, it is optimal. For these distillation tasks, the protocols included in our survey are as follows; this is intended to be comprehensive over the protocols that could be competitive for the present tasks. We consider concatenations of the following:

- Our novel protocols: Our CCZ-to-CCZ distillation protocols — see Section 4.3 — based on $\mathbb{F}_8$ -Galois qudit codes, our CS-to-CS distillation protocols — see Section 4.5 — based on $\mathbb{F}_4$ -Galois qudit codes, our novel $64T$ -to- $2CCZ$ ($d = 4$) protocol⁴ — see Section 5.5. Note that this paper contains many more novel protocols than these; we are only considering the ones here that could be competitive for the present tasks.
- Existing protocols: Synthillation protocols [CH17], including the $(7k + 4 + (k \bmod 2))T \rightarrow kCS$ and the $(6k + 2)T \rightarrow kCCZ$ protocols, both at distance 2, the well-known $15T \rightarrow T$ protocol at distance 3 [BK05], $(3k + 8)T \rightarrow kT$ protocols, for even k , at distance 2 [BH12], the Haah-Hastings $64T \rightarrow 2CCZ$ protocol at distance 4 [HH18a], and the known synthesis (non-distillation) conversions of $3T \rightarrow CS$ [HC17], and $2CS \rightarrow CCZ$ [Bev+20].

We further comment that, in finding the competitive protocols, we restrict ourselves to concatenated distance at most 8, and spatial footprint at most 75.⁵ The resulting Pareto frontiers are shown in Figures 1 and 2. We find that our novel protocols feature heavily in the competitive schemes at the target error rates of interest; the only exception is when considering $|T\rangle$ state injection at physical error rate 10^{-6} (assuming some small amount of cultivation), and where the metric for success is volumetric rate. In this case, either the $8T \rightarrow CCZ$ synthillation protocol [CH17], or the $64T \rightarrow 2CCZ$ Haah-Hastings protocol [HH18a] are found to be the competitive protocols.

1.4 Open Questions

- Can one design larger codes *maximally* satisfying the twisted three-orthogonality condition of Eq. 41 for distilling the single-qudit U_7 gates [WHY25]? This is particularly relevant since the U_7 gate is equivalent to the widely-considered qubit CCZ gate over the field $\mathbb{F}_8$; see Section 3.2.5. Similar twisted products also appear when considering trace codes of AG codes. In this work, for this problem we develop a numerical approach, which is to build a forbidden hypergraph, and solve for its maximum independent set (MIS); see Construction 4.11 and 5.8. However, MIS is an NP-hard problem, and so this approach does not scale.

Even in the small-codes examples we developed in this work, we have to restrict ourselves to codewords coming from monomial evaluations for finding CCZ-to-CCZ (U_7 in $\mathbb{F}_8$) distillation protocols. A better understanding, or more geometrical picture of the twisted three-orthogonality condition, might help in generalizing our constructions to larger lengths and alphabet sizes. Such an understanding might help in establishing the best asymptotic achievable rate and distance of U_7 distillation protocols. Moreover, once we have subspaces satisfying the twisted three-orthogonality condition over large alphabets, can one take the trace [Coh+25], subfield subcode [DGM70], or the subspace code [HMS98], to obtain codes in a smaller alphabet (since U_7 is simpler there)?

⁴Note that our novel distance-4 $64T$ -to- $2CCZ$ protocol has the same parameters as the existing Haah-Hastings protocol [HH18a]. However, our novel protocol has a lower leading-order total error rate: $2720p^4$ versus $2944p^4$ (albeit an equal per-state average error rate of $2368p^4$). Unfortunately, our novel protocol also requires greater space footprint, and thus does not appear in any competitive protocols.

⁵Competitive protocols for time rate tend to be larger, as better rates can be achieved at larger sizes, especially using our novel protocol families. We cap the allowed spatial footprint at 75 to model a situation in which a quantum architecture sacrifices a moderate number, in this case 75, of logical qubits, for the given distillation task.

- Can two- and multipoint divisors [Mat01; DK11; Cer+25], and improved order bounds [Bee07] improve the parameters of our distillation protocols? Can other algebraic geometry objects, such as curves of positive-defect, or higher-dimensional varieties [Han01] help with the code design?
- For real-phase qudit gates, we have only classified their Clifford hierarchy level, but not classified them up to Clifford equivalence; see Section 3.2.4. For example, in Sec. 3.2.5, we leave the question open whether the three possible U_7^β gates in $\mathbb{F}_{2^s}$, where $3 \mid s$, are Clifford inequivalent in general. For non-real-phase gates, due to the difference between $\mathbb{F}_{2^s}$ and $\mathbb{Z}_{2^s}$, we find it difficult to even classify the Clifford hierarchy levels of the corresponding gates. This is problematic, since if one wants to distill some multi-qubit magic gate, one naturally tries to pack such a multi-qubit gate into a Galois qudit gate, and derive the corresponding orthogonality condition for transversality. A better understanding of the non-real-phase magic gate would make this process easier.
- For our CCZ and CS-input protocols, we are mostly enforcing the logical action to be disjoint CCZ and CS (except Con. 4.22 and Con. 4.21). However, hypergraph state outputs also provide an interesting design space. Can they be directly used for some algorithmic subroutines (for example, we show that qudit-CCZ gate can be used to perform multiplication in the corresponding binary fields)? If not, how can one extract relevant simple-enough hypergraph states (e.g. TOF#) from a complicated hypergraph magic state? We gave such examples in Con. 4.1 & 5.11), but a systematic understanding is lacking, except for extracting disjoint CCZs from a CCZ-only circuit, which was shown to be equivalent to the subrank problem of trilinear forms [Men+26].
- How can one implement the distillation factories we found most efficiently in a broader architecture? Especially, how can one compile these protocols at the logical level in relevant qLDPC codes? One might need to optimize the weight of our protocols, co-optimize $\mathbb{F}_2$ -bases and qLDPC code logical bases to reduce surgery overhead.
- The distillation protocols constructed in this work are mostly algebraically motivated, and we know little about their optimality. Recent SAT-solver [Jac+26] or symmetry-based protocol discovery approaches [SGJ26] might help either certifying the optimality, or find smaller factories.

1.5 Organization

We begin in Section 2 by presenting preliminary material necessary for Sections 3 and 4. In Section 3, we prove results on Galois qudits and their gates which will support our protocol constructions. A result in this section which lies outside our main contributions can be found in Section 3.2.4, where we give a classification of the diagonal gates in the Galois qudit Clifford hierarchy with real phases in a way that is native to those gates, resolving an open question in [He+25c]. Section 4 then constructs our most important protocols. From there, Section 5 constructs more exotic protocols that are outside our primary scope. These use more advanced algebraic geometric techniques, and we begin that section with the necessary preliminary material.

Appendix A proves several facts around our distillation of $|\text{CS}\rangle$ using $\mathbb{F}_4$ codes. Appendix B then presents complete preliminaries on algebraic geometry codes in order to supplement Section 5. Appendix C discusses how our Galois qudit magic state distillation protocols may be run most compactly in hardware, by showing how known tricks for the regular qubit factories transfer seamlessly to this case. Finally, Appendix D goes into detail on our error rate and spacetime footprint calculations, finishing with tables of statistics on some of the competitive schemes.

The SageMath notebooks containing all the necessary tools to reproduce and verify our protocols are available online at <https://github.com/gongaa/MSD-binary-fields>.

2 Preliminaries

2.1 Diagonal Gates in the Clifford hierarchy

The Clifford hierarchy is a recursively defined hierarchy of quantum gates [GC99], whose exact classification remains a major open problem. While the k -th level of the Clifford hierarchy does not form a group, the diagonal gates in the k -th level do form a group, and furthermore the diagonal gates in the Clifford hierarchy have been classified [CGK17]. We consider the following diagonal n -qubit gates (and their products):

$$U_{m,\mathbf{a}} := \sum_{\mathbf{x} \in \mathbb{F}_2^n} \exp\left(\frac{2\pi i}{2^m} x_1^{a_1} \dots x_n^{a_n}\right) |\mathbf{x}\rangle \langle \mathbf{x}|, \quad (1)$$

where m is a positive integer and $\mathbf{a} = (a_1, \dots, a_n) \in \mathbb{F}_2^n$. One immediate example is $n = 2, m = 1$ and $\mathbf{a} = (1, 1)$, which gives the CZ gate. $U_{m,\mathbf{a}}$ is in the Clifford hierarchy for n qubits and its level is $(m - 1) + \text{wt}(\mathbf{a})$. Moreover, these gates can be used to generate the diagonal Clifford hierarchy. The magic state corresponding to $U_{m,\mathbf{a}}$, often denoted $|U_{m,\mathbf{a}}\rangle$, is defined as the gate $U_{m,\mathbf{a}}$ acting on $|+\rangle^{\otimes n}$. Using gate teleportation [GC99][Bev+20, Alg. A.1], one may consume the magic state corresponding to $U_{m,\mathbf{a}}$ to execute the gate $U_{m,\mathbf{a}}$ on an arbitrary n -qubit state using only operations in lower levels of the Clifford hierarchy. In particular, a gate in the third level can be teleported using Clifford operations.

The most famous gates (and corresponding states) of this kind are T , CS and CCZ. More general $U_{m,\mathbf{a}}$ gates, and their products, can be described by “phase polynomials”. For example, when $n = 5$ and $m = 1$, the phase polynomial $x_1x_2x_3 + x_3x_4x_5$ corresponds to the multiplication of two CCZ gates (overlapping on a single qubit), and is known as the TOF# gate:

$$\text{TOF}\#_{12345} := \sum_{\mathbf{x} \in \mathbb{F}_2^5} (-1)^{x_1x_2x_3 + x_3x_4x_5} |\mathbf{x}\rangle \langle \mathbf{x}| \quad (2)$$

This gate is useful in many algorithmic subroutines, for example, in a log-depth adder [Dra+04]. Additionally, TOF# can also be used to execute a shared-controlled-SWAP; indeed, $\text{TOF}\#_{12345} = \text{CCZ}_{123}\text{CCZ}_{145}$ is Clifford-equivalent to C_1 -(SWAP₂₃SWAP₄₅) via

$$C_1\text{-(SWAP}_{23}\text{SWAP}_{45}) = \text{CNOT}_{23}H_3\text{CNOT}_{45}H_5 \cdot (\text{TOF}\#_{12345}) \cdot H_5\text{CNOT}_{45}H_3\text{CNOT}_{23}. \quad (3)$$

The shared-controlled swap is prevalent in fermionic Hamiltonian simulation [Bab+18], and other algorithmic subroutines [Chi+03].

More generally than the above, one can consider the notion of Clifford equivalence:

Definition 2.1 (Clifford Equivalence). *We say two unitary gates U and V are Clifford equivalent, if there exist Clifford unitaries C and C' , such that $U = CVC'$.*

It is possible to exhaustively classify (at least at small sizes) classes of phase polynomial gates up to Clifford equivalence. For example, considering only gates formed from products of CCZ gates, one can check that there is only one such gate up to Clifford equivalence on four qubits, and it is Clifford equivalent to a single CCZ gate. Further, on five qubits, any product of CCZ gates is Clifford equivalent to either a single CCZ gate, or the TOF# gate. Such examples are illustrated in Figure 3.

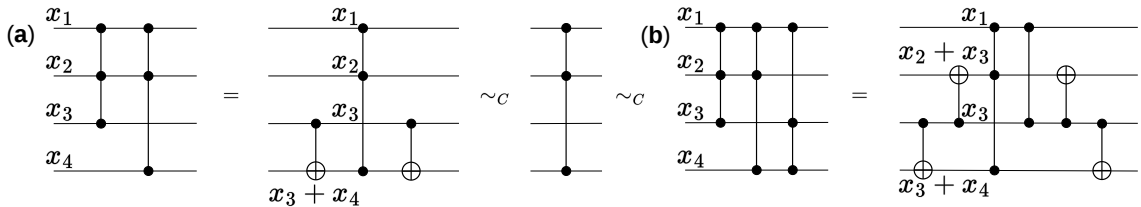


Figure 3: Examples of Clifford equivalence. (a) and (b) are both Clifford equivalent to a single CCZ gate. (a) $x_1x_2x_3 + x_1x_2x_4 = x_1x_2(x_3 + x_4)$. (b) $x_1x_2x_3 + x_1x_2x_4 + x_1x_3x_4 = x_1(x_2 + x_3)(x_3 + x_4) + x_1x_3$.

Relatedly to Clifford equivalence, it is useful to establish the most efficient means to synthesize various gates from others. For example, the ancilla-free T count for a diagonal gate in the third level of

the Clifford hierarchy is defined in [CH17] to be the *minimum* number of T gates required to synthesize the gate unitarily with T gates, CNOT gates, and S gates, and using no ancillas. The ancilla-free T count of CCZ is 7. Such transformations can be discovered by manipulating phase polynomials. For example, considering variables $x_i \in \{0, 1\}$, one verifies the identity

$$4x_1x_2x_3 \equiv x_1 + x_2 + x_3 - (x_1 \oplus x_2) - (x_1 \oplus x_3) - (x_2 \oplus x_3) + (x_1 \oplus x_2 \oplus x_3), \quad (4)$$

where $+$ denotes regular integer addition, and $\oplus$ denotes integer addition modulo 2.⁶ It follows from this identity that the CCZ state (or gate) can be unitarily synthesized using 7 T gates, as well as CNOT operations, by considering various $U_{m,a}$ gates with $n = 3$ and $m = 3$. Because $m > 1$, we must differentiate between regular integer addition and addition modulo 2. Explicitly, we have

$$\exp\left(\frac{2\pi i}{2^m}(x_1 + x_2)\right) \neq \exp\left(\frac{2\pi i}{2^m}(x_1 \oplus x_2)\right) \text{ for } m > 1. \quad (5)$$

This is unlike when we consider products of CCZ gates like the TOF# gate, in which case addition in the phase polynomial is both regular integer addition and modulo 2 addition. For example, for the TOF# gate,

$$\exp(\pi i(x_1x_2x_3 + x_3x_4x_5)) = \exp(\pi i(x_1x_2x_3 \oplus x_3x_4x_5)). \quad (6)$$

It is also found in [CH17] that the ancilla-free T count of the TOF# gate mentioned above is 11. We note that this is less than twice the ancilla-free T count of twice that of CCZ, showing that it is more efficient to directly synthesize TOF# than to synthesize two CCZ gates separately, at least in this setting. Synthillation techniques [CH17] also allow one to distill 12 T gates to 1 TOF# gate at distance 2 (see App.C for the explicit example).

Hadamard gates and measurement-and-feedback add more power over merely using CNOT+ T . The most famous example is Jones' construction of a CCZ gate from four T gates [Jon13a]. An interpretation [Bev+20] is that, applying $HS\bar{H} = \sqrt{X}$ on the third qubit on the $|\text{CCZ CS}_{1,2}\rangle = \text{CCZ CS}_{12}|+++\rangle$ state (can be unitarily synthesized using CNOTs and four T) leads to the $|\text{CCZ}\rangle$ state [Bev+20, Fig. 12(b)], which can be further teleported to realize a CCZ gate. As another example, while unitary synthesis of a CCZ gate (using CNOT+CS) requires 3 CS gates, only 2 CS gates are required for the $|\text{CCZ}\rangle$ state [Bev+20, Fig. 13].

2.2 Introduction to rings, finite fields and Reed-Muller codes

We now give a brief introduction to rings and finite fields. The following definitions and facts can be found in many textbooks such as [LN96]; we thus state the most relevant ones without proof.

A *ring* is a set R equipped with two binary operations $+$ (addition) and $\cdot$ (multiplication) such that: R is an abelian group with respect to $+$; $\cdot$ is associative, that is, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$, $\forall a, b, c \in R$; multiplication is distributive over addition, $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(b + c) \cdot a = b \cdot a + c \cdot a$. In this work, we only work with commutative rings (those for which $a \cdot b = b \cdot a$ for all $a, b \in R$) with a (multiplicative) identity (those containing an element called 1 satisfying $1 \cdot a = a$ for all $a \in R$).

Given a positive integer n and $r \in R$, we write nr for the element of R obtained by summing r n times. If, for a ring R , there exists a positive integer n such that $nr = 0$ for every $r \in R$, then the least such positive integer n is called the *characteristic* of R .

In a ring, multiplicative inverses are not required to exist. A ring in which every nonzero element has a multiplicative inverse is called a *field*. A subset of fields are integral domains, which are integral domains. These are rings in which there are no zero divisors, meaning that if $ab = 0$, then $a = 0$ or $b = 0$. Given an integral domain R , one may construct a field from it called its fraction field, $\text{Frac}(R)$, which contains a copy of R . The elements of the fraction field are the equivalence classes of $R \times (R \setminus \{0\})$, under the equivalence relation $(a, b) \sim (c, d) \iff ad = bc$. Canonically, the equivalence class of (a, b) is denoted $\frac{a}{b}$, and the operations of addition and multiplication are defined via $\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd}$, and $\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}$.

An example of a field is $\mathbb{F}_p = \{0, 1, \dots, p-1\}$ for a prime p , where the operations are achieved by arithmetic modulo p . More generally, a finite field (also known as a Galois field) K must have $q = p^s$ elements for some $s \in \mathbb{N}$, where the prime $p := \text{char } K$ is the characteristic of K , and s is the *extension*

⁶The minimality of the various ancilla-free T counts is established by a relation to the decoding of a Reed-Muller code [AM19].

degree of K over $\mathbb{F}_p$. For all primes p and $s \in \mathbb{N}$, $\mathbb{F}_{p^s}$ exists and it is unique up to isomorphism. It is canonical to denote the corresponding K as $\mathbb{F}_q$ or alternatively $\text{GF}(q)$. The *multiplicative group* $\mathbb{F}_q^\times$ of nonzero elements of $\mathbb{F}_q$ is cyclic, meaning that there exists at least one *primitive element* α such that $\mathbb{F}_q = \{0, \alpha, \alpha^2, \dots, \alpha^{q-1} = 1\}$. Since any power of 1 is 1 itself, one has $\gamma^{q-1} = \begin{cases} 1 & \gamma \in K^\times \\ 0 & \gamma = 0 \end{cases}$, and hence also $\gamma^q = \gamma$, $\forall \gamma \in \mathbb{F}_q$. q is called the *order* of the finite field $\mathbb{F}_q$.

A subset F of K that is itself a field is called a *subfield* of K , and K is called an *extension field* of F . It turns out that every *subfield* of $\mathbb{F}_{p^{s'}}$ has order p^s , where s is a positive integer dividing s' . Let $q = p^s$ and $m = s'/s$ in the following.

A field isomorphism is a bijective map from one field to another that preserves multiplication and addition. A field automorphism on a field K is a field isomorphism from K to itself. The group of automorphisms of $\mathbb{F}_{q^m}$ that fix every element of $\mathbb{F}_q$ ($q = p^s$) is also referred to as the *Galois group* of $\mathbb{F}_{q^m}$ over $\mathbb{F}_q$, and it is generated cyclically by the *Frobenius transform* $\sigma_1 : \alpha \mapsto \alpha^q$ ($\alpha \in \mathbb{F}_{q^m}$). To be more explicit, we can write the elements of the Galois group as the mappings $\sigma_0, \sigma_1, \dots, \sigma_{m-1}$ defined by $\sigma_j(\alpha) = \alpha^{q^j}$ for $\alpha \in \mathbb{F}_{q^m}$. The images $\sigma_j(\alpha)$ are called the *Galois conjugates* of α . Note that indeed $\sigma_j(\gamma) = \gamma$ for $\gamma \in \mathbb{F}_q$.

The *trace* $\text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha)$ of $\alpha \in \mathbb{F}_{q^m}$ is the sum of the conjugates of α , i.e.,

$$\text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha) = \alpha + \alpha^q + \alpha^{q^2} + \dots + \alpha^{q^{m-1}}. \quad (7)$$

The trace maps into $\mathbb{F}_q$ surjectively. The trace function $\text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}$ is $\mathbb{F}_q$ -linear,

$$\text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\gamma_1 \alpha_1 + \gamma_2 \alpha_2) = \gamma_1 \text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha_1) + \gamma_2 \text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha_2), \quad \forall \gamma_1, \gamma_2 \in \mathbb{F}_q \text{ and } \forall \alpha_1, \alpha_2 \in \mathbb{F}_{q^m}, \quad (8)$$

where the additivity follows from the fact that $(a+b)^p = a^p + b^p$ (and hence $(a+b)^{p^i} = a^{p^i} + b^{p^i}$) for $a, b \in \mathbb{F}_{p^s}$ since $\binom{p}{i} = \frac{p!}{i!(p-i)!} \equiv 0 \pmod{p}$ for $i = 1, 2, \dots, p-1$. We have $\text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha^q) = \text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha)$. A further, less trivial fact, is that $\text{tr}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha) = 0$ if and only if $\alpha = \beta^q - \beta$ for some $\beta \in \mathbb{F}_{q^m}$.

The *norm* $\text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}$ of α is the product of all its conjugates:

$$\text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha) = \alpha \cdot \alpha^q \cdot \dots \cdot \alpha^{q^{m-1}} = \alpha^{(q^m-1)/(q-1)}. \quad (9)$$

The norm function is multiplicative: $\text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha\beta) = \text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha)\text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\beta)$, $\forall \alpha, \beta \in \mathbb{F}_{q^m}$. As for the trace, $\text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha^q) = \text{Nm}_{\mathbb{F}_{q^m}/\mathbb{F}_q}(\alpha)$, and the norm function maps into $\mathbb{F}_q$ surjectively.

For a general ring (not necessarily a field), an ideal I of a ring R is a subset of R that is itself a ring and, moreover, $\forall a \in I$ and $r \in R$ we have $ar = ra \in I$. We will only be working with finitely generated ideals. Such ideals are formed by considering $\{a_1, \dots, a_k\}$ a subset of R , and the ideal generated by them is $(a_1, \dots, a_k) := \{a_1 r_1 + \dots + a_k r_k \mid \forall r_1, \dots, r_k \in R\}$. We will sometimes use $\langle a_1, \dots, a_k \rangle$ to denote the ideal generated by $\{a_1, \dots, a_k\}$.

R/I is the *quotient ring* of R by the ideal I . Its addition and multiplication are given by $(r+I) + (s+I) = (r+s)+I$, $(r+I) \cdot (s+I) = rs+I$. In the quotient ring when $I = \langle a_1, \dots, a_k \rangle$, it is convenient to write $R/I = R/(a_1, \dots, a_k)$. In this quotient ring, one can identify $a_1 = 0, \dots, a_k = 0$.

In coding theory, e.g., when defining the Reed-Muller codes, we will frequently deal with *polynomial rings* and their quotient rings. Letting x be an indeterminate variable, the polynomial ring $R[x]$ is the set of formal sums $a_n x^n + \dots + a_1 x + a_0$ under usual addition and multiplication. Multivariate polynomial rings are defined inductively by $R[x_1, x_2, \dots, x_n] := R[x_1, x_2, \dots, x_{n-1}][x_n]$. In the case that $R = K$ for some field K of characteristic p , we have that p times anything equals zero.

To define Reed-Muller codes, we consider the quotient ring

$$\mathbb{F}_q[x_1, \dots, x_m]/(x_1^q - x_1, \dots, x_m^q - x_m), \quad q = 2^s.$$

The following definitions and facts about Reed-Muller codes may be found in [MS77]. The q -ary Reed-Muller code in m variables of degree r is denoted $RM_q(r, m)$, and is the length- q^m code over $\mathbb{F}_q$ formed by evaluating all polynomials in the above ring of degree at most r at all points $(x_1, x_2, \dots, x_m) \in \mathbb{F}_q^m$. Because $\eta^q = \eta$ for all $\eta \in \mathbb{F}_q$, quotienting by the terms $(x_i^q - x_i)$ in the above ring allows us to identify the given polynomials with the corresponding codewords of the Reed-Muller code in a one-to-one fashion. For example, note that the degree of any individual variable in any polynomial in the quotient ring does not exceed $q-1$.

For Reed-Muller codes, a particularly important example for us will be the binary Reed-Muller

code with $q = 2$, defined over the quotient ring

$$\mathbb{F}_2[x_1, \dots, x_m] / (x_1^2 - x_1, \dots, x_m^2 - x_m).$$

The distance of $\text{RM}_2(r, m)$ turns out to be 2^{m-r} ; indeed, one may quickly see that it must be at most this by considering the degree- r polynomial $x_1 x_2 \dots x_r$, whose evaluation has weight 2^{m-r} . We may denote by $\text{ev}(f)$ the codeword corresponding to the polynomial f . Then we have that the coordinate-wise multiplication of $\text{ev}(f)$ and $\text{ev}(g)$ is $\text{ev}(fg)$. More generally, one can see that the weight of a codeword corresponding to the evaluation of a monomial formed from the product of v variables is 2^{m-v} . The code dual to $\text{RM}_2(r, m)$ is $\text{RM}_2(m - r - 1)$.

Analogous facts hold more generally for q -ary Reed-Muller codes. For example, the distance of $\text{RM}_q(r, m)$ is $(\mu + 1)q^\nu$, if we write $(q - 1)m - r = \nu(q - 1) + \mu$ for $0 \leq \mu < q - 1$. The coordinate-wise multiplication of $\text{ev}(f)$ and $\text{ev}(g)$ is $\text{ev}(fg)$ in the general q -ary case, also.

More general codes, called *affine monomial codes*, can be considered by evaluating different sets of polynomials (than those of degree at most r) at the points of $\mathbb{F}_q^m$. A key example for us will be the hyperbolic codes. These can be obtained from Reed-Muller codes by deleting parity checks, increasing the code dimension while keeping the same minimum distance [HLP98, Ch. 4.4]. Concretely, a basis of the hyperbolic code of designed distance d is formed by the monomials $X_1^{\alpha_1} \dots X_m^{\alpha_m}$ where $\prod_{i=1}^m (\alpha_i + 1) < d$. A more general class of affine monomial codes will be considered in App. A.3.

2.3 Affine and projective space

The *affine space* over the field K , is simply the vector space K^n over K ; this is usually denoted $\mathbb{A}_K^n$ or just $\mathbb{A}^n$ when the field K is implicit. We write all its elements as $\mathbb{A}_K^n = \{(x_1, \dots, x_n) \mid x_i \in K\}$. The *projective space* over a field K , denoted as $\mathbb{P}_K^n$, or just $\mathbb{P}^n$ when K is implicit, is defined by quotienting K^{n+1} by scalar multiplication by elements of $K^\times = K \setminus \{0\}$. Explicitly,

$$\mathbb{P}^n = (K^{n+1} \setminus \{0\}) / \sim,$$

where the equivalence relation $\sim$ on K^{n+1} is defined as

$$(x_0 : \dots : x_n) \sim (\lambda x_0 : \dots : \lambda x_n) \quad \forall \lambda \in K^\times.$$

Take $\mathbb{P}_{\mathbb{F}_4}^2$ as an example, one can enumerate all its elements in the following way: $\{(1 : y : z) \mid y, z \in \mathbb{F}_4\} \cup \{(0 : 1 : z) \mid z \in \mathbb{F}_4\} \cup \{(0 : 0 : 1)\}$.

Let $U_i \subset \mathbb{P}^n$ be the subset of points $(X_0 : \dots : X_n)$ with $X_i \neq 0$. Then on U_i the ratios $x_j = X_j / X_i$ are well-defined and give a bijection $U_i \cong \mathbb{A}^n$. Notice that $U_0, \dots, U_n$ together cover $\mathbb{P}^n$. We will later refer to U_i as the $X_i = 1$ affine chart covering $\mathbb{P}^n$.

A homogeneous polynomial is a polynomial for which all terms have a common degree. An example would be

$$f^h(x_0, x_1, x_2) = x_0^2 x_1 x_2 + x_1^3 x_2,$$

which is a homogeneous polynomial of degree 4. Notice that if F is a homogeneous polynomial of degree d then

$$f^h(\lambda x_0, \lambda x_1, \dots, \lambda x_n) = \lambda^d \cdot f^h(x_0, x_1, \dots, x_n). \quad (10)$$

Such functions are not in general well-defined functions on $\mathbb{P}^n$, but what is well-defined, because of Eq. (10), is the zero locus of such functions. The zero locus of F is

$$V(F) := \{(x_0 : \dots : x_n) \in \mathbb{P}^n : f^h(x_0, x_1, \dots, x_n) = 0\}.$$

Clearly, given any polynomial on the affine space $\mathbb{A}^n$, the polynomial is a genuinely well-defined function on $\mathbb{A}^n$, and it always makes sense to talk about the zero locus. Later, we will use capitalized variables like X, Y, Z to emphasize that they are variables taking values in a projective space, see for example Construction 4.10, but lower-case variables like x, y to emphasize that they take values in affine space, see for example Construction 4.11.

3 Galois Qudits over $\text{GF}(2^s)$

3.1 Galois Qudits and their Implementation in Qubits

A Galois qudit over the finite field $\text{GF}(2^s) = \mathbb{F}_{2^s}$ is a 2^s -dimensional qudit with a choice of Pauli group deliberately made to encode the arithmetic of the finite field $\mathbb{F}_{2^s}$. In making this choice, the

2^s -dimensional qudit is made equivalent to a set of s qubits in its states, Pauli group and Clifford hierarchy [Got24; Wil26]. Building quantum codes for Galois qudits over $\text{GF}(2^s)$ with desirable properties often allows one to build qubit codes with similarly desirable properties [WHY25; GG25a; Ngu25; NP25; He+25c; He+25b]. In particular, by constructing qudit codes over $\text{GF}(2^s)$ with transversal third-level gates, we can construct distillation protocols for qubit magic states that only require qubit Clifford operations [WHY25; NP25] to execute on qubits. Thus, we study qubit magic state distillation protocols by constructing and analyzing quantum codes over $\mathbb{F}_{2^s}$.

The mapping between the 2^s -dimensional Galois qudit and the set of s qubits goes via a “qudit-to-qubit mapping”, for which all the details are laid out in [Wil26]. In particular, isomorphisms may be constructed for the states of the two systems, CSS codes on them, as well as the levels of the Clifford hierarchy, which specialises to diagonal gates in each level of the Clifford hierarchy. All of these isomorphisms are compatible with each other in the natural ways. Accordingly, by constructing distillation protocols for Galois qudits, we obtain distillation protocols for qubits. We make this part of the transformation explicit shortly.

Throughout, we specify distillation protocols via CSS codes, where the CSS codes are specified by matrices in a way that is now standard [BH12], where the rows of the given matrix specify the X -logical operators in its first k rows, and its remaining rows specify X stabilizer generators. This is an important enough notion for us that we give it a name here. One may see [Wil26] for the construction of Galois qudit CSS codes from spaces of X and Z stabilizers/logical operators. In what follows, two vectors $x, y \in \mathbb{F}_q^n$ are said to be *orthogonal* if $\langle x, y \rangle \equiv \sum_i x_i y_i = 0$.

Definition 3.1 (*X-generator matrix*). *An X-generator matrix for a quantum CSS code is some matrix $G \in \mathbb{F}_q^{m \times n}$, with an associated positive integer $1 \leq k \leq m$. We require that G is full rank over $\mathbb{F}_q$. This matrix defines a quantum CSS code where the first k rows are non-trivial logical X operators (referred to as logical rows), and the remaining rows are X stabilizer generators. Formally, let $\mathcal{G}$ be the $\mathbb{F}_q$ -vector space generated by the rows of G , and $\mathcal{G}_0$ be the $\mathbb{F}_q$ -vector space generated by the latter $m - k$ rows of G . The CSS code has X stabilizers given by the space $\mathcal{G}_0$ and Z stabilizers given by the space $\mathcal{G}^\perp$.*

It is straightforward to compute code properties from the X -generator matrix.

Proposition 3.2. *Let $G \in \mathbb{F}_q^{m \times n}$ be an X-generator matrix for a quantum CSS code over $\mathbb{F}_q$ with k logical rows. The quantum CSS code has k logical qudits and n physical qudits. Its Z -distance is equal to the minimum Hamming weight of a nonzero vector in $\mathbb{F}_q^n$ orthogonal to all of the latter $m - k$ rows of G , but not orthogonal to at least one of the first k rows of G .*

Proof. In the notation of Definition 3.1, the number of logical qudits of the corresponding CSS code is $\dim_{\mathbb{F}_q} \mathcal{G} - \dim_{\mathbb{F}_q} \mathcal{G}_0 = m - (m - k) = k$, where the first equality follows from the requirement that G is full rank. The Z distance is then the minimum Hamming weight of an element of $\mathcal{G}_0^\perp \setminus \mathcal{G}^\perp$. By linearity, a vector $z \in \mathbb{F}_q^n$ is in $\mathcal{G}_0^\perp \setminus \mathcal{G}^\perp$ if and only if it is orthogonal to every row of G_0 , but not every row of G . $\square$

For distilling magic states of diagonal gates, only the Z -distance is relevant, since one can twirl the magic states [BK05; BH12].

Proposition 3.3 (Generalization of [BK05; BH12] twirling). *Let U be a diagonal third-level s -qubit operator. For $a \in \mathbb{F}_2^s$, define the operators $M_a \equiv UX^a U^\dagger$ and states $|A_a\rangle \equiv Z^a U |+\rangle^s$. Then, for any state, ρ , its twirl with respect to $(M_a)_{a \in \mathbb{F}_2^s}$ picked uniformly at random is $\frac{1}{2^s} \sum_{a \in \mathbb{F}_2^s} M_a \rho M_a^\dagger = \sum_{a \in \mathbb{F}_2^s} \rho_a |A_a\rangle \langle A_a|$ where $\rho_a \equiv \langle A_a | \rho | A_a \rangle$.*

Proof. First, note that $\langle A_a | A_b \rangle = \langle +|^s Z^{a+b} |+\rangle^s = \delta_{ab}$, so the states $(|A_a\rangle)_{a \in \mathbb{F}_2^s}$ form an orthonormal basis. Additionally, they are eigenvectors of the operators $(M_a)_{a \in \mathbb{F}_2^s}$ with eigenvalues $M_a |A_b\rangle = UX^a U^\dagger Z^b U |+\rangle^s = (-1)^{a \cdot b} |A_b\rangle$. For an arbitrary density matrix, we can expand it in the basis $\rho = \sum_{a, b \in \mathbb{F}_2^s} \rho_{ab} |A_a\rangle \langle A_b|$, where $\rho_{ab} = \langle A_a | \rho | A_b \rangle$. After passing this state through the twirling channel,

we get

$$\begin{aligned}
\frac{1}{2^s} \sum_{a \in \mathbb{F}_2^s} M_a \rho M_a^\dagger &= \frac{1}{2^s} \sum_{c \in \mathbb{F}_2^s} \sum_{a, b \in \mathbb{F}_2^s} \rho_{ab} M_c |A_a\rangle \langle A_b| M_c^\dagger \\
&= \frac{1}{2^s} \sum_{c \in \mathbb{F}_2^s} \sum_{a, b \in \mathbb{F}_2^s} \rho_{ab} (-1)^{c \cdot (a+b)} |A_a\rangle \langle A_b| \\
&= \sum_{a \in \mathbb{F}_2^s} \rho_{aa} |A_a\rangle \langle A_a|
\end{aligned}$$

□

Typically, we will refer to the rows of G as $\mathbf{g}_1, \dots, \mathbf{g}_m$, and the j -th entry of the i -th row as g_{ij} . We will usually think of G as a generator matrix for a classical code, i.e., the rows of G form a basis for the code. We restrict ourselves to $\mathbb{F}_{2^s}$ -linear codes, i.e., if $\mathbf{g}_1, \mathbf{g}_2 \in \mathbb{F}_{2^s}^n$ are codewords, then so is $\gamma_1 \mathbf{g}_1 + \gamma_2 \mathbf{g}_2$ for all $\gamma_1, \gamma_2 \in \mathbb{F}_{2^s}$.

If G is a binary matrix, one immediately has a quantum CSS code for qubits. If $s > 1$, then we may binarize the matrix G as follows to produce a qubit code.⁷ Consider some basis $B = (\alpha_i)_{i=1}^s$ for $\mathbb{F}_{2^s}$ over $\mathbb{F}_2$, i.e., every $\gamma \in \mathbb{F}_{2^s}$ can be written uniquely as $\gamma = \sum_{i=1}^s b_i \alpha_i$ for $b_i \in \mathbb{F}_2$, $i = 1, \dots, s$. The generator matrix $G \in \mathbb{F}_{2^s}^{m \times n}$ may be expanded into a binary matrix in $\mathbb{F}_2^{ms \times ns}$, simply by taking any entry γ of G and replacing it with an $\mathbb{F}_2^{s \times s}$ matrix whose (i, j) -th entry is $\text{tr}(\gamma \alpha_i \alpha_j)$ [MS77]⁸. The resultant binary matrix specifies a qubit CSS code with its first ks rows being logical X operators and its latter $(m - k)s$ rows being X stabilizer generators.

One may use different bases B for the mapping of qudits to qubits. This yields different isomorphisms between the qudits and qubits. Changing the basis B used in the mapping is equivalent to performing a CNOT circuit on the s qubits making up the qudit. A particularly convenient choice of basis is a *self-dual basis*, one for which $\text{tr}(\alpha_i \alpha_j) = \delta_{ij}$. Such a basis exists for all s , and can be found in polynomial time using Lempel factorization [SL80]. It is particularly convenient to extract the components of a field element in a self-dual basis:

$$\mathbb{F}_{2^s} \ni \gamma = \sum_{i=1}^s b_i \alpha_i, \quad b_i = \text{tr}(\gamma \alpha_i) \in \mathbb{F}_2. \quad (11)$$

Using this expression and the phase polynomial language, we can build up the connection between qudit and qubit diagonal gates. In the rest of this section, we will study particular qudit gates, including the qudit-CCZ gates acting on three qudits, as well as single qudit gates that include CS (specifically over $\mathbb{F}_4$), norm gates and U_7 gates for arbitrary binary extension fields. These are all non-Clifford gates. The aim is therefore to construct qudit codes, encoded in matrices $G \in \mathbb{F}_{2^s}^{m \times n}$, having these gates transversal, in order to distill their magic states. This immediately gives us qubit protocols distilling the equivalent of these gates under the qudit-to-qubit mapping.

Before proceeding, however, let us state some simple observations about the Clifford hierarchy for Galois qudits of dimension 2^s (regardless of the extension degree s). We focus particularly on the operations that perform arithmetic over $\mathbb{F}_{2^s}$, for which we note that the level of the Clifford hierarchy are very different to arithmetic over $\mathbb{Z}$, originating from the fact that addition does not cause “carrying” in $\mathbb{F}_{2^s}$.

Remark 3.4. *Clifford hierarchy of arithmetic operations in binary extension fields.*

1. *In-place addition over $\mathbb{F}_{2^s}$, that is, $|x\rangle |y\rangle \mapsto |x\rangle |x + y\rangle$ for $x, y \in \mathbb{F}_{2^s}$, is a Clifford operation.*
2. *Multiplying an element in $\mathbb{F}_{2^s}$ by a known constant $\beta \in \mathbb{F}_{2^s}^\times$ is a Clifford operation. This can be also seen as a change of basis, i.e., from $(\alpha_i)_{i=1}^s$ to $(\beta \alpha_i)_{i=1}^s$. There are many more changes of basis than these, however, and they are all Clifford.*
3. *Multiplication of two unknown elements $x, y \in \mathbb{F}_{2^s}$, that is, $|x\rangle |y\rangle |0\rangle \mapsto |x\rangle |y\rangle |xy\rangle$, is in the third level of the Clifford hierarchy. We will see this explicitly in subsection 3.2.1.*

⁷It is equivalent to construct a qudit code from the matrix over $\mathbb{F}_{2^s}$, and binarize the code itself [Wil26].

⁸One can think of this $\gamma \mapsto (\text{tr}(\gamma \alpha_i \alpha_j))_{i,j=1}^s$ mapping as multiplying each codeword of the $\mathbb{F}_{2^s}$ -linear code by α_i and then subsequently binarizing each entry; see [Wil26] for more details on mapping qudit codes to qubit codes.

4. The in-place Frobenius transform $|\gamma\rangle \mapsto |\gamma^2\rangle$ is a Clifford operation. A way to see this is to expand in the normal basis⁹ $\alpha_i = \alpha^{2^i}$ for some primitive element α of $\mathbb{F}_{2^s}$. Writing $\gamma = \sum_{i=1}^s b_i \alpha_i$, we have $\gamma^2 = (\sum_{i=1}^s b_i \alpha_i)^2 = \sum_{i=1}^s b_i^2 \alpha_i^2 = \sum_{i=1}^s b_i \alpha_{i+1 \pmod{s+1}}$, and so the Frobenius transform is just cyclically shifting the s qubits making up the qudit (with this basis B), which is Clifford.¹⁰

3.2 Expansion of gates

3.2.1 Qudit-CCZ gate

As mentioned, gates on $\text{GF}(2^s)$ qudits can be expanded into qubit gates as well, where diagonal gates are mapped to diagonal gates, and they retain their level in the Clifford hierarchy. Let us examine this further. Full details can be found in [Wil26], but we give the idea here as well. Let us start with the example of the three-qudit CCZ gate [GG25a; Ngu25], defined as follows for $x, y, z \in \text{GF}(2^s)$:

$$\text{CCZ} : |x\rangle|y\rangle|z\rangle \mapsto (-1)^{\text{tr}(xyz)} |x\rangle|y\rangle|z\rangle \quad (12)$$

For simplicity, let $B = (\alpha_i)_{i=1}^s$ be a self-dual basis for $\mathbb{F}_q$ over $\mathbb{F}_2$. To expand the gate, first expand x, y, z in the self-dual basis: $x = \sum_{i=1}^s x_i \alpha_i$, $y = \sum_{i=1}^s y_i \alpha_i$, and $z = \sum_{i=1}^s z_i \alpha_i$, where $x_i, y_i, z_i \in \{0, 1\}$. The computational basis states $|x\rangle|y\rangle|z\rangle$ on 3 qudits can be expanded to computational basis states on $3s$ qubits by writing x, y, z out in their components. Next, using the fact that $\text{tr}(\cdot)$ is $\mathbb{F}_2$ -linear:

$$\text{tr}(xyz) = \text{tr} \left(\sum_{i,j,k=1}^s x_i y_j z_k \alpha_i \alpha_j \alpha_k \right) = \sum_{i,j,k=1}^s x_i y_j z_k \text{tr}(\alpha_i \alpha_j \alpha_k). \quad (13)$$

When we expand CCZ to a $3s$ -qubit gate, it gives a phase $\text{tr}(xyz) = \sum_{i,j,k=1}^s x_i y_j z_k \text{tr}(\alpha_i \alpha_j \alpha_k)$ to the computational basis state $|x\rangle|y\rangle|z\rangle$. Therefore, at the qubit level, this qudit CCZ gate applies a qubit CCZ to every triplet (x_i, y_j, z_k) for which $\text{tr}(\alpha_i \alpha_j \alpha_k) = 1$.

Let us show an explicit example of the qudit CCZ gate for $\mathbb{F}_4 = \{0, 1, \omega, \omega^2\}$, where arithmetic in this field is specified by $\omega + \omega^2 = 1$. One can check that (ω, ω^2) forms a self-dual basis. The left side of figure 4 denotes the example of the $\mathbb{F}_4$ -qudit CCZ gate expanded in this basis. Claim 3.5 shows that the qudit CCZ gate can be used to coherently multiply elements in the field $\mathbb{F}_q$ on a quantum computer.

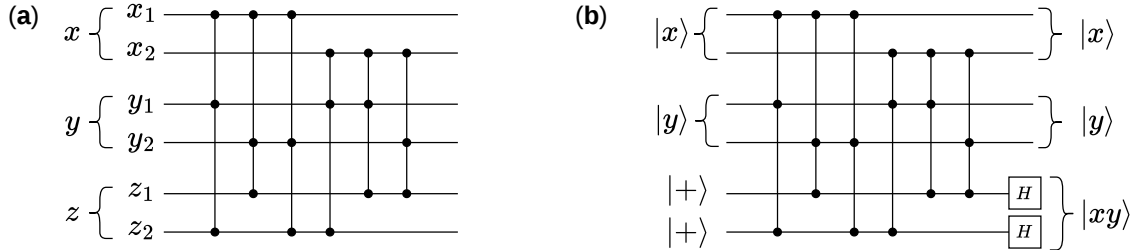


Figure 4: (a) $\mathbb{F}_4$ -qudit CCZ expanded as qubit CCZs. Expand $x = x_1\omega + x_2\omega^2$, where $x_1, x_2 \in \mathbb{F}_2$, and similarly for y, z . (b) The magic state corresponding to $\mathbb{F}_4$ -CCZ can be consumed to perform multiplication in $\mathbb{F}_4$ – one just needs to conjugate the third pair of qubits by Hadamard gates.

Claim 3.5. The $\mathbb{F}_{2^s}$ -qudit CCZ gate allows one to multiply two elements in that field on a quantum computer. That is, one use of a CCZ gate, and Clifford operations, is sufficient to perform the arithmetic operation $|x\rangle|y\rangle|0\rangle \mapsto |x\rangle|y\rangle|xy\rangle$, where $x, y \in \mathbb{F}_{2^s}$, and each ket is an s -qubit register (equivalently, a one 2^s -dimensional qudit register. Fig. 4(b) shows an example for $\mathbb{F}_4$.

Proof. We will show that the following sequence of operations executes the desired operation on the 3-qudit state $|x\rangle|y\rangle|0\rangle$:

⁹A normal basis of $\mathbb{F}_{q^m}$ over $\mathbb{F}_q$, which is a basis of the form $\{\alpha, \alpha^q, \dots, \alpha^{q^{m-1}}\}$, always exists, see e.g. [LN96, Thm. 2.35]. As we commented before, for $\mathbb{F}_{2^s}$ over $\mathbb{F}_2$, a self-dual basis always exists; however, a self-dual normal basis exists if and only if s is not divisible by 4. [MP13]

¹⁰Recall that picking a different basis B for $\mathbb{F}_q$ over $\mathbb{F}_2$ yields different isomorphisms from qudits to qubits, however each give valid isomorphisms of the diagonal gates in each level of the Clifford hierarchy. Thus, we may choose a particular basis B , establish the level of the corresponding qubit gate, and conclude the level of the initial qudit gate.

1. Perform Hadamard on every qubit making up the third register, which all begin in the $|0\rangle$ state;
2. Perform the qudit CCZ gate on the three qudits ($3s$ qubits);¹¹
3. Perform Hadamard again on all s qubits making up the last register.

A fact we will be using is that

$$H^{\otimes s} |z\rangle = \frac{1}{\sqrt{2^s}} \sum_{t \in \mathbb{F}_{2^s}} (-1)^{\text{tr}(zt)} |t\rangle. \quad (14)$$

Here, $|z\rangle$ denotes the qudit computational basis state corresponding to $z \in \mathbb{F}_q$, but $H^{\otimes s}$ corresponds to performing the qubit Hadamard operation on the s qubits making up the qudit. To see this equation, consider expanding $z \in \mathbb{F}_{2^s}$ in a self-dual basis $(\alpha_i)_{i=1}^s$ as $z = z_1\alpha_1 + \dots + z_s\alpha_s$; one also expands $t \in \mathbb{F}_{2^s}$ as $t = t_1\alpha_1 + \dots + t_s\alpha_s$: $z_i, t_i \in \{0, 1\}$. Then, $H^{\otimes s} |z\rangle$ is a uniform superposition over $|T\rangle$ with phase $(-1)^{z_1t_1 + \dots + z_st_s}$. This exponent is exactly

$$\text{tr}(zt) = \text{tr} \left(\left(\sum_i z_i \alpha_i \right) \left(\sum_j t_j \alpha_j \right) \right) = \sum_{i,j} z_i t_j \text{tr}(\alpha_i \alpha_j) = \sum_i z_i t_i.$$

The desired circuit is then performed as follows:

$$|x\rangle|y\rangle|0^s\rangle \xrightarrow{H^{\otimes s} \text{ on } z} |x\rangle|y\rangle \frac{1}{\sqrt{2^s}} \sum_{z \in \mathbb{F}_{2^s}} |z\rangle \xrightarrow{\text{Qudit CCZ}} \frac{1}{\sqrt{2^s}} \sum_z (-1)^{\text{tr}(xyz)} |x\rangle|y\rangle|z\rangle. \quad (15)$$

Apply $H^{\otimes s}$ again on the z -register:

$$\begin{aligned} H^{\otimes s} \left(\frac{1}{\sqrt{2^s}} \sum_z (-1)^{\text{tr}(xyz)} |z\rangle \right) &= \frac{1}{\sqrt{2^s}} \sum_z (-1)^{\text{tr}(xyz)} \left(\frac{1}{\sqrt{2^s}} \sum_t (-1)^{\text{tr}(zt)} |t\rangle \right) \\ &= \frac{1}{2^s} \sum_t \left(\sum_z (-1)^{\text{tr}(z(xy+t))} \right) |t\rangle \\ &= |xy\rangle, \end{aligned}$$

where in the last step, we have used

$$\sum_{z \in \mathbb{F}_{2^s}} (-1)^{\text{tr}(zu)} = \begin{cases} 2^s, & \text{if } u = 0, \\ 0, & \text{if } u \neq 0. \end{cases}$$

□

This multiplication in the binary extension field finds usage in, for example, decoded quantum interferometry [Jor+25]. In particular, [Kha+25] provides a detailed resource analysis for solving the optimal polynomial intersection problem in binary extension fields. The binary extension field multiplication might also be useful in preparing certain catalyst states [Kim25].

Later, we will only present qudit-CCZ distillation protocols (Table 3) up to $\mathbb{F}_{64}$. For multiplication in larger binary extension fields, e.g., $\mathbb{F}_{2^{163}}$, one can reduce the problem into many multiplications in small binary extension fields by paying with some additional CNOT gates [CC88; CÖ10]. We present an example of this reduction in App. B.3, after introducing relevant concepts for algebraic curves in Sec. 5.2 and App. B.1.

Note that, as a consequence of remark 3.4, the qudit CCZ gates CCZ^{β_1} and CCZ^{β_2} are Clifford equivalent for any $\beta_1, \beta_2 \in \mathbb{F}_{2^s}^\times$, where

$$\text{CCZ}^\beta |\eta_1\rangle|\eta_2\rangle|\eta_3\rangle = (-1)^{\text{tr}(\beta\eta_1\eta_2\eta_3)} |\eta_1\rangle|\eta_2\rangle|\eta_3\rangle. \quad (16)$$

This is because an equivalent way of doing CCZ^{β_1} is to perform multiplication on the first qudit¹² $|\eta_1\rangle \mapsto |\frac{\beta_2}{\beta_1}\eta_1\rangle$ before applying CCZ^{β_2} . This fact will later play a role in Sec. 5.3 when we distill qudit-CCZ gates.

¹¹This step can also be performed by consuming a $|\text{CCZ}\rangle$ state, and using Clifford operations.

¹²In fact, any of the three qudits work.

3.2.2 Control- S gate in $\text{GF}(4)$

Besides expanding a qudit diagonal gate into a qubit one, one can also do the reverse in some cases. If the qubit gate only involves multi-control- Z gates, i.e., it is a ± 1 -phase gate, then it turns out to always be possible; this boils down to the fact that arithmetic in $\mathbb{F}_2$ is simply integer arithmetic modulo 2, that is, $\mathbb{Z}_2 = \mathbb{F}_2$. However, if we are considering phase polynomial gates with non-real phases, i.e., involving phases $e^{i\pi/2^t}$, for some $t \in \mathbb{N}$, to the power of some polynomial, then due to the difference between $\mathbb{Z}_{2^t}$ and $\mathbb{F}_{2^t}$, complications arise in general.

Nevertheless, let us show a simple viable example for $\mathbb{F}_4 = \{0, 1, \omega, \omega^2\}$, as it leads to interesting MSD protocols in Sec. 4.5. In $\mathbb{F}_4$, we have that $\omega^2 = \omega + 1$ and $\omega^3 = 1$; also $\text{tr}(1) = 0 = \text{tr}(0)$ and $\text{tr}(\omega) = \text{tr}(\omega^2) = 1$. We consider applying the CS gate to the 2 qubits of an $\mathbb{F}_4$ qudit that has been expanded in the self-dual basis (ω, ω^2) .

$$\gamma = b_1\omega + b_2\omega^2, \quad \text{CS}|\gamma\rangle := \text{CS}|b_1\rangle|b_2\rangle = i^{b_1b_2}|\gamma\rangle. \quad (17)$$

Since the basis (ω, ω^2) is self dual, one can extract b_1 and b_2 as $b_1 = \text{tr}(\gamma\omega)$ and $b_2 = \text{tr}(\gamma\omega^2)$ (see Equation (11)). Using arithmetic over $\mathbb{F}_4$, $b_1b_2 = (\gamma\omega + \gamma^2\omega^2)(\gamma\omega^2 + \gamma^2\omega) = \gamma^3 + \text{tr}(\gamma)$. Therefore,

$$\text{CS} : |\gamma\rangle \mapsto i^{\gamma^3 + \text{tr}(\gamma)}|\gamma\rangle, \quad (18)$$

where we emphasise that the arithmetic in the exponent is over $\mathbb{F}_4$.

3.2.3 The norm gate

Consider the multi-control- Z gate $C^{s-1}Z$ acting on the s qubits constituting a $\text{GF}(2^s)$ qudit:

$$C^{s-1}Z|\gamma\rangle := C^{s-1}Z|b_1\rangle|b_2\rangle \cdots |b_s\rangle = (-1)^{b_1b_2 \cdots b_s} |b_1\rangle|b_2\rangle \cdots |b_s\rangle. \quad (19)$$

Considering a self-dual basis $(\alpha_i)_{i=0}^{s-1}$ for $\mathbb{F}_{2^s}$ over $\mathbb{F}_2$, we can use $b_i = \text{tr}(\gamma\alpha_i)$ and the explicit formula of trace in Eq. (8) to write the above qubit phase polynomial into a qudit one. However, a better way is to directly observe that such a gate is Pauli equivalent to

$$|\gamma\rangle \mapsto (-1)^{\text{Nm}(\gamma)}|\gamma\rangle = (-1)^{\gamma^{2^s-1}}|\gamma\rangle. \quad (20)$$

This is a gate for a single 2^s -dimensional qudit which we call the *norm gate*. Recall that besides $\text{tr}(\cdot)$, the norm function $\text{Nm}(\cdot)$ also maps $\mathbb{F}_{2^s}$ elements to $\{0, 1\}$; it takes a particularly simple form in $\mathbb{F}_{2^s}$ (cf. Eq. (9)): $\text{Nm}(\gamma) := \prod_{i=0}^{s-1} \gamma^{2^i} = \gamma^{2^s-1}$. Note that $\text{Nm}(\gamma) = 0$ if and only if $\gamma = 0$, and else $\text{Nm}(\gamma) = 1$; this is most easily seen from the fact that the multiplicative group of a finite field is cyclic.¹³ That is to say, $|\gamma\rangle$ gains a phase of $(-1)^0 = 1$ when $\gamma = 0$ and -1 when $\gamma \neq 0$. The Pauli equivalence of the norm gate to the $C^{s-1}Z$ gate is now obvious: the norm gate can be implemented by first applying X gates to each of the s qubits making up the qudit, giving us $b_i \mapsto b_i \oplus 1$, then applying a $C^{s-1}Z$ gate, and finally undoing the X gates. Then only in the case of $b_1 = \cdots = b_s = 0$ does the intermediate state gain a phase of -1 .

3.2.4 Classification of the diagonal Galois qudit Clifford hierarchy with real phases

Aside from the norm gate, we can examine more general diagonal Galois qudit gates in the Clifford hierarchy with real phases, i.e., with phases ± 1 . The aim is to classify the level of such gates in the Clifford hierarchy. Note that in some sense this has already been achieved, because each level of the diagonal Clifford hierarchy for (multiple) qudits of prime dimension is isomorphic to that of a single larger Galois qudit, as discussed, and the former has been classified [CGK17]. Thus, for any given Clifford hierarchy gate on one Galois qudit, one can use the isomorphism to write it as a gate for multiple qudits of a smaller dimension, and establish its level using that classification. The aim here, however, is to establish the level of such a gate in language which is native to the Galois qudits. Aside from giving us a deeper understanding of the real diagonal Clifford hierarchy gates for Galois qudits, it is operationally useful as it allows us to derive transversality conditions for the corresponding magic states naturally. This treatment resolves an open question in [He+25c].

¹³Strictly speaking, this would be true irrespective of the structure of the multiplicative group given Lagrange's theorem for finite groups.

All single-qudit diagonal real-phase gates are in the Clifford hierarchy, and can be written

$$|\gamma\rangle \mapsto (-1)^{f(\gamma)} |\gamma\rangle, \quad f(\gamma) = \sum_{i=0}^{2^s-1} c_i \gamma^i \in \{0, 1\}, \quad c_i \in \mathbb{F}_q. \quad (21)$$

Writing $q = 2^s$, we note that the polynomial only has terms of degree less than q , because $\gamma^q = \gamma$, $\forall \gamma \in \mathbb{F}_q$.

Now, since $f : \mathbb{F}_q \rightarrow \mathbb{F}_2$, we have $f^2 = f$, so the coefficients c_i satisfy $c_{2i} = c_i^2$. Therefore, if c_i is nonzero, then so is c_j for any j in the 2-cyclotomic coset C_i of i , which is defined by

$$C_i = \{i, 2i, 4i, \dots, 2^{\ell_i-1}i\} \bmod q-1, \quad \text{where } \ell_i = \min\{j \geq 1 \mid 2^j i \equiv i \bmod q-1\}. \quad (22)$$

For example, if $s = 4$ then $q = 2^4 = 16$, and considering $i = 3$, we have $C_i = \{3, 6, 12, 9\}$, and $\ell_i = 4$. In this case, we notice that $\ell_i = s$, but more generally, we must always have $\ell_i \mid s$, following from the fact that $2^s i \equiv i \bmod q-1$. Moreover, using $c_{2i} = c_i^2$, we have

$$\sum_{j=0}^{\ell_i-1} c_{2^j i} \gamma^{2^j i} = \sum_{j=0}^{\ell_i-1} c_i^{2^j} \gamma^{2^j i} = \sum_{j=0}^{\ell_i-1} (c_i \gamma^i)^{2^j} = \text{tr}_{\mathbb{F}_{2^{\ell_i}}/\mathbb{F}_2}(c_i \gamma^i). \quad (23)$$

Note that the right-hand side makes sense, indeed, $c_i \gamma^i \in \mathbb{F}_{2^{\ell_i}}$ follows from

$$c_i = c_{2^{\ell_i} i} = c_i^{2^{\ell_i}} \implies c_i \in \mathbb{F}_{2^{\ell_i}} \quad (24)$$

and

$$2^{\ell_i} i \equiv i \bmod q-1 \implies \gamma^i = (\gamma^i)^{2^{\ell_i}}. \quad (25)$$

Now, notationally, we always let i , the smallest element in C_i , represent C_i , and we then let Γ be the set of such representatives of the 2-cyclotomic cosets modulo $q-1$. In total, every Boolean function $f : \mathbb{F}_{2^s} \rightarrow \mathbb{F}_2$ can be written uniquely in the form of Equation (21), and in turn uniquely as

$$f(\gamma) = c_0 + \sum_{i \in \Gamma} \text{tr}_{\mathbb{F}_{2^{\ell_i}}/\mathbb{F}_2}(c_i \gamma^i). \quad (26)$$

using this discussion.

In Theorem 3.6, we will start by establishing the level of the Clifford hierarchy of each term $(-1)^{\text{tr}_{\mathbb{F}_{2^{\ell_i}}/\mathbb{F}_2}(c_i \gamma^i)}$. Later, in Theorem 3.9, we will argue that the level of their product is simply the maximum of each of their levels.

Theorem 3.6. *Given $q = 2^s$ and a non-negative integer $n < q-1$, let ℓ_n be the size of the 2-cyclotomic coset C_n of n as in Eq. (22). For $c_n \in \mathbb{F}_{2^{\ell_n}}$, the single-qudit real-phase gate mapping $|\gamma\rangle$ to $(-1)^{\text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n \gamma^n)} |\gamma\rangle$ is in exactly the $\text{wt}(n)$ -th level of Clifford hierarchy¹⁴ if $c_n \neq 0$, where $\text{wt}(n)$ stands for the Hamming weight of the binary expansion of n . If $c_n = 0$, then the gate is trivial up to a global phase.*

Proof. Say $(\alpha_i)_{i=1}^s$ is a basis for $\mathbb{F}_{2^s}$, and expand $\gamma = \sum_{i=1}^s b_i \alpha_i$ where $b_i \in \{0, 1\}$. Then, the exponent is

$$\begin{aligned} \text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n \gamma^n) &= \text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n (b_1 \alpha_1 + \dots + b_s \alpha_s)^n) \\ &= \text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2} \left(c_n \sum_{e_1 + \dots + e_s = n} \binom{n}{e_1, \dots, e_s} \prod_{i=1}^s (b_i \alpha_i)^{e_i} \right), \end{aligned} \quad (27)$$

where we use the multinomial expansion formula, and the multinomial coefficients are $\binom{n}{e_1, e_2, \dots, e_s} := \frac{n!}{e_1! e_2! \dots e_s!}$. This expression is raised to the power of -1 , and so it is relevant to establish the parity of the multinomial coefficients.

Lemma 3.7. (Corollary of Lucas' theorem). *Consider the multinomial coefficient $\binom{n}{e_1, e_2, \dots, e_m}$, where e_i are positive integers and $e_1 + e_2 + \dots + e_m = n$. This is odd if and only if there is no carrying when adding up the binary expansions of $e_1, e_2, \dots, e_m$. That is, considering the expansions of each e_i in binary, the coefficient is odd if and only if, in each bit position of the binary expansion, there is at most one 1 across the e_i 's. An immediate corollary is that, if $m > \text{wt}(n)$, then the multinomial coefficient is even.*

¹⁴That is, the gate is in the $\text{wt}(n)$ -th level, but not in a lower level.

Proof. Lucas' theorem states that for non-negative integers A, B and a prime p , the binomial coefficient satisfies the following congruence relation:

$$\binom{A}{B} \equiv \prod_{i=1}^r \binom{A_i}{B_i} \pmod{p}, \quad (28)$$

where $A = A_r p^r + A_{r-1} p^{r-1} + \dots + A_1 p + A_0$ and $B = B_r p^r + B_{r-1} p^{r-1} + \dots + B_1 p + B_0$ are the base- p expansions of A, B . This uses the convention that $\binom{A}{B} = 0$ if $A < B$. It follows that $\binom{A}{B}$ is divisible by p if and only if $B_i > A_i$ for some i . We will only use the $p = 2$ case. The binomial coefficient $\binom{A}{B}$ is odd if and only if $B_i \leq A_i$ for every i , which is if and only if there is no carrying when B and $A - B$ are summed.

Notice that we can write the multinomial coefficient as a product of binomial coefficients:

$$\binom{n}{e_1, \dots, e_m} = \binom{n}{e_1} \binom{n-e_1}{e_2} \binom{n-e_1-e_2}{e_3} \dots \binom{e_{m-1}+e_m}{e_{m-1}}.$$

Equivalently, writing the suffix sum as $S_k = e_k + e_{k+1} + \dots + e_m$, the above formula is $\binom{n}{e_1, \dots, e_m} = \prod_{k=1}^{m-1} \binom{S_k}{e_k} = \prod_{k=1}^{m-1} \binom{S_k}{S_{k+1}}$. The multinomial coefficient on the left is odd if and only if all the binomial coefficients on the right-hand side are odd, and $\binom{S_k}{e_k}$ is odd if and only if there is no carrying when e_k and S_{k+1} are summed in binary to make S_k . Finally, there is no carrying when $e_1, \dots, e_m$ are summed to make n if and only if there is no carrying when e_k and S_{k+1} are summed to make S_k for all k . $\square$

Returning to the expression in Equation (27), we note that an s -term product of b_i 's corresponds to a $C^{(s-1)}Z$ gate. With this, and the above lemma, we find that the single-qudit phase gate $|\gamma\rangle \mapsto (-1)^{\text{tr}_{\mathbb{F}_{2^{\ell n}}/\mathbb{F}_2}(c_n \gamma^n)} |\gamma\rangle$ is in the $\text{wt}(n)$ -th level of the Clifford hierarchy. The remaining challenge is to establish that the gate lies precisely in that level. To this end, we consider the terms in Equation (27) with a $\text{wt}(n)$ -product of b_i 's. We look at the coefficient ($\in \{0, 1\}$) in front of all the $\text{wt}(n)$ -term products of b_i 's, and show that at least one of them is one. That is, we just want to consider terms in the sum of Equation (27) for which exactly $\text{wt}(n)$ of the e_i are positive, and of these we only need to consider terms for which the e_i have binary expansions of weight one, i.e., they are powers of two, since the multinomial coefficient is odd in exactly these cases.

Let $T \subseteq \{0, 1, \dots, s-1\}$ be the support of the binary expansion of n , i.e., $n = \sum_{t \in T} 2^t$. Let $D = |T| = \text{wt}(n)$, and write $T = \{t_1, \dots, t_D\}$. Then considering some distinct set of indices $\{i_1, \dots, i_D\} \subseteq [s]$, the coefficient of $b_{i_1} b_{i_2} \dots b_{i_D}$ is seen to be

$$B_{c_n}(\alpha_{i_1}, \dots, \alpha_{i_D}) := \text{tr}_{\mathbb{F}_{2^{\ell n}}/\mathbb{F}_2}(c_n M_T(\alpha_{i_1}, \dots, \alpha_{i_D})), \quad M_T(\alpha_{i_1}, \dots, \alpha_{i_D}) := \det \begin{pmatrix} \alpha_{i_1}^{2^{t_1}} & \alpha_{i_1}^{2^{t_2}} & \dots & \alpha_{i_1}^{2^{t_D}} \\ \vdots & \vdots & & \vdots \\ \alpha_{i_D}^{2^{t_1}} & \alpha_{i_D}^{2^{t_2}} & \dots & \alpha_{i_D}^{2^{t_D}} \end{pmatrix}.$$

We wish to show that there is some $\{i_1, \dots, i_D\}$ such that $B_{c_n}(\alpha_{i_1}, \dots, \alpha_{i_D}) = 1$. Note that the trace in its definition is well-defined because

$$M_T(h_1, \dots, h_D)^{2^{\ell n}} = \det \begin{pmatrix} h_1^{2^{t_1+\ell n}} & \dots & h_1^{2^{t_D+\ell n}} \\ \vdots & & \vdots \\ h_D^{2^{t_1+\ell n}} & \dots & h_D^{2^{t_D+\ell n}} \end{pmatrix} = M_{\{t_1+\ell n, \dots, t_D+\ell n\}}(h_1, \dots, h_D) = M_T(h_1, \dots, h_D).$$

The last step follows from $\{t_1 + \ell n, \dots, t_D + \ell n\} = T = \{t_1, \dots, t_D\}$. This fact follows from the definition of ℓn , since $\{t_1, \dots, t_D\}$ forms the support of the binary expansion of n , and multiplying n modulo $q-1$ is equivalent to cyclically shifting its binary expansion. The following properties of the determinant $M_T(h_1, \dots, h_D)$ are easy to verify:

1. If $\exists i \neq j$ such that $h_i = h_j$ are equal, then the determinant is zero (since two rows are equal).
2. It is an $\mathbb{F}_2$ -multilinear form since $(h_i + h'_i)^{2^{t_i}} = (h_i)^{2^{t_i}} + (h'_i)^{2^{t_i}}$, i.e.,

$$M_T(h_1, \dots, h_i + h'_i, \dots, h_D) = M_T(h_1, \dots, h_i, \dots, h_D) + M_T(h_1, \dots, h'_i, \dots, h_D).$$

Since the trace function is also $\mathbb{F}_2$ -linear, $B_{c_n}(h_1, \dots, h_D)$ is also an $\mathbb{F}_2$ -multilinear form.

We claim that it suffices to show the existence of $h_1, \dots, h_D \in \mathbb{F}_q$ such that $B_{c_n}(h_1, \dots, h_D) = 1$.

Indeed, expanding $h_r = \sum_{i=1}^s \lambda_{r,i} \alpha_i$ with $\lambda_{r,i} \in \mathbb{F}_2$, and using the second property above, we have

$$1 = B_{c_n}(h_1, \dots, h_D) = \sum_{i_1, \dots, i_D} \lambda_{1,i_1} \cdots \lambda_{D,i_D} B_{c_n}(\alpha_{i_1}, \dots, \alpha_{i_D}). \quad (29)$$

There must therefore then be some $\{i_1, \dots, i_D\} \subseteq [s]$ such that $B_{c_n}(\alpha_{i_1}, \dots, \alpha_{i_D}) = 1$; clearly $i_1, \dots, i_D$ need to be pairwise disjoint for this to happen by the first property above.

To show that there exists some $h_1, \dots, h_D \in \mathbb{F}_q$ such that $1 = B_{c_n}(h_1, \dots, h_D)$, we first show that there exists $h_1, \dots, h_D \in \mathbb{F}_q$ such that $\mathbb{F}_{2^{\ell_n}} \ni M_T(h_1, \dots, h_D) \neq 0$. Indeed, suppose that $M_T(h_1, \dots, h_D) = 0$ for all $h_1, \dots, h_D \in \mathbb{F}_q$. For $h \in \mathbb{F}_q$, let $r(h)$ be the row vector $(h^{2^{t_1}}, h^{2^{t_2}}, \dots, h^{2^{t_D}})$, and consider the vector space $W := \text{span}_{\mathbb{F}_q} \{r(h) : h \in \mathbb{F}_q\} \subseteq \mathbb{F}_q^D$. Since $M_T(h_1, \dots, h_D) = 0$ for all $h_1, \dots, h_D$, we have that $r(h_1), \dots, r(h_D)$ are linearly dependent over $\mathbb{F}_q$ for all $h_1, \dots, h_D$. This means that $\dim_{\mathbb{F}_q} W < D$, since otherwise there are vectors $r(h_1), \dots, r(h_D)$ spanning the whole space. This means that there is some fixed non-zero vector $(a_1, \dots, a_D) \in \mathbb{F}_q^D$ orthogonal to every vector in W , i.e., for which $\sum_{j=1}^D a_j h^{2^{t_j}} = 0$ for all $h \in \mathbb{F}_q$. However, no non-zero polynomial of degree $< q$ over $\mathbb{F}_q$ can vanish at all elements of $\mathbb{F}_q$, yielding a contradiction.

Now given $h_1, \dots, h_D \in \mathbb{F}_q$ for which $v := M_T(h_1, \dots, h_D) \neq 0$, consider scaling them simultaneously by $a \in \mathbb{F}_q$. Since $\sum_{t \in T} 2^t = n$, we have $M_T(ah_1, \dots, ah_D) = a^n M_T(h_1, \dots, h_D)$. We show that there exists a such that $B_{c_n}(ah_1, \dots, ah_D) = \text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n a^n v) = 1$. It is clear that $a^n \in \mathbb{F}_{2^{\ell_n}}$ since $(a^n)^{2^{\ell_n}} = a^n$. We now use the fact that the $\mathbb{F}_2$ -linear span of $\{a^n \mid a \in \mathbb{F}_q\}$ is all of $\mathbb{F}_{2^{\ell_n}}$, which we show at the end. Using this fact, the surjectivity of $\text{tr}(\cdot)$ means that there exists $a' \in \mathbb{F}_{2^{\ell_n}}$ that $\text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n a' v) = 1$. Expanding a' into an $\mathbb{F}_2$ -sum of elements in $\{a^n \mid a \in \mathbb{F}_q\}$, there must be some a^n such that $\text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n a^n v) = 1$ since $\text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(c_n a' v) = 1$.

Let us conclude by showing that

$$U := \text{span}_{\mathbb{F}_2} \{a^n \mid a \in \mathbb{F}_q\} = \mathbb{F}_{2^{\ell_n}}. \quad (30)$$

First, we know that the left-hand side is contained in the right-hand side, and one readily checks that U forms a field. U is a subfield of $\mathbb{F}_{2^s}$, and must therefore be isomorphic to $\mathbb{F}_{2^m}$ for some m . Moreover, since U is a subfield of $\mathbb{F}_{2^{\ell_n}}$, we must have $m \leq \ell_n$. We can consider some primitive element p of $\mathbb{F}_q$, that is, some generator of the (cyclic) multiplicative group $\mathbb{F}_q^*$, and note that $p^n \in U$. This means that $(p^n)^{2^m} = p^n$, which implies that $2^m n \equiv n \pmod{q-1}$. By the minimality of ℓ_n in its definition, and since $m \leq \ell_n$, we have $m = \ell_n$, as required. $\square$

Remark 3.8. Usually we would just define phase gates via $|\gamma\rangle \mapsto (-1)^{\text{tr}(\beta \gamma^n)} |\gamma\rangle$, where the trace is from $\mathbb{F}_q = \mathbb{F}_{2^s}$ down to $\mathbb{F}_2$. In this case, to make use of Thm. 3.6, one first uses the following identity, which can be proved by plugging in the definition of trace, cf. Eq. (7)

$$\text{tr}_{\mathbb{F}_{2^s}/\mathbb{F}_2}(\beta \gamma^n) = \text{tr}_{\mathbb{F}_{2^{\ell_n}}/\mathbb{F}_2}(\text{tr}_{\mathbb{F}_{2^s}/\mathbb{F}_{2^{\ell_n}}}(\beta) \gamma^n), \quad (31)$$

and then we may use $c_n = \text{tr}_{\mathbb{F}_q/\mathbb{F}_{2^{\ell_n}}}(\beta)$ in the theorem.

Theorem 3.9. The single-qudit real-phase gate $|\gamma\rangle \mapsto (-1)^{f(\gamma)} |\gamma\rangle$, where $f(\gamma) = c_0 + \sum_{i \in \Gamma} \text{tr}_{\mathbb{F}_{2^{\ell_i}}/\mathbb{F}_2}(c_i \gamma^i)$, lies in exactly the $\max_{i \in \Gamma, c_i \neq 0} \text{wt}(i)$ -th level of the Clifford hierarchy.

Proof. The proof is similar to that of Theorem 3.6.

Let $D = \max_{i \in \Gamma, c_i \neq 0} \text{wt}(i)$, and let $\Gamma_D = \{i \in \Gamma \mid c_i \neq 0, \text{wt}(i) = D\}$. For $i \in \{0, 1, \dots, s-1\}$, we use $T(i)$ to denote the support of the binary expansion of i . For $\{i_1, \dots, i_D\} \subseteq [s]$ pairwise distinct, the coefficient of $b_{i_1} b_{i_2} \cdots b_{i_D}$ is

$$B_f(\alpha_{i_1}, \dots, \alpha_{i_D}) = \sum_{m \in \Gamma_D} \text{tr}_{\mathbb{F}_{2^{\ell_m}}/\mathbb{F}_2}(c_m M_{T(m)}(\alpha_{i_1}, \dots, \alpha_{i_D})). \quad (32)$$

The strategy is again to show that there exists $h_1, \dots, h_D \in \mathbb{F}_q$ such that $B_f(h_1, \dots, h_D) = 1$, and then by the same reasoning as in Eq. (29), there exist $i_1, \dots, i_D$ pairwise distinct such that $B_f(\alpha_{i_1}, \dots, \alpha_{i_D}) = 1$.

Pick an arbitrary $m \in \Gamma_D$ and choose $h_1, \dots, h_D \in \mathbb{F}_q$ such that $B_{c_m}(h_1, \dots, h_D) = 1$. Fix this

choice of $h_1, \dots, h_D$ and further expand Eq. (32) using the definition of trace,

$$\begin{aligned} \{0, 1\} \ni B_f(h_1, \dots, h_D) &= \sum_{m \in \Gamma_D} \sum_{j=0}^{\ell_m-1} (c_m M_{T(m)}(h_1, \dots, h_D))^{2^j} \\ &= \sum_{i \in \Lambda_D} c_i M_{T(i)}(h_1, \dots, h_D), \end{aligned}$$

where $\Lambda_D = \bigsqcup_{m \in \Gamma_D} C_m$, we use $M_{T(m)}(h_1, \dots, h_D)^{2^j} = M_{T(2^j m)}(h_1, \dots, h_D)$, and we define $c_{2^j m} = c_m^{2^j}$.

We have

$$B_f(ah_1, \dots, ah_D) = \sum_{i \in \Lambda_D} c_i M_{T(i)}(h_1, \dots, h_D) a^i.$$

Since the cyclotomic cosets are disjoint, the coefficient of a^i only gets a contribution from C_i . Since $c_i \neq 0$ for all $i \in \Lambda_D$ and $M_{T(m)}(h_1, \dots, h_D) \neq 0$, this is a non-zero polynomial in a . No non-zero polynomial over $\mathbb{F}_q$ of degrees at most $q-1$ can vanish on all q elements of $\mathbb{F}_q$. Therefore, there exists $a \in \mathbb{F}_q$ such that $B_f(ah_1, \dots, ah_D) \neq 0$, meaning that $B_f(h_1, \dots, h_D) = 0$. $\square$

Remark 3.10. The above results provide a classification of the diagonal gates in the Galois qudit Clifford hierarchy with real phases, that is, those with phases ± 1 . While the full diagonal Clifford hierarchy is classified for qubits, it is not clear how to extend the above proof to general single-Galois qudit phase gates,¹⁵ for example, those with a phase

$$(-1)^{f_1(\gamma)} \cdot i^{f_2(\gamma)} \cdot (e^{i\pi/4})^{f_3(\gamma)} \dots,$$

where $f_1, f_2, f_3, \dots$ are $\mathbb{F}_2$ -valued functions. One might naively think that we can classify each $f_i(\gamma)$ individually and combine the results (adding $t-1$ to the classification result of $f_t(\gamma)$ to account for the phase $e^{2\pi i/2^t}$ and taking the max). However, it is difficult to even transfer the above classification to the case of phases $i^{f_2(\gamma)}$, and this is because $\mathbb{F}_4$ is distinct from $\mathbb{Z}_4$. Think of an $\mathbb{F}_{16}$ qudit expanded as $\gamma = \sum_{i=1}^s b_i \alpha_i$ for $b_i \in \{0, 1\}$. Consider the phase gate $|\gamma\rangle \mapsto i^{b_1 b_2 \oplus b_3 b_4} |\gamma\rangle$, where $\oplus$ denotes modulo 2 addition. This gate is $CS_{12}CS_{34}CCCZ_{1234}$ on the four qubits, and therefore is in exactly the fourth level of the Clifford hierarchy. However,

$$b_1 b_2 \oplus b_3 b_4 = \text{tr}(\gamma \alpha_1) \text{tr}(\gamma \alpha_2) + \text{tr}(\gamma \alpha_3) \text{tr}(\gamma \alpha_4)$$

only involves terms of degree at most 2 in γ . Even with a +1 contribution accounting for base i , it will be wrongly classified into the third level.

Having said this, there might be a way to constrain each $f_i(\gamma)$ into a certain form, so that one can classify each term separately and combine the results, but the form of the above proof for real phases will be lost.

3.2.5 U_7 gate

A single-qudit non-Clifford introduced in [WHY25] is the so-called U_7 gate.

$$U_7 : |\gamma\rangle \mapsto (-1)^{\text{tr}(\gamma^7)} |\gamma\rangle. \quad (33)$$

We will also consider the U_7^β generalization of it [He+25c]:

$$U_7^\beta : |\gamma\rangle \mapsto (-1)^{\text{tr}(\beta \gamma^7)} |\gamma\rangle. \quad (34)$$

As per Remark 3.8, if $\text{tr}_{\mathbb{F}_{2^s}/\mathbb{F}_{2^{\ell_7}}}(\beta) = 0$ then this gate is effectively trivial, otherwise U_7^β lies in the $\text{wt}(7) = \text{wt}(1112) = 3$ -rd level of the Clifford hierarchy. Recall from Eq. (22) that

$$\ell_7 = \min\{j \geq 1 \mid (2^j - 1) \cdot 7 \equiv 0 \pmod{q-1}\}, \quad q = 2^s. \quad (35)$$

Besides the level of U_7^β , we also want to consider U_7^β for different β , and their Clifford equivalence.

If s is not divisible by three, then $\gcd(2^s - 1, 2^3 - 1) = 1$. Thus, in this case, any $\beta \in \mathbb{F}_{2^s}^\times$ can be written as η^7 for some $\eta \in \mathbb{F}_{2^s}$. So U_7^β is Clifford equivalent to U_7 because $U_7^\beta : |\gamma\rangle \rightarrow |\gamma\eta\rangle \xrightarrow{U_7} |\gamma^7\eta^7\rangle$

¹⁵Again, the diagonal Galois qudit Clifford hierarchy is already classified in the sense that the diagonal qubit Clifford hierarchy is classified, and the two are isomorphic. The challenge here is to classify the diagonal Clifford hierarchy for Galois qudits in a way that is native to the Galois qudits.

where the first step is a change of basis and is Clifford. Therefore, if s is not divisible by 3, all U_7^β are Clifford equivalent to each other as long as $\beta \neq 0$.

If s is divisible by three, let us first determine ℓ_7 . It is clear that for $s = 3$ we have $\ell_7 = 1$, and thus U_7^β becomes trivial if $\text{tr}(\beta) = 0$. Also, $U_7^\beta |\gamma\rangle = (-1)^{\text{tr}(\beta)\gamma^7} |\gamma\rangle$, and thus all U_7^β where $\text{tr}(\beta) = 1$ are the same gate, namely a CCZ gate supported on the three qubits constituting the qudit.

For $3 \mid s$ and $s > 3$, let us first show that $\ell_7 = s$. Of course, $(2^s - 1) \cdot 7 \equiv 0 \pmod{q - 1}$. On the other hand, suppose we have $(2^j - 1) \cdot 7 \equiv 0 \pmod{2^s - 1}$ for $1 \leq j < s$. Write $M = (2^s - 1)/7$, then $M \mid 2^s - 1$ and $M \mid 2^j - 1$ and thus $M \mid \gcd(2^s - 1, 2^j - 1) = 2^{\gcd(s,j)} - 1$. Since $\gcd(s, j) \leq s/2$ (given that $j < s$), and $M = \frac{2^s - 1}{7} > 2^{s/2} - 1$ (this may be checked for $3 \mid s$ and $s > 3$), we have a contradiction.

Thus, whenever $3 \mid s$ and $s > 3$, $\ell_7 = s$, U_7^β is a non-trivial third-level gate for every $\beta \neq 0$. Let α be a primitive root of $\mathbb{F}_{2^s}$. First notice that if $\beta_1 = \beta_2 \cdot \alpha^{7i}$ for some $i \in \mathbb{Z}_{\geq 0}$, then $U_7^{\beta_1}$ and $U_7^{\beta_2}$ are Clifford equivalent. This is because $\text{tr}(\beta_1 \gamma^7) = \text{tr}(\beta_2 (\alpha^i \gamma)^7)$ and according to Remark 3.4, multiplying by the known constant α^i is Clifford. We therefore focus on the multiplicative group $\mathbb{F}_{2^s}^\times = \{1, \alpha, \alpha^2, \alpha^3, \dots, \alpha^{q-2}\}$, where we emphasise $7 \mid q - 1$. Different β 's that are equal under multiplication by some power of α^7 lead to Clifford equivalent U_7^β , and so one only needs to focus on U_7^β with $\beta \in \{1, \alpha, \alpha^2, \alpha^3, \dots, \alpha^6\}$. Moreover, again from Remark 3.4, we know that the Frobenius automorphism is Clifford. This further tells us that the three U_7^β with $\beta \in \{\alpha, \alpha^2, \alpha^4\}$ are all Clifford equivalent, and similarly the U_7^β with $\beta \in \{\alpha^3, \alpha^5, \alpha^6\}$ are Clifford equivalent. We conclude that for $3 \mid s$ and $s > 3$, there are at most 3 Clifford inequivalent U_7^β .

The first non-trivial case of this is the $\mathbb{F}_{64}$ case shown in Fig. 5. For the three cases in $\mathbb{F}_{64}$, we perform minimum-weight decoding of a length 63 Reed-Muller code to determine their minimal ancilla-free T count [AM19] and found them to be unequal, and therefore the three cases in $\mathbb{F}_{64}$ must be Clifford inequivalent to each other. It is worth mentioning that, through brute-force basis change¹⁶, we observe that the case (d3) in Fig. 5 is Clifford equivalent to the $\mathbb{F}_4$ -qudit CCZ gate plotted in Fig. 4(a).

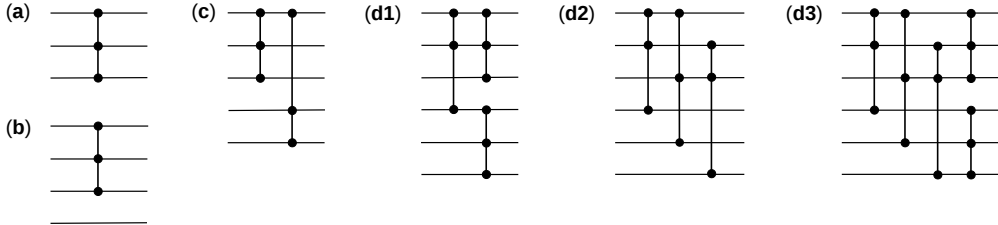


Figure 5: U_7^β gate expanded in qubit CCZ gates up to Clifford equivalence. (a) U_7^β in $\mathbb{F}_8$ with β satisfying $\text{tr}(\beta) \neq 0$. If $\text{tr}(\beta) = 0$, U_7^β is the identity. (b) In $\mathbb{F}_{16}$, all U_7^β for which $\beta \neq 0$ are Clifford equivalent to a single CCZ. (c) In $\mathbb{F}_{32}$, all U_7^β for which $\beta \neq 0$ are Clifford equivalent to a TOF# gate. (d1-d3) In $\mathbb{F}_{64}$; (d1) For $\beta = \alpha^{7i}$; (d2) For $\beta = \alpha^{1+7i}$ or α^{2+7i} or α^{4+7i} ; (d3) For $\beta = \alpha^{3+7i}$ or α^{5+7i} or α^{6+7i} . The three are Clifford inequivalent because their minimal ancilla-free T count [CH17] is not equal. The T count for (d1-d3) is 13, 15, 17, respectively.

As a final remark, note that one can use a qudit-CCZ gate in $\mathbb{F}_{2^{3s}}$ to implement the U_7 gate in $\mathbb{F}_{2^s}$; copy $|\gamma\rangle$ to the other two all-zero registers using CNOT gates and apply the Frobenius transform to raise the two copies to $|\gamma^2\rangle$ and $|\gamma^4\rangle$. Apply qudit-CCZ across the three registers, and finally undo the CNOT gates. Since the ancillae need to start in the zero state, this does not tell us that $U_7 \otimes I \otimes I$ and qudit-CCZ are Clifford equivalent.

¹⁶For small fields $\mathbb{F}_{2^s}$, we may attempt to establish Clifford equivalence of two CCZ circuits by randomly sampling a basis transform described by an $\text{GL}(s, \mathbb{F}_2)$ matrix (A_{ij}) . Each variable x_i occurring in the phase polynomial transforms as $x_i \mapsto \sum_j A_{ij} x_j$. We can then obtain the transformed phase polynomial (using $x_i^2 = x_i$, 2 times anything equals zero), removing any degree ≤ 2 terms (since they correspond to CZ and Z gates). We obtain Clifford equivalence if what is left over is the target phase polynomial. The procedure is illustrated in Fig. 3(b).

4 Distillation: transversality conditions and protocol constructions

4.1 Generalized triorthogonality on qubit codes and TOF# protocols

The generalized triorthogonality framework [CH17, App. D] gives a flexible way to construct distillation protocols based on binary CSS codes. In particular, acting with T gates transversally on the physical qubits, we can establish whether the codespace is preserved (possibly after Clifford corrections), and if so, what the induced logical action is. Canonically, the binary CSS code is written into a matrix $G = \begin{pmatrix} G_1 \\ G_0 \end{pmatrix} \in \mathbb{F}_2^{m \times n}$ matrix, where rows of $G_1 \in \mathbb{F}_2^{k \times n}$ and $G_0 \in \mathbb{F}_2^{(m-k) \times n}$ describe the X -logicals and X stabilizers of the $[[n, k, d]]$ code. From G , the procedure to check the logical action of the transversal T gate on the physical qubits is efficient, only requiring checks on the weights of overlaps of various rows.

Let us now explain how this works, meanwhile slightly extending the framework with a metric $\Gamma = [\Gamma_1, \dots, \Gamma_n] \in \mathbb{Z}_8^n$. The physical meaning of the metric is that, instead of applying transversal T , we apply $T^{\Gamma_1} \otimes \dots \otimes T^{\Gamma_n}$ to the physical qubits.

The binary matrix $G \in \mathbb{F}_2^{m \times n}$ is a generator matrix for some code. Its rows are denoted $\mathbf{g}_1, \dots, \mathbf{g}_k, \mathbf{g}_{k+1}, \dots, \mathbf{g}_m$. We associate binary variables $z_1, \dots, z_m$ with each row from G ; codewords of the code for which G is a generator matrix may be written $\sum_{i=1}^m z_i \mathbf{g}_i$.

Applying the physical gate $T^{\Gamma_1} \otimes \dots \otimes T^{\Gamma_n}$, the following transformation occurs:

$$|\sum_{i=1}^m z_i \mathbf{g}_i\rangle \mapsto \exp\left[\frac{i\pi}{4} p(z_1, z_2, \dots, z_m)\right] |\sum_{i=1}^m z_i \mathbf{g}_i\rangle. \quad (36)$$

The phase gained, as described by the phase polynomial $p(z_1, \dots, z_m) \bmod 8$ can be calculated using the general fact that [CH17, Eq. D7]

$$\bigoplus_{i=1}^m a_i = \sum_{i=1}^m a_i - 2 \sum_{1 \leq i < j \leq m} a_i a_j + 4 \sum_{1 \leq i < j < l \leq m} a_i a_j a_l - 8 \sum \dots, \quad (37)$$

where $\oplus$ denotes addition modulo 2, and $a_i \in \{0, 1\}$. The expression is up to m terms, but the coefficient in front of the m' -th term is $2^{m'-1}$ ($m' < m$), and thus for us to determine the phase polynomial mod 8, we can ignore all intersections involving more than three terms. We therefore have

$$\begin{aligned} p(z_1, \dots, z_m) &= \sum_{j=1}^n \Gamma_j \left(\bigoplus_{i=1}^m z_i g_{ij} \right) \\ &\equiv \sum_{j=1}^n \Gamma_j \left[\sum_{i=1}^m z_i g_{ij} - 2 \sum_{1 \leq i < l \leq m} z_i z_l g_{ij} g_{lj} + 4 \sum_{1 \leq i < l < r \leq m} z_i z_l z_r g_{ij} g_{lj} g_{rj} \right] \bmod 8 \\ &\equiv \sum_{i=1}^m \text{wt}_{\Gamma}(\mathbf{g}_i) z_i - 2 \sum_{1 \leq i < l \leq m} \text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l) z_i z_l + 4 \sum_{1 \leq i < l < r \leq m} \text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l \wedge \mathbf{g}_r) z_i z_l z_r \bmod 8, \end{aligned} \quad (38)$$

where $\mathbf{g}_i \wedge \mathbf{g}_j := (g_{i1}g_{j1}, \dots, g_{in}g_{jn})$ denotes entry-wise AND of the two binary vectors, and $\text{wt}_{\Gamma}(\mathbf{u}) := \sum_{j=1}^n \Gamma_j u_j$ denotes a Hamming weight in the metric Γ .

For a moment, think of the X stabilizers as additional X logicals fixed to the $|+\rangle$ state and treat these m logicals as m wires. Then Eq. 38 says that the logical circuit created on these m wires is

1. $T^{\text{wt}_{\Gamma}(\mathbf{g}_i)}$ acting on logical qubit i , which becomes trivial if $\text{wt}_{\Gamma}(\mathbf{g}_i) \equiv 0 \bmod 8$, and becomes Clifford if $\text{wt}_{\Gamma}(\mathbf{g}_i) \equiv 0 \bmod 2$.
2. $\text{CS}^{\text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l)}$ acting on logical qubits i and l , which becomes trivial if $\text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l) \equiv 0 \bmod 4$, and Clifford if $\text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l) \equiv 0 \bmod 2$.
3. $\text{CCZ}^{\text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l \wedge \mathbf{g}_r)}$ acting on logical qubits i, l, r , which becomes trivial if $\text{wt}_{\Gamma}(\mathbf{g}_i \wedge \mathbf{g}_l) \equiv 0 \bmod 2$.

Therefore, the physical gate $T^{\Gamma_1} \otimes \dots \otimes T^{\Gamma_n}$ is a valid logical gate if the non-trivial part of this circuit is only on the first k wires (authentic logical qubits), which is equivalent to saying that $p(z_1, \dots, z_m) \bmod 8$ depends only on $z_1, \dots, z_k$, but not $z_{k+1}, \dots, z_m$ (the variables associated to the stabilizer rows).

However, as pointed out in [CH17], when this binary code is used as a distillation protocol, it is enough to request only that the *non-Clifford part* of the logical circuit lies on the first k wires. This is because we can control the encoding circuit of this code. The encoding into logical all-plus state can be executed by, initializing wires 1 to m in $|+\rangle$ and other $n - m$ wires in $|0\rangle$ and then apply some CNOT circuit. The Clifford corrections (even in the case of a CZ gate between wire 1 and $k + 1$) can be done just before the CNOT circuit (but after the initialization).

We follow the same logic of generalized triorthogonality in deriving all our $\mathbb{F}_{2^s}$ -code distillation protocols. Associate a variable with each row of the generator matrix, calculate the phase polynomials under some physical magic gates, and enforce that the *non-Clifford part* of the phase polynomial be independent of the variables associated to the stabilizer rows.

We want to remark that it is straightforward to extend this framework for calculating the logical effect of applying $Z^{1/2^t}$ gates (or its power) to the physical qubits. One just needs to consider the phase polynomial modulo 2^{t+1} , and thus change the cutoff in the expansion [WQB23, Prop. A.1]

$$\bigoplus_{i=1}^m a_i = \sum_{s=1}^m 2^{s-1} \sum_{\substack{S \subseteq [m] \\ |S|=s}} \prod_{j \in S} a_j. \quad (39)$$

One immediately sees that only overlaps between every $\leq (t + 1)$ rows from G contribute to the calculation.

Construction 4.1 ($64T \rightarrow 2\text{TOF}\#$ at $d = 4$). *For this construction, we recall the preliminary material on Reed-Muller codes, see Section 2.2, where we wish to consider binary Reed-Muller codes in the present case. In [HH18a], several $T \rightarrow \text{CCZ}$ protocols based on Reed-Muller codes are developed. Now, we consider a length-64 Reed-Muller code, and letting X logicals correspond to the 15 degree-2 monomials $\{x_i x_j\}_{1 \leq i < j \leq 6}$, and letting X stabilizers be the degree < 2 monomials, i.e., $(x_i)_{1 \leq i \leq 6}$, and the constant function 1. This code has a Z -distance of 4, and an X -distance of 16.*

To calculate the effect of the physical transversal T gate on this code, we may consider the generalized triorthogonality framework, as well as the information about the overlaps and weights of Reed-Muller codewords given in the preliminaries. For this code, the logical action of the transversal gate turns out to be a CCZ circuit, with CCZ applied to every triplet of X -logicals (represented by monomials) multiplying to the full monomial $x_1 x_2 x_3 x_4 x_5 x_6$, whose evaluation vector has Hamming weight one.^a We can simplify this hypergraph state into $2\text{TOF}\#$ gates, see Figure 6. One can set the logical qubits 3, 4, 7, 8, 12 (wires associated to $x_1 x_4$, $x_1 x_5$, $x_2 x_4$, $x_2 x_5$, $x_3 x_6$) to $|0\rangle$ prior to applying the physical T gates. CCZ gates that involve these qubits become trivial, and what is left is two decoupled $\text{TOF}\#$ gates on logicals $\{1, 10, 11, 14, 15\}$ and $\{2, 5, 6, 9, 13\}$.

^aThis logical hypergraph magic state, as shown in Figure 6, has also been derived using other methods [Ren+20; Bar+26].

4.2 Qudit CCZ gate protocols

Beyond $\mathbb{F}_2$, we may use matrices over higher fields, $G \in \mathbb{F}_q^{m \times n}$, to encode qudit quantum codes, where we only consider binary extension fields (the case $q = 2^s$) in our work, so that we can ultimately obtain qubit magic state distillation protocols. The first k rows serve as logical X operators, and the latter $m - k$ rows are X stabilizer generators; see Definition 3.1. Generalizing the case for qubits, we denote the two sub-matrices given by the former k rows and the latter $m - k$ rows as G_1 and G_0 , and the $\mathbb{F}_q$ -row span of G_1, G_0, G as $\mathcal{G}_1, \mathcal{G}_0, \mathcal{G}$ respectively.

[Ngu25; GG25a; NP25] showed that if a qudit code encoded by a matrix G satisfies the three-orthogonal property

$$\sum_{i=1}^n (g_{a,i})(g_{b,i})(g_{c,i}) = \begin{cases} 1 & \text{if } 1 \leq a = b = c \leq k \\ 0 & \text{otherwise,} \end{cases} \quad (40)$$

then applying transversal qudit CCZ gates to three patches of this qudit code leads to qudit CCZ

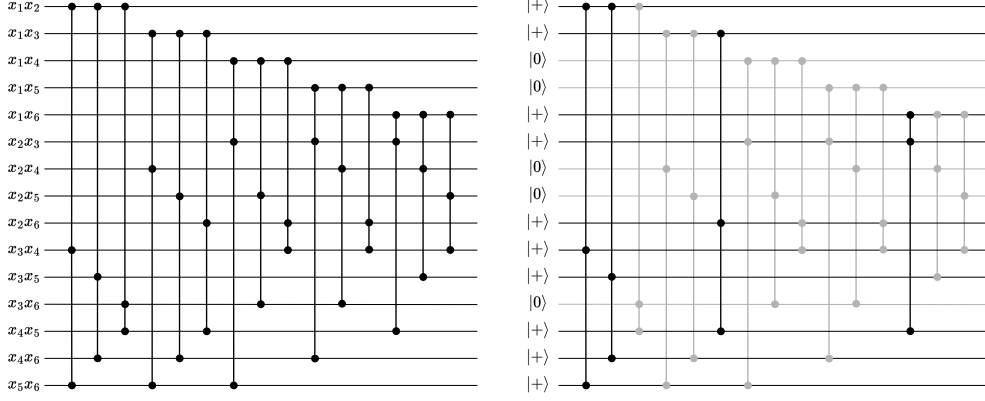


Figure 6: Left: the resulting logical circuit when applying transversal T to the $[[64, 15, 4]]$ Reed-Muller code. The 15 logicals are represented by the $\binom{6}{2}$ degree-two monomials in six variables. There is a CCZ for every three monomials that multiply to $x_1x_2x_3x_4x_5x_6$. Right: by initializing some of the logicals to the $|0\rangle$ state, two disjoint TOF# states can be obtained. The error suppression of this protocol is $10256p^4$.

gates on the logical level. A generalized version yielding a transversal qudit $C^{\ell-1}Z$ gate is found in Eq. (43) [He+25b]; qudit CCZ is the special case of $\ell = 3$.

We defer a discussion of our protocols distilling qudit CCZ states to Sec. 5, after we have introduced the necessary notions on algebraic geometry codes. The parameters of the corresponding finite-length constructions are summarized in Table 3. Note that those protocols take qudit CCZ states as input and output, making them seem unwieldy, since synthesizing one input qudit CCZ state over $\mathbb{F}_{2^s}$ naively requires $O(s^3)$ qubit CCZ states. However, using the connection between CCZ states and multiplication in their corresponding field, it is in fact possible to synthesize such a qudit CCZ state with only $O(s)$ qubit CCZ states and Clifford operations; see the discussion in App. B.3, as well as [BP11].

4.3 U_7 gate protocols

The U_7^β gate, discussed in Section 3.2.5, is a non-Clifford gate [WHY25]. Moreover, they proved that if a qudit quantum code satisfies the following *twisted three-orthogonal* criterion

$$\sum_{i=1}^n (g_{a,i})^4 (g_{b,i})^2 (g_{c,i}) = \begin{cases} 1 & \text{if } 1 \leq a = b = c \leq k \\ 0 & \text{otherwise,} \end{cases} \quad (41)$$

then U_7 ($\beta = 1$) is a transversal gate for this code. We consider here a more generalized framework that is useful for deriving protocols. If the code is twisted three-orthogonal with respect to $(\mathbf{\Gamma}, \boldsymbol{\tau})$, that is,

$$\sum_{i=1}^n \Gamma_i (g_{a,i})^4 (g_{b,i})^2 (g_{c,i}) = \begin{cases} \tau_a & \text{if } 1 \leq a = b = c \leq k \\ 0 & \text{otherwise} \end{cases}, \quad (42)$$

then $\forall \beta \in \mathbb{F}_q$, $U_7^{\beta\mathbf{\Gamma}} := \otimes_{i=1}^n U_7^{\beta\Gamma_i}$ implements $\overline{U_7^{\beta\mathbf{\Gamma}}} := \overline{\otimes_{i=1}^k U_7^{\beta\tau_i}}$. The proof is almost identical to [WHY25], but before going into that, we remark that introducing the metric gamma $\mathbf{\Gamma}$ is a useful generalization when dealing with elliptic curve codes that are in general not self-orthogonal. Moreover, considering β is important in $\mathbb{F}_{2^s}$ where s is a multiple of 3, see Section 3.2.5. This generalization is similar to the ℓ -orthogonality framework developed in [He+25c]:

$$\sum_{i=1}^n \Gamma_i (g_{a_1,i}) (g_{a_2,i}) \cdots (g_{a_\ell,i}) = \begin{cases} \tau_{a_1} & \text{if } 1 \leq a_1 = a_2 = \cdots = a_\ell \leq k \\ 0 & \text{otherwise} \end{cases}. \quad (43)$$

Notice that a matrix G satisfying Eq. 43 satisfies Eq. 42. In [WHY25], distillation protocols for U_7^β gates were built by simply defining matrices G that satisfied the stronger statement of Eq. 43 with $\ell = 7$. However, this is inherently wasteful, because Eq. 42 is sufficient for U_7^β transversality, and requires the matrix to satisfy fewer conditions. For example, one does not need to check whether

$\sum_{i=1}^n \Gamma_i(g_i^1)^4(g_i^2)(g_i^3)(g_i^4)$ is zero¹⁷. We will exploit this later (in Sec. 4.3 and Con. 5.8), only aiming to satisfy Eq. 42 in order to construct improved protocols for the U_7^β gate.

Lemma 4.2. *On a quantum code over $\mathbb{F}_8$ constructed from a matrix satisfying Eq. (42), the physical $U_7^{\beta\Gamma}$ gate implements the logical gate $\overline{U_7^{\beta\tau}}$.*

Proof. Write a classical codeword of $\mathcal{G}$ as a linear combination of its rows: $\mathbf{f} = \sum_{a=1}^m u_a \mathbf{g}_a \in \mathbb{F}_q^n$ for $u_a \in \mathbb{F}_q$, $a = 1, \dots, m$. The i -th entry of this codeword is $f_i = \sum_{a=1}^m u_a g_{a,i}$ and so $f_i^7 = (\sum_{a=1}^m u_a g_{a,i})^7$. Applying Eq. (42) and the multinomial formula (where note that all even multinomial coefficients vanish, and all odd multinomial coefficients reduce to 1, since we work in a binary extension field), one has

$$\begin{aligned} \sum_{i=1}^n \Gamma_i f_i^7 &= \sum_{a=1}^m \sum_{i=1}^n \Gamma_i (u_a g_{a,i})^7 + \sum_{\substack{1 \leq a,b \leq m \\ a \neq b}} \sum_{i=1}^n \Gamma_i [(u_a g_{a,i})^6 (u_b g_{b,i}) + (u_a g_{a,i})^5 (u_b g_{b,i})^2 + (u_a g_{a,i})^4 (u_b g_{b,i})^3] \\ &\quad + \sum_{\substack{1 \leq a,b,c \leq m \\ a,b,c \text{ pairwise distinct}}} \sum_{i=1}^n \Gamma_i (u_a g_{a,i})^4 (u_b g_{b,i})^2 (u_c g_{c,i}) = \sum_{a=1}^k \tau_a u_a^7. \end{aligned} \quad (44)$$

Notice that many of the terms in the expression vanish by applying Eq. (41) and contribute trivially. The effect of applying $U_7^{\beta\Gamma}$ on the product state $|\mathbf{f}\rangle = \otimes_{i=1}^n |f_i\rangle$ is thus

$$\otimes_{i=1}^n U_7^{\beta\Gamma_i} |\mathbf{f}\rangle = (-1)^{\sum_{i=1}^n \text{tr}(\beta \Gamma_i f_i^7)} |\mathbf{f}\rangle = (-1)^{\text{tr}(\beta \sum_{i=1}^n \Gamma_i f_i^7)} |\mathbf{f}\rangle = (-1)^{\text{tr}(\beta \sum_{a=1}^k \tau_a u_a^7)} |\mathbf{f}\rangle$$

The logical action on $|\mathbf{u}\rangle$ is thus

$$\begin{aligned} \otimes_{i=1}^n U_7^{\beta\Gamma_i} |\mathbf{u}\rangle &= \sum_{\mathbf{g} \in \mathcal{G}_0} \otimes_{i=1}^n U_7^{\beta\Gamma_i} \left| \sum_{a=1}^k u_a \mathbf{g}_a + \mathbf{g} \right\rangle \\ &= \sum_{\mathbf{g} \in \mathcal{G}_0} (-1)^{\text{tr}(\beta \sum_{a=1}^k \tau_a u_a^7)} \left| \sum_{a=1}^k u_a \mathbf{g}_a + \mathbf{g} \right\rangle \\ &= (-1)^{\text{tr}(\beta \sum_{a=1}^k \tau_a u_a^7)} |\mathbf{u}\rangle \\ &= \overline{\otimes_{i=1}^k U_7^{\beta\tau_i} |\mathbf{u}\rangle}, \end{aligned}$$

giving the result. $\square$

Throughout, when we say CCZ, we mean the regular 3-qubit CCZ gate unless otherwise specified. We emphasize that the U_7 gate in $\mathbb{F}_8$ is equivalent to a single such CCZ gate on the three qubits of the $\mathbb{F}_8$ qudit; see Section 3.2.5.

Construction 4.3 ($8\text{CCZ} \rightarrow 2\text{CCZ}$ at $d = 2$). *We construct an $8\text{CCZ} \rightarrow 2\text{CCZ}$ ($d = 2$) protocol derived from the $\mathbb{F}_8$ Reed-Solomon code. We choose the two rows $\{x^2, x\}$ to form the X logical generators, and the 1 row to form the X stabilizer. Explicitly, we write $\mathbb{F}_8$ as $\mathbb{F}_2(\alpha)$, where α is an element satisfying $\alpha^3 = \alpha + 1$, the matrix defining the qudit code is*

$$\begin{array}{ccccccc} \text{ev}(x^2) & \begin{pmatrix} 0 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 \end{pmatrix} \\ \text{ev}(x) & \begin{pmatrix} 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 \end{pmatrix} \\ \text{ev}(1) & \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \end{array} \quad (45)$$

These three rows $\{x^2, x, 1\}$ satisfy Eq. (41) with $k = 2$. One can verify this by hand: notice that $x^7 = 1$, $\forall x \in \mathbb{F}_8^\times$ and hence $\sum_{x \in \mathbb{F}_8} (x^2)^7 = \sum_{x \in \mathbb{F}_8} (x^1)^7 = 1$ and $\sum_{i=1}^8 (1)^7 = 8 \cdot 1 = 0$. One can also verify that $\sum_{i=1}^8 (g_{a,i})^4 (g_{b,i})^2 (g_{c,i})^1 = 0$ for $a, b, c \in \{0, 1, 2\}$ not all equal using $\sum_{i=1}^7 \alpha^{bi} = 0$ for $\text{gcd}(7, b) = 1$ (in this case α^b is also a primitive element of this field). For example, $\sum_{i=1}^8 (g_{1,i})^6 (g_{2,i})^1 = \sum_{i=1}^7 (\alpha^{2i})^6 (\alpha^i)^1 = \sum_{i=1}^7 \alpha^{13i} = 0$. We generalize this construction to multi-control-Z distillation in Con. 4.13, and an algebraic proof can be found there.

¹⁷This is because by lemma 3.7, the multinomial coefficient $\binom{7}{4,1,1,1}$ is even and hence disappear in $\mathbb{F}_2$ arithmetic. In fact, there is no need to calculate the overlap of more than three rows.

Let us show concretely how the above matrix can be binarized to obtain a distillation protocol for the qubit CCZ gate. A self-dual normal basis for $\mathbb{F}_8$ is

$$\alpha_0 = \alpha^2 + \alpha + 1, \quad \alpha_1 = \alpha + 1 = \alpha_0^2, \quad \alpha_2 = \alpha^2 + 1 = \alpha_0^4, \quad (46)$$

where $\alpha^3 = \alpha + 1$. Each GF(8) qudit entry γ in the generator matrix can be expanded into a 3×3 binary matrix as:

$$\begin{pmatrix} \text{tr}(\gamma\alpha_0\alpha_0) & \text{tr}(\gamma\alpha_0\alpha_1) & \text{tr}(\gamma\alpha_0\alpha_2) \\ \text{tr}(\gamma\alpha_1\alpha_0) & \text{tr}(\gamma\alpha_1\alpha_1) & \text{tr}(\gamma\alpha_1\alpha_2) \\ \text{tr}(\gamma\alpha_2\alpha_0) & \text{tr}(\gamma\alpha_2\alpha_1) & \text{tr}(\gamma\alpha_2\alpha_2) \end{pmatrix}. \quad (47)$$

The resulting expanded qubit code of $[[8, 2, 2]]_8$ is as follows. As always, the upper rows (in this case six) are X logicals, and the latter rows (in this case three) are X stabilizers.

$$\begin{pmatrix} 000 & 111 & 110 & 011 & 001 & 101 & 010 & 100 \\ 000 & 100 & 111 & 110 & 011 & 001 & 101 & 010 \\ 000 & 101 & 010 & 100 & 111 & 110 & 011 & 001 \\ 000 & 001 & 111 & 101 & 110 & 010 & 011 & 100 \\ 000 & 011 & 100 & 001 & 111 & 101 & 110 & 010 \\ 000 & 111 & 101 & 110 & 010 & 011 & 100 & 001 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 010 & 010 & 010 & 010 & 010 & 010 & 010 & 010 \\ 001 & 001 & 001 & 001 & 001 & 001 & 001 & 001 \end{pmatrix} \quad (48)$$

Applying CCZ to every adjacent triplet of qubits (columns) yield two logical CCZ gates, supported on rows 1, 2, 3 and rows 4, 5, 6 respectively.

Remark 4.4. *There is an $8\text{CCZ} \rightarrow 2\text{CCZ}$ ($d = 2$) protocol [Cha+22, Fig. 16] constructed from three different binary codes, each removing one of the three logical rows of the $[[8, 3, 2]]$ Reed-Muller code. We find this protocol to be inequivalent to our Eq. (48). In fact, when concatenating with the $8T \rightarrow \text{CCZ}$ ($d = 2$) first-level factory, we find their resulting protocol to be equivalent to the $64T \rightarrow 2\text{CCZ}$ ($d = 4$) protocol from [HH18a] (the equivalency can be proved similarly to our App. A.3). Their error suppression $2944p^4$ is different from what is obtained by concatenating our code with the $8T \rightarrow \text{CCZ}$ code, which is $3136p^4$. Notice that the error coefficient is worse in this case, but need not be generally so.*

Construction 4.5 ($9\text{CCZ} \rightarrow 1\text{CCZ}$ at $d = 3$). *The $9 \rightarrow 1$ ($d = 3$) protocol is obtained by padding the row x with a one and rows $x^2, 1$ with a zero, see below. Padded x^2 is treated as logical, and $x, 1$ are stabilizers.*

$$\begin{pmatrix} 0 & 0 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 \\ 1 & 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad (49)$$

Remark 4.6. *Another way to obtain a protocol with these parameters is via a projective Reed-Solomon code. The nine points of $\mathbb{P}_{\mathbb{F}_8}$ are $(0 : 1)$ and $\{(1 : \gamma) \mid \gamma \in \mathbb{F}_8^\times\}$. One can construct a matrix as the evaluation of the homogeneous polynomials XY (logical), Y^2 , X^2 on these nine $\mathbb{P}_{\mathbb{F}_8}$ points¹⁸:*

$$\begin{matrix} \text{ev}(XY) \\ \text{ev}(Y^2) \\ \text{ev}(X^2) \end{matrix} \begin{pmatrix} 0 & 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 \\ 1 & 0 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad (50)$$

Construction 4.7 ($14\text{CCZ} \rightarrow 4\text{CCZ}$ at $d = 2$). *We can double the above $\mathbb{F}_8$ RS construction as follows to obtain a $14 \rightarrow 4$ ($d = 2$) protocol*

$$\left(\begin{array}{cccccc|cccccccc} \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{array} \right) \quad (51)$$

¹⁸As we mentioned in Sec. 2.3, for a homogeneous function F of degree d , one has $f^h(\lambda x_0, \dots, \lambda x_n) = \lambda^d \cdot f^h(x_0, \dots, x_n)$. So scaling by $\lambda \in \mathbb{F}_8^\times$ of a projective point leads to a scalar multiplication of the corresponding column by λ . However, since $\lambda^7 = 1$, the scaling has no effect in Eq. 42.

Construction 4.8 ($16\text{CCZ} \rightarrow 2\text{CCZ}$ at $d = 3$). A $16 \rightarrow 2$ ($d = 3$) protocol can be obtained by doubling the projective RS code

$$\left(\begin{array}{cccccc|cccc} 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 & 0 \\ \hline 1 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{array} \right) \quad (52)$$

Construction 4.9 ($(6k+2)\text{CCZ} \rightarrow 2k\text{CCZ}$ at $d = 2$). Inspired by the $(3k+8)T \rightarrow kT$ protocol from [BC15] and the $(6k+2)T \rightarrow k\text{CCZ}$ protocol from [CH17] (especially the latter), here we give a $(6k+2)\text{CCZ} \rightarrow 2k\text{CCZ}$ protocol.

$$\left(\begin{array}{cccc|ccc} L & M & 0 & 0 & \cdots & 0 \\ L & S & M & 0 & \cdots & 0 \\ L & S & S & M & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \hline L & S & S & S & \cdots & M \\ \hline 1 & 1 & \mathbf{1}_6 & \mathbf{1}_6 & \cdots & \mathbf{1}_6 \end{array} \right) \quad (53)$$

where $L = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$, $M = \begin{pmatrix} \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^{10} & \alpha^{12} \\ \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 \end{pmatrix}$, $S = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$ and $\mathbf{1}_6 = (1 \ 1 \ 1 \ 1 \ 1 \ 1)$.

Construction 4.10 ($21\text{CCZ} \rightarrow 3\text{CCZ}$ at $d = 3$). Considering gluing three projective lines $X = 0$, $Y = 0$, $Z = 0$ in $\mathbb{P}_{\mathbb{F}_8}^2$ along a triangle. That is, we consider 21 evaluation points

$$\{(0 : 1 : \gamma) \mid \gamma \in \mathbb{F}_8^\times\} \cup \{(\gamma : 0 : 1) \mid \gamma \in \mathbb{F}_8^\times\} \cup \{(1 : \gamma : 0) \mid \gamma \in \mathbb{F}_8^\times\}.$$

and we consider constructing a stabilizer matrix whose columns correspond to these evaluation points. The matrix is constructed as follows, with three logical X rows and three stabilizer X rows (and 21 columns):

	$(0 : 1 : \gamma)$	$(\gamma : 0 : 1)$	$(1 : \gamma : 0)$
$\text{ev}(YZ^2)$	γ^2	0	0
$\text{ev}(ZX^2)$	0	γ^2	0
$\text{ev}(XY^2)$	0	0	γ^2
$\text{ev}(X)$	0	γ	1
$\text{ev}(Y)$	1	0	γ
$\text{ev}(Z)$	γ	1	0

(54)

One checks that this satisfies Eq. (41). The distance of this code is three; indeed, one checks that it is at least three by observing that no two stabilizer columns are equal up to scalar multiplication.^a

^aNote that strictly speaking this verifies only that the Z -distance is at least three. However, the Z -distance is the relevant distance for distillation because one can twirl away X errors [BH12] (one may also check that the X -distance is greater than the Z -distance any way).

Construction 4.11 ($64\text{CCZ} \rightarrow 8\text{CCZ}$ at $d = 4$). Here we construct a protocol using affine monomial codes (see the end of Sec. 2.2) defined over the quotient ring

$$\mathbb{F}_8[x, y]/(x^8 - x, y^8 - y).$$

There are $8^2 = 64$ points on the affine grid; denote them as $(x, y) \in \mathbb{F}_8^2$. All the X -logicals and stabilizers we choose for the distillation matrix are in the form of evaluation of monomials $x^a y^b$, $0 \leq a, b \leq 6$ over the affine grid $\mathbb{F}_8^2$.

To satisfy Eq. (41), we choose the X stabilizers as $1, x, y, x^2, y^2$ (whose dual code is the hyperbolic

code of designed distance 4)^a. We are able to find eight compatible logicals by solving for the maximum independent set in a certain hypergraph, constructed as follows.

We start by searching for all exponents^b (a, b) , $1 \leq a, b \leq 6$ such that $x^a y^b$ together with the five chosen stabilizers, satisfy Eq. (41) with $k = 1$. There are 29 such exponents (a, b) . Think of each of these exponents, as well as the five stabilizer exponents, as vertices. We then add a hyperedge to every triplet and pair of vertices that would cause a violation of Eq. 42. For example, for a triplet of exponents $(a_1, b_1), (a_2, b_2), (a_3, b_3)$, we add a hyperedge if $\sum_{(x,y) \in \mathbb{F}_8^2} (x^{a_1} y^{b_1})^1 (x^{a_2} y^{b_2})^2 (x^{a_3} y^{b_3})^4 \neq 0$. These (hyper)edges specify which simultaneous choice of exponents are forbidden. Therefore, finding a maximum number of logicals is equivalent to finding a maximum independent set within this forbidden hypergraph. While finding a maximum independent set in a hypergraph is NP-hard in general, it is tractable in this small instance via a depth-first search. As a result, we find three maximum independent sets all of size 8:

$$\{(1, 2), (1, 5), (2, 5), (2, 6), (4, 2), (4, 6), (5, 1), (5, 5)\}, \quad (55)$$

$$\{(1, 3), (2, 2), (2, 4), (3, 1), (3, 4), (4, 2), (4, 3), (4, 4)\}, \quad (56)$$

$$\{(1, 5), (2, 1), (2, 4), (5, 1), (5, 2), (5, 5), (6, 2), (6, 4)\}. \quad (57)$$

Remark 4.12. The logical exponent set in Eq. (56) is symmetric with respect to x and y , and in this case, we can use the footprint bound (see App. A.3) to show that the X -distance of the protocol is lower bounded by 16.

^aOne can see that xy cannot be an X stabilizer (in the metric of all-one), because raised to its seventh-power, all coordinates of $x^7 y^7$ sum up to one.

^bNotice here we can restrict ourselves to both a, b being positive. This is because, we want $\sum_{(x,y) \in \mathbb{F}_8^2} (x^a y^b)^7 =$
1. The left-hand side can be rewritten into $\left(\sum_{x \in \mathbb{F}_8} x^{7a}\right) \cdot \left(\sum_{y \in \mathbb{F}_8} y^{7b}\right)$ and is nonzero only when both $a, b > 0$.

4.4 Norm gate protocols

Besides $\text{tr}(\cdot)$, the norm function $\text{Nm}(\cdot)$ also maps $\text{GF}(2^s)$ elements to $\{0, 1\}$; it takes a particularly simple form in $\text{GF}(2^s)$: $\text{Nm}(\gamma) := \prod_{i=0}^{s-1} \gamma^{2^i} = \gamma^{2^s-1}$. Note that $\text{Nm}(\gamma) = 0$ if and only if $\gamma = 0$ and else $\text{Nm}(\gamma) = 1$, which follows from the fact that the multiplicative group of a finite field is cyclic. We define the norm gate as follows,

$$|\gamma\rangle \mapsto (-1)^{\text{Nm}(\gamma)} |\gamma\rangle = (-1)^{\gamma^{2^s-1}} |\gamma\rangle. \quad (58)$$

That is to say, $|\gamma\rangle$ gains a phase of $(-1)^0 = 1$ when $\gamma = 0$ and -1 when $\gamma \neq 0$. One can see that the norm gate, as an s -qubit gate, is equivalent to the $C^{s-1}Z$ gate, up to Paulis, regardless of the way the qudit is decomposed into qubits. Indeed, to implement the norm gate on the qubit level (up to a global phase), expand $\gamma = \sum_{i=1}^s b_i \alpha_i$. One first applies qubit-wise X gates on the s qubits, such that $b_i \mapsto b_i \oplus 1$, then a $C^{s-1}Z$ gate, and finally the X gates again. Then, only in the case of $b_1 = \dots = b_s = 0$ does the intermediate state gain a phase of -1 .

Associating $u_1, \dots, u_m$ with each row from G , then the computational basis state $|\sum_{i=1}^m u_i \mathbf{g}_i\rangle$ corresponding to a codeword of the classical code gains a phase of $(-1)^{p(u_1, \dots, u_m)}$, where the phase polynomial reads

$$p(u_1, \dots, u_m) = \sum_{j=1}^n \left(\sum_{i=1}^m u_i g_{i,j} \right)^{2^s-1}. \quad (59)$$

To expand this, we use the multinomial expansion¹⁹

$$(a_1 + a_2 + \dots + a_m)^{2^s-1} = \sum_{e_1 + \dots + e_m = 2^s-1} \binom{2^s-1}{e_1, e_2, \dots, e_m} a_1^{e_1} \dots a_m^{e_m} \quad (60)$$

¹⁹In the following equation, the sum is over tuples of non-negative integers $(e_i)_{i=1}^m$ that sum to $2^s - 1$.

and Lemma 3.7, which states that the multinomial coefficient

$$\binom{2^s - 1}{e_1, e_2, \dots, e_m} = \begin{cases} \text{odd} & \text{no carrying when adding } e_1 + \dots + e_m = 2^s - 1 \text{ in base 2} \\ \text{even} & \text{otherwise} \end{cases} \quad (61)$$

We therefore have

$$p(u_1, \dots, u_m) = \sum_{\substack{e_1 + \dots + e_m = 2^s - 1 \\ \text{supp}(\text{bin}(e_i)) \text{ pairwise disjoint} \\ \sqcup_{i=1}^m \text{supp}(\text{bin}(e_i)) = \{1, \dots, s\}}} \left(\prod_{i=1}^m u_i^{e_i} \right) \left[\sum_{j=1}^n \prod_{i=1}^m g_{i,j}^{e_i} \right], \quad (62)$$

where $\text{bin}(\cdot)$ denotes the binary expansion of the integer e_i , and $\text{supp}(\text{bin}(\cdot))$ denotes the position of the ones in the binary expansion.

For the logical action to also be a norm gate on each of the k logical qudits, the phase polynomial should simplify to $\sum_{i=1}^k u_i^{2^s - 1}$.

One may check that this is ensured by the following condition, which we call *twisted s -orthogonality*

$$\sum_{i=1}^n (g_{a_1,i})^{2^0} (g_{a_2,i})^{2^1} \dots (g_{a_s,i})^{2^{s-1}} = \begin{cases} 1 & \text{if } 1 \leq a_1 = a_2 = \dots = a_s \leq k \\ 0 & \text{otherwise} \end{cases} \quad (63)$$

Note that the U_7 gate is the norm gate in $\mathbb{F}_8$, and indeed we can see that this condition specializes to the U_7 gate's transversality condition (see Eq. 41) when we set $s = 3$.

Construction 4.13 ($2^s C^{s-1}Z \rightarrow 2C^{s-1}Z$ at distance 2). *Take the Reed-Solomon code over $\mathbb{F}_{2^s}$ and use x^2, x^1 as X logicals, and 1 as the X stabilizer. Let us show that these three rows satisfy the the above norm-gate orthogonality condition Eq. (63).*

Call α a primitive element of this field. First note that

$$\sum_{i=1}^{2^s-1} \alpha^{ij} = \begin{cases} 1 & \text{if } j \equiv 0 \pmod{2^s - 1} \\ 0 & \text{otherwise} \end{cases} \quad (64)$$

This is to say, the entry-wise sum of the evaluation vector $\text{ev}(x^j) \in \mathbb{F}_{2^s}^{2^s}$ is one if and only if $j \equiv 0 \pmod{2^s - 1}$. Therefore, the logical rows x^2 and x^1 each satisfy $\sum_{i=1}^n (g_{a,i})^{2^0} (g_{a,i})^{2^1} \dots (g_{a,i})^{2^{s-1}} = \sum_{i=1}^n (g_{a,i})^{2^s-1} = 1$ for $a = 1, 2$.

For the other conditions in Eq. (63), suppose $\text{ev}(x^2)$ is raised entry-wise to the e_1 -th power, $\text{ev}(x^1)$ to the e_2 -th power, and $\text{ev}(1)$ to the e_3 -th power, where $e_1 + e_2 + e_3 = 2^s - 1$, and $\text{supp}(\text{bin}(e_1)) \sqcup \text{supp}(\text{bin}(e_2)) \sqcup \text{supp}(\text{bin}(e_3)) = \{1, \dots, s\}$. Multiplying these and summing, we are simply considering the sum of the evaluation vector $\text{ev}(x^{2e_1+e_2})$. As above, we know that this is zero unless $2^s - 1 \mid 2e_1 + e_2$. Notice that $2e_1 + e_2 = e_1 + (e_1 + e_2) \leq 2(2^s - 1)$ and the equality holds if and only if $e_1 = 2^s - 1$ and $e_2 = e_3 = 0$, which has already been discussed. Also $2e_1 + e_2 = 0$ if and only if $e_3 = 2^s - 1$ and $e_1 = e_2 = 0$, which again has already been discussed. The only remaining possibility to consider is $2e_1 + e_2 = 2^s - 1$. One can actually see that the already-discussed case of $(e_1, e_2, e_3) = (0, 2^s - 1, 0)$ is the only possibility here. Indeed, aside from that case, e_1 must be non-zero. Let i be the position corresponding to the smallest power of 2 that is supported in the binary expansion of e_1 . Since $\text{supp}(e_1) \cap \text{supp}(e_2) = \emptyset$, and since $2e_1$ has the binary expansion of e_1 shifted by one, the binary expansion of $2e_1 + e_2$ is not supported at position i , meaning that $2e_1 + e_2 = 2^s - 1 = 11 \dots 1_2$ is impossible.

Construction 4.14 ($(2^s + 1)C^{s-1}Z \rightarrow 1C^{s-1}Z$ at distance 3). *One can pad the above Reed-Solomon to its projective version just as in Construction 4.5. That is, the $\text{ev}(x^2)$ row is padded with a 0 and treated as a logical, the $\text{ev}(x)$ row is padded with a 1 and treated as a stabilizer, and the $\text{ev}(1)$ row is padded with a 0 and treated as a stabilizer. The satisfaction of Equation 63 and the distance follows from the same argument there.*

²⁰Recall that saying that there is no carrying when adding $e_1 + \dots + e_m = 2^s - 1$ in base 2 is equivalent to saying that there is no overlap in the support of the binary expansion of the e_i . Note that the binary expansion of $2^s - 1$ is simply $11 \dots 1$.

4.5 Qubit CS protocols from $\mathbb{F}_4$ -linear codes

Here, we only consider codes over $\mathbb{F}_4 = \{0, 1, \omega, \omega^2\}$. As a reminder, we have $\omega^2 = \omega + 1$ and $\omega^3 = 1$, $\text{tr}(1) = 0 = \text{tr}(0)$, and $\text{tr}(\omega) = \text{tr}(\omega^2) = 1$. We can expand an $\mathbb{F}_4$ qudit using two qubits in the self-dual basis $\{\omega, \omega^2\}$, and consider applying a CS gate between these two qubits.

$$\gamma = b_1\omega + b_2\omega^2, \quad \text{CS}|\gamma\rangle := \text{CS}|b_1\rangle|b_2\rangle = i^{b_1b_2}|\gamma\rangle. \quad (65)$$

One can extract b_1, b_2 as $b_1 = \text{tr}(\gamma\omega)$ and $b_2 = \text{tr}(\gamma\omega^2)$ using the self-duality of the basis. Then, $b_1b_2 = (\gamma\omega + \gamma^2\omega^2)(\gamma\omega^2 + \gamma^2\omega^4) = \gamma^3 + \text{tr}(\gamma)$. Now, note that in this expression, $\gamma^3, \text{tr}(\gamma) \in \{0, 1\}$ are summed as elements of $\mathbb{F}_4$. Since they are exponentiated with base i , we need to convert the addition to $\mathbb{Z}_4$ addition. With $\oplus$ denoting the addition in $\mathbb{F}_4$, and $+$ denoting the addition in $\mathbb{Z}_4$, one checks that $\gamma^3 \oplus \text{tr}(\gamma) = \gamma^3 - \text{tr}(\gamma) \pmod{4}$. Therefore,

$$\text{CS} : |\gamma\rangle \mapsto i^{\gamma^3 - \text{tr}(\gamma)}|\gamma\rangle, \quad (66)$$

with the subtraction performed in $\mathbb{Z}_4$. The condition for physical CS to implement logical CS is

$$\sum_{i=1}^n g_{a,i} g_{b,i} g_{c,i} = \begin{cases} 1 & \text{if } 1 \leq a = b = c \leq k \\ 0 & \text{otherwise} \end{cases}, \quad (67)$$

in terms of the matrix G over $\mathbb{F}_4$ that defines the protocol. The proof is in App. A; we show two ways of deriving the condition. The first way in App. A.1 converts the calculation to one of generalized triorthogonality in a modified binary code and is most easy to understand. The second way in App. A.2 is much more involved but also more insightful, since we directly compute the phase polynomial over $\mathbb{Z}_4$ (instead of $\mathbb{F}_4$).

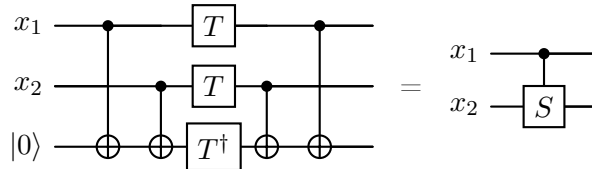
Notice that this condition is the same as for the $\mathbb{F}_4$ -qudit CCZ, i.e., three-orthogonality. Therefore, the following protocols presented for CS-to-CS distillation can also be used for $\mathbb{F}_4$ -qudit CCZ distillation.

We now present our first construction for CS-to-CS distillation using codes over $\mathbb{F}_4$.

Construction 4.15 ($4\text{CS} \rightarrow 1\text{CS}$ at $d = 2$). *The Reed-Solomon code over $\mathbb{F}_4$ naturally satisfies the above three-orthogonality condition.*

$$\begin{pmatrix} 0 & 1 & \omega & \omega^2 \\ 1 & 1 & 1 & 1 \end{pmatrix} \quad (68)$$

It turns out that any of our protocols that takes CS as input can be turned into a protocol that takes T (and $T^\dagger$) as input, as follows. Given a qubit code, imagine we are about to apply physical CS gates on disjoint pairs of physical qubits, such as for our binarized codes over $\mathbb{F}_4$. There is a way to turn these two-qubit gates into transversal $T/T^\dagger$ gates on an embedded code [WQB23], constructed as follows. For each such pair, add one ancilla qubit initialized in $|0\rangle$, and perform CNOT gates from the two qubits onto this ancilla (see the circuit below). Then, when acting with $T \otimes T \otimes T^\dagger$ on this embedded code, it gives a phase of $x_1 + x_2 + 7(x_1 \oplus x_2) = 2x_1x_2 \pmod{8}$. This means that a CS gate has been performed on the two non-ancillary qubits.



Using this idea, it is easy to determine the X logical/stabilizer matrix of the embedded code, simply by stabilizer propagation. Note that the logical action of physical $T/T^\dagger$ on the embedded code directly translates back to the logical action on the original code. It should be clear that the distance of the embedded code is lower bounded by that of the original code, because the weight after the stabilizer propagation is non-decreasing.

Let us demonstrate the idea using our Reed-Solomon example:

$$\begin{pmatrix} 0 & 1 & \omega & \omega^2 \\ 1 & 1 & 1 & 1 \end{pmatrix} \xrightarrow{\text{binarize}} \begin{pmatrix} 00 & 10 & 01 & 11 \\ 00 & 01 & 11 & 10 \\ 10 & 10 & 10 & 10 \\ 01 & 01 & 01 & 01 \end{pmatrix} \xrightarrow{\text{gadgetize}} \begin{pmatrix} 000 & 101 & 011 & 110 \\ 000 & 011 & 110 & 101 \\ 101 & 101 & 101 & 101 \\ 011 & 011 & 011 & 011 \end{pmatrix} \quad (69)$$

One can use (38) to verify that $TTT^\dagger TTT^\dagger TTT^\dagger TTT^\dagger$ on this code, i.e., transversal T with metric

$$\mathbf{\Gamma} = (1, 1, -1, 1, 1, -1, 1, 1, -1, 1, 1, -1),$$

indeed gives logical CS (up to Clifford corrections).

Doubling the above $4 \rightarrow 1$ construction gives a $6\text{CS} \rightarrow 2\text{CS}$ ($d = 2$) protocol.

$$\begin{pmatrix} 1 & \omega & \omega^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & \omega & \omega^2 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Adding the all-one stabilizer row to the second logical row, one can see that this protocol is the $k = 2$ special case of the following construction.

Construction 4.16 ($(2k + 2)\text{CS} \rightarrow k\text{CS}$ at $d = 2$). *Again inspired by [CH17, Sec. IV.E], one can design a family of distance-two protocols of asymptotic overhead 2.*

$$\begin{pmatrix} 0 & 1 & \omega & \omega^2 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 1 & 1 & \omega & \omega^2 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & \omega & \omega^2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & \cdots & \omega & \omega^2 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & \cdots & 1 & 1 \end{pmatrix} \quad (70)$$

Similar to [NH22, Lemma 7], we show in App. A.4 that 2 is asymptotically optimal for $d = 2$ CS-to-CS protocols derived from $\mathbb{F}_4$ -linear triorthogonal codes.

One can concatenate these schemes with themselves to obtain higher distance protocols. For example, concatenating the $4 \rightarrow 1$ scheme with itself leads to a $16\text{CS} \rightarrow 1\text{CS}$ protocol at $d = 4$, and more generally, one can have a $4^t\text{CS} \rightarrow 1\text{CS}$ at distance $d = 2^t$. On the other hand, the situation is more subtle when $k > 1$ protocols are involved in the concatenation. For example, one might naively think that first distilling 4CS states from the $10\text{CS} \rightarrow 4\text{CS}$ protocol, then further distilling them using the $4 \rightarrow 1$ protocol would yield a $10 \rightarrow 1$ protocol at distance 4. This is not true in general ($d = 2$ in this case, in fact), because the errors on the outputs of the $k > 1$ protocols are correlated. Protocols with $k > 1$ can, however, be used with care in concatenation. For example, a $24 \rightarrow 2$; ($d = 4$) protocol is possible by beginning with four $6 \rightarrow 2$ protocols, and then using two copies of the $4 \rightarrow 1$ protocol, where each run of the $6 \rightarrow 2$ protocol is routed to the two second-level protocols. We now list some possible schemes: some concatenated, and some not.

Construction 4.17 ($16\text{CS} \rightarrow 1\text{CS}$ at $d = 4$, more generally, $4^t\text{CS} \rightarrow 1\text{CS}$ at $d = 2^t$). *One can either concatenate the $4 \rightarrow 1$ ($d = 2$) protocol (construction 4.15) with itself, or use the two-variable Reed-Muller code over $\mathbb{F}_4$ to obtain a distance-four $16\text{CS} \rightarrow 1\text{CS}$ protocol. For the latter construction, take xy as the X logical, and $y^3, y^2, y, x, 1$ as X stabilizers. The two methods give equivalent protocols. In fact, it turns out that this is a general connection. The $4^t \rightarrow 1$ ($d = 2^t$) protocol obtained by iteratively concatenating $4 \rightarrow 1$ with itself is always equivalent to a protocol constructed using a decreasing monomial code; we provide the exact construction and proof in App. A.3.*

Construction 4.18 ($12\text{CS} \rightarrow 1\text{CS}$ at $d = 3$). *We first provide the matrix and then explain how*

we obtained it.

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & \omega^2 & 1 & \omega & \omega^2 & \omega \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & \omega & \omega^2 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & \omega & \omega^2 & 0 & 0 & 1 & \omega & \omega^2 & 1 & \omega & \omega^2 \end{pmatrix} \quad (71)$$

The X stabilizer space is obtained as follows. One first writes down the 21 points of the projective space $\mathbb{P}_{\mathbb{F}_4}^2$ as the columns. That is, we consider the 21 points of $\mathbb{P}_{\mathbb{F}_4}^2$, written canonically as

$$\{(1 : y : z) : y, z \in \mathbb{F}_4\} \cup \{(0 : 1 : z) : z \in \mathbb{F}_4\} \cup \{(0 : 0 : 1)\}, \quad (72)$$

and consider a matrix in $\mathbb{F}_4^{3 \times 21}$ whose rows are $\text{ev}(X)$, $\text{ev}(Y)$ and $\text{ev}(Z)$ on these points. Of course, given that points in the projective space are only defined up to scalar multiplication by non-zero elements of $\mathbb{F}_4$, we have only defined this matrix up to scalar multiplication of any column by a non-zero element of $\mathbb{F}_4$. It turns out that any choice works, however.^a At this point, we have a 3×21 matrix over $\mathbb{F}_4$, which is a parity-check matrix for a code of distance 3, since no two distinct points $(x : y : z)$ and $(x' : y' : z')$ in $\mathbb{P}_{\mathbb{F}_4}^2$ can be related by a scalar, by definition.

We next consider the nine points in $\mathbb{P}_{\mathbb{F}_4}^2$ lying on the Hermitian curve over $\mathbb{F}_4$ in Hermitian space, that is, the 9 points $(x : y : z)$ satisfying $Y^2Z + YZ^2 = X^3$ (we introduce this curve in detail later in Sec. 5.2). Removing the corresponding 9 columns from the matrix yields a 3×12 matrix over $\mathbb{F}_4$ whose rows are 3-orthogonal.^b

Given these stabilizer rows, we wish to add as many logical rows as possible, that is, we wish to complete the code to a maximal triorthogonal space [HH18a]. To do this, we can consider a matrix whose rows are the vectors $(g_{i,1}g_{j,1}, \dots, g_{i,n}g_{j,n})$ for every pair of (not necessarily distinct) indices (i, j) labelling two stabilizer rows. The kernel of this matrix contains vectors $(s_1, \dots, s_n)$ that can be added as logical rows, as long as they satisfy $\sum_{i=1}^n s_i^3 \neq 0$. We find that the resulting kernel has dimension 6, including the three-dimensional X -stabilizer space itself. It also turns out that one cannot add more than one logical row in this case, because adding one logical row adds in more constraints, which reduces the kernel to just the three-dimensional X -stabilizer space, that is, we have found a maximal triorthogonal space. One choice of the logical (which satisfies $\sum_{i=1}^n s_i^3 = 1$) is chosen and displayed in the resulting matrix. The protocol has distance 3 since the X -stabilizer rows are a parity-check matrix for a code of distance 3, as discussed.

^aAny choice works because Equation 67 remains true even after re-scaling a column by a non-zero element of $\mathbb{F}_4$ (since for any $\eta \in \mathbb{F}_4^\times$, we have $\eta^3 = 1$). Note that the condition for U_7 -orthogonality in $\mathbb{F}_8$ is special for the same reason.

^bWhen we say the rows of a matrix are 3-orthogonal, we mean that those rows satisfy Eq. 67 with $k = 0$. It turns out that the original 3×21 matrix is 3-orthogonal, and the 3×9 sub-matrix formed by the 9 points on the Hermitian curve is 3-orthogonal. In turn, the 3×12 matrix that we have formed by removing those 9 points is 3-orthogonal.

One may wonder what would have happened if we removed a different set of points from the original 21. For all $n < 12$, we enumerate all the possible sets of n points in the projective space and repeat the procedure. In each case, we are not able to add a logical row, and so we cannot find an $n \rightarrow 1$ ($d = 3$) CS-to-CS protocol with these methods, although this does not rule out that one exists (we constrained ourselves to three-dimensional stabilizer spaces).

The kernel method above can be applied iteratively in order to add multiple logical rows: randomly select one codeword from the appropriate kernel space whose triple self-intersection is one, build a new set of linear equations and solve for the kernel. We repeat until there is no valid logical row.

The following two constructions are found by mixing the iterative kernel search together with geometric intuitions.

Construction 4.19 ($16\text{CS} \rightarrow 2\text{CS}$ at $d = 3$). We begin by considering the regular $\mathbb{F}_4$ Reed-Muller codes in two variables; the 16 points of the affine plane $\mathbb{A}_{\mathbb{F}_4}^2$ are taken as evaluation points. The evaluations of $1, x, y$ naturally serve as stabilizers, and xy is a logical row. However, no other monomial rows can be added as a logical row. It turns out, however, that it is possible

to add a further row by considering linear combinations of monomials. Our kernel search, which searches over the $\mathbb{F}_4$ -linear combinations over the monomials (spanned by $\{x^2, y^2, x^3, y^3\}$ together with the existing rows in this case), gives many possible solutions, including the example shown, which is $\text{ev}(y^3 + \omega x^3)$. It further turns out that, no matter which extra logical row is picked, the new kernel reduces to the three-dimensional stabilizer space, i.e., we cannot add a third logical row.

$$\begin{array}{l} \text{ev}(y^3 + \omega x^3) \\ \text{ev}(xy) \\ \text{ev}(x) \\ \text{ev}(y) \\ \text{ev}(1) \end{array} \begin{pmatrix} 0 & 1 & 1 & 1 & \omega & \omega^2 & \omega^2 & \omega^2 & \omega & \omega^2 & \omega^2 & \omega^2 & \omega & \omega^2 & \omega^2 & \omega^2 \\ 0 & 0 & 0 & 0 & 0 & \omega^2 & 1 & \omega & 0 & 1 & \omega & \omega^2 & 0 & \omega & \omega^2 & 1 \\ 0 & 0 & 0 & 0 & \omega & \omega & \omega & \omega & \omega^2 & \omega^2 & \omega^2 & \omega^2 & 1 & 1 & 1 & 1 \\ 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad (73)$$

Construction 4.20 ($21\text{CS} \rightarrow 4\text{CS}$ at $d = 3$). We begin by writing down all the 21 points of $\mathbb{P}_{\mathbb{F}_4}^2$ in (72) as columns to form a set of X stabilizers. The rows can be viewed as $\text{ev}(X)$, $\text{ev}(Y)$ and $\text{ev}(Z)$.^a The kernel has dimension 15 over $\mathbb{F}_4$, and can be interpreted as the 15 degree-four (quartic) monomial evaluations. Among those, $\text{ev}(X^4) = \text{ev}(X)$, $\text{ev}(Y^4) = \text{ev}(Y)$, $\text{ev}(Z^4) = \text{ev}(Z)$ are the stabilizers. Since the projective plane can be viewed as three affine charts glued together, the previous construction can be seen as being evaluated on the $Z = 1$ chart; this naturally leads us to choose $\text{ev}(XYZ^2)$ and $\text{ev}(Z(Y^3 + \omega X^3))$ as the two logical rows to match that construction. The new kernel is spanned by $\{XZ^3, XY^3, YX^3, YZ^3\}$ besides the existing rows. We find through brute-force iterative search that we can add at most two more logical rows; the most symmetric solution is presented below:

$$\begin{array}{l} \text{ev}(X(Z^3 + \omega Y^3)) \\ \text{ev}(Y(X^3 + \omega Z^3)) \\ \text{ev}(Z(Y^3 + \omega X^3)) \\ \text{ev}(XYZ^2) \\ \text{ev}(X) \\ \text{ev}(Y) \\ \text{ev}(Z) \end{array} \begin{pmatrix} 0 & 1 & 1 & 1 & \omega & \omega^2 & \omega^2 & \omega^2 & \omega & \omega^2 & \omega^2 & \omega^2 & \omega & \omega^2 & \omega^2 & \omega^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \omega & 1 & 1 & 1 & \omega^2 & \omega & \omega & \omega & 1 & \omega^2 & \omega^2 & \omega^2 & 0 & \omega & \omega & \omega & 0 \\ 0 & \omega^2 & 1 & \omega & 0 & 1 & \omega & \omega^2 & 0 & 1 & \omega & \omega^2 & 0 & 1 & \omega & \omega^2 & 0 & \omega & \omega^2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & \omega^2 & \omega & 0 & \omega & 1 & \omega^2 & 0 & \omega^2 & \omega & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \omega & \omega & \omega & \omega & \omega^2 & \omega^2 & \omega^2 & \omega^2 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 & 0 & \omega & \omega^2 & 1 & 1 \end{pmatrix}$$

^aAs always, because we are evaluating these polynomials over the points in $\mathbb{P}_{\mathbb{F}_4}^2$, the matrix will only be well-defined up to multiplications of each column by elements in $\mathbb{F}_4^\times$, because doing so does not affect Eq. (67), since $\eta^3 = 1$ for all $\eta \in \mathbb{F}_4^\times$.

Construction 4.21 ($16\text{CS} \rightarrow 1\text{TOF}\#$ at $d = 3$). Similar to the generalized triorthogonality for binary codes, one can relax the intersection of logical rows in three-orthogonality for CS protocols. The resulting logical action is usually a hypergraph magic gate. In the present instance, we use the two-variable RM code over $\mathbb{F}_4$, taking xy, x^2, y^2 as X logicals, and $x, y, 1$ as X stabilizers. The logical action of applying physical CS gates transversally can be computed using the techniques described in App. A.1. On the binary level, it is a CS on qubits 1,2 together with an $\mathbb{F}_4$ -qudit CCZ on qubits 1 to 6 (cf. Fig. 4(a)). One can then measure the first qubit in Z basis, and upon Clifford correction, the remaining circuit on qubits 2 to 6 is Clifford-equivalent to a TOF# gate. Another way to see this is to set the first logical qubit to the zero state before applying the physical CS gates, such that all CCZ gates and CS gate involving this logical qubit have trivial effect.

Below we show the binary matrix describing the protocol for qubits. Applying the CS gate to adjacent pairs of physical qubits yields the logical gate $\text{CCZ}_{013}\text{CCZ}_{014}\text{CCZ}_{023}$, which one checks

is Clifford equivalent to $\text{TOF}\# \sim_c \text{CCZ}_{0,1,4} \text{CCZ}_{0,2,3}$ by acting with CNOT_{43} .

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \quad (74)$$

Construction 4.22 ($8\text{CS} \rightarrow 1\text{TOF}\#$ at $d = 2$). One can also obtain an $8\text{CS} \rightarrow 1\text{TOF}\#$ ($d = 2$) protocol from the Hermitian curve $y^2 + y = x^3$ over $\mathbb{F}_4$ by choosing y, x^2, x as X logicals and 1 as X stabilizer. The logical intersection pattern (on the binary level) is the same as the above RM construction. Hence, one can use the same method to obtain $\text{TOF}\#$. Again, we write down the binary matrix giving the protocol at the qubit level, where one checks that the application of the CS gate to adjacent pairs of physical qubits yields the same logical gate as in the previous construction.

$$\begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \quad (75)$$

Construction 4.23 ($8\text{CCS} \rightarrow 1\text{CCS}$ at $d = 2$). The Reed-Solomon code over $\mathbb{F}_8$ can also be used for CCS-to-CCS distillation.^a

$$\begin{pmatrix} 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad (76)$$

That is, this matrix can be binarized (using the self-dual normal basis for $\mathbb{F}_8$, see Eq. (46)), and the action of CCS on the 8 sets of 3 physical qubits yields a logical CCS gate on the three logical qubits.

To see the CCS transversality, imagine attaching the following embedded code gadget (a $k = 3$ classical simplex code) to each triplet of qubits expanding an $\mathbb{F}_8$ qudit.

$$|x_1\rangle|x_2\rangle|x_3\rangle \underbrace{|0\rangle^{\otimes 4}}_{\text{gadget qubits}} \mapsto |x_1\rangle|x_2\rangle|x_3\rangle \underbrace{|x_1 \oplus x_2\rangle|x_1 \oplus x_3\rangle|x_2 \oplus x_3\rangle|x_1 \oplus x_2 \oplus x_3\rangle}_{\text{gadget qubits}}. \quad (77)$$

Applying $\sqrt{T}\sqrt{T}\sqrt{T}\sqrt{T}^\dagger\sqrt{T}^\dagger\sqrt{T}^\dagger\sqrt{T}$ leads to a CCS gate on $|x_1\rangle|x_2\rangle|x_3\rangle$, i.e., this state gains a phase of $i^{x_1x_2x_3}$ because of the following identity.

$$4x_1x_2x_3 = (x_1 \oplus x_2 \oplus x_3) - (x_1 \oplus x_2) - (x_1 \oplus x_3) - (x_2 \oplus x_3) + x_1 + x_2 + x_3. \quad (78)$$

One may use the self-dual normal basis for $\mathbb{F}_8$ in Eq. (46) to binarize the qudit code in Equation (76) before gadgetizing it with the simplex code. Then, on the resulting binarized and gadgetized $[[56, 3, 2]]$ qubit code, one can use the framework of Sec. 4.1 (but with mod 16 cutoff rather than mod 8 to calculate the induced logical action, noting that the logical action of CCS on the binarized code is the same as the logical action of $\sqrt{T}\sqrt{T}\sqrt{T}\sqrt{T}^\dagger\sqrt{T}^\dagger\sqrt{T}^\dagger\sqrt{T}$ on the binarized and gadgetized code. It can be calculated that this logical action is $\text{CCS}_{123} \text{CS}_{12}^\dagger \text{CS}_{13}^\dagger \text{CS}_{23}^\dagger S_1 S_2 S_3$ (with no action between any stabilizer row and logical row). This logical action is Pauli equivalent to a $\text{CCS}_{123}^\dagger$ (up to a global phase):

$$(X_1 X_2 X_3) \text{CCS}_{123}^\dagger (X_1 X_2 X_3) = -i \text{CCS}_{123} \text{CS}_{12}^\dagger \text{CS}_{13}^\dagger \text{CS}_{23}^\dagger S_1 S_2 S_3.$$

^aWe have already seen that this code admits transversal CCZ, as a subcode of (45).

5 Protocols from Algebraic Geometry Codes

5.1 Introduction to algebraic curves

Here, we intend to provide an intuitive but non-rigorous introduction to algebraic curves and how they are used to define error-correcting codes. We leave the more rigorous treatment to App. B.1, where one will find the bulk of the definitions. Our presentation in this section aims to provide some motivation for those abstract definitions later.

Let $K = \mathbb{F}_{2^s}$ throughout this section, and denote by $\bar{K} = \bar{\mathbb{F}}_{2^s} = \bar{\mathbb{F}}_2$ the algebraic closure of K .²¹ Informally, a curve is an algebraic set with one degree of freedom. In this paper, the curves are usually presented by one equation $f(x, y) = 0$ in two affine variables; they are so-called *plane curves*. All the curves we consider are presented in Table 3. Later, when we say a curve χ is ‘defined by’ an equation f (or multiple equations when there are more variables), we mean χ ‘is the zero set of’ that equation(s). Points on the curve χ with all their coordinates²² in K are called *K-rational points*, or simply *rational points* when the field K is clear from context.

To give an explicit example, consider the affine plane curve

$$\mathcal{H} : y^2 + y = x^3 \quad \text{over } \mathbb{F}_4 = \{0, 1, \omega, \omega^2\}. \quad (79)$$

This is the $q = 2$ Hermitian curve (in the form $y^q + y = x^{q+1}$ over $\mathbb{F}_{q^2}$), which also happens to be an elliptic curve²³ in this case. It has eight $\mathbb{F}_4$ -rational points:

$$x = 0 : (0, 0), (0, 1), \quad (80)$$

$$x = 1 : (1, \omega), (1, \omega^2), \quad (81)$$

$$x = \omega : (\omega, \omega), (\omega, \omega^2), \quad (82)$$

$$x = \omega^2 : (\omega^2, \omega), (\omega^2, \omega^2). \quad (83)$$

These eight $\mathbb{F}_4$ -rational points (α, β) in the affine plane $\mathbb{A}_K^2 = \{(x, y) \mid x, y \in \bar{K}\}$ all satisfy $\beta^2 + \beta = \alpha^3$; we will later refer to them as $P_{\alpha, \beta}$. Later, we shorthand $\mathbb{A}_K^2$ as $\mathbb{A}^2$ for simplicity. There is one additional $\mathbb{F}_4$ -rational point at infinity, denoted Q_∞ , obtained after projective closure, which we discuss momentarily. This distinguished point Q_∞ will be the point used in the construction of a one-point code.

One can embed the affine space $\mathbb{A}^2$ into the projective space $\mathbb{P}^2 = \{(x : y : z) \neq (0 : 0 : 0)\} / (x : y : z) \sim (\lambda x : \lambda y : \lambda z)$ by identifying the point $(x, y) \in \mathbb{A}^2$ with the equivalence class of $(x : y : 1) \in \mathbb{P}^2$. If an affine plane curve is defined by $f(x, y) = 0$, then its projective closure in $\mathbb{P}^2$ is defined by the homogenization

$$f^h(x, y, z) = z^\ell \cdot f(x/z, y/z), \quad \ell = \deg f. \quad (84)$$

The above Hermitian curve example $\mathcal{H}$ has projective closure

$$\mathcal{H}' : Y^2 Z + Y Z^2 = X^3. \quad (85)$$

Projective points $(x : y : z)$ with $z = 0$ are *points at infinity*. In this example, there is exactly one such $\mathbb{F}_4$ -rational point, namely

$$Q_\infty = (0 : 1 : 0). \quad (86)$$

Together with this unique point at infinity, this gives nine $\mathbb{F}_4$ -rational points on $\mathcal{H}'$ in total.

5.1.1 Genus of a smooth plane curve

Let $\chi \subseteq \mathbb{P}^2$ be a projective plane curve defined by a homogeneous polynomial $f^h(X, Y, Z) \in K[X, Y, Z]$. A point $P = (a : b : c)$ on χ is *singular* if all three partial derivatives²⁵ of f^h vanish at P , i.e.,

²¹The algebraic closure of a field K is K extended by the roots of all polynomials with coefficients in K . That is, for any polynomial $p \in K[T]$, there is a factorization $p(T) = a(T - \alpha_1) \dots (T - \alpha_{\deg p})$ where each $\alpha_i \in \bar{K}$. By considering the definition of $\mathbb{F}_{2^s}$, one can see that the algebraic closure of $\mathbb{F}_{2^s}$ is identical to that of $\mathbb{F}_2$ for all $s \geq 1$; that is, $\bar{\mathbb{F}}_{2^s} = \bar{\mathbb{F}}_2$.

²²For $(x_1, x_2, \dots, x_m) \in \chi$, we refer to the i -th entry in the tuple as the i -th coordinate. When clear from context, we will simply refer to each coordinate by a unique variable e.g. (x, y, z) .

²³For us, an elliptic curve will be one with ‘genus’ $g = 1$; we will define the genus shortly.

²⁴Note that capitalized variables are used to emphasize the fact that they are variables in the projective space, whereas lower-case variables denote that they take values in the affine space.

²⁵The derivative of a polynomial $\sum_i a_i x^i$ is defined to be the formal derivative $\sum_i i a_i x^{i-1}$, where $i a_i$ denotes the summation of i copies of a_i . The partial derivative in x_i of a multivariate polynomial in $(x_1, \dots, x_n)$ is computed by treating it as a univariate polynomial in x_i .

$f_X^h(P) = f_Y^h(P) = f_Z^h(P) = 0$. Otherwise P is called *smooth* or *nonsingular*. The curve is *smooth* or *nonsingular* if all of its points over $\bar{K}$ are smooth. Otherwise, it is *singular*. For an affine curve defined by $f(x, y) = 0$, the same condition says that a point (a, b) is *smooth* if $f_x(a, b) \neq 0$ or $f_y(a, b) \neq 0$. As an example, one can show by explicit calculation that the Hermitian curve example in Eq. 85 is smooth.²⁶ In particular, it is smooth at the point at infinity Q_∞ . Its affine version is also smooth.

In fact, almost all of the curves we consider, listed in Table 3, and their projective versions, are smooth. There is one exception: the $y^2 + y = x^5$ curve. Despite the affine curve being smooth ($f_y = 1$), its projective closure $Y^2Z^3 + YZ^4 = X^5$ is singular at the point at infinity $(0 : 1 : 0)$. While smooth projective curves are easiest to make statement about, we will be able to treat this curve separately by noting that it has the form $y^h + \mu y = f(x)$ with $h = p^e > 1$, and satisfies the assumptions of Thm. 5.1; see [Sti08, Prop. 6.4.1].²⁷

The genus (plural: genera), which we denote by g , is an important property of a given curve, and is a non-negative integer. Forgoing a formal definition for now, the genus characterizes the geometric complexity of a curve. For example, the projective line $\mathbb{P}^1$ has genus $g = 0$, whereas elliptic curves have genus $g = 1$. It is a deep fact that, as well as characterizing its topology, the genus also controls how many rational points a curve may have. For example, the Hasse-Weil bound [Sti08, Sec. V] states that for a smooth, irreducible²⁸ projective curve, one has

$$|\#\chi(\mathbb{F}_q) - (q + 1)| \leq 2g\sqrt{q},$$

where $\#\chi(\mathbb{F}_q)$ is the number of $\mathbb{F}_q$ -rational points on χ .

The genus of a smooth projective plane curve defined by a degree d polynomial is

$$g = \frac{1}{2}(d-1)(d-2), \quad (87)$$

and a singular plane curve has a genus smaller than this value (see the Plücker formula). In particular, for projective curves of the form $y^h + \mu y = f(x)$ with $h = p^e > 1$ satisfying the assumptions in Thm. 5.1 [Sti08, Prop. 6.4.1], the genus equals

$$\frac{1}{2}(h-1)(\deg f - 1). \quad (88)$$

5.1.2 Curves over finite fields with many points

We aim to construct distillation protocols for $\mathbb{F}_q$ -qudit CCZ states using curves over $\mathbb{F}_q$. We want to have curves with many rational points over $\mathbb{F}_q$, since later we will want to puncture away points (see Prop. 5.2) to generate logical qudits while retaining good parameters. In this direction, let us consider curves with many rational points.

A curve of genus g is called $\mathbb{F}_q$ -maximal, or simply maximal, if it attains the Hasse-Weil upper bound:

$$\#\chi(\mathbb{F}_q) = q + 1 + 2g\sqrt{q}. \quad (89)$$

For genus $g = 0$, the projective line $\mathbb{P}^1$ is maximal. If $g > 0$, maximality may only be attained if q is square. A tightening of the Hasse-Weil upper bound exists, called the Serre bound [Sti08, Thm. 5.3.1]:

$$\#\chi(\mathbb{F}_q) \leq q + 1 + g \cdot \lfloor 2\sqrt{q} \rfloor \quad (90)$$

A curve attaining this bound is called a *defect-zero* curve. In particular, a maximal curve is a defect-zero curve. We will only work with defect-zero curves, and we will now move towards a classification of defect-zero curves of small genus over the field sizes of interest to us. This will lead us to the curves in Table 3. Note that the field sizes of interest to us are small powers of 2, $q = 4, 8, 16, 32, 64$.

We start by constraining the possible genera for maximal curves over $\mathbb{F}_4, \mathbb{F}_{16}$ and $\mathbb{F}_{64}$. Indeed, the following facts are true for maximal curves over $\mathbb{F}_{q^s}$.

1. Any smooth irreducible curve of genus $g > q(q-1)/2$ cannot be maximal [Iha82].²⁹

²⁶Given $F = Y^2Z + YZ^2 + X^3$, we have $f_X^h = X^2$, $f_Y^h = Z^2$, $f_Z^h = Y^2$, noting that $2 = 0$ in extension fields of $\mathbb{F}_2$, and these functions cannot all be zero in the projective plane.

²⁷One may alternatively treat this curve by considering its smooth version, that is, its smooth projective model [Wik][Ful08, Ch. 7]. This is equivalent to considering the function field of the curve; we defer the discussion on this to App. B.1.

²⁸An irreducible plane curve has a defining polynomial that is irreducible, i.e., cannot be factored into two or more non-constant polynomials.

²⁹We need not worry about non-smooth or reducible curves. In the first case, it is possible to smooth the

2. There is a unique maximal curve over $\mathbb{F}_{q^2}$ of genus $g = q(q-1)/2$ up to isomorphism, namely the Hermitian curve $y^q + y = x^{q+1}$ [RS94];
3. If $g < q(q-1)/2$ then $g \leq \lfloor (q-1)^2/4 \rfloor$ [FT96];
4. For $q = 2^s$, the second-largest genus is $q(q-2)/4$, which is achieved by the curve $\sum_{i=1}^s y^{q/2^i} = x^{q+1}$ [AT98].

It then follows that the possible genera for maximal curves in $\mathbb{F}_4$ are $\{0, 1\}$, and in $\mathbb{F}_{16}$ are $\{0, 1, 2, 6\}$. There are defect-zero curves over these fields with all these genera [VV00], and they are listed in Table 3.³⁰ For maximal curves over $\mathbb{F}_{64}$, we summarize the known constructions for certain genera in Table 4. However, for constructing distillation protocols for qubits, we restrict ourselves to genus 0 and 1, in order to keep the resulting qubit distillation protocols reasonably small. We list the corresponding curves for $g = 0$ and 1 in Table 3.

Moving onto $\mathbb{F}_8$ and $\mathbb{F}_{32}$, we are not aware of defect-zero curves other than the elliptic curves (those with $g = 1$) and the Klein quartic curve over $\mathbb{F}_8$ [VV00] (which has genus $g = 3$). The defect-zero elliptic curves that we use for these small alphabets ($s \leq 6$), as summarized in our Table 3, come from [DM87] and they happen to have binary coefficients in their defining equations.

As it will be helpful later, we comment that for the elliptic curves of each type, the number of rational points, including the point at infinity, is equal to $2^s - \omega_1^s - \omega_2^s + 1$, where the complex numbers ω_1, ω_2 are called Frobenius eigenvalues, as displayed in Table 2.

Type	Equation	ω_1, ω_2
I	$y^2 + y = x^3 + x^2 + 1$	$1 \pm i$
II	$y^2 + xy = x^3 + x^2 + x$	$\frac{1}{2}(1 \pm i\sqrt{7})$
III	$y^2 + y = x^3$	$\pm i\sqrt{2}$
IV	$y^2 + xy = x^3 + x$	$\frac{1}{2}(-1 \pm i\sqrt{7})$
V	$y^2 + y = x^3 + x^2$	$-1 \pm i$

Table 2: The five types of nonsingular elliptic curves over $\mathbb{F}_2$ [DM87], and the corresponding Frobenius eigenvalues. The number of $\mathbb{F}_{2^s}$ -rational points on the curve is $2^s - \omega_1^s - \omega_2^s + 1$ (including the point at infinity).

5.2 Introduction to one-point codes

Here, we introduce the most elementary way of constructing a (classical) algebraic geometry code from an algebraic curve: the one-point code construction. Let us provide motivation for this construction by considering how algebraic geometry codes extend Reed-Solomon codes; this intuition was how Goppa first discovered algebraic geometry codes [Gop83].

Typically, we think of a (generalized) Reed-Solomon code as defined by a set of points $\alpha \subseteq \mathbb{F}_q$, where $|\alpha| = n \leq q$, and some integer $k \leq n$. The Reed-Solomon code is then formed by taking every univariate polynomial over $\mathbb{F}_q$ of degree less than k , and evaluating this polynomial at the points α . One codeword is the evaluation of one polynomial, and it turns out that we obtain a code of length n and dimension k over $\mathbb{F}_q$. Note that for a “full-length” Reed-Solomon code, one takes $n = q$ (and $\alpha = \mathbb{F}_q$).

An initially strange, but ultimately valid way to talk about polynomials of degree less than k is to talk about the polynomials which “blow up” fewer than k times at infinity (the polynomial $x + a$ blows up once at infinity, the polynomial $x^2 + ax + b$ blow up twice at infinity, etc.). Slightly more formally, the polynomials of degree less than k are those polynomials whose *pole order* at infinity is less than k . The object that we are secretly considering is the projective line over $\mathbb{F}_q$ (which has $q+1$ $\mathbb{F}_q$ -rational points, namely $\mathbb{F}_q \cup \{Q_\infty\}$), and we are considering the set of polynomials defined on this

curve [Wik][Ful08, Ch. 7], and in the second case the curve may be decomposed into irreducible pieces.

³⁰It is unknown whether there are many than one up to isomorphism, outside of the genus $g = 0$ case (the projective line), and the case of the Hermitian curves.

curve with pole order less than k at infinity. We then form a code by evaluating these polynomials at some choice of (non-infinite points) $\alpha \subseteq \mathbb{F}_q$ on the projective line.

With this intuition, passing from Reed-Solomon codes to more general algebraic geometry codes is natural. Indeed, the one-point code construction over an algebraic curve calls for us to pick a point Q on the curve and construct codewords by considering polynomials with pole order below some prescribed value at Q . A codeword is then exactly the evaluation of one of these polynomials at some prescribed points on the curve. While the Reed-Solomon case of $g = 0$ is very transparent, passing to higher genera g allows us to construct longer codes over the same field $\mathbb{F}_q$, because raising the genus g yields curves with more points over $\mathbb{F}_q$.

There are two immediate questions. First, which point Q to pick, and second, how can one compute the pole order of a polynomial at Q ? For the first question, we take Q to be $(0 : 1 : 0)$, which can be verified to be a point on the projective closures of all the curves we present in Table 3. Since this is a point at infinity, we write Q_∞ instead of Q for this special point, like we did for the Reed-Solomon/projective line example above. For the second question, given an irreducible smooth projective curve, the pole order of a function at Q_∞ can be computed using *intersection theory*.³¹ We will elucidate the general procedure behind this calculation through examples, starting with an example of the Hermitian curve $\mathcal{H} : y^2 + y = x^3$ over $\mathbb{F}_4$. Note that all content between now and Thm. 5.1 requires the projective curve to be smooth.

Suppose χ is an affine plane curve defined by an irreducible equation $f(x, y) = 0$. Its *affine coordinate ring* is $K[\chi] = K[x, y]/(f)$ (see Sec. 2.2 for the preliminary material on ring theory). The *function field* of χ is the fraction field $K(\chi) = \text{Frac}(K[\chi])$.³² Elements of $K(\chi)$ are called *rational functions* on χ . One may view them as quotients of polynomial functions, subject to the relation $f(x, y) = 0$, where the denominator is not identically zero. For example, on the curve $y^2 + y = x^3$ and its function field, one can replace any expression containing y^2 by $y + x^3$, thus reducing the degree of y to ≤ 1 .

On the projective closure of the curve, the rational functions are quotients of homogeneous polynomials of the same degree, again subject to the equation defining the curve, and the denominator not being identically zero. A simple example of a rational function is X/Z , where $(X : Y : Z)$ are homogeneous coordinates on $\mathbb{P}^2$.

The projective plane is three affine charts³³ glued together; we define our affine curves in the $Z = 1$ affine chart. Thus, on the projective closure of the curve, the affine coordinates x and y can be viewed as the rational functions

$$x = \frac{X}{Z}, \quad y = \frac{Y}{Z}. \quad (91)$$

Take the projective closure of $\mathcal{H}$, which is $\mathcal{H}' : Y^2Z + YZ^2 = X^3$. We seek the pole orders of x and y at Q_∞ on the curve $\mathcal{H}'$. We can calculate these using intersections of $\mathcal{H}'$ with lines in $\mathbb{P}^2$. Consider the line $\mathcal{L} : X = 0$. Substituting $X = 0$ into $\mathcal{H}'$ gives $YZ(Y + Z) = 0$, so the intersection points of $\mathcal{H}'$ and $\mathcal{L}$ are $(0 : 0 : 1) = P_{0,0}$, $(0 : 1 : 1) = P_{0,1}$, $(0 : 1 : 0) = Q_\infty$, each occurring with multiplicity 1. This situation is denoted

$$\mathcal{H}' \cdot \mathcal{L} = P_{0,0} + P_{0,1} + Q_\infty, \quad (92)$$

What we have on the right-hand side is an example of *divisor notation*. For our purposes, a *divisor* is simply a bookkeeping device telling us at which points some function f has zeros and poles. Slightly more formally, the divisor of a function f , denoted (f) , is an integer linear combination of points/-places,³⁴ where the coefficient of a place is the multiplicity of its zero at the place (if positive) or minus the order of its pole (if negative). In this case, we use $\mathcal{H}' \cdot \mathcal{L}$ to denote the divisor of the function $YZ(Y + Z)$, which has three zeros, each of multiplicity one, and no poles.

Intuitively, it is clear that if functions f, g have zeros of multiplicities o_1, o_2 at a point P , then the

³¹The one curve we consider whose projective closure is not smooth is $y^2 + y = x^5$, as mentioned. However, in this case, one can use the smooth projective model of the curve [Wik][Ful08, Ch. 7], or use the places of the function field. We defer this to App. B.1.

³²Note that $K[\chi]$ is an integral domain, following from the fact that f is irreducible.

³³The notion of a chart will be familiar to those have worked with manifolds, where some patch of an unfamiliar topology like projective space may be mapped using familiar coordinates: in this case affine coordinates.

³⁴For the purposes of this section, the words point and place may be used interchangeably, since we only consider points/places of degree one, that is, rational places. For the more general notions, see App. B.1.

multiplication of the two functions fg has a zero of multiplicity $o_1 + o_2$ at P . A similar statement holds for poles and their orders, leading us to the statement $(fg) = (f) + (g)$.

Next, consider the line at infinity $\mathcal{N} : Z = 0$. Substituting $Z = 0$ into $\mathcal{H}'$ gives $X^3 = 0$, so $\mathcal{N}$ meets $\mathcal{H}'$ only at Q_∞ , with multiplicity 3. Now we have

$$\mathcal{H}' \cdot \mathcal{N} = 3Q_\infty. \quad (93)$$

Since $x = X/Z$, its divisor is obtained by subtracting the intersection divisor with $\mathcal{N} : Z = 0$ from that with $\mathcal{L} : X = 0$:

$$(x) = \left(\frac{X}{Z}\right) = (X) - (Z) = \mathcal{H}' \cdot \mathcal{L} - \mathcal{H}' \cdot \mathcal{N} = P_{0,0} + P_{0,1} - 2Q_\infty. \quad (94)$$

We have found that x has a pole of order 2 at Q_∞ .

One can do a similar calculation for (y) by considering the intersection of $\mathcal{H}'$ with $\mathcal{M} : Y = 0$, which is $\mathcal{H}' \cdot \mathcal{M} = 3P_{0,0}$. Hence,

$$(y) = \left(\frac{Y}{Z}\right) = 3P_{0,0} - 3Q_\infty, \quad (95)$$

and y has a pole of order 3 at Q_∞ .

This motivates the notion of the valuation of a function at a given place. Considering, for example, the place Q_∞ , for a nonzero rational function f , define $v_{Q_\infty}(f)$ to be its order of vanishing at Q_∞ , with poles recorded as negative values. In our example,

$$v_{Q_\infty}(x) = -2, \quad v_{Q_\infty}(y) = -3. \quad (96)$$

We also have $v_{Q_\infty}(x^i y^j) = -(2i + 3j)$.

More generally, for any point P on the curve and any nonzero rational function, the integer $v_P(f)$ measures the order of vanishing or pole of f at P . The divisor of f is

$$(f) = \sum_P v_P(f)P. \quad (97)$$

Given a divisor D , write $D \geq 0$ if all its integer coefficients are nonnegative, and write $\text{supp}(D)$ for the set of points with a nonzero coefficient in D . For our purpose, the degree $\deg(D)$ is the sum of the coefficients.³⁵

Given a divisor G , its Riemann-Roch space is the set of functions with divisor $\geq -G$.

$$\mathcal{L}(G) = \{f \in K(\chi)^\times : (f) + G \geq 0\} \cup \{0\} \quad (98)$$

One can see that the Riemann-Roch is a vector space over the field K , and its dimension over K is denoted $\ell(G) := \dim \mathcal{L}(G)$. For the one-point codes we are particularly interested in, we take $G = rQ_\infty$, so the Riemann-Roch space is

$$\mathcal{L}(rQ_\infty) = \{f \in K(\chi)^\times : v_{Q_\infty}(f) \geq -r, v_P(f) \geq 0, \forall P \neq Q_\infty\} \cup \{0\}. \quad (99)$$

This consists of all rational functions having pole order at most r at Q_∞ and *no poles* (that is, the functions are regular) away from Q_∞ . Stepping back for a moment, in our elementary Reed-Solomon example, the underlying projective curve was the projective line, and the Riemann-Roch space used to define the Reed-Solomon code of dimension k was the vector space of polynomials whose pole order at the point at infinity, Q_∞ , was less than k . In other words, we had $G = (k-1)Q_\infty$. In that case, the Riemann-Roch space was a k -dimensional vector space, as was the resulting code.

Just as in the Reed-Solomon case, the dimension of the Riemann-Roch space turns out to control the dimension of the corresponding code, and so we need tools to calculate the dimension of the Riemann-Roch space.³⁶ In full generality, this can be hard to do precisely, since for a given pole order r , there might not exist a rational function that has a pole of order r at Q_∞ , so the dimension may be smaller than one would otherwise expect. This is the situation that $\mathcal{L}(rQ_\infty) = \mathcal{L}((r-1)Q_\infty)$ for some positive integer r , in which case r is called a *gap*; proving the non-existence of these gaps is beyond the

³⁵In general, if $D = \sum_P n_P P$, then $\deg(D) = \sum_P n_P \cdot \deg(P)$, but for our purpose, all points appearing in a divisor are places of degree one, i.e., we only consider places P with $\deg(P) = 1$. The formal definition of the degree of a place can be found in App. B.1.

³⁶The dimension of the code will equal the dimension of the Riemann-Roch space when the evaluation map is injective, which we always take to be the case. The evaluation map is injective when the degree of the divisor defining the Riemann-Roch space is less than the number of points at which we evaluate the functions, that is, $\deg(G) < n$. This is the natural analogue of the familiar statement in the Reed-Solomon case, where we have that a polynomial of degree $< n$ is uniquely specified by its evaluations at n points.

scope of this introduction. For our purposes, we just utilize the following corollary of Riemann-Roch theorem [HLP98, Cor. 2.58].

Let D be a divisor on a smooth projective curve of genus g and let $\deg(D) > 2g - 2$. Then, the dimension of the corresponding Riemann-Roch space is

$$\ell(D) = \deg(D) - g + 1. \quad (100)$$

This formula may be understood through the lens of the Weierstrass gap theorem which states that the number of gaps is equal to the genus g of the curve, and all the integers $r \geq 2g$ are non-gaps. For example, consider our above $\mathcal{H}'$ case, which had genus $g = 1$. There must be $g = 1$ gaps, but all the integers greater than or equal to 2 are non-gaps, meaning that the only gap is 1 itself. This gives us our conclusion on the dimension of the Riemann-Roch space in this case:

$$l(rQ_\infty) = \begin{cases} 1 & , \text{ if } r = 0 \\ r & , \text{ if } r > 0 \end{cases}.$$

We now have the tools to calculate the dimension of certain Riemann-Roch spaces arising from one-point code constructions over curves given in Table 3. Moreover, using the intersection theory tools we have developed, we can establish the pole orders of x and y at Q_∞ , and thus argue for the existence of monomial bases of the Riemann-Roch spaces of the form $\{x^i y^j\}$, for certain combinations of (i, j) . For all curves in our Table 3 except $y^2 + y = x^5$, one can apply the above intersection theory approach to establish the pole order of x and y at Q_∞ . Indeed, to do this, we first take all the eligible monomials that could be in the Riemann-Roch space by considering their pole orders at Q_∞ . In some cases (see the example in the next paragraph for the Klein quartic) we need to further filter out the ones that have poles at some point other than Q_∞ .³⁷ To form the monomial basis for the Riemann-Roch space, we also need to reduce the monomials by the equation defining the curve (e.g. replace y^2 by $y + x^3$ in the $\mathcal{H}'$ example above, thus decreasing the degree of y below 2), thus guaranteeing linear independence between the monomials. To show that the monomials span the Riemann-Roch space, take $r = 2g$ and do dimension counting; the number of gaps must equal the genus of the curve.

To further illustrate the above procedure, consider the Klein quartic curve $x^3 y + y^3 + x = 0$ over $\mathbb{F}_8$ [HLP98, Example 2.34, 2.75, 2.76]. Let $\chi : Y^3 Z + X^3 Y + Z^3 X = 0$ be its projective closure. One can see $P = (1 : 0 : 0)$, $Q_\infty = (0 : 1 : 0)$, $R = (0 : 0 : 1)$ are on χ ; among them, P and Q_∞ are points at infinity because $z = 0$. Let $\mathcal{L}$ be the line with equation $X = 0$. Substituting $X = 0$ into χ , one is left with $Y^3 Z = 0$, and therefore $\mathcal{L}$ intersects χ in points R and Q_∞ ; the intersection divisor reads $\chi \cdot \mathcal{L} = 3R + Q_\infty$. Similarly, one can consider the intersection of $\mathcal{M} : Y = 0$ or $\mathcal{N} : Z = 0$ with χ , which give $\chi \cdot \mathcal{M} = 3P + R$ and $\chi \cdot \mathcal{N} = 3Q_\infty + P$. Therefore, $(X/Z) = 3R - P - 2Q_\infty$, $(Y/Z) = R + 2P - 3Q_\infty$, and $(x^i y^j) = -(2i + 3j)Q_\infty + (2j - i)P + (3i + j)R$. If we consider Riemann-Roch spaces defined by the place Q_∞ , i.e., a divisor $G = rQ_\infty$, the functions in $\mathcal{L}(rQ_\infty)$ should satisfy $(f) + rQ_\infty \geq 0$. For some $x^i y^j$ to be in the Riemann-Roch space, the coefficient of P in $(x^i y^j)$ must be non-negative, meaning $i \leq 2j$. Now we know that $\{x^i y^j \mid 0 \leq 2i + 3j \leq r, 0 \leq i \leq 2j\}$ are in $\mathcal{L}(rQ_\infty)$, take $r = 2g = 6$, we have four functions $1, y, xy, y^2$ with poles in Q_∞ of order 0, 3, 5, 6, respectively. Hence they are independent and since $\ell(6Q_\infty) = 6 - g + 1 = 4$, they form a basis of $\mathcal{L}(6Q_\infty)$. If $i > 3$ in $x^i y^j$, since $i \leq 2j$, we can replace $x^3 y$ by $y^3 + x$ and decrease i . Therefore, we should take $0 \leq i < 3$ for basis. For any $r \geq 6$, it is clear that we can find i, j such that $2i + 3j = r$ and $0 \leq i < 3$, $0 \leq j \leq 2j$. Therefore, $\{x^i y^j \mid 0 \leq 2i + 3j \leq r, 0 \leq i < 3, 0 \leq j \leq 2j\}$ form a basis of $\mathcal{L}(rQ_\infty)$.

One can repeat the above procedure to the two elliptic curves with a cross term xy in our Table 3, and establishes that $\{x^i y^j \mid j \in \{0, 1\}, 2i + 3j \leq r\}$ forms a basis of $\mathcal{L}(rQ_\infty)$. For the rest of the curves in our Table 3, including $y^2 + y = x^5$ (whose projective closure is not smooth), the following theorem (we only care about $p = 2$) establishes a monomial basis for them. This result is useful for actually constructing codes, because we can write down a basis for the Riemann-Roch space corresponding to the code, and therefore a basis for the code as the evaluations of those basis functions.

Theorem 5.1. [Sti08, Prop. 6.4.1] *Let K be a field of characteristic $p > 0$. Consider a function field $F = K(x, y)$ defined by $y^h + \mu y = f(x) \in K[x]$, where $h = p^e > 1$, and $\mu \in K^\times$. Assume that $\deg f =: m > 0$ is coprime to p , and that all roots of $y^h + \mu y = 0$ are in K . Then, there is a unique*

³⁷These are not allowed in the Riemann-Roch space $\mathcal{L}(rQ_\infty)$, because the definition of this Riemann-Roch space only allows its members to have poles at Q_∞ .

place at infinity, Q_∞ , of degree one.³⁸ Moreover, the elements $x^i y^j$ with $0 \leq i, 0 \leq j \leq h-1$,³⁹ $hi + mj \leq r$ form a basis of the space $\mathcal{L}(rQ_\infty)$ over K . The genus is $g = (h-1)(m-1)/2$.

We may now consider applying Theorem 5.1 to the curves we consider. In the $m > h$ case, $Q_\infty = (0 : 1 : 0)$ because the projective closure has degree m . Indeed, writing $f(x) = \sum_{i=0}^m a_i x^i$ for $a_m \neq 0$, the projective closure is $Y^h Z^{m-h} + \mu Y Z^{m-1} = a_m X^m + \sum_{i=0}^{m-1} a_i X^i Z^{m-i}$, which is clearly solved by $(0 : 1 : 0)$. The case $h = m$ is ruled out by the fact that m and p are coprime. The final case to consider is that of $h > m$. In this case, $(1 : 0 : 0)$ is the unique point at infinity.

Now, the one-point codes we are interested in take the following form. Given a smooth projective curve over K , or a curve satisfying the conditions of Thm. 5.1, take $G = rQ_\infty$ and let $D = P_1 + P_2 + \dots + P_n$ be the sum of all K -rational points on the affine curve ($\text{supp } G \cap \text{supp } D = \emptyset$). The algebraic geometry (AG) code $C_{\mathcal{L}}(D, G)$ associated with the divisors D and G is defined to be the evaluations of elements of the Riemann-Roch space

$$C_{\mathcal{L}}(D, G) := \{(f(P_1), \dots, f(P_n)) \mid f \in \mathcal{L}(G)\} \subseteq \mathbb{F}_q^n. \quad (101)$$

Let us give an explicit example again using $\mathcal{H}'$. Here D is the sum of the eight affine $\mathbb{F}_4$ -rational points listed in Eq. (80). The code $C_{\mathcal{L}}(D, 3Q_\infty)$ consists of the evaluation of $\{1, x, y\}$:

	$P_{0,0}$	$P_{0,1}$	$P_{1,\omega}$	P_{1,ω^2}	$P_{\omega,\omega}$	P_{ω,ω^2}	$P_{\omega^2,\omega}$	P_{ω^2,ω^2}
$\text{ev}(y)$	0	1	ω	ω^2	ω	ω^2	ω	ω^2
$\text{ev}(x)$	0	0	1	1	ω	ω	ω^2	ω^2
$\text{ev}(1)$	1	1	1	1	1	1	1	1

(102)

The dimension of a code $C_{\mathcal{L}}(D, G)$ is determined as follows. The evaluation map $\text{ev}_D : \mathcal{L}(G) \rightarrow K^n$ has kernel⁴⁰ $\mathcal{L}(G - D)$. Hence,

$$k = \dim C_{\mathcal{L}}(D, G) = \ell(G) - \ell(G - D). \quad (103)$$

It turns out in general that if $\deg(A) < 0$ for any divisor A , we have $\ell(A) = 0$, and therefore if $\deg(G) < n$, then $\deg(G - D) < 0$, so $\ell(G - D) = 0$, from which it follows that the evaluation map is injective. Therefore, $k = \ell(G)$.

If, moreover $\deg(G) > 2g - 2$, $\ell(G) = \deg(G) - g + 1$ and thus

$$k = \deg(G) - g + 1, \quad (2g - 2 < \deg(G) < n). \quad (104)$$

For the one-point code $G = rQ_\infty$, we have

$$k = r - g + 1, \quad (2g - 2 < r < n). \quad (105)$$

We state without proof the following distance lower bounds:

1. [Sti08, Cor. 2.2.3] $C_{\mathcal{L}}(D, G)$ has distance $d \geq n - \deg(G)$.
2. [Sti08, Thm. 2.2.7, 2.2.8] The dual code $C_{\mathcal{L}}(D, G)^\perp$ has distance $d^\perp \geq \deg(G) - (2g - 2)$.

Since the divisor of the product of two functions is the sum of the individual divisors, $(fg) = (f) + (g)$, one can see that one-point codes satisfy a multiplication property. That is, for two codewords of a one-point code $(c_1, \dots, c_n) \in C_{\mathcal{L}}(D, rQ_\infty)$ and $(c'_1, \dots, c'_n) \in C_{\mathcal{L}}(D, r'Q_\infty)$, their entrywise-product is also contained in a one-point code.

$$(c_1 c'_1, \dots, c_n c'_n) \in C_{\mathcal{L}}(D, (r + r')Q_\infty). \quad (106)$$

This is convenient for our purpose of constructing e.g., triorthogonal spaces. Puncturing such a space allows us to obtain qudit CCZ distillation protocols, as we will see shortly.

5.3 Qudit distillation protocols from punctured one-point codes

We now show how to construct distillation protocols from one-point codes by puncturing. Recall the triorthogonality conditions from Sec. 4. Our procedure will take a space satisfying one of these

³⁸For the purposes of this section, it suffices to only think about rational places, which have degree one.

³⁹It is natural to restrict the degree of y to less than h , because y^h can be replaced with $y + f(x)$.

⁴⁰A function $f \in \mathcal{L}(G)$ lies in $\ker(\text{ev}_D)$ when $f(P_i) = 0$ for every i , equivalently $v_{P_i}(f) \geq 1$ for every i . Since $\text{supp}(G) \cap \text{supp}(D) = \emptyset$, this is saying $(f) + G - D \geq 0$. Thus $\ker(\text{ev}_D) = \mathcal{L}(G - D)$.

triorthogonality conditions in some trivial way (i.e. $k = 0$) and puncturing to obtain a non-trivial basis. A map $F : \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ is called *multi-additive* if it is additive in each argument separately: $\forall j$,

$$F(x_1, \dots, x_j + x'_j, \dots, x_r) = F(x_1, \dots, x_j, \dots, x_r) + F(x_1, \dots, x'_j, \dots, x_r).$$

Let $p = \text{char } \mathbb{F}_q$. If s, t, u are powers of p , then

$$\lambda(x, y, z) = x^s y^t z^u, \quad (107)$$

is multi-additive, equivalently $\mathbb{F}_p$ -trilinear, because $(x_1 + x_2)^{p^i} = x_1^{p^i} + x_2^{p^i}$ for non-negative integers i and $x_1, x_2 \in \mathbb{F}_q$. It need not be $\mathbb{F}_q$ -linear in any argument. In the following, we will restrict λ to always be a map of this form.

For a metric $\mathbf{\Gamma} \in \mathbb{F}_q^n$, define a map $\Lambda_{\mathbf{\Gamma}} : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$,

$$\Lambda_{\mathbf{\Gamma}}(\mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3) = \sum_{i=1}^n \Gamma_i \lambda(g_{1,i}, g_{2,i}, g_{3,i}). \quad (108)$$

We call an $\mathbb{F}_q$ -linear code $C \subseteq \mathbb{F}_q^n$ $\Lambda_{\mathbf{\Gamma}}$ -orthogonal if

$$\Lambda_{\mathbf{\Gamma}}(\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c) = 0, \quad \forall \mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c \in C. \quad (109)$$

If $(g_a)_{a=1}^m$ is an $\mathbb{F}_q$ -basis of C , it suffices to check all m^3 basis triples, since

$$\Lambda_{\mathbf{\Gamma}}\left(\sum_a \alpha_a g_a, \sum_b \beta_b g_b, \sum_c \gamma_c g_c\right) = \sum_{a,b,c} \alpha_a^s \beta_b^t \gamma_c^u \Lambda_{\mathbf{\Gamma}}(g_a, g_b, g_c).$$

Thus the condition depends on the space C , rather than on a choice of generator matrix. The qudit-CCZ and the twisted-triorthogonal conditions for U_7 of Sec. 4 are instances of this construction.

It is typically easier to first construct a triorthogonal space and then puncture it to obtain a basis satisfying the triorthogonality conditions from Sec. 4.

Proposition 5.2 (Punctured triorthogonal spaces). *Let $C \subseteq \mathbb{F}_q^n$ be an $[n, m]_q$ $\mathbb{F}_q$ linear code that is $\Lambda_{\mathbf{\Gamma}}$ -triorthogonal. Let $I = \{i_1, \dots, i_k\} \subseteq [n]$, put $J = [n] \setminus I$, and assume:*

1. *the restriction of C to the columns I , denoted $C|_I$, is the full space $C|_I = \mathbb{F}_q^k$; and*
2. *puncturing on I is injective on C , equivalently no nonzero codeword of C is supported fully in I .*

After reordering the columns $[I]$ first (permute the coordinates of $\mathbf{\Gamma}$ accordingly) and changing the row basis, a generator matrix of C has the form $\tilde{G} = \left(\begin{array}{c|c} I_k & G_1 \\ \hline 0 & G_0 \end{array} \right)$. The punctured matrix $G :=$

$\left(\begin{array}{c} G_1 \\ \hline G_0 \end{array} \right) \in \mathbb{F}_q^{m \times (n-k)}$ has rank m . Define an $[[n-k, k]]_q$ CSS code by taking the X -stabilizers to be the $\text{rowspan}(G_0)$, and the X -logical representatives to be G_1 (the Z -stabilizers are $\text{rowspan}(G^\perp)$). Call $(\mathbf{g}_a)_{a=1}^m$ the rows of G , then they satisfy the following relation

$$\Lambda_{\mathbf{\Gamma}|_J}(\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c) = \begin{cases} -\Gamma_a, & 1 \leq a = b = c \leq k, \\ 0, & \text{otherwise.} \end{cases} \quad (110)$$

Let $d(C)$ and $d(C^\perp)$ be the minimum distance of C and $C^\perp$, respectively. Then the qudit X and Z distances of the CSS code satisfy

$$d_X \geq d(C) - k, \quad d_Z \geq d(C^\perp) - k. \quad (111)$$

A sufficient condition for assumption 2 is $k < d(C)$.

Proof. Assumption 1 gives the systematic form; $\text{rank}(G) = m$ follows from assumption 2, and hence $\text{rank}(G_0) = m - k$.

For $a \leq k$, row a of $\tilde{G}$ is $(\mathbf{e}_a \mid \mathbf{g}_a)$ where $\mathbf{e}_a$ is the elementary vector that is one in the a -th coordinate and zero otherwise, while rows with $a > k$ are zero on I . Eq. (110) can be established by extending $\Lambda_{\mathbf{\Gamma}|_J}$ acting on rows of G to $\Lambda_{\mathbf{\Gamma}}$ acting on rows of $\tilde{G}$ and utilizing $\Lambda_{\mathbf{\Gamma}}$ -triorthogonality of the space.

d_X is lower bounded by the distance of the code spanned by the rows of G , $d(\text{rowspan}(G))$, and puncturing k coordinates lowers the weight of a codeword by at most k , establishing $d_X \geq d(C) - k$. For d_Z , by Prop. 3.2, $d_Z \geq d((\text{rowspan}(G_0))^\perp)$. Since the dual code distance can be interpreted as the least number of columns that are linearly dependent, and the columns of I_k are linearly independent among themselves, one has $d_Z \geq d((\text{rowspan}(G_0))^\perp) \geq d(C^\perp) - k$.

One can see that a sufficient condition for assumption 2 to hold is $k < d(C)$, since a codeword of a distance d code is uniquely recoverable from erasure of $d - 1$ coordinates. $\square$

Remark 5.3. If C is Λ -orthogonal w.r.t. some $\mathbf{\Gamma}$ that contains some zero coordinates. Let $Z = \{i \mid \Gamma_i = 0\}$ and $z = |Z|$. Then C punctured at Z , denoted as $\tilde{C} = \text{Punc}_Z(C)$. $\tilde{C}$ is $\Lambda_{\text{Punc}_Z(\mathbf{\Gamma})}$ -triorthogonal. If puncturing is injective, equivalently no nonzero codeword of C is supported fully in Z , then $\dim \tilde{C} = \dim C$. A sufficient condition for this assumption is $z < d(C)$; this will hold in our one-point codes below. We also have $d(\tilde{C}) \geq d(C) - z$, $d(\tilde{C}^\perp) \geq d(C^\perp)$.

After the puncturing of size k from $\tilde{C}$ in the above proposition, we have a $[[n - z - k, k]]_q$ CSS code with $d_X \geq d(C) - z - k$, $d_Z \geq d(C^\perp) - k$. Since the lower bound of Z distance is unaffected, and Z distance is the bottleneck (for all our protocols), puncturing Z allows us to pay z less magic input resources to distillation without compromising the distance. Therefore, after constructing $\mathbf{\Gamma}$, we always delete the zero coordinates.

We will now show that the one-point codes are $\Lambda_{\mathbf{\Gamma}}$ -triorthogonal spaces for some $\mathbf{\Gamma}$ that is not all-zero.

Lemma 5.4 (One-point codes are λ -triorthogonal spaces). Let χ be a smooth projective curve of genus g over $\mathbb{F}_q$ ($\text{char } \mathbb{F}_q = p$). Let $D = P_1 + \dots + P_n$ be a sum of distinct $\mathbb{F}_q$ -rational points on χ , and let $Q \notin \text{supp}(D)$ be rational. Assume $n > g$ and put $H = (n + g - 2)Q$. There exists a nonzero $\mathbf{\Gamma} \in C_{\mathcal{L}}(D, H)^\perp$ with $\text{wt}(\mathbf{\Gamma}) \geq n - g$. In particular, $\mathbf{\Gamma}$ has at most g zero entries. For $\Lambda_{\mathbf{\Gamma}}(\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c) = \sum_{i=1}^n \Gamma_i(g_{a,i})^s(g_{b,i})^t(g_{c,i})^u$ where s, t, u are powers of $p = \text{char } \mathbb{F}_q$. If $(s + u + t)r \leq n + g - 2$, then $C_{\mathcal{L}}(D, rQ)$ is a $\Lambda_{\mathbf{\Gamma}}$ -triorthogonal space.

Proof. Since $\deg H = n + g - 2 \geq 2g - 2$, the Riemann-Roch theorem, Eq. (100), gives $\ell(H) = \deg H - g + 1 = n - 1$. Hence $\dim C_{\mathcal{L}}(D, H) \leq n - 1$ by Eq. (103), so its dual code contains a nonzero codeword $\mathbf{\Gamma}$. By [Sti08, Thm. 2.2.7, 2.2.8], the dual code $C_{\mathcal{L}}(D, H)^\perp$ has distance $\geq \deg(H) - (2g - 2) = n - g$, and thus $\text{wt}(\mathbf{\Gamma}) \geq n - g$.

By the multiplication property of one-point codes Eq. (106), and the containment relationship: $C_{\mathcal{L}}(D, r'Q) \subseteq C_{\mathcal{L}}(D, rQ)$ if $r' \leq r$. We have

$$((g_{a,1})^s(g_{b,1})^t(g_{c,1})^u, \dots, (g_{a,n})^s(g_{b,n})^t(g_{c,n})^u) \in C_{\mathcal{L}}(D, (s + t + u)rQ) \subseteq C_{\mathcal{L}}(D, H),$$

for all $\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c \in C_{\mathcal{L}}(D, rQ)$. It follows that $\mathbf{\Gamma} \in C_{\mathcal{L}}(D, H)^\perp \subseteq C_{\mathcal{L}}(D, (s + t + u)rQ)^\perp$, and therefore, $\Lambda_{\mathbf{\Gamma}}(\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c) = 0$. $\square$

We are now ready to use the defect-zero curves in Table 3 to construct qudit-CCZ distillation protocols. For all these curves, define D as sum of distinct $\mathbb{F}_q$ -rational points disjoint from $Q = (0 : 1 : 0)$. The explicit calculation of $\mathbf{\Gamma}$ for them can be found in App. B.2. As a summary of the calculations there, $\mathbf{\Gamma}$ can be taken to be all-one for all curves in Tab. 3 except for the Klein quartic and elliptic curves over $\mathbb{F}_{2^s}$, $s > 2$. For the latter, one will see later that some coordinates of $\mathbf{\Gamma}$ can be zero. By remark 5.3, we can remove those places from D .

Corollary 5.5 (punctured one-point CSS code parameters). Let χ be a smooth projective curve of genus g over $\mathbb{F}_q$. Let $D = P_1 + \dots + P_n$ be a sum of distinct $\mathbb{F}_q$ -rational points on χ , and let $Q \notin \text{supp}(D)$ be rational. Suppose $C = C_L(D, rQ)$ is $\Lambda_{\mathbf{\Gamma}}$ -triorthogonal, where $2g - 2 < r < n$. Let z be the number of zero entries of $\mathbf{\Gamma}$. If

$$k \leq r - g + 1, \quad z + k < n - r,$$

then one can delete the z zero coordinates and choose k remaining coordinates to puncture so as to obtain a CSS code $[[n - z - k, k, (d_X, d_Z)]]_q$ with

$$d_X \geq n - z - r - k, \quad d_Z \geq r - 2g + 2 - k.$$

Proof. The Riemann-Roch theorem and previous distance lower bounds give

$$\dim C = r - g + 1, \quad d(C) \geq n - r, \quad d(C^\perp) \geq r - 2g + 2.$$

The inequality $z + k < n - r$ guarantees puncturing to be injective. After deleting the zero coordinates of $\mathbf{\Gamma}$, the code still has dimension $r - g + 1$, and one can further chooses k columns to puncture. Prop. 5.2 and remark 5.3 give the result. $\square$

Field	g	$\#\chi(\mathbb{F}_q)$	Δn	Curve	Protocols (k, d)
$\mathbb{F}_4$	0	5	0	projective line	$(k, 3 - k)$
	1	9	0	$y^2 + y = x^3$	$(k, 3 - k)$
$\mathbb{F}_8$	0	9	1	projective line	$(k, 4 - k)$
	1	14	2	$y^2 + xy = x^3 + x^2 + x$	$(1, 3), (3, 2)$
	3	24	1	$y^3 + x^3y = x$	$(2, 4), (4, 3), (6, 2)$
$\mathbb{F}_{16}$	0	17	0	projective line	$(k, 7 - k)$
	1	25	1	$y^2 + y = x^3 + x^2 + 1$	$(k, 8 - k), 1 \leq k \leq 5,$ $(7, 2)$
	2	33	1	$y^2 + y = x^5$	$(k, 9 - k), 1 \leq k \leq 5,$ $(6, 4), (7, 3), (9, 2)$
	6	65	0	$y^4 + y = x^5$	$(k, 15 - k), 1 \leq k \leq 7, (k, 16 - k), k \in \{8, 9\},$ $(k, 17 - k), 10 \leq k \leq 12, (k, 18 - k), k \in \{13, 14\},$ $(15, 4), (16, 3), (18, 2)$
$\mathbb{F}_{32}$	0	33	1	projective line	$(k, 12 - k)$
	1	44	2	$y^2 + xy = x^3 + x$	$(k, 14 - k), 1 \leq k \leq 11,$ $(13, 2)$
$\mathbb{F}_{64}$	0	65	0	projective line	$(k, 23 - k)$
	1	81	0	$y^2 + y = x^3$	$(k, 27 - k), 1 \leq k \leq 25,$ $(26, 2)$

Table 3: The defect-zero curves (cf. Sec. 5.1.2) we use to define qudit CCZ distillation protocols, and the parameters of the resulting qudit CCZ distillation protocols. g denotes the genus of the curve, $\#\chi(\mathbb{F}_q)$ the number of rational points on the curve, see Sec. 5.1 for an introduction. The discussion on the transformation from a curve to a qudit CCZ distillation protocol is contained in Sections 5.2 to 5.4. For the protocols, a tuple (k, d) implies an $(\#\chi(\mathbb{F}_q) - \Delta n - k) \rightarrow k$ distillation protocol, with distance d , taking qudit CCZ states as input and output. $\Delta n = 1 + z - h$, where z is the number of zero in metric (cf. Cor. 5.5), and h is the number of natural logical row (cf. remark 5.7). For the distances, lower bounds come from Cor. 5.5, and upper bounds are found by QDistRnd [PSK22; PSK24]. For each k , we randomly puncture k columns a hundred times and pick the largest obtained distance. When the upper bound is larger than the lower bound, we brute-force certify if the upper bound holds.

For the purpose of distilling qudit-CCZ gates, take $\lambda(x, y, z) = xyz$ in Eq. (108). Lemma 5.4 applies when $3r \leq n + g - 2$. Thus one is free to choose $r_* = \lfloor \frac{n+g-2}{3} \rfloor$. Assume $n > 5g$, which is the case for defect-zero curves over binary fields. Then r_* satisfies $r_* > 2g - 2$, and Cor 5.5 applies. For any admissible k , we get a $(n - z - k) \rightarrow k$ qudit-CCZ distillation protocol with distance bounded below by $r_* - 2g + 2 - k$ (the d_X lower bound is no smaller than this).

In fact, when working with the curves in Table 3, we find that sometimes one can take a bigger r_* , especially when g becomes larger. In Lemma 5.4, one can take H such that $\dim C_{\mathcal{L}}(D, H)^\perp = 1$ and take $\mathbf{\Gamma}$ to be the nonzero codeword of $C_{\mathcal{L}}(D, H)^\perp$. We do not know how to write down a closed formula for H , because this depends on where the gaps are. One must have $\deg H \leq n + 2g - 1$ (see the next subsection for the equality case), however. This is because $C_{\mathcal{L}}(D, (n + 2g - 1)Q_\infty) = \mathbb{F}_q^n$: one has $\dim C_{\mathcal{L}}(D, G) = \ell(G) - \ell(G - D)$ and take $G = (n + 2g - 1)Q_\infty$ in this case. Since $\deg(G) > \deg(G - D) = n + 2g - 1 - n > 2g - 2$, by Riemann-Roch, one has $\ell(G) = (n + 2g - 1) - g + 1 = n + g$ and $\ell(G - D) = (2g - 1) - g + 1 = g$.

Remark 5.6. *One can also use one-point codes to distill U_7 gates. A naive way to do so is to take $\lambda(x, y, z) = x^4 y^2 z$ in Eq. (108), and this interpreting the U_7 orthogonality condition Eq. (42) as seven-orthogonality (instead of twisted orthogonality). As we commented in 4.3, this is highly inefficient, because one actually never needs to test the overlap of more than three rows. This suboptimality is the reason why we choose to not include a table of U_7 gate distillation protocols parameters.*

Nevertheless, we do want to comment on the fact that metric can make the parameters of this naive seven-orthogonality construction better. For example, by considering a generalized RS code over $\mathbb{F}_{32}$ (U_7 in this field is Clifford equivalent to $TOF\#$), the naively punctured $31TOF\# \rightarrow 1TOF\#$ ($d = 5$) protocol can be made $29TOF\# \rightarrow 1TOF\#$ ($d = 5$) using metric. One can also use metrics to obtain $15 \rightarrow 1$ ($d = 3$) and $8 \rightarrow 1$ ($d = 2$) $TOF\# \rightarrow TOF\#$ protocols.

5.4 Distillation without puncturing

We now give some exceptional distillation protocols that do not appear to be puncturings of some Λ -triorthogonal spaces. We constrain ourselves to one-point code with monomial basis in this subsection and leave a precise understanding of the most general phenomenon to further work.

We call a monomial evaluation row a *natural logical row*, if it has non-zero self-intersection and has zero $\Lambda_{\mathbf{\Gamma}}$ intersection with the other rows. An example would be $\text{ev}(x)$ in the $[[4, 1, 2]]_4$ Reed-Solomon code, where $\text{ev}(1)$ serves as the stabilizer row. More generally, let $q = 2^s$ with s even, then $(2^2 - 1)$ divides $(2^s - 1) = q$. Let $a = \frac{q-1}{3}$, then $\text{ev}(x^a)$ naturally has non-zero triple self-intersection: $\sum_{\alpha \in \mathbb{F}_q} (\alpha^{(q-1)/3})^3 = \sum_{\alpha \in \mathbb{F}_q} \alpha^{q-1} = \sum_{\alpha \in \mathbb{F}_q^\times} 1 = 1$. Take the X stabilizers to be generated by $\text{ev}(1), \text{ev}(x), \dots, \text{ev}(x^{a-1})$. This gives a $[[q, 1, (d_X = q - a, d_Z = a + 1)]]_q$ quantum Reed-Solomon code with transversal qudit-CCZ. In fact, when s is even, one can construct a $[[4, 1, (3, 2)]]_q$ generalized Reed-Solomon code with transversal qudit-CCZ: choose four distinct points $\alpha_1, \dots, \alpha_4 \in \mathbb{F}_q$, and define $\Gamma_i = \left(\prod_{j \neq i} (\alpha_i - \alpha_j) \right)^{-1}$; use $(\alpha_i)_{1 \leq i \leq 4}$, which is $\text{ev}(x)$, as the logical row and all-ones as the stabilizer row, together with the metric $(\Gamma_i)_{1 \leq i \leq 4}$.

Besides Reed-Solomon, other AG codes can demonstrate the same behavior, although our understanding of them is case-by-case.

The one-point code over the Hermitian curve $y^4 + y = x^5$ over $\mathbb{F}_{16}$ is of genus 6 and has 64 affine $\mathbb{F}_{16}$ -rational points. One also has $v_{Q_\infty}(x) = -4$ and $v_{Q_\infty}(y) = -5$, thus $\{x^i y^j \mid 0 \leq i, 0 \leq j \leq 3, 4i + 5j \leq r\}$ form a basis of $\mathcal{L}(rQ_\infty)$, cf. Thm. 5.1. For this one-point code, we have $\dim C_{\mathcal{L}}(D, 74Q_\infty)^\perp = 1$ and the only codeword (up to scalar multiplication) in $C_{\mathcal{L}}(D, 74Q_\infty)^\perp$ is all-one (cf. App. B.2.3); we also have $\dim C_{\mathcal{L}}(D, 75D_\infty) = 0$ by our comments before Remark 5.6 because $75 = n + 2g - 1$. Thus, we can take $C_{\mathcal{L}}(D, 24Q_\infty)$ to be the stabilizer space and $\mathbf{\Gamma} = \mathbf{1}$. We find that $\text{ev}(y^5)$ of pole order 25 at Q_∞ serves as a natural logical row: $\sum_{(x,y) \in D} (y^5)^3 = 1$.

Remark 5.7. *In general, for qudit-CCZ distillation, if we use $C_{\mathcal{L}}(D, r_0 Q)$ as X stabilizers, and a basis $\{\mathbf{g}_1, \dots, \mathbf{g}_h\}$ of $C_{\mathcal{L}}(D, r_1 Q) \setminus C_{\mathcal{L}}(D, r_0 Q)$ serving as natural logical rows, i.e. $\Lambda_{\mathbf{\Gamma}}(\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c) \neq 0$ if and only if $1 \leq a = b = c \leq h$. Then, after deleting the z zero coordinates of $\mathbf{\Gamma}$ and puncturing $k - h$*

columns (with the injectivity assumption), one obtains an $(n - z - (k - h)) \rightarrow k$ qudit-CCZ distillation protocol with

$$d_X \geq n - r_1 - z - (k - h), \quad d_Z \geq r_0 - 2g + 2 - (k - h).$$

A sufficient condition for such a non-zero Γ to exist is $\dim C_{\mathcal{L}}(D, (2r_0 + r_1)Q)^\perp \geq 1$.

For U_7 on Reed-Solomon codes, one can get a natural logical row when s is divisible by 3, since then $(2^3 - 1)$ divides $(2^s - 1)$. Let $a = \frac{q-1}{7}$, then $\text{ev}(x^a)$ naturally serves as a logical row and choosing the span of $\text{ev}(1), \text{ev}(x), \dots, \text{ev}(x^{a-1})$ as stabilizers gives a $[[q, 1, (q - a, a + 1)]]_q$ quantum Reed-Solomon code with transversal U_7 gate. Our $\mathbb{F}_8$ Reed-Solomon construction in 4.3 is an example, where $\text{ev}(x^1)$ is a natural logical row. However, it turns out that in this case, $\text{ev}(x^2)$ can also be added to the logical subspace. This hints at the fact that, naively following the degree (interpreting the twisted-three orthogonality as seven orthogonality) gives only suboptimal protocols for U_7 gate, and we believe this is the case for distilling gates whose orthogonality conditions involve twisting.

For this reason, we developed the forbidden hypergraph method in Con. 4.11. We impose the Ansatz of rows being evaluations of monomials for this numerical method to be tractable. The next construction based on Klein-quartic $Y^3Z + X^3Y + Z^3X = 0$ over $\mathbb{F}_8$ is another demonstration of this method:

Construction 5.8 ($24\text{CCZ} \rightarrow 4\text{CCZ}$ at $d = 3$). For the X stabilizers, we write down its columns as the 24 points in $\mathbb{P}_{\mathbb{F}_8}^2$ that lie on the Klein quartic curve $Y^3Z + X^3Y + Z^3X = 0$, which is the evaluation of X, Y, Z . Again, since the columns are not related to each other by a scalar, the dual code distance is at least three. For the X -logicals, we choose $X^1Y^2Z^5$, $X^2Y^5Z^1$, $X^5Y^1Z^2$ and $X^5Y^5Z^5$. They are obtained similar to Con. 4.11, that is to solve for a maximum independent set in a certain forbidden hypergraph. However, we put a restriction on the total degree of the monomials, that is $a + b + c$ in $X^aY^bZ^c$ to be $a + b + c \equiv 1 \pmod{7}$. This is because, normalization is required when mixing the evaluation of homogeneous polynomials (in projective space; a monomial is automatically a homogeneous polynomial) of different degree. This is because $(X : Y : Z)$ and $(\alpha X : \alpha Y : \alpha Z)$ define the same projective point. To normalize a homogeneous polynomial of total degree d at each point, one can divide the evaluation by α^d , where α is the first coordinate of the projective point that is nonzero^a. As we commented before, the twisted three-orthogonality is special for $\mathbb{F}_8$ in the sense that scaling a column of the distillation matrix by $\gamma \in \mathbb{F}_8^\times$ does not change the intersection pattern. Therefore, normalizing by the same scalar at a projective point implies that the total degree of the homogeneous polynomials should be the same modulo seven.

After committing to X, Y, Z serving as the stabilizers, we search over all monomials $X^aY^bZ^c$ where $0 \leq a, b, c \leq 6$ and $a + b + c \equiv 1 \pmod{7}$ that each individually form a valid logical with stabilizers X, Y, Z . There are 16 such candidates. Then we build a forbidden hypergraph just like in Con. 4.11 and then solve for the maximum independent set, the unique solution of which is the exponent set $(a, b, c) = \{(1, 2, 5), (2, 5, 1), (5, 1, 2), (5, 5, 5)\}$.

^aThis is not the only way of doing normalization. Say in our case, since $X + Y + Z \neq 0$ on any of the points lying on the Klein quartic, we can choose to normalize the evaluation of $X^aY^bZ^c$ by $(X + Y + Z)^d$ where $d = a + b + c$.

5.5 Protocols based on trace codes

Let $K = \mathbb{F}_{q^m}$ and $k = \mathbb{F}_q$. For a K -linear code $C \subseteq K^n$, its *subfield subcode* and *trace code* are

$$C|_{\mathbb{F}_q} := C \cap k^n, \quad \text{tr}(C)_{K/k} := \{(\text{tr}_{K/k}(c_i))_{i=1}^n \mid (c_1, \dots, c_n) \in C\} \subseteq k^n \quad (112)$$

Delsarte's theorem relates the two by duality

$$(C|_k)^\perp = \text{tr}_{K/k}(C^\perp), \quad \text{hence } \text{tr}_{K/k}(C)^\perp = C^\perp|_k. \quad (113)$$

A consequence is that, if $\dim_K C = r$, then $r \leq \dim_k \text{tr}_{K/k}(C) \leq \min\{n, mr\}$. We refer the readers to [Sti08, Ch. 9] for proofs.

Our purpose here is to obtain distillation protocols with T inputs, thus we will only consider the case of tracing down to $k = \mathbb{F}_2$.

Let $q = 2^s$. For $\mathbf{c} \in \mathbb{F}_q^n$, put $T(\mathbf{c}) = \{\text{tr}_{\mathbb{F}_q/\mathbb{F}_2}(\lambda \mathbf{c}); \lambda \in \mathbb{F}_q\} \subseteq \mathbb{F}_2^n$, i.e., trace is applied coordinate-wise. If $(\alpha_i)_{i=1}^s$ is an $\mathbb{F}_2$ -basis of $\mathbb{F}_q$, then

$$T(\mathbf{c}) = \text{span}_{\mathbb{F}_2} \{\text{tr}_{\mathbb{F}_q/\mathbb{F}_2}(\alpha_i \mathbf{c}); 1 \leq i \leq s\}.$$

Consequently, if $\mathbf{c}_1, \dots, \mathbf{c}_r$ generate an $\mathbb{F}_q$ -linear code C , then

$$\text{tr}(C) := \text{tr}(C)_{\mathbb{F}_q/\mathbb{F}_2} = \text{span}_{\mathbb{F}_2} \{\text{tr}_{\mathbb{F}_q/\mathbb{F}_2}(\alpha_i \mathbf{c}_a) \mid 1 \leq i \leq s, 1 \leq a \leq r\}.$$

These sr rows need not be independent. As an example, for $\mathbf{c} = (c_1, \dots, c_n)$, the two subspaces $T((c_1^{2^t}, \dots, c_n^{2^t}))$ and $T(\mathbf{c})$ are equivalent, because $\text{tr}(\lambda c_j^{2^t}) = \text{tr}(\lambda^{2^{s-t}} c_j)$, and $\lambda \mapsto \lambda^{2^{s-t}}$ is an automorphism on $\mathbb{F}_q$. Thus the (evaluation of) functions $f, f^2, f^4, \dots$ generate the same subspace after taking trace.

The reason we consider trace code is because a cyclic code can be obtained as trace code of Reed-Solomon code [Sti08, Prop. 9.2.4], and the punctured Reed-Muller code is a cyclic code [MS77, Ch. 13, Th. 11]⁴¹. Motivated by the $64T \rightarrow 2\text{CCZ}$ at $2944p^4$ protocol constructed using Reed-Muller code [HH18a], we next give a $64T \rightarrow 2\text{CCZ}$ protocol at $2720p^4$ based on the trace code of Hermitian code.

Example 5.9 (Trace code of the Hermitian code). *Let D be the set of affine $\mathbb{F}_{q^2}$ -rational points on the Hermitian curve $y^q + y = x^{q+1}$, then D contains q^3 points. For $x^i y^j$ with $0 \leq i \leq q^2 - 1$ and $0 \leq j \leq q - 1$, one has*

$$\sum_{P \in D} \text{ev}_P(x^i y^j) = \begin{cases} 1, & \text{if } i = q^2 - 1, j = q - 1, \\ 0, & \text{otherwise.} \end{cases} \quad (114)$$

This is because for a fixed $x \in \mathbb{F}_{q^2}$, the q solutions of $y^q + y = x^{q+1}$ form an affine translate of subfield

$y_0 + \mathbb{F}_q$. For $0 \leq j \leq q - 1$, $\sum_{u \in \mathbb{F}_q} (y_0 + u)^j = \begin{cases} 1, & j = q - 1, \\ 0, & 0 \leq j < q - 1. \end{cases}$ The sum over D therefore vanishes

unless $j = q - 1$. In that case it becomes $\sum_{x \in \mathbb{F}_{q^2}} x^i = \begin{cases} 0, & 0 \leq i < q^2 - 1, \\ 1, & i = q^2 - 1. \end{cases}$

Construction 5.10 ($64T \rightarrow 2\text{CCZ}$ at $2720p^4$). *Here we give a $64T \rightarrow 2\text{CCZ}$ protocol, for which the probability of any error is $2720p^4$. This is based on the trace code of the Hermitian one-point code derived from $y^4 + y = x^5$ over $\mathbb{F}_{16}$. This improves over the error rate of the known $64T \rightarrow 2\text{CCZ}$ protocol, which is $2944p^4$ (although both protocols have a per-state average error probability equal to $2368p^4$).*

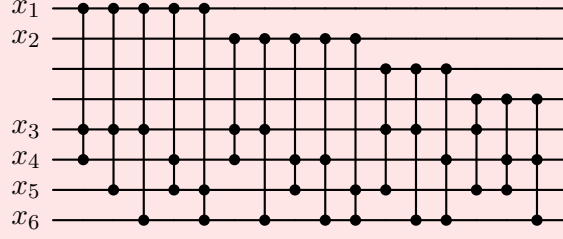
From Th. 5.1, one can see that the monomial basis for $\mathcal{L}(12Q_\infty)$ is $\{1, x, y, x^2, xy, y^2, x^3\}$. Moreover, recall that for trace codes, if row x and y are chosen, then x^2 and y^2 are redundant. Consider the trace code of $\{1, x, y, xy, x^3\}$, the only odd triple intersections occur between trace codes of row xy and x^3 ; treat them as the logical operators and $\{1, x, y\}$ as stabilizers.

Only xy and x^3 intersects non-trivially because among all the monomials $x^i y^j$ where $0 \leq i \leq 15, 0 \leq j \leq 3$, only $\sum_{P \in D} \text{ev}_P(x^{15} y^3) = 1$ and $(xy)^1 (xy)^2 (x^3)^4 = x^{15} y^3$. The exact intersection between their trace subspaces can be calculated as (for $\lambda, \mu, \nu \in \mathbb{F}_{16}$):

$$\sum_{P \in D} \text{tr}(\lambda xy(P)) \text{tr}(\mu xy(P)) \text{tr}(\nu x^3(P)) = \text{tr}((\lambda \mu^2 + \lambda^2 \mu) \nu^4).$$

Using the self-dual basis $(\alpha^3, \alpha^{12}, \alpha^7, \alpha^{13})$ for $\mathbb{F}_{16}$ defined by $\alpha^4 = \alpha + 1$ and expanding each qudit row $\mathbf{c}$ into four qubit rows as $\text{tr}(\alpha_i \mathbf{c})$, we obtain the odd triple-intersections on the eight logical generators derived from xy, x^3 is as follows.

⁴¹The binary $\text{RM}(r, m)^*$ is the trace code of $\text{ev}_{\mathbb{F}_q^\times} \text{span}_{\mathbb{F}_q} \{x^a \mid \text{wt}(a) \leq r\}$ where $q = 2^m$, which is strictly speaking not a Reed-Solomon code, but a monomial evaluation code on $\mathbb{F}_q$.



Next, we purge two logicals (third and fourth wire), and the CCZ circuit on the rest six logicals is Clifford equivalent to two decoupled CCZs; in terms of phase polynomial, it differs by only degree ≤ 2 terms to $x_1(x_3 + x_4 + x_6)(x_5 + x_4 + x_6) + x_2(x_4 + x_3 + x_5)(x_6 + x_3 + x_5)$. This can be seen by observing $x_3x_4 + x_3x_5 + x_3x_6 + x_4x_5 + x_5x_6 = (x_3 + x_4 + x_6)(x_5 + x_4 + x_6) + x_4 + x_6$, and similarly for the CCZs involving x_2 .

Construction 5.11 ($64T \rightarrow \mathbb{F}_4$ -qudit CCZ at $d = 4$). Set the first two wires to $|0\rangle$ and so that all the CCZs associated with them are removed. What is left, i.e., the last six CCZ gates on the last six logicals, form an $\mathbb{F}_4$ qudit CCZ gate (cf. 4).

Acknowledgments

The authors thank Jonathan Moussa for pointing out to us the 8CCZ-to-2CCZ distance two protocol from [Cha+22]. The authors acknowledge inspiring discussions with Shraddha Singh.

The authors acknowledge the use of large language models, in particular ChatGPT 5.6 Sol, and Claude Opus 5, to generate code for the calculation of error rates, and spacetime footprints, of concatenated protocols. The code was checked and tested by the authors, and the authors take responsibility for all content.

A.G. acknowledges funding from the Swiss State Secretariat for Education, Research and Innovation (SERI) under contract No. 20QU-1_225224. A.G. is affiliated with the Institute of Theoretical Physics of ETH Zurich. C.A.P. is currently a Simons-CIQC postdoctoral fellow at the Simons Institute for the Theory of Computing, supported by NSF QLCI Grant 2016245. A.W. acknowledges support from the MIT Department of Physics, from the MIT-IBM Watson AI Lab, and from NSG grant PHY-2325080. A.W. is affiliated with the Center for Theoretical Physics — a Leinweber Institute Massachusetts Institute of Technology, Cambridge, MA. This pre-print is assigned number MIT-CTP/6086.

References

- [AT98] Miriam Abdon and Fernando Torres. *On maximal curves in characteristic two*. 1998. arXiv: math/9811091 [math.AG] (cit. on p. 39).
- [AM19] Matthew Amy and Michele Mosca. “T-Count Optimization and Reed–Muller Codes”. In: *IEEE Transactions on Information Theory* 65.8 (Aug. 2019), pp. 4771–4784. ISSN: 1557-9654 (cit. on pp. 11, 23, 73).
- [Bab+18] Ryan Babbush, Craig Gidney, Dominic W Berry, Nathan Wiebe, Jarrod McClean, Alexandru Paler, Austin Fowler, and Hartmut Neven. “Encoding electronic spectra in quantum circuits with linear T complexity”. In: *Physical Review X* 8.4 (2018), p. 041015 (cit. on pp. 3, 10).
- [BP11] Stéphane Ballet and Julia Pielant. “On the tensor rank of multiplication in any extension of $\mathbb{F}_2$ ”. In: *Journal of Complexity* 27.2 (2011), pp. 230–245 (cit. on p. 26).

- [Bar+26] Alexander Barg, Nolan J. Coble, Dominik Hangleiter, and Christopher Kang. “Geometric Structure and Transversal Logic of Quantum Reed–Muller Codes”. In: *IEEE Transactions on Information Theory* 72.1 (Jan. 2026), pp. 415–436. ISSN: 1557-9654 (cit. on p. 25).
- [Bee07] Peter Beelen. “The order bound for general algebraic geometric codes”. In: *Finite Fields and Their Applications* 13.3 (2007), pp. 665–680. ISSN: 1071-5797 (cit. on p. 9).
- [BM18] Peter Beelen and Maria Montanucci. “A new family of maximal curves”. In: *Journal of the London Mathematical Society* 98.3 (2018), pp. 573–592 (cit. on p. 67).
- [Bev+20] Michael Beverland, Earl Campbell, Mark Howard, and Vadym Kliuchnikov. “Lower bounds on the non-Clifford resources for quantum computations”. In: *Quantum Science and Technology* 5.3 (2020), p. 035009 (cit. on pp. 5, 8, 10, 11, 78, 82).
- [BC15] Sergey Bravyi and Andrew Cross. “Doubled color codes”. In: *arXiv preprint arXiv:1509.03239* (2015) (cit. on p. 29).
- [BH12] Sergey Bravyi and Jeongwan Haah. “Magic-state distillation with low overhead”. In: *Physical Review A—Atomic, Molecular, and Optical Physics* 86.5 (2012), p. 052329 (cit. on pp. 2, 8, 14, 29, 75, 81, 82).
- [BK05] Sergey Bravyi and Alexei Kitaev. “Universal quantum computation with ideal Clifford gates and noisy ancillas”. In: *Physical Review A* 71.2 (Feb. 2005). ISSN: 1094-1622 (cit. on pp. 2, 4, 8, 14, 81).
- [BSS16] Sergey Bravyi, Graeme Smith, and John A. Smolin. “Trading Classical and Quantum Computational Resources”. In: *Physical Review X* 6.2 (June 2016). ISSN: 2160-3308 (cit. on pp. 73, 74).
- [Cai+26] Madelyn Cain, Qian Xu, Robbie King, Lewis RB Picard, Harry Levine, Manuel Endres, John Preskill, Hsin-Yuan Huang, and Dolev Bluvstein. “Shor’s algorithm is possible with as few as 10,000 reconfigurable atomic qubits”. In: *arXiv preprint arXiv:2603.28627* (2026) (cit. on p. 4).
- [CH17] Earl T Campbell and Mark Howard. “Unified framework for magic state distillation and multiqubit gate synthesis with reduced resource cost”. In: *Physical Review A* 95.2 (2017), p. 022316 (cit. on pp. 4, 8, 11, 23–25, 29, 33, 73, 81).
- [CÖ10] Murat Cenk and Ferruh Özbudak. “On multiplication in finite fields”. In: *Journal of Complexity* 26.2 (2010), pp. 172–186. ISSN: 0885-064X (cit. on pp. 17, 71).
- [Cer+25] Michael J Cervia, Henry Lamm, Diyi Liu, Edison M Murairi, and Shuchen Zhu. “Magic State Distillation using Asymptotically Good Codes on Qudits”. In: *arXiv preprint arXiv:2512.21874* (2025) (cit. on pp. 4, 9).
- [Cha+22] Christopher Chamberland, Kyungjoo Noh, Patricio Arrangoiz-Arriola, Earl T Campbell, Connor T Hann, Joseph Iverson, Harald Putterman, Thomas C Bohdanowicz, Steven T Flammia, Andrew Keller, et al. “Building a fault-tolerant quantum computer using concatenated cat codes”. In: *PRX Quantum* 3.1 (2022), p. 010329 (cit. on pp. 28, 50).
- [Chi+03] Andrew M. Childs, Richard Cleve, Enrico Deotto, Edward Farhi, Sam Gutmann, and Daniel A. Spielman. “Exponential algorithmic speedup by a quantum walk”. In: *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*. STOC03. ACM, June 2003, pp. 59–68 (cit. on pp. 3, 10).
- [CC88] David V Chudnovsky and Gregory V Chudnovsky. “Algebraic complexities and algebraic curves over finite fields”. In: *Journal of complexity* 4.4 (1988), pp. 285–316 (cit. on pp. 17, 71).
- [Coh+25] Gil Cohen, Dean Doron, Noam Goldgraber, and Tomer Manket. “Tracing ag codes: Toward meeting the gilbert-varshamov bound”. In: *arXiv preprint arXiv:2511.08788* (2025) (cit. on p. 8).

- [CKT00] Antonio Cossidente, Gabor Korchmaros, and F Torres. “Curves of large genus covered by the Hermitian curve”. In: *Communications in Algebra* 28.10 (2000), pp. 4707–4728 (cit. on p. 67).
- [CGK17] Shawn X Cui, Daniel Gottesman, and Anirudh Krishna. “Diagonal gates in the Clifford hierarchy”. In: *Physical Review A* 95.1 (2017), p. 012329 (cit. on pp. 10, 18).
- [DGM70] P. Delsarte, J.M. Goethals, and F.J. Mac Williams. “On generalized ReedMuller codes and their relatives”. In: *Information and Control* 16.5 (1970), pp. 403–442. ISSN: 0019-9958 (cit. on p. 8).
- [Dra+04] Thomas G Draper, Samuel A Kutin, Eric M Rains, and Krysta M Svore. “A logarithmic-depth quantum carry-lookahead adder”. In: *arXiv preprint quant-ph/0406142* (2004) (cit. on p. 10).
- [DM87] Yves Driencourt and Jean Francis Michon. “Elliptic codes over fields of characteristics 2”. In: *Journal of Pure and Applied Algebra* 45.1 (1987), pp. 15–39. ISSN: 0022-4049 (cit. on p. 39).
- [DK11] Iwan M Duursma and Radoslav Kirov. “Improved two-point codes on Hermitian curves”. In: *IEEE transactions on information theory* 57.7 (2011), pp. 4469–4476 (cit. on p. 9).
- [FT96] Rainer Fuhrmann and Fernando Torres. “The genus of curves over finite fields with many rational points”. In: *manuscripta mathematica* 89.1 (1996), pp. 103–106 (cit. on p. 39).
- [Ful08] William Fulton. “Algebraic curves”. In: *An Introduction to Algebraic Geom* 54 (2008), p. 76 (cit. on pp. 38–40).
- [GGS10] Arnaldo Garcia, Cem Güneri, and Henning Stichtenoth. “A generalization of the Giulietti–Korchmáros maximal curve.” In: *Advances in Geometry* 10.3 (2010) (cit. on p. 67).
- [GQ01] Arnaldo Garcia and Luciane Quoos. “A construction of curves over finite fields”. In: *Acta Arithmetica* 98 (2001), pp. 181–195 (cit. on p. 67).
- [GT09] Arnaldo Garcia and Fernando Torres. “On unramified coverings of maximal curves”. In: 21 (Jan. 2009), pp. 35–42 (cit. on p. 67).
- [GV96] Gerard van der Geer and Marcel van der Vlugt. *How to Construct Curves over Finite Fields With Many Points*. 1996. arXiv: alg-geom/9511005 [alg-geom] (cit. on p. 67).
- [GH00] O. Geil and T. Hoholdt. “Footprints or generalized Bezout’s theorem”. In: *IEEE Transactions on Information Theory* 46.2 (2000), pp. 635–641 (cit. on p. 61).
- [Gei08] Olav Geil. “Evaluation codes from an affine variety code perspective”. In: *Advances in algebraic geometry codes, Ser. Coding Theory Cryptol* 5 (2008), pp. 153–180 (cit. on p. 61).
- [Gid25] Craig Gidney. *How to factor 2048 bit RSA integers with less than a million noisy qubits*. 2025. arXiv: 2505.15917 [quant-ph] (cit. on pp. 4, 6).
- [GSJ24] Craig Gidney, Noah Shutty, and Cody Jones. “Magic state cultivation: growing T states as cheap as CNOT gates”. In: *arXiv preprint arXiv:2409.17595* (2024) (cit. on p. 4).
- [GK09] Massimo Giulietti and Gábor Korchmáros. “A new family of maximal curves over a finite field”. In: *Mathematische Annalen* 343.1 (2009), pp. 229–245 (cit. on p. 67).
- [GG25a] Louis Golowich and Venkatesan Guruswami. “Asymptotically good quantum codes with transversal non-clifford gates”. In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*. 2025, pp. 707–717 (cit. on pp. 3, 4, 14, 16, 25).
- [GG25b] Louis Golowich and Venkatesan Guruswami. “Near-Asymptotically-Good Quantum Codes with Transversal CCZ Gates and Sublinear-Weight Parity-Checks”. In: *2025 IEEE 66th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2025, pp. 1561–1569 (cit. on p. 4).
- [GL25] Louis Golowich and Ting-Chun Lin. “Quantum ldpc codes with transversal non-clifford gates via products of algebraic codes”. In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*. 2025, pp. 689–696 (cit. on p. 4).

- [Gop83] Valerii Denisovich Goppa. “Algebraico-geometric codes”. In: *Mathematics of the USSR-Izvestiya* 21.1 (1983), pp. 75–91 (cit. on p. 39).
- [Got24] Daniel Gottesman. *Surviving as a Quantum Computer in a Classical World*. 2024 (cit. on p. 14).
- [GC99] Daniel Gottesman and Isaac L Chuang. “Demonstrating the viability of universal quantum computation using teleportation and single-qubit operations”. In: *Nature* 402.6760 (1999), pp. 390–393 (cit. on p. 10).
- [Haa+17] Jeongwan Haah, Matthew B Hastings, David Poulin, and Dave Wecker. “Magic state distillation with low space overhead and optimal asymptotic input count”. In: *Quantum* 1 (2017), p. 31 (cit. on p. 4).
- [HH18a] Jeongwan Haah and Matthew B. Hastings. “Codes and Protocols for Distilling T, controlled-S, and Toffoli Gates”. In: *Quantum* 2 (June 2018), p. 71. ISSN: 2521-327X (cit. on pp. 5, 8, 25, 28, 34, 49, 77, 79, 81, 82).
- [Han01] Søren Have Hansen. “Error-Correcting Codes from Higher-Dimensional Varieties”. In: *Finite Fields and Their Applications* 7.4 (2001), pp. 530–552. ISSN: 1071-5797 (cit. on p. 9).
- [HH18b] Matthew B. Hastings and Jeongwan Haah. “Distillation with Sublogarithmic Overhead”. In: *Phys. Rev. Lett.* 120 (5 2018), p. 050504 (cit. on p. 4).
- [HMS98] M. Hattori, R.J. McEliece, and G. Solomon. “Subspace subcodes of Reed-Solomon codes”. In: *IEEE Transactions on Information Theory* 44.5 (1998), pp. 1861–1880 (cit. on p. 8).
- [He+25a] Zhiyang He, Alexander Cowtan, Dominic J. Williamson, and Theodore J. Yoder. *Extractors: QLDPC Architectures for Efficient Pauli-Based Computation*. 2025. arXiv: 2503.10390 [quant-ph] (cit. on pp. 5, 73).
- [He+25b] Zhiyang He, Vinod Vaikuntanathan, Adam Wills, and Rachel Yun Zhang. “Asymptotically good quantum codes with addressable and transversal non-Clifford gates”. In: *arXiv preprint arXiv:2507.05392* (2025) (cit. on pp. 4, 14, 26).
- [He+25c] Zhiyang He, Vinod Vaikuntanathan, Adam Wills, and Rachel Yun Zhang. “Quantum codes with addressable and transversal non-Clifford gates”. In: *arXiv preprint arXiv:2502.01864* (2025) (cit. on pp. 4, 9, 14, 18, 22, 26).
- [HLP98] T. Høholdt, J. Lint, and R. Pellikaan. “Algebraic geometry of codes, handbook of coding theory”. In: *Amsterdam* (Jan. 1998), pp. 871–961 (cit. on pp. 13, 42).
- [HC17] Mark Howard and Earl Campbell. “Application of a resource theory for magic states to fault-tolerant quantum computing”. In: *Physical review letters* 118.9 (2017), p. 090501 (cit. on pp. 8, 78, 82).
- [Iha82] Yasutaka Ihara. “Some remarks on the number of rational points of algebraic curves over finite fields”. In: *Journal of the Faculty of Science, the University of Tokyo. Sect. 1 A, Mathematics* 28 (1982), pp. 721–724 (cit. on p. 38).
- [Jac+26] Hugo Jacinto, Xavier Valcarce, Victor Barizien, Élie Gouzien, and Nicolas Sangouard. “Exploring the landscape of compact magic-state distillation factories”. In: *arXiv preprint arXiv:2606.07734* (2026) (cit. on pp. 4, 9).
- [JMB25] Abraham Jacob, Campbell McLauchlan, and Dan E Browne. “Single-shot decoding and fault-tolerant gates with trivariate tricycle codes”. In: *arXiv preprint arXiv:2508.08191* (2025) (cit. on p. 4).
- [Jon13a] Cody Jones. “Low-overhead constructions for the fault-tolerant Toffoli gate”. In: *Physical Review A—Atomic, Molecular, and Optical Physics* 87.2 (2013), p. 022328 (cit. on p. 11).
- [Jon13b] Cody Jones. “Multilevel distillation of magic states for quantum computing”. In: *Physical Review A—Atomic, Molecular, and Optical Physics* 87.4 (2013), p. 042305 (cit. on p. 4).
- [Jor+25] Stephen P Jordan, Noah Shutty, Mary Wootters, Adam Zalcman, Alexander Schmidhuber, Robbie King, Sergei V Isakov, Tanuj Khattar, and Ryan Babbush. “Optimization by decoded quantum interferometry”. In: *Nature* 646.8086 (2025), pp. 831–836 (cit. on p. 17).

- [Kha+25] Tanuj Khattar, Noah Shutty, Craig Gidney, Adam Zalcman, Nouredin Yosri, Dmitri Maslov, Ryan Babbush, and Stephen P. Jordan. *Verifiable Quantum Advantage via Optimized DQI Circuits*. 2025. arXiv: 2510.10967 [quant-ph] (cit. on p. 17).
- [Kim25] Isaac H Kim. “Catalytic z -rotations in constant T -depth”. In: *arXiv preprint arXiv:2506.15147* (2025) (cit. on p. 17).
- [Koh+26] Jin Ming Koh, Anqi Gong, Andrei C Diaconu, Daniel Bochen Tan, Alexandra A Geim, Michael J Gullans, Norman Y Yao, Mikhail D Lukin, and Shayan Majidy. “Entangling logical qubits without physical operations”. In: *arXiv preprint arXiv:2601.20927* (2026) (cit. on pp. 62, 73).
- [KT18] Anirudh Krishna and Jean-Pierre Tillich. “Towards low overhead magic state distillation”. In: *arXiv preprint arXiv:1811.08461* (2018) (cit. on p. 4).
- [Lee+21] Joonho Lee, Dominic W Berry, Craig Gidney, William J Huggins, Jarrod R McClean, Nathan Wiebe, and Ryan Babbush. *Even More Efficient Quantum Computations of Chemistry Through Tensor Hypercontraction*. *PRX Quantum* 2, 3 (jul 2021). 2021 (cit. on p. 6).
- [Lee+25] Seok-Hyung Lee, Felix Thomsen, Nicholas Fazio, Benjamin J Brown, and Stephen D Bartlett. “Low-overhead magic state distillation with color codes”. In: *PRX Quantum* 6.3 (2025), p. 030317 (cit. on p. 4).
- [LPX26] Christine Li, John Preskill, and Qian Xu. *Transversal dimension jump for product qLDPC codes*. 2026. arXiv: 2510.07269 [quant-ph] (cit. on p. 4).
- [LN96] Rudolf Lidl and Harald Niederreiter. *Finite Fields*. 2nd ed. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1996 (cit. on pp. 11, 16).
- [Lit19a] Daniel Litinski. “A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery”. In: *Quantum* 3 (Mar. 2019), p. 128. ISSN: 2521-327X (cit. on pp. 5, 7, 72–74, 78).
- [Lit19b] Daniel Litinski. “Magic State Distillation: Not as Costly as You Think”. In: *Quantum* 3 (Dec. 2019), p. 205. ISSN: 2521-327X (cit. on pp. 5, 73, 75).
- [MS77] F. Jessie MacWilliams and N. J. A. Sloane. “The Theory of Error-Correcting Codes”. In: 1977 (cit. on pp. 12, 15, 49).
- [Mat01] Gretchen L Matthews. “Weierstrass pairs and minimum distance of Goppa codes”. In: *Designs, Codes and Cryptography* 22.2 (2001), pp. 107–121 (cit. on p. 9).
- [MEK12] Adam M Meier, Bryan Eastin, and Emanuel Knill. “Magic-state distillation with the four-qubit code”. In: *arXiv preprint arXiv:1204.4221* (2012) (cit. on p. 4).
- [Men+26] Varun Menon, J. Pablo Bonilla Ataides, Rohan Mehta, Andi Gu, Daniel Bochen Tan, and Mikhail D. Lukin. “Magic Tricycles: Efficient Magic-State Generation with Finite Block-Length Quantum LDPC Codes”. In: *Physical Review X* 16.2 (Apr. 2026). ISSN: 2160-3308 (cit. on pp. 4, 9).
- [MP13] Gary L Mullen and Daniel Panario. *Handbook of finite fields*. Vol. 17. CRC press Boca Raton, 2013 (cit. on p. 16).
- [NH22] Sepehr Nezami and Jeongwan Haah. “Classification of small triorthogonal codes”. In: *Physical Review A* 106.1 (2022), p. 012437 (cit. on pp. 4, 33, 62).
- [Ngu25] Quynh T Nguyen. “Good binary quantum codes with transversal CCZ gate”. In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*. 2025, pp. 697–706 (cit. on pp. 3, 4, 14, 16, 25).
- [NP25] Quynh T Nguyen and Christopher A Pattison. “Quantum fault tolerance with constant-space and logarithmic-time overheads”. In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*. 2025, pp. 730–737 (cit. on pp. 3, 4, 14, 25).
- [PSK24] Leonid P. Pryadko, V. A. Shabashov, and V. K. Kozin. *QDistRnd: Calculate the distance of a q -ary quantum stabilizer code*. Version v0.9.5. Nov. 2024. URL: <https://github.com/QEC-pages/QDistRnd> (cit. on p. 46).

- [PSK22] Leonid P. Pryadko, Vadim A. Shabashov, and Valerii K. Kozin. “QDistRnd: A GAP package for computing the distance of quantum error-correcting codes”. In: *Journal of Open Source Software* 7.71 (Mar. 2022), p. 4120. ISSN: 2475-9066 (cit. on p. 46).
- [Ren+20] Narayanan Rengaswamy, Robert Calderbank, Michael Newman, and Henry D. Pfister. “On Optimality of CSS Codes for Transversal T”. In: *IEEE Journal on Selected Areas in Information Theory* 1.2 (2020), pp. 499–514 (cit. on p. 25).
- [RS94] Hans-Georg Rück and Henning Stichtenoth. “A characterization of Hermitian function fields over finite fields.” In: *Journal für die reine und angewandte Mathematik (Crelles Journal)* 1994 (1994), pp. 185–188 (cit. on p. 39).
- [SL80] Gadiel Seroussi and Abraham Lempel. “Factorization of Symmetric Matrices and Trace-Orthogonal Bases in Finite Fields”. In: *SIAM Journal on Computing* 9.4 (1980), pp. 758–767 (cit. on p. 15).
- [SGJ26] Shraddha Singh, Craig Gidney, and Cody Jones. “Borrowed Identities: Malleable Distillation Factories and a Unified Numerical Search”. In: *arXiv preprint arXiv:2606.28518* (2026) (cit. on pp. 4, 9).
- [Sti08] Henning Stichtenoth. *Algebraic Function Fields and Codes*. 2nd. Springer Publishing Company, Incorporated, 2008. ISBN: 3540768777 (cit. on pp. 38, 42, 43, 45, 48, 49, 64, 66, 67, 69).
- [TB26] Ryan Tiew and Nikolas P Breuckmann. “Copy-cup Gates in Tensor Products of Group Algebra Codes”. In: *arXiv preprint arXiv:2602.23307* (2026) (cit. on p. 4).
- [VV00] Gerard Van Der Geer and Marcel Van Der Vlugt. “Tables of curves with many points”. In: *Mathematics of computation* 69.230 (2000), pp. 797–810. URL: manypoints.org (cit. on p. 39).
- [WQB23] Mark A Webster, Armanda O Quintavalle, and Stephen D Bartlett. “Transversal diagonal logical operators for stabiliser codes”. In: *New Journal of Physics* 25.10 (2023), p. 103018 (cit. on pp. 25, 32).
- [Web+26] Paul Webster, Lucas Berent, Omprakash Chandra, Evan T Hockings, Nouédyn Baspin, Felix Thomsen, Samuel C Smith, and Lawrence Z Cohen. “The Pinnacle Architecture: Reducing the cost of breaking RSA-2048 to 100 000 physical qubits using quantum LDPC codes”. In: *arXiv preprint arXiv:2602.11457* (2026) (cit. on p. 4).
- [Wik] Wikipedia contributors. *Resolution of singularities — Wikipedia, The Free Encyclopedia*. [Online; accessed 3-August-2026]. URL: https://en.wikipedia.org/wiki/Resolution_of_singularities#Resolution_of_singularities_of_curves (cit. on pp. 38–40).
- [Wil26] Adam Wills. “A Review of Galois Qudits”. In: *arXiv preprint arXiv:2605.18981* (2026) (cit. on pp. 3, 4, 14–16, 74).
- [Wil+26] Adam Wills, Michael E Beverland, Lev S Bishop, Jay M Gambetta, Patrick Rall, Vikesh Siddhu, and Andrew W Cross. “Concatenating Algebraic Codes over High-Rate Quantum LDPC Codes”. In: *arXiv preprint arXiv:2605.21898* (2026) (cit. on pp. 4, 81).
- [WHY25] Adam Wills, Min-Hsiu Hsieh, and Hayata Yamasaki. “Constant-overhead magic state distillation”. In: *Nature Physics* 21.11 (2025), pp. 1842–1846 (cit. on pp. 3, 4, 8, 14, 22, 26).
- [Xu+25a] Q. Xu, H. Zhou, G. Zheng, D. Bluvstein, J. Ataiades, M. Lukin, and L. Jiang. “Fast and Parallelizable Logical Computation with Homological Product Codes”. In: *Physical Review X* 15 (2 2025), p. 021065 (cit. on pp. 5, 73).
- [Xu+25b] Qian Xu, Hengyun Zhou, Dolev Bluvstein, Madelyn Cain, Marcin Kalinowski, John Preskill, Mikhail D Lukin, and Nishad Maskara. “Batched high-rate logical operations for quantum LDPC codes”. In: *arXiv preprint arXiv:2510.06159* (2025) (cit. on p. 4).
- [Xu+26] Shifan Xu, Kun Liu, Patrick Rall, Zhiyang He, and Yongshan Ding. *Distilling Magic States in the Bicycle Architecture*. 2026. arXiv: 2602.20546 [quant-ph] (cit. on pp. 4, 7, 77, 78).

[Yod+25] Theodore J. Yoder, Eddie Schoute, Patrick Rall, Emily Pritchett, Jay M. Gambetta, Andrew W. Cross, Malcolm Carroll, and Michael E. Beverland. *Tour de gross: A modular quantum computer based on bivariate bicycle codes*. 2025. arXiv: 2506.03094 [quant-ph] (cit. on pp. 5, 6, 73, 74).

Appendix A Orthogonality condition for the CS gate

In this appendix, we focus on codes over $\mathbb{F}_4$, deriving necessary and sufficient conditions for a transversal CS to be a valid logical gate on their binarized versions. The corresponding protocols were covered in Section 4.5. To be clear, the physical gate will be the CS gate applied on the pairs of qubits forming each $\mathbb{F}_4$ -qudit.

A.1 Method 1: Embedded code

After binarization, an entry $\gamma \in \mathbb{F}_4$ appearing in the matrix defining the $\mathbb{F}_4$ code, appears in the binarized X logical/stabilizer matrix as $M(\gamma) := \begin{pmatrix} \text{tr}(\gamma\omega^2) & \text{tr}(\gamma) \\ \text{tr}(\gamma) & \text{tr}(\gamma\omega) \end{pmatrix}$. Here, recall that we are representing $\mathbb{F}_4$ as $\{0, 1, \omega, \omega^2\}$, where $\omega^2 = \omega + 1$, and $\{\omega, \omega^2\}$ forms a self-dual basis (note that $\omega^3 = 1$). Further, after the embedded code gadget described in Section 4.5, its contribution becomes

$$\begin{pmatrix} \text{tr}(\gamma\omega^2) & \text{tr}(\gamma) & \text{tr}(\gamma\omega^2) + \text{tr}(\gamma) \\ \text{tr}(\gamma) & \text{tr}(\gamma\omega) & \text{tr}(\gamma) + \text{tr}(\gamma\omega) \end{pmatrix} = \begin{pmatrix} \text{tr}(\gamma\omega^2) & \text{tr}(\gamma) & \text{tr}(\gamma\omega) \\ \text{tr}(\gamma) & \text{tr}(\gamma\omega) & \text{tr}(\gamma\omega^2) \end{pmatrix}. \quad (115)$$

To be explicit, the new X logical / stabilizer matrix becomes

$$\begin{pmatrix} \mathbf{g}_1 \\ \vdots \\ \mathbf{g}_m \end{pmatrix} = \begin{pmatrix} g_{11} & \cdots & g_{1n} \\ \vdots & & \vdots \\ g_{m1} & \cdots & g_{mn} \end{pmatrix} \rightarrow \begin{pmatrix} \text{tr}(g_{11}\omega^2) & \text{tr}(g_{11}) & \text{tr}(g_{11}\omega) & \cdots & \text{tr}(g_{1n}\omega^2) & \text{tr}(g_{1n}) & \text{tr}(g_{1n}\omega) \\ \text{tr}(g_{11}) & \text{tr}(g_{11}\omega) & \text{tr}(g_{11}\omega^2) & \cdots & \text{tr}(g_{1n}) & \text{tr}(g_{1n}\omega) & \text{tr}(g_{1n}\omega^2) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \text{tr}(g_{m1}\omega^2) & \text{tr}(g_{m1}) & \text{tr}(g_{m1}\omega) & \cdots & \text{tr}(g_{mn}\omega^2) & \text{tr}(g_{mn}) & \text{tr}(g_{mn}\omega) \\ \text{tr}(g_{m1}) & \text{tr}(g_{m1}\omega) & \text{tr}(g_{m1}\omega^2) & \cdots & \text{tr}(g_{mn}) & \text{tr}(g_{mn}\omega) & \text{tr}(g_{mn}\omega^2) \end{pmatrix} \begin{matrix} x_1 \\ y_1 \\ \vdots \\ \vdots \\ x_m \\ y_m \end{matrix},$$

where on the right-hand side, we associate the binary variables $(x_1, y_1, \dots, x_m, y_m)$ to each row for computing the triorthogonal condition.

First, note that the weight of each row is even, i.e.,

$$\sum_{i=1}^n (\text{tr}(g_{ai}\omega^2) + \text{tr}(g_{ai}) + \text{tr}(g_{ai}\omega)) \mod 2 = \sum_{i=1}^n \text{tr}(g_{ai}(\omega^2 + 1 + \omega)) \mod 2 = 0,$$

where we use the fact that $\mathbb{Z}_2$ addition is the same as $\mathbb{F}_2$ addition.

Next, let us determine the double intersections.

1. The coefficient of $x_a y_a$ modulo two is

$$\begin{aligned} \sum_{i=1}^n (\text{tr}(g_{ai}\omega^2)\text{tr}(g_{ai}) + \text{tr}(g_{ai})\text{tr}(g_{ai}\omega) + \text{tr}(g_{ai}\omega)\text{tr}(g_{ai}\omega^2)) &= \sum_{i=1}^n [(g_{ai}\omega^2 + (g_{ai})^2\omega)(g_{ai} + (g_{ai})^2) \\ &+ (g_{ai} + (g_{ai})^2)(g_{ai}\omega + (g_{ai})^2\omega^2) + (g_{ai}\omega + (g_{ai})^2\omega^2)(g_{ai}\omega^2 + (g_{ai})^2\omega)] = \sum_{i=1}^n (g_{ai})^3 \end{aligned}$$

2. The coefficient of $x_a y_b$ modulo two ($a \neq b$) is $\sum_{i=1}^n \text{tr}(g_{ai}(g_{bi})^2\omega^2)$.

For the triple intersections ($a \neq b$ or a, b, c pairwise distinct):

1. The coefficient of $x_a y_a x_b$ modulo two is $\sum_{i=1}^n \text{tr}(g_{ai}(g_{bi})^2\omega^2)$.
2. The coefficient of $x_a y_a y_b$ modulo two is $\sum_{i=1}^n \text{tr}(g_{ai}(g_{bi})^2\omega)$.
3. The coefficient of $x_a x_b x_c$ modulo two is $\sum_{i=1}^n \text{tr}(g_{ai}g_{bi}g_{ci})$.
4. The coefficient of $x_a x_b y_c$ modulo two is $\sum_{i=1}^n \text{tr}(g_{ai}g_{bi}g_{ci}\omega)$.
5. The coefficient of $x_a y_b y_c$ modulo two is $\sum_{i=1}^n \text{tr}(g_{ai}g_{bi}g_{ci}\omega^2)$.

6. The coefficient of $y_a y_b y_c$ modulo two is $\sum_{i=1}^n \text{tr}(g_{ai} g_{bi} g_{ci})$.

We know that for $1 \leq a \neq b \leq m$ and a, b not both in $\{1, \dots, k\}$, we need the coefficient of $x_a y_b$, $x_a y_a x_b$, $x_a y_a y_b$ to be even. This enforces $\sum_{i=1}^n g_{ai} (g_{bi})^2 = 0$. Similarly, for $1 \leq a, b, c \leq m$ pairwise-distinct and a, b, c not all in $\{1, \dots, k\}$, the coefficient of $x_a x_b x_c, \dots, y_a y_b y_c$ needs to be even, so $\sum_{i=1}^n g_{ai} g_{bi} g_{ci} = 0$.

The necessary and sufficient condition for the non-Clifford action induced from physical transversal CS to be only on the logical space is thus also three-orthogonality, i.e., the same as for qudit CCZ.

If we further require the logical action to be disjoint CS (up to Clifford corrections), we can more compactly write the condition as

$$\sum_{i=1}^n g_{ai} g_{bi} g_{ci} = \begin{cases} 1 & \text{if } 1 \leq a = b = c \leq k \\ 0 & \text{otherwise} \end{cases}. \quad (116)$$

A.2 Method 2: Direct calculation

We have established that $\text{CS}|u\rangle = i^{u^3 - \text{tr}(u)}|u\rangle$, $u \in \text{GF}(4)$ in the main text. Note that both u^3 (norm of u) and $\text{tr}(u)$ only take on values in $\{0, 1\}$. The part that gives the phase $i^{-\text{tr}(u)}$ is Clifford, and corresponds to $\text{CZ}_{12} S_1^\dagger S_2^\dagger$. Since we allow Clifford correction between stabilizer and logical rows, we can ignore the $i^{-\text{tr}(u)}$ part and only focus on applying transversal i^{u^3} .

Write the X logical and stabilizer rows as $\mathbf{g}_a$, $a \in [m]$, and the first k are logical rows. A classical codeword can be written as $\sum_{a=1}^m u_a \mathbf{g}_a$ with $u_a \in \text{GF}(4)$, and the phase it gained after applying transversal i^{u^3} is $i^{p(u_1, \dots, u_m)}$. The phase polynomial reads

$$p(u_1, \dots, u_m) = \sum_{i=1}^n \left(\sum_{a=1}^m u_a g_{ai} \right)^3. \quad (117)$$

The inner summation, i.e., $\sum_{a=1}^m$ is over $\text{GF}(4)$, so that anything plus itself is zero. The outer summation, i.e., $\sum_{i=1}^n$ is over $\mathbb{Z}$; this is well defined since each individual term $\in \{0, 1\}$. To avoid confusion, we will use $+$ for $\mathbb{Z}$ addition, and $\hat{+}$ for $\text{GF}(4)$ addition (or $\oplus$ if the two summands are $\{0, 1\}$). Unless stated otherwise, the summation symbol $\sum$ will refer to regular addition over $\mathbb{Z}$.

Let us first prove a lemma:

Lemma A.1. For $a_1, \dots, a_m \in \text{GF}(4)$, then

$$(a_1 \hat{+} \dots \hat{+} a_m)^3 \equiv \sum_{i=1}^m a_i^3 + 2 \sum_{1 \leq i < j \leq m} a_i^3 a_j^3 + \sum_{1 \leq i < j \leq m} \text{tr}(a_i^2 a_j) + 2 \sum_{1 \leq i < j < k \leq m} \text{tr}(a_i a_j a_k) \pmod{4}.$$

Proof. We prove this lemma by induction, and we will be using the following identity to switch between addition in $\text{GF}(4)$ and $\mathbb{Z}_4$ (Eq. (37) modulo 4):

$$\bigoplus_{i=1}^{m'} b_i \equiv \sum_{i=1}^{m'} b_i + 2 \sum_{1 \leq i < j \leq m'} b_i b_j \pmod{4}, \quad (118)$$

where $b_1, \dots, b_{m'} \in \{0, 1\}$.

We show the base case for $m = 2$:

$$\begin{aligned} (a_1 \hat{+} a_2)^3 &= a_1^3 \hat{+} a_2^3 \hat{+} \text{tr}(a_1^2 a_2) \\ &\equiv a_1^3 + a_2^3 + \text{tr}(a_1^2 a_2) + 2a_1^3 a_2^3 + 2a_1^3 \text{tr}(a_1^2 a_2) + 2a_2^3 \text{tr}(a_1^2 a_2) \pmod{4} \\ &\equiv a_1^3 + a_2^3 + 2a_1^3 a_2^3 + \text{tr}(a_1^2 a_2) \pmod{4}, \end{aligned}$$

where in the last step, using $a_i^4 = a_i$, we have $a_1^3 \text{tr}(a_1^2 a_2) = a_1^3 (a_1^2 a_2 + a_1 a_2^2) = \text{tr}(a_1^2 a_2)$; similarly, $a_2^3 \text{tr}(a_1^2 a_2) = \text{tr}(a_1^2 a_2)$.

Assuming the formula is true for m , let us prove it for $m + 1$. Call $A_m := a_1 \hat{+} \dots \hat{+} a_m$ and $A_{m+1} := A_m \hat{+} a_{m+1}$.

$$\underbrace{(a_1 \hat{+} \dots \hat{+} a_m)}_{A_m} \hat{+} a_{m+1})^3 \equiv A_m^3 + a_{m+1}^3 + 2A_m^3 a_{m+1}^3 + \text{tr}(A_m a_{m+1}^2) \pmod{4},$$

where we used $\text{tr}(a^2 b) = \text{tr}(ab^2)$. Now, what we want to show is

$$A_{m+1}^3 \equiv \sum_{i=1}^{m+1} a_i^3 + 2 \sum_{1 \leq i < j \leq m+1} a_i^3 a_j^3 + \sum_{1 \leq i < j \leq m+1} \text{tr}(a_i^2 a_j) + 2 \sum_{1 \leq i < j < k \leq m+1} \text{tr}(a_i a_j a_k) \pmod{4}$$

but this is equivalent to

$$A_{m+1}^3 \equiv A_m^3 + a_{m+1}^3 + 2 \sum_{i=1}^m a_i^3 a_{m+1}^3 + \sum_{i=1}^m \text{tr}(a_i^2 a_{m+1}) + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i a_j a_{m+1}) \pmod{4}$$

It thus suffices to show that

$$2A_m^3 a_{m+1}^3 + \text{tr}(A_m a_{m+1}^2) \equiv 2 \sum_{i=1}^m a_i^3 a_{m+1}^3 + \sum_{i=1}^m \text{tr}(a_i^2 a_{m+1}) + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i a_j a_{m+1}) \pmod{4}$$

Indeed,

$$\begin{aligned} \text{LHS} &\equiv 2 \left(\sum_{i=1}^m a_i^3 \right) a_{m+1}^3 + 2 \left(\sum_{1 \leq i < j \leq m} \text{tr}(a_i^2 a_j) \right) a_{m+1}^3 + (\text{tr}(a_1 a_{m+1}^2) + \dots + \text{tr}(a_m a_{m+1}^2)) \pmod{4} \\ &\equiv 2 \sum_{i=1}^m a_i^3 a_{m+1}^3 + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i^2 a_j) a_{m+1}^3 + \sum_{i=1}^m \text{tr}(a_i a_{m+1}^2) + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i a_{m+1}^2) \text{tr}(a_j a_{m+1}^2) \pmod{4} \end{aligned}$$

We then use

$$\begin{aligned} \text{tr}(a_i a_{m+1}^2) \text{tr}(a_j a_{m+1}^2) &= (a_i a_{m+1}^2 + \hat{a}_i^2 a_{m+1})(a_j a_{m+1}^2 + \hat{a}_j^2 a_{m+1}) \\ &= a_i a_j a_{m+1} + \hat{a}_i a_j^2 a_{m+1}^3 + \hat{a}_i^2 a_j a_{m+1}^3 + \hat{a}_i^2 a_j^2 a_{m+1}^2 \\ &= \text{tr}(a_i a_j a_{m+1}) + \text{tr}(\hat{a}_i a_j^2) a_{m+1}^3 \\ &\equiv \text{tr}(a_i a_j a_{m+1}) + \text{tr}(a_i a_j^2) a_{m+1}^3 + 2 \text{tr}(a_i a_j a_{m+1}) \text{tr}(a_i a_j^2) a_{m+1}^3 \pmod{4} \end{aligned}$$

Substituting this in, we may conclude:

$$\begin{aligned} \text{LHS} &\equiv 2 \sum_{i=1}^m a_i^3 a_{m+1}^3 + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i^2 a_j) a_{m+1}^3 + \sum_{i=1}^m \text{tr}(a_i a_{m+1}^2) \\ &\quad + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i a_j a_{m+1}) + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i^2 a_j) a_{m+1}^3 \pmod{4} \\ &\equiv 2 \sum_{i=1}^m a_i^3 a_{m+1}^3 + \sum_{i=1}^m \text{tr}(a_i a_{m+1}^2) + 2 \sum_{1 \leq i < j \leq m} \text{tr}(a_i a_j a_{m+1}) \pmod{4}, \end{aligned}$$

as required. $\square$

With this lemma, we can continue calculating the phase polynomial:

$$p(u_1, \dots, u_m) = \sum_{i=1}^n \left(\sum_{a=1}^m u_a g_{ai} \right)^3 \quad (119)$$

$$\equiv \sum_{i=1}^n \left[\sum_{a=1}^m u_a^3 (g_{ai})^3 + 2 \sum_{1 \leq a < b \leq m} u_a^3 u_b^3 (g_{ai})^3 (g_{bi})^3 + \sum_{1 \leq a < b \leq m} \text{tr}(u_a^2 u_b (g_{ai})^2 g_{bi}) \right] \quad (120)$$

$$+ 2 \sum_{1 \leq a < b < c \leq m} \text{tr}(u_a u_b u_c g_{ai} g_{bi} g_{ci}) \pmod{4} \quad (121)$$

$$\equiv \sum_{a=1}^m u_a^3 \left[\sum_{i=1}^n (g_{ai})^3 \right] + 2 \sum_{1 \leq a < b \leq m} u_a^3 u_b^3 \left[\sum_{i=1}^n (g_{ai})^3 (g_{bi})^3 \right] \quad (122)$$

$$\sum_{1 \leq a < b \leq m} \sum_{i=1}^n \text{tr}(u_a^2 u_b (g_{ai})^2 g_{bi}) + 2 \sum_{1 \leq a < b < c \leq m} \sum_{i=1}^n \text{tr}(u_a u_b u_c g_{ai} g_{bi} g_{ci}) \pmod{4}$$

Notice that all additions here were over the integers. Next, to derive the triple intersection conditions over $\text{GF}(4)$, we need to turn the summation into $\text{GF}(4)$ addition given fixed a, b, c . We can again use

Eq. (118) for this purpose.

$$\begin{aligned}
& 2 \sum_{1 \leq a < b < c \leq m} \sum_{i=1}^n \text{tr}(u_a u_b u_c g_{ai} g_{bi} g_{ci}) \\
& \equiv 2 \sum_{1 \leq a < b < c \leq m} [\text{tr}(u_a u_b u_c g_{a1} g_{b1} g_{c1}) \hat{+} \cdots \hat{+} \text{tr}(u_a u_b u_c g_{an} g_{bn} g_{cn})] \pmod{4} \\
& \equiv 2 \sum_{1 \leq a < b < c \leq m} \text{tr}(u_a u_b u_c [g_{a1} g_{b1} g_{c1} \hat{+} \cdots \hat{+} g_{an} g_{bn} g_{cn}]) \pmod{4}
\end{aligned}$$

i to the power of $2 \text{tr}(u_a u_b u_c \gamma)$ is non-Clifford if $\gamma \neq 0$. In that case, binarizing u_a, u_b, u_c leads to CCZ gates across the three pairs of logical qubits.

The term with u_a^3 is a CS gate on the pair of qubits constituting the qudit, so instead of $\sum_{i=1}^n (g_{ai})^3 \pmod{4}$, we only need $\sum_{i=1}^n (g_{ai})^3 \pmod{2} = \sum_{i=1}^n (g_{ai})^3$ to tell us the non-Clifford logical action. The middle two terms in Eq. (122) that involves u_a, u_b require more work:

$$\begin{aligned}
& 2 \sum_{1 \leq a < b \leq m} u_a^3 u_b^3 \left[\sum_{i=1}^n (g_{ai})^3 (g_{bi})^3 \right] \equiv 2 \sum_{1 \leq a < b \leq m} u_a^3 u_b^3 [(g_{a1})^3 (g_{b1})^3 \hat{+} \cdots \hat{+} (g_{an})^3 (g_{bn})^3] \pmod{4} \\
& \sum_{1 \leq a < b \leq m} \sum_{i=1}^n \text{tr}(u_a^2 u_b (g_{ai})^2 g_{bi}) \equiv \sum_{1 \leq a < b \leq m} \left[\text{tr}(u_a^2 u_b [(g_{a1})^2 g_{b1} \hat{+} \cdots \hat{+} (g_{an})^2 g_{bn}]) \right. \\
& \quad \left. + 2 \sum_{1 \leq i < j \leq n} \text{tr}(u_a^2 u_b (g_{ai})^2 g_{bi}) \text{tr}(u_a^2 u_b (g_{aj})^2 g_{bj}) \right] \pmod{4} \\
& \equiv \sum_{1 \leq a < b \leq m} \left[\text{tr}(u_a^2 u_b [(g_{a1})^2 g_{b1} \hat{+} \cdots \hat{+} (g_{an})^2 g_{bn}]) \right. \\
& \quad \left. + 2 \sum_{1 \leq i < j \leq n} [\text{tr}(u_a^2 u_b g_{ai} g_{aj} (g_{bi} g_{bj})^2) + \text{tr}(g_{bi} g_{aj} (g_{ai} g_{bj})^2) u_a^3 u_b^3] \right] \pmod{4},
\end{aligned}$$

where we used Eq. (118) in the first step, and in the second we used

$$\begin{aligned}
& 2 \sum_{1 \leq i < j \leq n} \text{tr}(u_a^2 u_b (g_{ai})^2 g_{bi}) \text{tr}(u_a^2 u_b (g_{aj})^2 g_{bj}) \\
& \equiv 2 \sum_{1 \leq i < j \leq n} [\text{tr}(u_a^2 u_b g_{ai} g_{aj} (g_{bi} g_{bj})^2) \hat{+} \text{tr}(g_{bi} g_{aj} (g_{ai} g_{bj})^2) u_a^3 u_b^3] \pmod{4} \\
& \equiv 2 \sum_{1 \leq i < j \leq n} [\text{tr}(u_a^2 u_b g_{ai} g_{aj} (g_{bi} g_{bj})^2) + \text{tr}(g_{bi} g_{aj} (g_{ai} g_{bj})^2) u_a^3 u_b^3] \pmod{4}
\end{aligned}$$

Combining terms containing $u_a^3 u_b^3$ we get

$$\begin{aligned}
& 2 \left[u_a^3 u_b^3 \sum_{1 \leq i \leq n} (g_{ai})^3 (g_{bi})^3 + \sum_{1 \leq i < j \leq n} \text{tr}(g_{bi} g_{aj} (g_{ai} g_{bj})^2) u_a^3 u_b^3 \right] \\
& = 2 \left[u_a^3 u_b^3 \sum_{1 \leq i \leq n} (g_{ai})^3 (g_{bi})^3 \hat{+} \sum_{1 \leq i < j \leq n} \text{tr}(g_{bi} g_{aj} (g_{ai} g_{bj})^2) u_a^3 u_b^3 \right] \\
& = 2 u_a^3 u_b^3 \left[\sum_{1 \leq i \leq n} (g_{ai})^2 g_{bi} \right]^3
\end{aligned}$$

Merging terms containing $u_a^2 u_b$ leads to

$$\sum_{1 \leq a < b \leq m} \text{tr}(u_a^2 u_b \left[\sum_{1 \leq i \leq n} (g_{ai})^2 g_{bi} \right]) + 2 \text{tr}(u_a^2 u_b \left[\sum_{1 \leq i < j \leq n} g_{ai} g_{aj} (g_{bi} g_{bj})^2 \right]) \pmod{4}$$

The second term is Clifford, as will be seen next when we expand $\text{tr}(u_a^2 u_b \gamma) \pmod{4}$.

Call $\gamma := \sum_{1 \leq i \leq n} (g_{ai})^2 g_{bi}$; we show

$$2u_a^3 u_b^3 \gamma^3 + \text{tr}(u_a^2 u_b \gamma) \pmod{4} \quad (123)$$

is non-Clifford (in the third-level) if $\gamma \neq 0$. Let $u_a = a_1\omega + a_2\omega^2$, $u_b = b_1\omega + b_2\omega^2$ where $a_1, a_2, b_1, b_2 \in \{0, 1\}$.

$$\begin{aligned} 2u_a^3 u_b^3 \gamma^3 &\equiv 2(a_1 + a_2 - a_1 a_2)(b_1 + b_2 - b_1 b_2) \gamma^3 \pmod{4} \\ &\equiv 2(a_1 + a_2)(b_1 + b_2) \gamma^3 - 2(a_1 b_1 b_2 + a_2 b_1 b_2 + a_1 a_2 b_1 + a_1 a_2 b_2) \gamma^3 + 2a_1 a_2 b_1 b_2 \gamma^3 \pmod{4} \end{aligned}$$

$$\begin{aligned} &\text{tr}(u_a^2 u_b \gamma) \pmod{4} \\ &\equiv (a_1 \omega^2 + a_2 \omega)(b_1 \omega + b_2 \omega^2) \gamma + (a_1 \omega + a_2 \omega^2)(b_1 \omega^2 + b_2 \omega) \gamma^2 \pmod{4} \\ &\equiv a_1 b_1 \text{tr}(\gamma) + a_1 b_2 \text{tr}(\omega \gamma) + a_2 b_1 \text{tr}(\omega^2 \gamma) + a_2 b_2 \text{tr}(\gamma) \pmod{4} \\ &\equiv a_1 b_1 \text{tr}(\gamma) + a_1 b_2 \text{tr}(\omega \gamma) + a_2 b_1 \text{tr}(\omega^2 \gamma) + a_2 b_2 \text{tr}(\gamma) + 2a_1 a_2 b_1 b_2 \underbrace{(\text{tr}(\gamma) \text{tr}(\gamma) + \text{tr}(\omega \gamma) \text{tr}(\omega^2 \gamma))}_{\gamma^3} \end{aligned}$$

$$+ 2a_1 b_1 b_2 \text{tr}(\gamma) \text{tr}(\omega \gamma) + 2a_1 a_2 b_1 \text{tr}(\gamma) \text{tr}(\omega^2 \gamma) + 2a_1 a_2 b_2 \text{tr}(\omega \gamma) \text{tr}(\gamma) + 2a_2 b_1 b_2 \text{tr}(\omega^2 \gamma) \text{tr}(\gamma) \pmod{4}$$

Therefore, the potential CCZ part of Eq. (123) is

$$\begin{aligned} &2a_1 b_1 b_2 [\text{tr}(\gamma) \text{tr}(\omega \gamma) - \gamma^3] + 2a_1 a_2 b_1 [\text{tr}(\gamma) \text{tr}(\omega^2 \gamma) - \gamma^3] + \\ &2a_1 a_2 b_2 [\text{tr}(\omega \gamma) \text{tr}(\gamma) - \gamma^3] + 2a_2 b_1 b_2 [\text{tr}(\omega^2 \gamma) \text{tr}(\gamma) - \gamma^3] \pmod{4} \end{aligned}$$

For this part to disappear, we must have $\text{tr}(\gamma) \text{tr}(\omega \gamma) = \gamma^3 = \text{tr}(\gamma) \text{tr}(\omega^2 \gamma)$, which holds only when $\gamma = 0$.

A.3 Concatenated $[[4, 1, 2]]_4$ and decreasing monomial code

We show that the m -fold concatenation of the $[[4, 1, 2]]_4$ code (Eq. (68)) can be interpreted as a monomial code, where the evaluation is over the affine space $\mathbb{F}_4^m = \{(x_1, x_2, \dots, x_m) \mid x_1, \dots, x_m \in \mathbb{F}_4\}$. The X logical of the code is (the evaluation of) $x_1 x_2 \dots x_m$, and the Z logical is $x_1^2 x_2^2 \dots x_m^2$. The X (resp. Z) stabilizers are (the evaluation of) all the monomials that are below the X (resp. Z) logical $x_1 x_2 \dots x_m$ (resp. $x_1^2 x_2^2 \dots x_m^2$) in the lexicographic order $x_m \succ x_{m-1} \succ \dots \succ x_1$: write $x^\alpha := x_1^{\alpha_1} \dots x_m^{\alpha_m}$, where $\alpha = (\alpha_1, \dots, \alpha_m)$ and $\alpha_i \in \mathbb{Z}_{\geq 0}$, then $x^\alpha \succ x^\beta$ if the right-most non-zero entry in the difference $\alpha - \beta \in \mathbb{Z}^m$ is positive⁴². This is to say, the concatenated $[[4, 1, 2]]$ code can be interpreted as a decreasing monomial code.

One can easily verify the commutation relationship by noticing that the overlap of the two evaluation vectors $\text{ev}(x^\alpha)$ and $\text{ev}(x^\beta)$, where $\alpha_i, \beta_i \in \{0, 1, 2, 3\}$ is 1 only when $\alpha_i + \beta_i \in \{3, 6\}$ for all $i \in [m]$, and zero otherwise. This can be seen by expanding the overlap as $\sum_{P \in \mathbb{F}_4^m} x^\alpha(P) x^\beta(P) = \prod_{i=1}^m (\sum_{t \in \mathbb{F}_4} t^{\alpha_i + \beta_i})$. One can also use the above overlap calculation to verify that the choice of X logical and stabilizers indeed form a triorthogonal space of $k = 1$.

We show the above correspondence with decreasing monomial codes by induction. The base case $[[4, 1, 2]]$ is easy to verify – the X logical is $\text{ev}(x)$ and the X stabilizer is $\text{ev}(1)$. To facilitate understanding, we show the $m = 2$ case by explicitly writing down the X -logical and X -stabilizers of the $[[16, 1, 4]]_4$ code (see below). Call the two variables x_1, x_2 , and evaluation is over the affine space $\mathbb{F}_4^2$. With the concatenation approach, the logical row is $x_1 x_2$. There shall be five X stabilizer rows, one is x_2 , and the other four come from the $\text{ev}(1) = (1, 1, 1, 1)$ stabilizer of the $[[4, 1, 2]]_4$ code, one can write

them as $\delta_0(x), \delta_1(x), \delta_\omega(x), \delta_{\omega^2}(x)$, where the delta function $\delta_\gamma(x) = \begin{cases} 1 & \text{if } x = \gamma \\ 0 & \text{if } x \neq \gamma \end{cases}$ can be expressed

as a polynomial: $\delta_\gamma(x) = 1 - (x - \gamma)^3$, $\gamma \in \mathbb{F}_4$.

One observes that $\text{span}_{\mathbb{F}_4}\{\delta_0(x), \delta_1(x), \delta_\omega(x), \delta_{\omega^2}(x)\} = \text{span}_{\mathbb{F}_4}\{1 - x^3, 1 - (x - 1)^3, 1 - (x - \omega)^3, 1 - (x - \omega^2)^3\} = \text{span}_{\mathbb{F}_4}\{1, x, x^2, x^3\}$ from $\delta_\gamma(x) = 1 - (x - \gamma)^3$. Therefore, one can equivalently write the X stabilizers as (the evaluation of) $1, x_1, x_1^2, x_1^3, x_2$, which are indeed all the monomials below the X logical $x_1 x_2$.

⁴²Here the order is defined for monomials in $\mathbb{F}_4[x_1, \dots, x_m]$ rather than $\mathbb{F}_4[x_1, \dots, x_m]/\langle x_1^4 - x_1, \dots, x_m^4 - x_m \rangle$. That is to say, the exponent vectors live in $\mathbb{Z}_{\geq 0}^m$. This is a well-defined monomial order, i.e., if monomials $u \prec v$ and w is any other monomial, then $uw \prec vw$.

	(0,0)	(0,1)	(0, ω)	(0, ω^2)	(1,0)	(1,1)	(1, ω)	(1, ω^2)	(ω ,0)	(ω ,1)	(ω , ω)	(ω , ω^2)	(ω^2 ,0)	(ω^2 ,1)	(ω^2 , ω)	(ω^2 , ω^2)
$\text{ev}(x_1 x_2)$	0	0	0	0	0	1	ω	ω^2	0	ω	ω^2	1	0	ω^2	1	ω
$\delta_0(x_1)$	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
$\delta_1(x_1)$	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
$\delta_\omega(x_1)$	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0
$\delta_{\omega^2}(x_1)$	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
$\text{ev}(x_2)$	0	1	ω	ω^2	0	1	ω	ω^2	0	1	ω	ω^2	0	1	ω	ω^2

Now suppose we iteratively concatenate $[[4, 1, 2]]_4$ with itself m times; the last step is to encode each of the GF(4)-qudits of the $[[4^{m-1}, 1, 2^{m-1}]]_4$ code into the $[[4, 1, 2]]_4$ code. Therefore, the new X logical is obtained by replacing each of its entry $\gamma \in \text{GF}(4)$ with the vector $\gamma \cdot (0, 1, \omega, \omega^2)$, where $(0, 1, \omega, \omega^2) = \text{ev}(x)$ is the X logical of the $[[4, 1, 2]]_4$ code. Therefore, the X logical $\text{ev}(x_1 x_2 \cdots x_{m-1})$ gets encoded into $\text{ev}(x_1 x_2 \cdots x_m)$, now being the X logical of the $[[4^m, 1, 2^m]]_4$ code.

We can similarly apply induction to the X stabilizers, which we denote S_m for the $[[4^m, 1, 2^m]]_4$ code. We need to show that S_m is formed by $\text{Fun}(x_1, \dots, x_{m-1})$ which consists of all functions $\mathbb{F}_4^{m-1} \rightarrow F_4$ on the variables $x_1, \dots, x_{m-1}$, and $x_m \cdot S_{m-1}$. Here, S_{m-1} consists of (the evaluation of) all monomials $\prec x_1 x_2 \cdots x_{m-1}$, which is the set of X stabilizers of the $[[4^{m-1}, 1, 2^{m-1}]]_4$ code, by the inductive hypothesis. Note that $\text{Fun}(x_1, \dots, x_{m-1}) \cup (x_m \cdot S_{m-1})$ indeed form *all* the monomials $\prec x_1 x_2 \cdots x_m$.

There are two equivalent generating sets of $\text{Fun}(x_1, \dots, x_{m-1})$: the first is by all the monomials generated by $x_1, \dots, x_{m-1}$, the second is the delta functions $\{\delta_P(x_1, \dots, x_{m-1}) \mid P \in \mathbb{F}_4^{m-1}\}$. The delta functions precisely provide the 4^{m-1} local stabilizers of the form $(0, \dots, 0, 1, 1, 1, 1, 0, \dots, 0)$ (weight four) coming from encoding each of the GF(4)-qudits of the $[[4^{m-1}, 1, 2^{m-1}]]_4$ code into the $[[4, 1, 2]]_4$ code. The other part of the contribution to S_m , namely $x_m \cdot S_{m-1}$, arises from encoding each of the entry of an X stabilizer of the $[[4^{m-1}, 1, 2^{m-1}]]_4$ using the logical row $(0, 1, \omega, \omega^2)$ of $[[4, 1, 2]]_4$.

Although the distances of the code are natural to see from the concatenation, namely $[[4^m, 1, d_X = 3^m / d_Z = 2^m]]$, we can alternatively prove the distances via the footprint bound [GH00; Gei08], which says the following. Fix any monomial order ' $\prec$ ' (which should be valid with respect to multiplication, i.e., if monomials $u \prec v$ and w is any other monomial, then $uw \prec vw$), we can define the *leading monomial* $\text{LM}(f)$ for a given polynomial $f \in \mathbb{F}_q[x_1, \dots, x_m]$ (take $q = 4$ for our case) as the largest monomial appearing in f with respect to ' $\prec$ '. For an ideal J of polynomials, define its *footprint* $\Delta_\prec(J)$ to be the set of those monomials which are not the leading monomial of any polynomial in J . Equivalently, $\Delta_\prec(J)$ contains the monomials that are not contained in the ideal $\langle \text{LM}(J) \rangle$ (here we use $\langle \cdot \rangle$ instead of $(\cdot)$ to denote the ideal generated by the polynomials surrounded by the brackets), where $\text{LM}(J) = \{\text{LM}(f) \mid f \in J\}$. The footprint bound says that if $\Delta_\prec(J)$ is finite, then $|\Delta_\prec(J)| \geq |V(J)|$, where $V(J) := \{x \in \mathbb{F}_q^m \mid f(x) = 0, \forall f \in J\}$.

Knowing the number of zeros of f on the affine space $\mathbb{F}_q^m$ allows us to determine the weight of $\text{ev}(f)$, since the weight counts the number of coordinates where the evaluation is nonzero. Call $V(f)$ the set of zeros of f , then $\text{wt}(\text{ev}(f)) = q^m - |V(f)| = q^m - |V(\langle f \rangle)|$ since $V(f) = V(\langle f \rangle)$. Moreover, let $I := \langle x_1^q - x_1, \dots, x_m^q - x_m \rangle$; its zero locus⁴³ $V(I)$ is the full affine space $\mathbb{F}_q^m$, and therefore, $|V(\langle f \rangle)| = |V(\langle f, x_1^q - x_1, \dots, x_m^q - x_m \rangle)|$.

Now we are ready to apply the footprint bound to determine the X distance, which is the lowest weight among all evaluation vectors $\{\text{ev}(x_1 x_2 \cdots x_m + s) \mid s \in S_m\}$. The leading monomial of all these polynomials $f = x_1 x_2 \cdots x_m + s$ is $\text{LM}(f) = x_1 x_2 \cdots x_m$. We have

$$\text{wt}(\text{ev}(f)) = 4^m - |V(\langle f, x_1^4 - x_1, \dots, x_m^4 - x_m \rangle)| \geq 4^m - |\Delta_\prec(\langle f, x_1^4 - x_1, \dots, x_m^4 - x_m \rangle)|, \quad (124)$$

where the footprint bound is applied in the second step.

Recall that the footprint $\Delta_\prec(\langle f, x_1^4 - x_1, \dots, x_m^4 - x_m \rangle)$ contains those monomials that are not in $\langle \text{LM}(\langle f, x_1^4 - x_1, \dots, x_m^4 - x_m \rangle) \rangle \supseteq \langle \text{LM}(f), \text{LM}(x_1^4 - x_1), \dots, \text{LM}(x_m^4 - x_m) \rangle = \langle x_1 x_2 \cdots x_m, x_1^4, \dots, x_m^4 \rangle$. Therefore, the footprint is contained in the set of monomials x^β with $0 \leq \beta_i < 4$ that are not divisible by $x_1 x_2 \cdots x_m$, of which there are $4^m - 3^m$. Eq. (124) then implies $\text{wt}(\text{ev}(f)) \geq 3^m$, and consequently $d_X \geq 3^m$. One can similarly show that $d_Z \geq 2^m$. Moreover, the X logical (resp. Z logical) $x_1 x_2 \cdots x_m$

⁴³Given an ideal I of a polynomial ring, its zero locus $V(I)$ are exactly the points in the affine space $\mathbb{F}_q^m$ on which every polynomial in I vanishes.

(resp. $\prod_{i=1}^m (1 + x_i + x_i^2)$) achieve these bounds.

Clifford aspects of the binarized $[[4^m, 1, 2^m]]_4$ code. The binarized code has parameters $[[2 \cdot 4^m, 2, d_X = 4 \cdot 3^{m-1} / d_Z = 2^m]]_2$, which are the same parameters as two patches of surface code. We will now show how this code admits implementations of the full Clifford group in full qudit distance, which equals the qubit distance in this case. Note that we already know for this code that the logical CS between the two logical qubits may be implemented using qudit-transversal physical CS, thereby retaining the qudit distance.

As for the Cliffords, we first note that individual S gates can be done with full distance on the two logical qubits, using the identity $X_2 \text{CS}_{12} X_2 \text{CS}_{12} = S_1$. Next, we show that all CNOT-type circuits between the two logical qubits can also be obtained in a qudit-transversal way (together with qudit permutations). It is observed in [Koh+26] that, for an $\mathbb{F}_4$ -qudit code, for any $\gamma \in \mathbb{F}_4$, $|\gamma\rangle_L \rightarrow |\alpha\gamma\rangle_L$ can be achieved through scaling each physical $\mathbb{F}_4$ -qudit by $\alpha \in \mathbb{F}_4^\times$. To complete the set of CNOT-type circuits, one only needs to find an implementation of the logical Frobenius transform $|\gamma\rangle_L \mapsto |\gamma^2\rangle_L$. Note that in the self-dual basis that we use for binarization, $\{\omega, \omega^2\}$, the Frobenius transform on one $\mathbb{F}_4$ -qudit just corresponds to swapping the two qubits. For our $[[4^m, 1, 2^m]]_4$ code, we show that the logical Frobenius transform can be achieved through physical Frobenius transforms (swapping the two physical qubits making up each physical qudit), and permuting the physical $\text{GF}(4)$ -qudits, as we now show through induction. In other words, the two logical qubits can be swapped by simply re-labeling the physical qubits.

For $[[4, 1, 2]]_4$, whose X -stabilizer is $(1, 1, 1, 1)$ and X -logical is $\ell_X = (0, 1, \omega, \omega^2)$. After individual physical Frobenius transforms, one swaps the third and fourth Galois qudit. The X stabilizer is preserved, and the X logical also transforms as desired:

$$\gamma \ell_X = (0, \gamma, \gamma\omega, \gamma\omega^2) \mapsto (0, \gamma^2, \gamma^2\omega^2, \gamma^2\omega) \mapsto (0, \gamma^2, \gamma^2\omega, \gamma^2\omega^2) = \gamma^2 \ell_X.$$

Now suppose we know how to do the logical Frobenius transform on $[[4^{m-1}, 1, 2^{m-1}]]_4$ using physical Frobenius and permutation. Then for the $[[4^m, 1, 2^m]]_4$ code, think of it as encoding each physical Galois-qudit of $[[4, 1, 2]]_4$ using a $[[4^{m-1}, 1, 2^{m-1}]]_4$ code. We first do the Frobenius transform on each of the four patches of $[[4^{m-1}, 1, 2^{m-1}]]_4$ using the induction hypothesis, then swap the third and fourth patch.

With the full CNOT-type circuits and individual S achieved with full qudit distance, we can now use the Hadamard teleportation gadget in [Koh+26, Fig. 13] to get individual Hadamard. Therefore, the full Clifford group on the two logical qubits can be done in full qudit distance, which equals the qubit distance in this case.

A.4 Optimality of asymptotic overhead two in $d = 2$ CS-to-CS distillation

We show that a $\mathbb{F}_4$ -linear code satisfying the triorthogonal constraints for CS-to-CS distillation (Eq. (67)) with $d \geq 2$ must satisfy $n \geq 2k$. Moreover, if there is an X stabilizer of full support (weight n), we can strengthen the bound to $n \geq 2k + 2$; in this case our Con. 4.16 is asymptotically optimal.

We only need to use the two-logical-one-stabilizer zero-overlap condition to show $n \geq 2k$. Interestingly, our proof is different from [NH22] for T -to- T distillation; in fact, our proof can serve as an alternative proof for their statement.

The convenience of the two-logical-one-stabilizer zero-overlap condition lies in the independence of the choice of basis for the logical and stabilizer space, respectively. This is because the overlap expression $T(\mathbf{g}_a, \mathbf{g}_b, \mathbf{g}_c) = \sum_{i=1}^n g_{ai} g_{bi} g_{ci}$ is a trilinear form.

We prove the following lemma by induction.

Lemma A.2. *Consider two $\mathbb{F}_q$ -linear spaces $L, S \subset \mathbb{F}_q^n$, where S is non-zero. S has support at every point, that is, given a basis for S , there is no coordinate $i \in [n]$ at which every basis element is zero.⁴⁴ If L and S satisfy $T(\ell_1, \ell_2, \mathbf{s}) = 0$ for any $\ell_1, \ell_2 \in L, \mathbf{s} \in S$, then $n \geq 2 \cdot \dim L$.*

⁴⁴This property may also be referred to by saying that S “has no zero column”. In this description, we imagine a generator matrix for S , that is, a matrix whose rows form a basis for S . This property means that the generator matrix for S has no zero column.

Proof. To facilitate understanding, we identify L and S with their generator matrices, that is, matrices whose rows form an $\mathbb{F}_4$ -basis for their respective spaces (the choice of basis does not matter). We call $k = \text{rank}(L)$.

We proceed via induction on n . For $n = 1$, the only option for S to be non-zero is $S = (\gamma)$ for $\gamma \in \mathbb{F}_4^\times$ and hence $L = (0)$, so $k = 0$. For $n = 2$, if $k = 2$, then we can write $L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, and no nonzero solution of S satisfies the overlap zero constraint. Note that $k = 1$ for $n = 2$ is possible, though: consider $L = S = (1 \ 1)$.

In the general case, assume the induction hypothesis holds for all n' smaller than n , which is the number of columns of L and S . Take an arbitrary nonzero row $\mathbf{s}$ from S , and let $P = \text{supp}(\mathbf{s})$. Turn L into a reduced row-echelon form on columns P through row operations on L (row operations on L preserve $T(\ell_1, \ell_2, \mathbf{s}) = 0$). This partitions L into row space L_0 , which vanishes on P , and L_1 , which contains the pivots.

We now consider the following lemma.

Lemma A.3. *Suppose we have $\mathbf{s} \in \mathbb{F}_q^m$ and $s_i \neq 0, \forall i \in [m]$ and a matrix A with entries in $\mathbb{F}_q$ having m columns, such that for any two rows $\mathbf{a}_i, \mathbf{a}_j$ of A , we have $T(\mathbf{a}_i, \mathbf{a}_j, \mathbf{s}) = \sum_{l=1}^m a_{il}a_{jl}s_l = 0$. Then $\text{rank}(A) \leq \lfloor \frac{m}{2} \rfloor$.*

Proof. The vanishing overlap condition can be written equivalently as $ADA^T = 0$, where $D = \text{diag}(s_1, \dots, s_m)$. Therefore, $A(AD)^T = 0$, which tells us that $\text{im}((AD)^T) \subseteq \ker(A)$. This implies that $\text{rank}(AD) = \text{rank}((AD)^T) \leq \dim(\ker(A)) = m - \text{rank}(A)$. Since D is invertible, we have $\text{rank}(AD) = \text{rank}(A)$. It follows that $\text{rank}(A) \leq m - \text{rank}(A)$. $\square$

Let us apply this lemma with m being $|P|$, A being $L_1[P]$, and $\mathbf{s}$ being $\mathbf{s}[P]$, where appending $[P]$ refers to the restriction of a matrix/row-vector to the columns indexed by a set P . We get $\text{rank}(L_1[P]) = \text{rank}(L_1) \leq |P|/2$.

If $P = [n]$, then the result follows. Otherwise, put $\bar{P} = [n] \setminus P$, we would next like to apply the inductive hypothesis of Lemma A.2 to $L_0[\bar{P}]$ and $S[\bar{P}]$, which we may do since they have $n - |P| < n$ columns. In order to do this, let us verify the assumptions of the inductive hypothesis. Since S has no zero columns, so does $S[\bar{P}]$. We need to verify the assumptions before applying the induction hypothesis to $L_0[\bar{P}]$ and $S[\bar{P}]$ (taking the columns $\bar{P} = [n] \setminus P$ from the matrices L_0, S). They both have $n - |P| < n$ columns. Since S contains no all-zero column, so does $S[\bar{P}]$, and also we have that $S[\bar{P}]$ is a non-zero space. Recall that L_0 vanishes on P , and thus $T(\ell_1[\bar{P}], \ell_2[\bar{P}], \mathbf{s}'[\bar{P}]) = T(\ell_1, \ell_2, \mathbf{s}') = 0$ for any $\ell_1, \ell_2 \in L_0$, $\mathbf{s}' \in S$. The induction hypothesis gives $\text{rank}(L_0[\bar{P}]) = \text{rank}(L_0) \leq (n - |P|)/2$, where $\text{rank}(L_0[\bar{P}]) = \text{rank}(L_0)$ follows from the fact that $L_0[P]$ is the zero matrix.

Since L_0, L_1 are obtained via row operations from the full-rank matrix L , we have $\text{rank}(L) = \text{rank}(L_0) + \text{rank}(L_1) \leq n/2$. $\square$

To apply Lemma A.2, we consider $L = L_X$ and $S = S_X$, the $\mathbb{F}_4$ -linear spaces spanned by the rows of the logical part of the matrix (the upper k rows), and the stabilizer part (the remaining rows), where all these rows together form a matrix for a code with $k = \dim L_X$ logical qudits.⁴⁵ Note also that L_X, S_X forming the X -logicals and X -stabilizers of a quantum code of distance ≥ 2 implies that no column of S_X is all-zero, that is, S_X has full support.⁴⁶ From here, we may conclude $n \geq 2k$.

In the present $\mathbb{F}_4$ case, we can further show that, if $\exists \mathbf{s} \in S_X$ with full support, then $n \geq 2k + 2$ (no induction is needed). Note first that we can rescale $\mathbf{s}$ to all-one, because scaling a column by $\gamma \in \mathbb{F}_4^\times$ preserves the triorthogonality condition (since $\gamma^3 = 1$). Then, considering the matrix $A := \begin{pmatrix} L_X \\ \mathbf{1}_n \end{pmatrix}$, the presence of the all-ones row gives us $AA^T = 0 \Rightarrow \text{im}(A^T) \subseteq \ker(A) \Rightarrow \text{rank}(A^T) = \text{rank}(A) \leq$

⁴⁵It is interesting that Lemma A.2 applies in a stronger setting, however, since it does *not* assume that $L \cap S = \{0\}$, or impose any triorthogonal constraint on S alone.

⁴⁶Strictly speaking, distance ≥ 2 is possible with S_X having a zero column, as long as L_X has the same column being zero. However, in this case, the all zero column may simply be removed from both spaces, which leaves all parameters unchanged, except the length of the code is lower, and we may apply our result to this new code, thus lower-bounding the length of the original code.

$n - \text{rank}(A) \Rightarrow 2\text{rank}(A) \leq n$. The conclusion then follows from the fact that $\text{rank}(A) = k + 1$, where $k = \text{rank}(L_X)$.

Appendix B Algebraic curves: definitions and applications

B.1 Advanced preliminaries / Formal Algebraic Geometry Code definitions

We now make some of the notions in Sec. 5 more precise and introduce basic notions such as function fields, extension and ramification, differentials and residue theorem, so that we can understand the dual codes better. It is recommended to read Sec. 5.1 and 5.2 before reading this section.

A particularly special type of ideal is called a maximal ideal. An ideal I of R is called a *maximal ideal* if $I \neq R$ and the only ideals containing I are I and R . It is possible to create a field by quotienting any ring by a maximal ideal; in fact, R/I is a field if and only if I is a maximal ideal of R . Another important type of ideal is a prime ideal. An ideal I of R is called a prime ideal if $I \neq R$ and whenever $ab \in I$, we have $a \in I$ or $b \in I$. A maximal ideal is a prime ideal.

Let us now explain the notion of *localization* of a ring. Let R be a ring. Consider a multiplicatively closed set $S \subset R$, that is, a subset S for which $1 \in S$ and $a, b \in S \Rightarrow ab \in S$. One defines an equivalence relation on $R \times S$:

$$(a, s) \sim (a', s') \Leftrightarrow \exists u \in S \text{ s.t. } u(as' - a's) = 0. \quad (125)$$

We denote the equivalence class of a pair $(a, s) \in R \times S$ by $\frac{a}{s}$. The set of all equivalence classes

$$S^{-1}R = \left\{ \frac{a}{s} \mid a \in R, s \in S \right\} \quad (126)$$

is called the *localization* of R at the multiplicatively closed set S . It is a ring together with the addition and multiplication

$$\frac{a}{s} + \frac{a'}{s'} := \frac{as' + a's}{ss'}, \quad \frac{a}{s} \cdot \frac{a'}{s'} := \frac{aa'}{ss'}. \quad (127)$$

A particularly important example is localization of R at a prime⁴⁷ ideal P of R . Let $S = R \setminus P$, one can verify that S is multiplicatively closed, since $a \notin P$ and $b \notin P$ implies $ab \notin P$. One usually denotes the localization $S^{-1}R$ by R_P . This localization R_P has exactly one maximal ideal $\{\frac{a}{s} \mid a \in P, s \notin P\}$. A ring is called *local* if it has exactly one maximal ideal. Therefore, the localization R_P is a local ring.

Every function field F over K can be regarded as a finite field extension of a rational functional field $K(x)$. Throughout, think of K as a finite field $\mathbb{F}_q$ and is therefore perfect. Assume that K is the full constant field of F . For a plane model $f(x, y) = 0$, one has $F = K(x, y)$ subject to this relation.

Local parameters and valuations. Let P be a smooth point. A *local parameter* at P is a function t that vanishes to *first order* at P . Equivalently, t generates the maximal ideal $\mathfrak{m}_P$ of the local ring $\mathcal{O}_P$. In particular, every nonzero rational function $f \in K(\chi)^\times$ can be written locally as $f = t^m u$ where $m \in \mathbb{Z}$ and u is a unit in $\mathcal{O}_P$, meaning that $u(P) \neq 0$. Define $v_P(f) = m$. Depending on $v_P(f)$ greater or smaller than 0, we say f has a zero or a pole at P , and $|v_P(f)|$ is the multiplicity. $v_P(f) = 0$ means $f(P) \neq 0$ and is finite.

The valuation satisfies

$$v_P(fg) = v_P(f) + v_P(g), \quad v_P(f + g) \geq \min\{v_P(f), v_P(g)\}. \quad (128)$$

A corollary is the strict triangle inequality [Sti08, Lemma 1.1.11]:

$$v_P(f + g) = \min\{v_P(f), v_P(g)\} \text{ if } v_P(f) \neq v_P(g). \quad (129)$$

For a smooth affine or projective plane curve, one can write down the local parameters as follows.

Given a smooth affine plane curve $f(x, y) = 0$ and a point $P = (a, b)$. One of $x - a$ or $y - b$ is usually a local parameter, but not always both. Since $f_x dx + f_y dy = 0$, we have $f_x(a, b)(x - a) + f_y(a, b)(y - b) + \text{higher terms} = 0$. If $f_y(a, b) \neq 0$, then locally y is a function of x , and $t = x - a$ is a local parameter. Similarly, if $f_x(a, b) \neq 0$ then $t = y - b$ is a local parameter. P being smooth implies that at least one of $f_x(a, b), f_y(a, b)$ is nonzero.

⁴⁷Since a maximal ideal is prime, one can define localization at a maximal ideal. This plays an important role in the function fields, because a place (introduced later) is a maximal ideal of certain valuation ring. See Eq. (130) for an example of localization at a maximal ideal.

As an example, consider the point $P = (0, 0)$ on $y^2 + y = x^3$. Here $f_x(0, 0) = 0$ and $f_y(0, 0) = 1$. So x is a local parameter, not y . Indeed, near P , $y = x^3 + \text{higher-order terms}$.

For this curve, let us also show how to apply the valuation rules to re-derive $v_{Q_\infty}(x) = -2$ and $v_{Q_\infty}(y) = -3$ where $Q_\infty = (0 : 1 : 0)$ (cf. Sec 5.2). Since the y coordinate of Q_∞ is nonzero, we work in the $Y = 1$ affine chart. Define $u = X/Y$ and $v = Z/Y$, then the curve equation becomes $v + v^2 = u^3$. Since $\partial(v + v^2 + u^3)/\partial v = 1$, u is a local parameter at Q_∞ and $v_{Q_\infty}(u) = 1$. From $v(1+v) = u^3$, since $1+v$ is a unit, i.e., $v_{Q_\infty}(1+v) = 0$, we get $v_{Q_\infty}(v) = 3$. Therefore, $x = X/Z = u/v$ and $y = Y/Z = 1/v$ has valuation $v_{Q_\infty}(x) = 1 - 3 = -2$ and $v_{Q_\infty}(y) = -3$, respectively.

For a smooth projective curve, suppose $P = (a : b : c) \in \mathbb{P}^2$. One of a, b, c is nonzero, suppose $c \neq 0$ and we work in the $Z = 1$ affine chart: $x = X/Z$, $y = Y/Z$, $P = (a/c, b/c)$. Then the candidate affine local parameters are: $x - \frac{a}{c} = \frac{cX - aZ}{cZ}$ and $y - \frac{b}{c} = \frac{cY - bZ}{cZ}$. Which one to use again depends on the partial derivative.

Places and degree. Let F/K be the function field of a curve over the constant field K .

In the function-field literature, a *place* of F/K is often defined valuation-theoretically. Namely, a *valuation ring* of F/K is a proper subring $\mathcal{O} \subset F$ such that $K \subseteq \mathcal{O}$ and, for every $0 \neq z \in F$, either $z \in \mathcal{O}$ or $z^{-1} \in \mathcal{O}$.

For a nonsingular curve, the valuation rings that occur are discrete valuation rings. Such a ring has a unique maximal ideal and is a local ring. One often calls this maximal ideal the *place*.

Equivalently, a place P gives a discrete valuation $v_P : F^\times \rightarrow \mathbb{Z}$ such that

$$\mathcal{O}_P = \{f \in F \mid v_P(f) \geq 0\}$$

is the corresponding valuation ring, and

$$P := \mathfrak{m}_P = \{f \in F \mid v_P(f) > 0\}$$

is its maximal ideal. The elements of $\mathcal{O}_P$ are the functions that are regular, i.e., have no pole, at P , while the elements of $\mathfrak{m}_P$ are the functions vanishing at P .

The *residue field* of P is $F_P := \mathcal{O}_P/\mathfrak{m}_P$ and the *degree* of the place is $\deg(P) = [F_P : K]$.

The K -rational points of a nonsingular curve are exactly the places of degree one. The above valuation-theoretic definition agrees with this more geometric picture as follows.

Suppose the affine curve χ is defined by $f(x, y) = 0$, and let $A = K[x, y]/(f)$ be its affine coordinate ring. If the point $(\alpha, \beta) \in \mathbb{A}^2$ is on the curve, i.e., $f(\alpha, \beta) = 0$, $\alpha, \beta \in K$, then its corresponding maximal ideal of A is $\mathfrak{m}_P = (x - \alpha, y - \beta)$, the ideal generated by the polynomials $x - \alpha$ and $y - \beta$. Indeed, $\mathfrak{m}_P$ is maximal in A because $A/\mathfrak{m}_P \cong K$ is a field. The corresponding valuation ring is the local ring (localization of A at the maximal ideal $\mathfrak{m}_P$)

$$\mathcal{O}_P = A_{\mathfrak{m}_P} \cong K[x, y]_{(x-\alpha, y-\beta)}/(f) = \left\{ \frac{g(x, y)}{h(x, y)} \mid g, h \in K[x, y]/(f), h(\alpha, \beta) \neq 0 \right\}. \quad (130)$$

Its maximal ideal is $\mathfrak{m}_P \mathcal{O}_P = (x - \alpha, y - \beta) A_{\mathfrak{m}_P}$. The residue map is evaluation at P : $\frac{g(x, y)}{h(x, y)} \mapsto \frac{g(\alpha, \beta)}{h(\alpha, \beta)}$.

For the places at infinity. Take $Q_\infty = (0 : 1 : 0)$ on $y^2 + y = x^5$ as an example. To write its local ring in ideal form, use the affine chart $Y \neq 0$. Put $u = \frac{X}{Y}$, $v = \frac{Z}{Y}$. In terms of the original affine functions, $x = \frac{X}{Z} = \frac{u}{v}$, $y = \frac{Y}{Z} = \frac{1}{v}$. The projective equation becomes $v^3 + v^4 = u^5$. The point Q_∞ corresponds to $u = 0$, $v = 0$, but the curve equation is still singular at $(u, v) = (0, 0)$. For the first blow-up, use $v = uz$, thus $u^5 + u^3 z^3 + u^4 z^4 = 0 \Rightarrow u^2 + z^3 + uz^4 = 0$. Q_∞ corresponds to $(u, z) = (0, 0)$, and the curve equation is still singular here. For the second blow-up, use $u = zw$, and $z^2 w^2 + z^3 + z^5 w = 0 \Rightarrow w^2 + z + z^3 w = 0$. Q_∞ corresponds to $(z, w) = (0, 0)$, and since $\frac{\partial(w^2 + z + z^3 w)}{\partial z} = 1$, the curve equation is smooth here and w is a local parameter. In particular, $v_{Q_\infty}(w) = 1$. We have $z = \frac{w^2}{1 + wz^2}$. Since $1 + wz^2$ is a unit, we have $v_{Q_\infty}(z) = 2$. Tracing back the previous changes of variables, we have $z = \frac{v}{u} = \frac{1/y}{x/y} = \frac{1}{x}$ and $w = \frac{u}{z} = \frac{x/y}{1/x} = \frac{x^2}{y}$. The valuation is $\mathcal{O}_{Q_\infty} = K[z, w]_{(z, w)}/(w^2 + z + wz^3)$, and the maximal ideal is $(z, w) \mathcal{O}_{Q_\infty} = (\frac{1}{x}, \frac{x^2}{y}) \mathcal{O}_{Q_\infty}$.

Here we also give an example of places of higher degree, since they are useful in multiplication in the finite field, which is the magic we are distilling. Again, we use the Hermitian curve example, $y^2 + y = x^3$ over $\mathbb{F}_4$. This curve has no degree-two places: all of its $\mathbb{F}_{4^2}$ -rational points are already defined over $\mathbb{F}_4$. We therefore give an example of a degree-three place. Such a place can be written as an ideal $Q = (p(x), y + q(x))$, where $p(x) \in \mathbb{F}_4[x]$ is irreducible of degree 3, and $q(x) \in \mathbb{F}_4[x]$

satisfies $q(x)^2 + q(x) \equiv x^3 \pmod{p(x)}$ since $y = q(x)$ in this ideal. Then $A/Q \cong \mathbb{F}_4[x]/(p(x))$ and thus $\deg(Q) = 3$. An explicit example is $Q = (x^3 + x^2 + 1, y + x^2 + x)$. Geometrically, if α is a root of $x^3 + x^2 + 1$ in $\mathbb{F}_{4^3}$, then this degree-three place represents the Frobenius orbit (the Frobenius map here is $z \mapsto z^4$)

$$(\alpha, \alpha^2 + \alpha), (\alpha^4, \alpha^8 + \alpha^4), (\alpha^{16}, \alpha^{32} + \alpha^{16}).$$

Readers interested only in the reduction of large field multiplication into smaller ones may proceed directly to App. B.3. The remainder of this subsection is needed for the metric $\mathbf{\Gamma}$ calculation (cf. Lemma 5.5) in App. B.2.

Function field extension and ramification [Sti08, Sec. 3]. Every function field over K (a perfect field, e.g., a finite field) can be regarded as a finite field extension F of a rational function field $K(x)$. Over the field extension $K(x, y)/K(x)$, think of the defining equation of the curve as a polynomial of y with coefficients in $K(x)$. For example, for the elliptic curve $y^2 + xy = x^3 + x^2 + x$, rewrite the equation as $f(y) = y^2 + xy - (x^3 + x^2 + x)$. Informally, a point above (*lies over*) a given $x = \alpha$ is ramified when $f(y)$ has a double root in y . In our example, the point $P = (0, 0)$ above $x = 0$ is ramified, with a *ramification index* $e(P | x = 0) = 2$, because $f(y) = y^2 = 0$ has a double root. More generally, let P be a place of F above a place Q of $K(x)$. The ramification index $e(P|Q)$ is defined by $v_P(h) = e(P|Q) \cdot v_Q(h)$, $h \in K(x)^\times$; Q is unramified if $e(P|Q) = 1$, and ramified if $e(P|Q) > 1$. In this example, the point at infinity $Q_\infty = (0 : 1 : 0)$ is also ramified and lies over P_∞ (the pole of x in $K(x)$). One has $v_{P_\infty}(x) = -1$, since x has a simple pole at infinity. Using intersection theory, we can obtain $v_{Q_\infty}(x) = -2$, which is a pole of order 2. Therefore, the ramification index is $e(Q_\infty | P_\infty) = 2$.

Differentials and residues. We only state the definitions and important theorems here. Examples of these notions appear in Sec. B.2, where we calculate the metric for elliptic curves and Klein quartic.

Here we restrict ourselves to a degree-one place P ; this is enough for our metric calculation later. A *Weil differential* is an object that can locally be written in the form

$$\omega = \left(\sum_{i \geq m} a_i t^i \right) dt, \quad (131)$$

where t is a local parameter at P .

Define $v_P(\omega) = m$, and the *residue* of ω at P is the coefficient of $t^{-1}dt$, i.e., $\text{res}_P(\omega) = a_{-1}$. This definition is independent of the choice of local parameter.

The *residue theorem* says that for every Weil differential ω , $\text{res}_P(\omega) = 0$ for almost all places P , and

$$\sum_P \text{res}_P(\omega) = 0. \quad (132)$$

The divisor of a nonzero differential is $(\omega) = \sum_P v_P(\omega)P$. For a divisor A , define

$$\Omega(A) = \{\omega \mid (\omega) \geq A\} \cup \{0\}. \quad (133)$$

Differential AG codes and duality [Sti08, Thm. 2.2.8-Cor. 2.2.11]. Let $D = P_1 + \dots + P_n$ be a sum of distinct K -rational places, and let G be a divisor whose support is disjoint from $\text{supp}(D)$. The evaluation AG code is

$$C_{\mathcal{L}}(D, G) = \{(f(P_1), \dots, f(P_n)) \mid f \in \mathcal{L}(G)\}. \quad (134)$$

The differential AG code is

$$C_{\Omega}(D, G) = \{(\text{res}_{P_1}(\omega), \dots, \text{res}_{P_n}(\omega)) \mid \omega \in \Omega(G - D)\}. \quad (135)$$

The residue theorem implies $C_{\Omega}(D, G) \subset C_{\mathcal{L}}(D, G)^\perp$ as follows. If $f \in \mathcal{L}(G)$ and $\omega \in \Omega(G - D)$, recall

$$\mathcal{L}(G) = \{f \in K(\chi)^\times \mid (f) + G \geq 0\} \cup \{0\}, \quad \Omega(G - D) = \{\omega \mid (\omega) \geq G - D\} \cup \{0\}, \quad (136)$$

then $(f\omega) \geq -D$, i.e., $f\omega$ has possible simple poles only at the points P_i .

Since $\text{res}_{P_i}(f\omega) = f(P_i)\text{res}_{P_i}(\omega)$, we have

$$\sum_{i=1}^n f(P_i)\text{res}_{P_i}(\omega) = \sum_P \text{res}_P(f\omega) = 0. \quad (137)$$

In fact, dimension counting using the Riemann-Roch theorem gives the equality

$$C_{\Omega}(D, G) = C_{\mathcal{L}}(D, G)^\perp. \quad (138)$$

Now suppose there exists a differential η such that

$$v_{P_i}(\eta) = -1 \quad \text{and} \quad \Gamma_i := \text{res}_{P_i}(\eta) \neq 0, \quad \forall i. \quad (139)$$

Then every differential in $\Omega(G - D)$ can be written as $h\eta$ for some $h \in \mathcal{L}(D - G + (\eta))$. Therefore,

$$C_{\mathcal{L}(D, G)}^\perp = C_\Omega(D, G) = \mathbf{\Gamma} \cdot C_{\mathcal{L}}(D, D - G + (\eta)), \quad (140)$$

where $\mathbf{\Gamma} \cdot (a_1, \dots, a_n) = (\Gamma_1 a_1, \dots, \Gamma_n a_n)$.

g	Curve(s)	Comments
2	$y^2 + y = x^5 + x^3 + \theta^5 x$ [GV96]	-
3	Two curves are known: $y^4 + y^2 + y = x^3$ [GQ01, Example 12], $y^4 + y = x^3$.	These two curves are not $\mathbb{F}_{64}$ -isomorphic [GT09].
4	$y^2 + y = x^9$ [GV96, Method I]	-
6	$\begin{cases} y_1^2 + y_1 = \theta x^2 + \theta^4 x^3 + \theta^2 x^5 \\ y_2^2 + y_2 = \theta^2 x^2 + \theta x^3 + \theta^4 x^5 \end{cases}$ [GV96, Method II]	A singular plane model $z^4 + (\theta + 1)z^2 + \theta z = \theta^6 x^3(x + 1)^3$ can be obtained by defining $z = y_1 + \theta^6 y_2$.
7	$y^8 + y = x^3$	Follows from [Sti08, Prop. 6.4.1].
9	$c \cdot s(U, V, W) = 0$ where $s(U, V, W) = U^8 W + W^8 V + V^8 U + (UVW)^3$, $U = \beta X + Y + \beta^{65} Z$, $V = \beta^{65} X + \beta Y + Z$, $W = X + \beta^{65} Y + \beta Z$. [CKT00, Remark 2.2 & 2.3].	$c = \beta^{27}$, $\beta \in \mathbb{F}_{512}$ satisfies $\beta^9 + \beta + 1 = 0$. The multiplication by c makes the coefficients in $c \cdot s(U, V, W) = 0$ land in $\mathbb{F}_{64}$.
10	$y^9 = x^6 + x^3$	This is a singular plane model of the $q = 2$, $n = 3$ case of [BM18; GGS10; GK09].
12	$\begin{cases} y_1^2 + y_1 = \theta^4 x^9 \\ y_2^2 + y_2 = \theta^2 x^9 \end{cases}$ [GV96, Method I]	A singular plane model $z^4 + z^2 + z = x^9$ can be obtained by defining $z = \theta y_1 + \theta^2 y_2$. This curve also appears as the (II.1) case of [CKT00, Thm. 2.1].
28	$y^8 + y = x^9$	Hermitian curve

Table 4: A summary of maximal curves over $\mathbb{F}_{64}$. g is the genus and a maximal curve of that genus has $64 + 16g + 1$ points. α is a primitive element of $\mathbb{F}_{64}$ that satisfies $\alpha^6 + \alpha + 1 = 0$, taking $\theta = \alpha^{27}$, then θ is a generator of the $\mathbb{F}_8$ subfield and satisfies $\theta^3 + \theta + 1 = 0$.

B.2 Calculation of metric

B.2.1 Klein quartic codes

Recall that the Klein quartic curve $x^3 y + y^3 + x = 0$ over $\mathbb{F}_8$ has two points at infinity: $Q_\infty = (0 : 1 : 0)$ and $P = (1 : 0 : 0)$. Call $R = (0 : 0 : 1)$, and recall from the calculation below Thm. 5.1 that

$$(x) = 3R - P - 2Q_\infty, \quad (y) = R + 2P - 3Q_\infty. \quad (141)$$

We construct the one-point codes by taking $G = rQ_\infty$ in $C_{\mathcal{L}}(D, G)$. Let D_0 be the sum of the 21 affine rational places excluding $P_{0,0} = R = (0 : 0 : 1)$. Denote these 21 points by $P_{\alpha, \beta}$ for $\alpha \in \mathbb{F}_8^\times, \beta \in \mathbb{F}_8$ that satisfy $\alpha^3 \beta + \beta^3 + \alpha = 0$. We explore the two possibilities that $D = D_0 + R$ and $D = D_0 + R + P$.

Random puncturing (a hundred trials) gives the following distillation protocol parameters

$$D = D_0 + R : \quad 20 \rightarrow 1 \ (d = 4), \quad 18 \rightarrow 3 \ (d = 3), \quad 16 \rightarrow 5 \ (d = 2); \quad (142)$$

$$D = D_0 + R + P : \quad 21 \rightarrow 2 \ (d = 4), \quad 19 \rightarrow 4 \ (d = 3), \quad 17 \rightarrow 6 \ (d = 2). \quad (143)$$

Metric for $C_{\mathcal{L}}(D_0 + R, rQ_{\infty})$. The metric is the single codeword (up to multiplication by a scalar) of $C_{\mathcal{L}}(D, 23Q_{\infty})^{\perp} = C_{\Omega}(D, 23Q_{\infty} - D) = (\text{res}_{P_{0,0}}(\omega), \dots)$, where ω satisfies

$$(\omega) \geq 23Q_{\infty} - D = 23Q_{\infty} - D_0 - R.$$

Let $h(x) = \prod_{\alpha \in A} (x - \alpha)$ where A is the set of x -coordinates that actually occur among the affine rational points. One can verify that $A = \mathbb{F}_8$, i.e., $\forall \alpha \in \mathbb{F}_8, \exists \beta \in \mathbb{F}_8$ such that $\alpha^3\beta + \beta^3 + \alpha = 0$ by taking $\beta = \alpha^5\eta$ for η that satisfies $\eta^3 + \eta + 1 = 0$. Therefore,

$$h(x) = x^8 + x.$$

We seek ω in the form $\frac{x^i y^j dx}{h(x)}$ for integer i, j . Eq. (141) implies that

$$(x^i y^j) = -(2i + 3j)Q_{\infty} + (2j - i)P + (3i + j)R, \quad (144)$$

and we still need to calculate $(h(x))$ and (dx) .

At each affine point $P_{\alpha,\beta}$ with $\alpha \neq 0$, the function $x - \alpha$ is a local parameter because $F_y = y^2 + x^3 \neq 0$ (assume $F_y(P_{\alpha,\beta}) = 0$, then the curve equation gives $\alpha = 0$, contradiction). Writing $t = x - \alpha$, we have

$$h(x) = x^8 + x = (\alpha + t)^8 + \alpha + t = t^8 + t = t(1 + t^7),$$

so $h(x)$ has a simple zero at each point of D_0 .

At R , we have $v_R(x) = 3$ from Eq. (141), so $h(x) = x^8 + x = x(1 + x^7)$ and $1 + x^7$ is a unit at R . Hence $v_R(h(x)) = v_R(x) = 3$.

At the two points at infinity, x has a pole. Recall from Eq. (128) that

$$v(f + g) = \min\{v(f), v(g)\} \text{ whenever } v(f) \neq v(g).$$

Since $v_P(x) = -1$, we have $v_P(x^8 + x) = v_P(x^8) = 8v_P(x) = -8$. Similarly, since $v_{Q_{\infty}}(x) = -2$, $v_{Q_{\infty}}(x^8 + x) = -16$. Thus

$$(h(x)) = D_0 + 3R - 8P - 16Q_{\infty}. \quad (145)$$

Now we compute (dx) . Differentiating the curve equation $F(x, y) = x^3y + y^3 + x = 0$ gives

$$F_x dx + F_y dy = 0 \Rightarrow F_x dx = F_y dy, \quad F_x = x^2y + 1, \quad F_y = y^2 + x^3. \quad (146)$$

At every affine point other than R , one have $F_y \neq 0$, so $x - \alpha$ is a local parameter, since $dx = d(x - \alpha)$, dx has valuation 0.

At $P = (1 : 0 : 0)$, since $v_P(x) = -1$, $\frac{1}{x}$ is a local parameter. We have $du = \frac{dx}{x^2}$ and hence $dx = x^2 du$. Therefore $v_P(dx) = v_P(x^2) = -2$.

At R , since $v_R(y) = 1$, y is a local parameter. $du = dy = \frac{F_x}{F_y} dx = \frac{x^2y+1}{y^2+x^3} dx$. We have $v_R(y^2 + x^3) = \min\{v_R(y^2), v_R(x^3)\} = 2$, and $v_R(x^2y + 1) = \min\{2v_R(x) + v_R(y), 0\} = 0$. Thus $v_R(dx) = v_R(\frac{y^2+x^3}{x^2y+1}) = v_R(y^2 + x^3) - v_R(x^2y + 1) = 2$. In fact, one can also use $u = \frac{x}{y^2}$ as a local parameter at R , then $du = \frac{y^2 dx - 2xy dy}{y^4} = \frac{dx}{y^2}$. Hence $dx = y^2 du$ and $v_R(dx) = v_R(y^2) = 2$.

The subtle point is $Q_{\infty} = (0 : 1 : 0)$. Here $v_{Q_{\infty}}(x) = -2$ and $v_{Q_{\infty}}(y) = -3$ and so $u = \frac{x}{y}$ is a local parameter. $du = \frac{x dy - y dx}{y^2} = \frac{x^3}{y(y^2+x^3)} dx$. $v_{Q_{\infty}}(y^2 + x^3)$ requires care to compute – since $v_{Q_{\infty}}(y^2) = v_{Q_{\infty}}(x^3) = -6$, cancellation may occur. Dividing the affine equation $x^3y + y^3 + x = 0$ by y yields $x^3 + y^2 + \frac{x}{y} = 0$. Therefore, $v_{Q_{\infty}}(y^2 + x^3) = v_{Q_{\infty}}(x/y) = 1$. Consequently, $v_{Q_{\infty}}(dx) = v_{Q_{\infty}}(y) + v_{Q_{\infty}}(y^2 + x^3) - 3v_{Q_{\infty}}(x) = 4$. Thus

$$(dx) = 2R - 2P + 4Q_{\infty}. \quad (147)$$

Hence

$$(\omega) = \left(\frac{x^i y^j dx}{h(x)} \right) = (3i + j - 1)R + (2j - i + 6)P + (20 - 2i - 3j)Q_{\infty} - D_0. \quad (148)$$

Requiring $(\omega) \geq 23Q_{\infty} - D_0 - R$, yields a unique solution $i = 1, j = -2$.

Let us now calculate the residue of $\omega = \frac{x dx}{y^2(x^8+x)} = \frac{dx}{y^2(x^7+1)}$.

At $P_{\alpha,\beta}$ where $\alpha \neq 0$, use the local parameter $t = x - \alpha$, then $\omega = \frac{1}{t} \cdot \frac{x}{y^2(t^7+1)} \cdot dt$, so

$$\text{res}_{P_{\alpha,\beta}} = \frac{\alpha}{\beta^2}, \quad \alpha \neq 0. \quad (149)$$

At $R = (0 : 0 : 1)$, recall that $u = \frac{x}{y^2}$ is a local parameter and $du = \frac{dx}{y^2}$. Therefore

$$\omega = \frac{x dx}{y^2(x^8+x)} = \frac{du}{x^7+1}.$$

Since $v_R(x) = 3 > 0$, $x^7 + 1$ is a unit. The Laurent expansion of ω in the local parameter u has no

$u^{-1}du$ term. Therefore

$$\text{res}_R(\omega) = 0. \quad (150)$$

Metric for $C_{\mathcal{L}}(D_0 + R + P, rQ_{\infty})$. Let us first comment on the evaluation of the basis functions

$$\mathcal{L}(rQ_{\infty}) = \{x^i y^j \mid i, j \in \mathbb{Z}_{\geq 0}, 2i + 3j \leq r, i \leq 2j\} \quad (151)$$

at the infinity P . The condition $i \leq 2j$ forces that $v_P(x^i y^j) = 2j - i \geq 0$. For those i, j such that $v_P(x^i y^j) > 0$, the evaluation is zero since the order of vanishing is positive. One needs some extra computation for $i = 2j$ though. For example, to evaluate $x^2 y$ at P : notice that dividing the affine equation $x^3 y + y^3 + x = 0$ by x gives $x^2 y + \frac{y^3}{x} + 1 = 0$. $v_P(y^3/x) = 7 > 0$ and thus $\text{ev}_P(y^3/x) = 0$. Therefore,

$$\text{ev}_P(x^2 y) = \text{ev}_P(y^3/x + 1) = \text{ev}_P(1) = 1. \quad (152)$$

The metric is the single codeword of $C_{\mathcal{L}}(D_0 + R + P, 25Q_{\infty})^{\perp}$. We still seek ω in the form of $\frac{x^i y^j dx}{h(x)}$, and that Eq. (148) still holds. Imposing $(\omega) \geq 25Q_{\infty} - D_0 - R - P$ yields a unique solution $i = 1, j = -3$. The residue of $\omega = \frac{x dx}{y^3(x^8 + x)}$ at each place is

$$\text{res}_{P_{\alpha, \beta}}(\omega) = \frac{\alpha}{\beta^3} \text{ for } \alpha \neq 0, \quad \text{res}_P(\omega) = 1, \quad \text{res}_R(\omega) = 1. \quad (153)$$

We only show $\text{res}_P(\omega) = 1$ and leave the rest to the readers. We calculate in Eq. (152) that $\text{ev}_P(x^2 y) = 1$, therefore, $\text{ev}_P(x^6 y^3) = 1$. At P , $u = \frac{1}{x}$ is a local parameter.

$$\omega = \frac{x dx}{y^3(x^8 + x)} = \frac{u^6}{y^3(1 + u^7)} \cdot \frac{du}{u},$$

and $\text{ev}_P(1 + u^7) = 1 + \text{ev}_P(u^7) = 1$ since $v_P(u^7) = 7 > 0$. Also, $\frac{u^6}{y^3} = \frac{1}{x^6 y^3} = 1$. Hence $\text{res}_P(\omega) = \text{ev}_P\left(\frac{u^6}{y^3(1 + u^7)}\right) = 1$.

B.2.2 Elliptic curve codes

Elliptic curves have genus $g = 1$, by Riemann-Roch, the dimension of $\mathcal{L}(rQ_{\infty})$ is $\ell(rQ_{\infty}) = r$ for $r > 0$. Take $r = n - 1$, then the metric is the single codeword in $C_{\mathcal{L}}(D, (n - 1)Q_{\infty})^{\perp} = C_{\Omega}(D, (n - 1)Q_{\infty}) = \{(\text{res}_{P_1}\omega, \dots, \text{res}_{P_n}\omega)\}$ for a differential ω whose divisor satisfies $(\omega) \geq (n - 1)Q_{\infty} - D$. An explicit choice is $\omega = \frac{dx}{h(x)}$, where $h(x) = \prod_{\alpha \in A} (x - \alpha)$, and A is the set of x -coordinates that occur among the

K -rational affine points of the curve. We distinguish between two cases from Table. 2:

Type I, III, V. The curve has no cross term xy and is of the form $y^2 + y = x^3 + a_2 x^2 + a_6$. Here $(dx) = 0$ and each $\alpha \in A$ gives two affine points of the curve (if $y = \beta$ is a solution, then $y = \beta + 1$ is another), so $(h(x)) = D - nQ_{\infty}$. thus $(\omega) \geq (n - 1)Q_{\infty} - D$. At each place $P_i = (\alpha, \beta) \in \mathbb{P}_F$ where $\alpha \in A$, $(x - \alpha)$ is a local paramter and $\text{res}_{P_i}\omega = 1/h'(\alpha) = \prod_{\alpha' \in A, \alpha' \neq \alpha} (\alpha - \alpha')^{-1}$.

Type II, IV. The curve has a cross term and is of the form $y^2 + xy = x^3 + a_2 x^2 + x$. Let $H(x) = \prod_{\alpha \in A \setminus \{0\}} (x - \alpha)$, then $h(x) = xH(x)$. One calculates that $(\frac{dx}{x}) = 0$ (in fact, $(dx) = (x) = 2P - 2Q$ where $Q = (0 : 1 : 0)$ and $P = (0 : 0 : 1)$ is the unique affine point above $x = 0$. So $(\omega) = (n - 1)Q_{\infty} - D + P \geq (n - 1)Q_{\infty} - D$. At each place $P_i = (\alpha, \beta) \in \mathbb{P}_F$ where $\alpha \in A \setminus \{0\}$, $\text{res}_{P_i}\omega = 1/h'(\alpha) = \prod_{\alpha' \in A, \alpha' \neq \alpha} (\alpha - \alpha')^{-1}$. At $x = 0$ where $F_y = 0$ but $F_x = 1$, y is a local parameter

instead of x . Differentiating the curve equation gives $x dy = (x^2 + y + 1) dx$, so $\frac{dx}{h(x)} = \frac{dx}{x} \cdot \frac{1}{H(x)} = \frac{dy}{y+1} \cdot \frac{1}{H(x)}$. Since $H(0) = \prod_{\alpha \in A \setminus \{0\}} (-\alpha) \neq 0$ and $y + 1 = 1$ at P , ω is regular at P and hence $\text{res}_P(\omega) = 0$. For the three-orthogonality purpose, we can remove P from D since the metric here is 0; this reduces the code length by one.

B.2.3 One-point codes from [Sti08, Prop. 6.4.1]

Consider a function field $F = K(x, y)$ with $y^h + \mu y = f(x) \in K[x]$ over $K = \mathbb{F}_{2^s}$ for $\mu \in K^{\times}$, where $h = p^e > 1$ and $\deg f =: 0 > 0$ is coprime to p , and that all roots of $y^h + \mu y = 0$ are in K .

[Sti08, Prop. 6.4.1] shows that $P_{\infty} = (x)$ is the only place of $K(x)$ that ramifies in $F/K(x)$, and Q_{∞} is unique place of F/K above P_{∞} . Moreover, $Q_{\infty}|P_{\infty}$ is totally ramified with $e(Q_{\infty}|P_{\infty}) = h$. Since x has a simple pole at P_{∞} , we thus have $v_{Q_{\infty}}(x) = -h$. Since $v_{Q_{\infty}}(y^h + \mu y) = v_{Q_{\infty}}(y^h) = h$, $v_{Q_{\infty}}(y) = v_{Q_{\infty}}(f(x)) = m \cdot v_{Q_{\infty}}(x)$, we have $v_{Q_{\infty}}(y) = -m$. Finally, $(dx) = (2g - 2)Q_{\infty}$.

Let $A \subseteq K$ be the set of x -coordinates that occur among the K -rational affine points of the curve, and $h(x) = \prod_{\alpha \in A} (x - \alpha)$. Since all roots of $y^h + \mu y = 0$ lie in K , if there is one solution for y , then there are exactly h solutions in K , and $(h(x)) = D - nQ_\infty$, where $D = \sum_{\alpha} \sum_{\substack{\beta \in K \\ \beta^q + \mu\beta = f(\alpha)}} P_{\alpha,\beta}$ is the sum of all affine K -rational places and there are $n = h|A|$ many. Put $\omega = \frac{dx}{h(x)}$, then its divisor

$$(\omega) = (2g - 2)Q_\infty - (D - nQ_\infty) = (n + 2g - 2)Q_\infty - D.$$

Since $F_y = \mu \neq 0$, $x - \alpha$ is a local parameter at $P_{\alpha,\beta}$, and $\text{res}_{P_{\alpha,\beta}}(\omega) = \frac{1}{h'(\alpha)}$.

In the special case that $A = K$, one can show that the metric is all-one because $h(x) = x^{|K|} - x$ and $h'(x) = |K|x^{|K|-1} - 1 = 1$. This is the case for, e.g., the Hermitian curve $y^q + y = x^{q+1}$ over $K = \mathbb{F}_{q^2}$, since the defining equation is effectively $\text{tr}_{\mathbb{F}_{q^2}/\mathbb{F}_q}(y) = \text{Nm}_{\mathbb{F}_{q^2}/\mathbb{F}_q}(x)$ and both sides can take all values in $\mathbb{F}_q$.

B.3 Multiplication in binary extension fields

The general recipe to do multiplication in $\mathbb{F}_{2^n}$ using a curve of genus g is to choose Q to be a degree- n place on the curve and D to be a divisor of degree $n + g - 1$ that has non-overlapping support with Q . Here, we restrict ourselves to D being a single place of degree $n + g - 1$. The place Q is chosen so that the evaluation at Q , $\text{ev}_Q : \mathcal{L}(D) \rightarrow \mathcal{O}_Q/Q \cong \mathbb{F}_{2^n}$ is bijective. Let $\{h_1, \dots, h_n\}$ be a basis of $\mathcal{L}(D)$. One can then use $\xi_i = \text{ev}_Q(h_i) \in \mathbb{F}_{2^n}$, $i = 1, \dots, n$ as a basis of the field $\mathbb{F}_{2^n}$. Suppose we want to multiply $a, b \in \mathbb{F}_{2^n}$ with the basis expansion $a = a_1\xi_1 + \dots + a_n\xi_n$ and $b = b_1\xi_1 + \dots + b_n\xi_n$. We identify them with the functions

$$f_a = a_1h_1 + \dots + a_nh_n \in \mathcal{L}(D), \quad f_b = b_1h_1 + \dots + b_nh_n \in \mathcal{L}(D).$$

The product $f_af_b \in \mathcal{L}(2D)$ can be expanded as⁴⁸

$$f_af_b = \sum_{j=1}^{2n+g-1} c_j h_j. \quad (154)$$

One can recover the product $ab \in \mathbb{F}_{2^n}$ by evaluating at Q , i.e., $ab = \text{ev}_Q(f_af_b) = \sum_{j=1}^{2n+g-1} c_j \text{ev}_Q(h_j)$. The only CCZ operations lie in determining $c_j \in \mathbb{F}_2$ from the unknowns $a_i, b_i \in \mathbb{F}_2$; all other operations, such as multiplying by constants $\text{ev}_Q(h_j)$ and adding things up, interpolation, are Clifford only.

To determine c_j , we first evaluate at N auxiliary places $P_1, \dots, P_N$ on the curve and then interpolate back. These places are picked so that $\deg(P_1) + \dots + \deg(P_N) = 2n + g - 1$, and the following map φ is injective.

$$\varphi : \mathcal{L}(2D) \rightarrow \prod_{i=1}^N \mathbb{F}_2^{\deg(P_i)} \cong \mathbb{F}_2^{2n+g-1}, \quad (155)$$

$$f \mapsto (\text{ev}_{P_1}(f), \dots, \text{ev}_{P_N}(f))^\top. \quad (156)$$

Then clearly

$$\varphi(f_af_b) = (\text{ev}_{P_1}(f_a) \cdot \text{ev}_{P_1}(f_b), \dots, \text{ev}_{P_N}(f_a) \cdot \text{ev}_{P_N}(f_b))^\top. \quad (157)$$

In other words, at individual places, say P_i , we are doing multiplication in $\mathbb{F}_{2^{\deg(P_i)}}$. Therefore, if $\mu_2(n)$ represents the number of CCZ gates required for multiplication in $\mathbb{F}_{2^n}$, then this evaluate-then-interpolate procedure requires $\mu_2(\deg(P_1)) + \dots + \mu_2(\deg(P_N))$ CCZs.

Note that $\text{ev}_{P_i}(f) \in \mathbb{F}_{2^{\deg(P_i)}}$, but we can binarize this into $\deg(P_i)$ bits. With this binarization applied to every place, we can write $\varphi(f_af_b) := (m_1, \dots, m_{2n+g-1})^\top$ for $m_i \in \mathbb{F}_2$.

φ is an $\mathbb{F}_2$ -linear map because evaluating the sum of two functions at a place is equivalent to evaluating each and then adding up the results. Therefore, given $f_af_b = \sum_{j=1}^{2n+g-1} c_j h_j$, we have

$$\varphi(f_af_b) = \sum_{j=1}^{2n+g-1} c_j \varphi(h_j) = G(c_1, \dots, c_{2n+g-1})^\top, \quad \text{where } G := \begin{pmatrix} | & & | \\ \varphi(h_1) & \cdots & \varphi(h_{2n+g-1}) \\ | & & | \end{pmatrix}. \quad (158)$$

One can recover $(c_1, \dots, c_{2n+g-1})^\top$ in Eq. (154) as $G^{-1}(m_1, \dots, m_{2n+g-1})^\top$.

⁴⁸Had we picked arbitrary functions $h_1, \dots, h_n$, their pairwise products $h_i h_j$ may span a space of dimension as large as roughly $n(n+1)/2$. Here, by choosing $h_1, \dots, h_n$ as a basis of the Riemann-Roch space $\mathcal{L}(D)$, the pairwise product space is much smaller because $\mathcal{L}(D) \cdot \mathcal{L}(D) \subseteq \mathcal{L}(2D)$.

One needs to verify that G is invertible. We will do so explicitly in the end for our following example of $\mathbb{F}_{2^5}$ multiplication using the elliptic curve $y^2 + y = x^3$ over $\mathbb{F}_2$. This curve has three $\mathbb{F}_2$ -rational places (degree one):

$$P_{00} = (x, y), \quad P_{01} = (x, y + 1), \quad P_{\infty},$$

and three degree-two places:

$$R_1 = (x + 1), \quad R_2 = (x^2 + x + 1, y + x), \quad R_3 = (x^2 + x + 1, y + x + 1).$$

We choose the degree-5 place $Q = (x^5 + x^4 + x^2 + x + 1, y + x^4 + x^2 + x)$. Furthermore, we choose a degree- $(n + g - 1) = 5$ place $D = (f(x), y + q(x))$ where,

$$f(x) = x^5 + x^3 + 1, \quad q(x) = x^3 + x^2 + x. \quad (159)$$

One can verify that $f(x)$ is irreducible over $\mathbb{F}_2$ and indeed $y^2 + y = q(x)^2 + q(x) \equiv x^3 \pmod{f(x)}$.

Instead of the (simplest) procedure described above, where each auxiliary place is used only once, we follow [CÖ10] which suggests using some places multiple times. In our example, we use the place P_{01} twice and all other places only once. This gives a 14 CCZ implementation of $\mathbb{F}_{2^5}$ multiplication (to be explained later), which is not optimal since 13 is known to be possible (and is the minimal for unitary synthesis) [CC88]. Nonetheless, we find it suitable for pedagogical purposes.

We can write down a basis of $\mathcal{L}(D)$ as follows. The functions are of the form $\frac{A(x)y+B(x)}{f(x)}$. Note that we do not need to include y^2 terms because they can be reduced via $y^2 = y + x^3$. The division by $f(x)$ is because, by definition, $\mathcal{L}(D)$ contains functions with poles only on D . Moreover, $\mathcal{L}(D)$ should not contain functions with poles on $D' = (f(x), y + \bar{q}(x))$ where

$$\bar{q}(x) := q(x) + 1 = x^3 + x^2 + x + 1.$$

D' is the other degree-five place, in fact, the conjugate place of D , because $\bar{q}(x)^2 + \bar{q}(x) = q(x)^2 + q(x) \equiv x^3 \pmod{f(x)}$. For the function $\frac{A(x)y+B(x)}{f(x)}$ to vanish on D' , we thus require

$$A(x)\bar{q}(x) + B(x) \equiv 0 \pmod{f(x)}.$$

As for a concrete basis of $\mathcal{L}(D)$, we can take $A(x)$ to be $1, x^1, x^2, x^3$ and let the corresponding $B(x)$ be $A(x)\bar{q}(x) \pmod{f(x)}$. To be more explicit:

$$\begin{aligned} h_1 &= 1, & h_2 &= \frac{y + x^3 + x^2 + x + 1}{f(x)}, & h_3 &= \frac{xy + x^4 + x^3 + x^2 + x}{f(x)}, \\ h_4 &= \frac{x^2y + x^4 + x^2 + 1}{f(x)}, & h_5 &= \frac{x^3y + x + 1}{f(x)}. \end{aligned}$$

Next, let us find $h_6, \dots, h_{10}$, such that together with $h_1, \dots, h_5$, they form a basis of $\mathcal{L}(2D)$. Similarly, the functions take the form $\frac{A(x)y+B(x)}{f(x)^2}$, and again, they need to vanish on D' .

$$A(x)\bar{Q}(x) + B(x) \equiv 0 \pmod{f(x)^2},$$

where $y \equiv \bar{Q}(x) \pmod{f(x)^2}$ needs to satisfy

$$\begin{aligned} \bar{Q}(x) &\equiv \bar{q}(x) = y \pmod{f(x)}, \\ y^2 + y &\equiv \bar{Q}(x)^2 + \bar{Q}(x) \equiv x^3 \pmod{f(x)^2}. \end{aligned}$$

The first equality implies $\bar{Q}(x) = \bar{q}(x) + f(x)s(x)$ for some $s(x)$. Substituting this form into the second equality and solving for $s(x)$ yields $s(x) \equiv x \pmod{f(x)}$. Therefore,

$$\bar{Q}(x) \equiv x^6 + x^4 + x^3 + x^2 + 1 \pmod{f(x)^2}.$$

A convenient choice of basis is again by taking $A(x)$ to be $x^4, \dots, x^8$ and set $B(x) \equiv A(x)\bar{Q}(x) \pmod{f(x)^2}$; concretely:

$$\begin{aligned} h_6 &= \frac{x^4y + x^8 + x^7 + x^4 + 1}{f(x)^2}, & h_7 &= \frac{x^5y + x^9 + x^8 + x^5 + x}{f(x)^2}, \\ h_8 &= \frac{x^6y + x^9 + x^2 + 1}{f(x)^2}, & h_9 &= \frac{x^7y + x^6 + x^3 + x + 1}{f(x)^2}, & h_{10} &= \frac{x^8y + x^7 + x^4 + x^2 + x}{f(x)^2}. \end{aligned}$$

Now, proceeding to the evaluation-then-interpolation part. We need to determine the matrix G from Eq. (158). Since we are going to use the place P_{01} twice, we need to slightly modify the $\mathbb{F}_2$ -linear

map φ .

$$\begin{aligned}\varphi : \mathcal{L}(2D) &\rightarrow (\mathbb{F}_{2^{\deg(P_{01})}})^{\times 2} \times \mathbb{F}_{2^{\deg(P_{00})}} \times \mathbb{F}_{2^{\deg(P_{\infty})}} \times \mathbb{F}_{2^{\deg(R_1)}} \times \mathbb{F}_{2^{\deg(R_2)}} \times \mathbb{F}_{2^{\deg(R_3)}} \\ &= \mathbb{F}_2^2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_4 \times \mathbb{F}_4 \times \mathbb{F}_4 \cong \mathbb{F}_2^{10}\end{aligned}\quad (160)$$

$$f \mapsto (\varphi_{P_{01}}(f), \text{ev}_{P_{00}}(f), \text{ev}_{P_{\infty}}(f), \text{ev}_{R_1}(f), \text{ev}_{R_2}(f), \text{ev}_{R_3}(f))^\top, \quad (161)$$

where $\varphi_{P_{01}}(f) := (\alpha_0, \alpha_1)$ which are the first two coefficients in expanding f in a local parameter t , i.e., $f = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \dots$, $\alpha_i \in \mathbb{F}_{2^{\deg(P_{01})}}$, at the place P_{01} .

A valid choice of local parameter at $P_{01} = (x, y + 1)$ is $t = x$ because $F_y = 1 \neq 0$. Moreover, to multiply $\varphi_{P_{01}}(f_a) = (\alpha_0, \alpha_1)$ and $\varphi_{P_{01}}(f_b) = (\beta_0, \beta_1)$, locally,

$$f_a = \alpha_0 + \alpha_1 x + O(x^2), \quad f_b = \beta_0 + \beta_1 x + O(x^2).$$

So $f_a f_b = \alpha_0 \beta_0 + (\alpha_0 \beta_1 + \alpha_1 \beta_0)x + O(x^2)$. Calculating $\varphi_{P_{01}} = (\alpha_0 \beta_0, \alpha_0 \beta_1 + \alpha_1 \beta_0)$ requires three CCZs, instead of four; this is an elementary example of Karatsuba multiplication.

For the degree-two places, multiplication in $\mathbb{F}_4 = \{0, 1, \omega, \omega^2\}$ (where $\omega^2 = \omega + 1$) takes also three CCZs only by Karatsuba:

$$\begin{aligned}(\alpha_0 + \alpha_1 \omega)(\beta_0 + \beta_1 \omega) &= (\alpha_0 \beta_0 + \alpha_1 \beta_1) + (\alpha_1 \beta_0 + \alpha_0 \beta_1 + \alpha_1 \beta_1) \omega \\ \gamma_1 &= \alpha_0 \beta_0, \quad \gamma_2 = \alpha_1 \beta_1, \quad \gamma_3 = (\alpha_0 + \alpha_1)(\beta_0 + \beta_1),\end{aligned}$$

and $\varphi_{R_i}(f_a f_b) = (\gamma_1 + \gamma_2, \gamma_1 + \gamma_3)$.

Multiplication in $\mathbb{F}_2$ takes one CCZ, so in total, $3 + 1 + 1 + 3 \times 3 = 14$ CCZs are used.

Finally, one can verify that the G matrix (cf. Eq. 158) below is invertible.

j	$\varphi_{P_{01}}(h_j)$	$h_j(P_{00})$	$h_j(P_{\infty})$	$h_j(R_1)$	$h_j(R_2)$	$h_j(R_3)$
1	(1, 0)	1	1	1	1	1
2	(0, 1)	1	0	ω	1	ω^2
3	(0, 0)	0	0	ω	ω	1
4	(1, 0)	1	0	ω^2	ω	ω^2
5	(1, 1)	1	0	ω	ω	0
6	(1, 0)	1	0	ω	ω^2	ω
7	(0, 1)	0	0	ω	1	ω^2
8	(1, 0)	1	0	ω^2	ω^2	0
9	(1, 1)	1	0	ω	0	1
10	(0, 1)	0	0	ω	0	ω

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

Appendix C Building Distillation Circuits

The majority of this manuscript is dedicated to leveraging the Galois qudit formalism to find novel distillation protocols. Since many of these protocols are quite small, we discuss concrete circuit realizations within the same formalism in order to aid practical implementation. In particular, we will discuss how qubit distillation protocols may be realized on fewer qubits than one naively expects using the transformation of [Lit19a], and we will show how the same transformation applies to our Galois qudit protocols.

C.1 Distillation protocols and Pauli-based computation (PBC)

Specializing what was discussed in Section 2.1, a third-level diagonal phase gate on k qubits can be represented by a degree at most three phase polynomial modulo 8. To be more concrete, $\text{CCZ}_{1,2,3}$, $\text{CS}_{1,2}$, T_1 correspond to $4x_1x_3x_3$, $2x_1x_2$, and $x_1 \bmod 8$, respectively, and similarly for CZ , S , Z . Such a phase polynomial can be rewritten as a weighted sum of linear Boolean functions

$$P(\mathbf{x}) = \sum_{\mathbf{u} \in \mathbb{F}_2^k \setminus \{0\}} a_{\mathbf{u}}(u_1 x_1 \oplus u_2 x_2 \oplus \dots \oplus u_k x_k), \quad (162)$$

where the coefficients $a_{\mathbf{u}}$ are integers modulo 8. Here, note that $\oplus$ denotes the summation of integers modulo 2, whereas $\sum$ denotes regular integer addition unless otherwise stated.

As an example, the CCZ gate

$$4x_1x_2x_3 \equiv x_1 + x_2 + x_3 + 7(x_1 \oplus x_2) + 7(x_1 \oplus x_3) + 7(x_2 \oplus x_3) + (x_1 \oplus x_2 \oplus x_3),$$

has coefficients $(a_{001}, a_{010}, a_{011}, \dots, a_{111}) = (1, 1, 7, 1, 7, 7, 1)$, where $+$ denotes regular integer addition unless otherwise stated.

In this form, an odd coefficient $a_{\mathbf{u}}$ corresponds to a T gate (up to S and Z) on the CNOT-

transformed bit $(u_1x_1 \oplus u_2x_3 \oplus \dots \oplus u_kx_k) = \bigoplus_{i \in \text{supp}(\mathbf{u})} x_i$ ⁴⁹, and we conclude from this identity that the CCZ gate may be unitarily synthesized from 7 T gates, and Cliffords.

In certain cases, the number of odd coefficients can be reduced by adding some phase polynomial identities, and such identities⁵⁰ are codewords of Reed-Muller codes $\text{RM}(k-4, k)$ punctured at the all-zero coordinate [AM19]. Minimizing the ancilla-free T count for such phase polynomial gates is therefore equivalent to decoding punctured Reed-Muller codes: a notoriously hard problem computationally.

Nevertheless, given Eq. 162 for a third-level diagonal phase gate, [CH17] observes that its non-Clifford part is equivalently represented by a so-called binary gate-synthesis matrix A which has k rows: the vector $\mathbf{u}$ occurs as a column precisely when $a_{\mathbf{u}}$ is odd. A can be imagined as the logical part of a binary X -generator matrix; see Definition 3.1. Associating binary variables $x_1, \dots, x_k$ to the rows of A , one may perform generalized triorthogonality calculations — see Sec. 4.1 — and go on to construct distillation protocols. In the case that the k -qubit third-level diagonal phase gate is CCZ-only, one only needs at most two qubits to convert this gate synthesis matrix into a $d = 2$ distillation protocol.

Lemma C.1. [CH17] *Let $A \in \mathbb{F}_2^{k \times t}$ be a full-rank gate-synthesis matrix for a CCZ-only unitary U . Define*

$$\delta = \begin{cases} 0, & t \text{ even and } \mathbf{1}_t^\top \notin \text{rowspan}(A), \\ 2, & t \text{ even and } \mathbf{1}_t^\top \in \text{rowspan}(A), \\ 1, & t \text{ odd,} \end{cases} \quad G = \begin{pmatrix} A & \mathbf{0}_{k \times \delta} \\ \mathbf{1}_{t+\delta}^\top \end{pmatrix}. \quad (163)$$

Then G defines a $[[t + \delta, k, d \geq 2]]$ code on which transversal T realizes U up to Clifford corrections.

The following is an example of the $\delta = 1$ case.

Example C.2 ($12T \rightarrow \text{TOF}\#$ at $d = 2$ [CH17]). *Consider a TOF# gate $\text{CCZ}_{123}\text{CCZ}_{145}$ whose phase polynomial is*

$$4x_1x_2x_3 + 4x_1x_4x_5 \equiv x_1 + x_5 + (x_2 \oplus x_4) + (x_3 \oplus x_4) - (x_1 \oplus x_5) - (x_4 \oplus x_5) - (x_1 \oplus x_2 \oplus x_4) \\ - (x_1 \oplus x_3 \oplus x_4) - (x_2 \oplus x_3 \oplus x_4) + (x_1 \oplus x_2 \oplus x_3 \oplus x_4) + (x_1 \oplus x_4 \oplus x_5) \pmod{8}.$$

Notice that, in the above, the right-hand side contains only terms with odd integer coefficients, namely 1 and 7. This leads us to the following generalized triorthogonal matrix, but note that in the general case when passing to the matrix, one ignores the even integer coefficient terms.

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ \hline 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{matrix} \quad (164)$$

For example, the third column of the logical part $(0, 1, 0, 1, 0)^T$ corresponds to $(x_2 \oplus x_4)$ in the phase polynomial above. Sometimes, one can avoid Clifford corrections by applying $T^{\Gamma_1} \otimes \dots \otimes T^{\Gamma_{12}}$ to the physical qubits. In this example, solving a linear equation in $\mathbf{\Gamma}$ over $\mathbb{Z}_8$ [Koh+26, App. H.2] shows that $\mathbf{\Gamma} = (1, 1, 1, 1, 7, 7, 7, 7, 7, 1, 1, 7)$ does the job.

C.1.1 Galois qudit gates as measurements

We will now proceed to the discussion of running our distillation protocols as circuits based on Pauli-based computation (PBC). We will show how all of the most important tricks and techniques for running and compressing qubit magic state distillation protocols apply naturally to the protocols based on codes over binary extension fields.

Many quantum error-corrected architectures leverage Pauli-based computation [BSS16] as a natural gate-set for logical operations [Lit19a; He+25a; Yod+25; Xu+25a]. Since for any n -qudit vector

⁴⁹[Lit19b] calls this a $\pi/8$ Pauli-product rotation $\exp\left[i\pi\left(\bigoplus_{i \in \text{supp}(\mathbf{u})} x_i\right)/8\right]$.

⁵⁰A phase polynomial identity in the form of Eq. (162) corresponds to a Reed-Muller codeword where the $\mathbf{u}$ coordinate is set to $(a_{\mathbf{u}} \bmod 2)$. [Lit19b, Fig. 2] provides an example of phase polynomial identity.

of field elements $u \in \mathbb{F}_{2^s}^n$ there is a natural definition of $X(u)$ and $Z(u)$ as nm -qubit Pauli matrices, generalization of this Galois qudit formalism to Pauli based computation initially looks trivial. However, especially when considering magic state distillation protocol implementation, we often care about measurements of the qubit operators corresponding to $X(\alpha u)$, for all $\alpha \in \mathbb{F}_{2^s}$, as a collection. The result of all these measurements is a map $\mathcal{M} : \mathbb{F}_{2^s} \rightarrow \mathbb{F}_2$, where $\mathcal{M}(\alpha)$ is the measurement outcome of $X(\alpha u)$. This map is clearly linear, motivating the following definition [Wil26].

Definition C.3. For some $u_x, u_z \in \mathbb{F}_{2^s}^n$, we call the n -qudit **measurement generated by** $i^{\text{tr}(u_x \cdot u_z)} X(u_x) Z(u_z)$ the measurement of all qubit operators corresponding to $i^{\text{tr}(u_x \cdot u_z)} X(\alpha u_x) Z(\alpha u_z)$ for all $\alpha \in \mathbb{F}_{2^s}$. We say the measurement **has outcome** $\gamma \in \mathbb{F}_{2^s}$ if the outcome of the qubit measurement $i^{\text{tr}(u_x \cdot u_z)} X(\alpha u_x) Z(\alpha u_z)$ is equal to $\text{tr}(\alpha \gamma)$. Given an n -qudit state $|\psi\rangle$, we say that $|\psi\rangle$ has eigenvalue $\gamma \in \mathbb{F}_{2^s}$ under $i^{\text{tr}(u_x \cdot u_z)} X(u_x) Z(u_z)$ if the outcome of the measurement generated by $i^{\text{tr}(u_x \cdot u_z)} X(u_x) Z(u_z)$ on $|\psi\rangle$ has the outcome γ with certainty.

While [BSS16] considered only magic state preparation and Pauli measurement, future works [Lit19a; Yod+25] have extended Pauli-based computation to also include gates of the form $\exp(i\phi Z) = \sum_{a \in \{0,1\}} \exp(i\phi(-1)^a) |a\rangle \langle a|$, which naturally generalizes to other Pauli bases and multiple qubits. As an example, we now work towards expressing the gate $U_7^\beta = \sum_{\eta \in \mathbb{F}_{2^s}} \exp(i\pi \text{tr}(\beta \eta^7)) |\eta\rangle \langle \eta|$ in a similar manner. The same procedure will allow us to execute Galois qudit gates in the PBC framework, which will go on to be used for running our distillation protocols based on binary extension fields in a small amount of space (on a smaller number of logical Galois qudits than one naively expects).

First, consider the following method for realizing $|\psi\rangle \rightarrow \exp(i\phi \frac{1+P}{2}) |\psi\rangle$, for some n -qubit Pauli P :

1. Initialize a qubit ancilla in the $|+\rangle$ state.
2. Apply the joint measurement $Z \otimes P$ to $|+\rangle \otimes |\psi\rangle$ to get outcome $\alpha \in \{0,1\}$.
3. Apply the map $|a\rangle \rightarrow e^{i\phi(a \oplus \alpha)} |a\rangle$ to the ancilla.
4. Measure the ancilla in the X basis to obtain outcome $\gamma \in \{0,1\}$.
5. Apply P to the $|\psi\rangle$ register when $\gamma = 1$.

Here it was natural to consider $\exp(i\phi \frac{1+P}{2})$ rather than $\exp(i\phi P)$ since the eigenvalues of $\frac{1+P}{2}$ live in $\mathbb{F}_2$, corresponding to the computational basis states of a qubit. This motivates the notation $\hat{P} = \frac{1+P}{2}$ as a Hermitian operator on n qubits with eigenvalues in $\mathbb{F}_2$. We generalize to n -qudit operators $\hat{P}$ for $P = i^{\text{tr}(u_x \cdot u_z)} X(u_x) Z(u_z)$ with eigenvalues in $\mathbb{F}_{2^s}$, by implementing an operation $\exp(i\pi \text{tr}(\hat{P}^7))$:

1. Initialize a qudit ancilla in the $|+\rangle$ state.
2. Apply the measurement generated by $Z \otimes P$ to $|+\rangle \otimes |\psi\rangle$ to get outcome $\alpha \in \mathbb{F}_{2^s}$.
3. Apply the map $|\eta\rangle \rightarrow \exp(i\pi \text{tr}((\eta - \alpha)^7)) |\eta\rangle$ to the ancilla.
4. Measure the ancilla in the X basis to obtain outcome $\gamma \in \mathbb{F}_{2^s}$.
5. If $P = i^{\text{tr}(u_x \cdot u_z)} X(u_x) Z(u_z)$, apply $i^{\text{tr}(\gamma^2 u_x \cdot u_z)} X(\gamma u_x) Z(\gamma u_z)$ to the $|\psi\rangle$ register.

As a specialization, we see that the gate U_7^β can be written as $\exp(i\pi \text{tr}(\beta \hat{Z}(1)^7))$, and is generalized to a larger family of operators $U_7^\beta[g] := \exp(i\pi \text{tr}(\beta \hat{Z}(g)^7))$ for $g \in \mathbb{F}_{2^s}^n$, which is the set of all conjugations of U_7^β by multi-qudit CNOT circuits. In particular:

$$U_7^\beta[g] = \sum_{u \in \mathbb{F}_{2^s}^n} \exp \left(i\pi \text{tr} \left(\beta \left(\sum_{i=1}^n g_a u_a \right)^7 \right) \right) |u\rangle \langle u|. \quad (165)$$

C.1.2 Realization of Distillation Protocols on Fewer Qudits

Suppose a matrix $G \in \mathbb{F}_q^{m \times n}$ (say, a matrix satisfying some triorthogonality property) is an X -generator matrix for a quantum error-correcting code (see Definition 3.1) with k logical qudits and a transversal U_7^β gate (for example). Traditionally, a magic state distillation protocol based on the code takes the following form:

1. Initialize n qudits into a logical $|+\rangle^{\otimes k}$ state within the codespace,
2. Apply U_7^β on all n qudits,
3. Measure the X stabilizers on the code and either post-select or decode,
4. Unencode from the code to obtain the final distilled $(U_7^\beta |+\rangle)^{\otimes k}$.

However, for the case of $\mathbb{F}_2$, it is known that there exists a quantum circuit supported on only m qubits (where m is the number of rows in the X -generator matrix) that realizes the protocol [Lit19b]. In this section we show that the same idea works for qudits, and given a matrix $G \in \mathbb{F}_q^{m \times n}$, the corresponding distillation protocol may be realized on m Galois qudits.

Proposition C.4. *Suppose $G \in \mathbb{F}_q^{m \times n}$ is an X -generator matrix for a code with k logical qudits. Consider the following quantum circuit:*

1. Initialize m qudits in the state $|+\rangle^{\otimes m}$.
2. For $i = 1$ through n , apply faulty twirled $U_7^\beta[g_i]$ gates,⁵¹ where g_i is the i 'th column of G .
3. Measure qudits $k + 1$ through m in the X basis.

Then, the measurement results correspond to the X stabilizers of the CSS code, and qudits 1 through k contain the final distilled $(U_7^\beta |+\rangle)^{\otimes k}$.

This result enables a realization of magic state distillation on fewer qubits, but perhaps more importantly enables a straightforward interpretation of G as a quantum circuit. The first k rows denote the output qudits, and the latter $m - k$ rows denote X -check qudits. Since the input magic states have been twirled, we can assume that only Z errors are present, so only the X checks are necessary. The columns of G denote multi-qudit Pauli degrees of freedom into which U_7^β are injected via $U_7^\beta[g_i]$.

Proof. Recall that the error correction code defined by G is given by $\text{CSS}(X, \mathcal{G}_0; Z, \mathcal{G}^\perp)$, where $\mathcal{G}_0$ is the $\mathbb{F}_q$ -span of the latter $m - k$ rows of G , and $\mathcal{G}$ is the $\mathbb{F}_q$ -span of all rows of G . Consider the code defined *just* by the Z checks, giving $m - k$ additional logical qudits since the X stabilizers are ignored. This code is spanned by vectors

$$|\bar{u}\rangle = \left| \sum_{a=1}^m u_a g^a \right\rangle \quad (166)$$

for $u \in \mathbb{F}_q^m$, since the Z checks of the form $Z(v)$ for $v \in \mathcal{G}^\perp$ satisfy $\sum_i v_i \cdot u_a g_i^a = 0$, implying $Z(v)|\bar{u}\rangle = |\bar{u}\rangle$. Now, let $\mathcal{E} : \mathbb{C}^{\mathbb{F}_q^m} \rightarrow \mathbb{C}^{\mathbb{F}_q^n}$ be an encoding isometry acting as $\mathcal{E}|u\rangle = |\bar{u}\rangle$. (This isometry could be realized by padding with $n - m$ many $|0\rangle$ states and then running a CNOT circuit.)

The traditional form of the distillation circuit is then expressed as follows. Recall that, for $u' \in \mathbb{F}_q^k$, a computational basis for the full $\text{CSS}(X, \mathcal{G}_0; Z, \mathcal{G}^\perp)$ code can be written as

$$|\bar{u}'\rangle = \frac{1}{\sqrt{|\mathcal{G}'|}} \sum_{g \in \mathcal{G}_0} \left| \sum_{a=1}^k u'_a g^a + g \right\rangle, \quad (167)$$

⁵¹The notion of twirling for Galois qudits is again entirely analogous to the qubit case, see for example [BH12], and allows us to only have to worry about Z errors on the inputted magic states, and not X errors.

so the state $|+\rangle^{\otimes k}$ encoded into $\text{CSS}(X, \mathcal{G}_0; Z, \mathcal{G}^\perp)$ can be expressed as

$$\frac{1}{\sqrt{q^k}} \sum_{u' \in \mathbb{F}_q^k} |\bar{u}'\rangle = \frac{1}{\sqrt{q^k}} \sum_{u' \in \mathbb{F}_q^k} \frac{1}{\sqrt{|\mathcal{G}_0|}} \sum_{g \in \mathcal{G}_0} \left| \sum_{a=1}^k u'_a g^a + g \right\rangle \quad (168)$$

$$= \frac{1}{\sqrt{q^m}} \sum_{u \in \mathbb{F}_q^m} \left| \sum_{a=1}^m u_a g^a \right\rangle \quad (169)$$

$$= \frac{1}{\sqrt{q^m}} \sum_{u \in \mathbb{F}_q^m} |\bar{u}\rangle \quad (170)$$

$$= \mathcal{E} \frac{1}{\sqrt{q^m}} \sum_{u \in \mathbb{F}_q^m} |u\rangle \quad (171)$$

$$= \mathcal{E} |+\rangle^m. \quad (172)$$

Let $(U_7^\beta)_{[i]}$ denote a unitary on n qudits, acting as U_7^β on qudit i and as identity elsewhere. Then, we derive:

$$\mathcal{E}^\dagger (U_7^\beta)_{[i]} \mathcal{E} |u\rangle = \mathcal{E}^\dagger (U_7^\beta)_{[i]} |\bar{u}\rangle \quad (173)$$

$$= \mathcal{E}^\dagger (U_7^\beta)_{[i]} \left| \sum_{a=1}^m u_a g^a \right\rangle \quad (174)$$

$$= \mathcal{E}^\dagger \exp \left(i\pi \text{tr} \left(\beta \left[\sum_{a=1}^k u_a g_i^a \right]^7 \right) \right) |\bar{u}\rangle \quad (175)$$

$$= \exp \left(i\pi \text{tr} \left(\beta \left[\sum_{a=1}^k u_a g_i^a \right]^7 \right) \right) |u\rangle. \quad (176)$$

$$= U_7^\beta [g_i] |u\rangle. \quad (177)$$

Consequently:

$$\mathcal{E}^\dagger \bigotimes_{i=1}^n U_7^\beta \mathcal{E} = \prod_{i=1}^n \mathcal{E}^\dagger (U_7^\beta)_{[i]} \mathcal{E} \quad (178)$$

$$= \prod_{i=1}^n U_7^\beta [g_i]. \quad (179)$$

Putting these pieces together, we see how the state at the second step of the traditional circuit is given by:

$$\bigotimes_{i=1}^n U_7^\beta \cdot \mathcal{E} |+\rangle^m = \mathcal{E} \cdot \prod_{i=1}^n U_7^\beta [g_i] \cdot |+\rangle^m, \quad (180)$$

which is of course the state $\bigotimes_{a=1}^k U_7^\beta |+\rangle$ encoded into $\text{CSS}(X, \mathcal{G}_0; Z, \mathcal{G}^\perp)$.

Finally, consider an X stabilizer corresponding to an element $g \in \mathcal{G}_0$. There exists a $v \in \mathbb{F}_q^m$ with support only on entries v_{k+1} through v_m such that $g = \sum_{a=1}^m v_a g_i^a$. The stabilizer $X(g)$ has the action on an encoded state:

$$\mathcal{E}^\dagger X(g) \mathcal{E} |u\rangle = \mathcal{E}^\dagger X(g) \left| \sum_{a=1}^m u_a g^a \right\rangle \quad (181)$$

$$= \mathcal{E}^\dagger \left| \sum_{a=1}^m u_a g^a + g \right\rangle \quad (182)$$

$$= \mathcal{E}^\dagger \left| \sum_{a=1}^m (u + v)_a g^a \right\rangle \quad (183)$$

$$= |u + v\rangle \quad (184)$$

$$= X(v) |i\rangle. \quad (185)$$

As a result, for $a = k + 1$ through m , the X stabilizer corresponding to row g^a is equivalent to Pauli X on the a 'th qubit in the code space that is the image of $\mathcal{E}$. We can conclude that (1) for an arbitrary state $|\psi\rangle$ on m qudits, measurements of qubits $a = k + 1$ through m in the X basis are equivalent to measuring $X(g^a)$ on $\mathcal{E}|\psi\rangle$, and (2) for an m qudit state $|\psi\rangle$ such that $\mathcal{E}|\psi\rangle$ is in the codespace of $\text{CSS}(X, \mathcal{G}_0; Z, \mathcal{G}^\perp)$, qubits $k + 1$ through m of $|\psi\rangle$ are all in the $|+\rangle$ state, so the logical information must be on qubits 1 through k . $\square$

Remark C.5. Note that, for certain special distillation factories based on Galois qudits, the protocol may be run on even fewer than m qudits by using the qudit analogue of the magic state distillation compression technique from [Xu+26]. To use this compression technique, one needs the check rows of the matrix G (the latter $m - k$ rows) to be writeable, after some row operations, into a form where some check rows have disjoint support. Using row operations over $\mathbb{F}_q$, rather than $\mathbb{F}_2$, the same conclusions hold for our Galois qudit protocols. For example, considering Construction 4.8, with matrix

$$\left(\begin{array}{cccccccc|cccccccc} 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & 1 & 0 \\ \hline 1 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha^2 & \alpha^4 & \alpha^6 & \alpha^8 & \alpha^3 & \alpha^5 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{array} \right),$$

after applying the above methodology, the protocol may be run on 5 qudits. However, because the first and second check rows are disjoint, one qudit used for the checks may be re-used, and in fact the protocol may be run on only 4 qudits.

Appendix D Time, Space, and Error Calculations

In this appendix, we provide detail on how we benchmark our various concatenated schemes, both their performance metrics — time rate, spatial footprint, volumetric rate — and their per-state average errors. The corresponding results were shown in Section 1.3.2.

D.1 Time and Space Calculations

To motivate our time and space calculations, let us begin by explaining our implementation approach for a given concatenated chain of protocols. We use, as a running example, the $(64T \rightarrow 2CCZ \text{ [HH18a]}) \cdot (8CCZ \rightarrow 2CCZ)$ concatenation. First, for the overall setup, note that magic state distillation protocols like $64T \rightarrow 2CCZ$ that have multiple outputs usually have correlations on the errors of their outputs. Because of this, in order to ensure that the concatenation has the full distance (in this case 8), if a protocol with multiple outputs is used in a chain, we must use multiple of any protocols following it (this does not affect situations where the given protocol is the last protocol in the chain). In the present example, we must run two of the $8CCZ \rightarrow 2CCZ$ protocols. In a three-level situation, where one run of the first-level protocol produces k_A outputs, and one run of the second-level protocol produces k_B outputs, we must run the second-level protocol k_A times, and the third-level protocol $k_A k_B$ times.

In terms of the actual setup in the quantum computer, our $(64T \rightarrow 2CCZ \text{ [HH18a]}) \cdot (8CCZ \rightarrow 2CCZ)$ has some logical qubits reserved for running one copy of the $64T \rightarrow 2CCZ$ protocol repeatedly, and logical qubits are reserved for two copies of the $8CCZ \rightarrow 2CCZ$. The spatial footprint of the concatenated scheme is therefore taken to be the spatial footprint of $64T \rightarrow 2CCZ$, plus twice the spatial footprint of $8CCZ \rightarrow 2CCZ$. We will come on to how the spatial footprint of an individual level, like $64T \rightarrow 2CCZ$ is calculated, momentarily. Let us note now that this methodology is used for all concatenated schemes. That is, there is one copy of the top-level factory that is run serially, feeding possibly multiple copies of lower-level factories. In particular, top-level states are injected serially. This is one implementation choice that we fix in this work for the sake of simplicity: others are possible.

Let us now consider the spatial footprint of an individual level, like $64T \rightarrow 2CCZ$ or $8CCZ \rightarrow 2CCZ$. For most protocols, this is simply the number of rows in the binary matrix describing the protocol (the number of qubit X logicals and X stabilizer generators). For protocols like $64T \rightarrow 2CCZ$ that are naturally defined over qubits, it is well-known that the protocol may be run on this many logical

qubits [Lit19a]. In Appendix C, we showed that the same ideas naturally transfer to our case of Galois qudit codes. That is, given a matrix $G \in \mathbb{F}_{2^s}^{m \times n}$ describing the distillation protocol, the protocol may be run on a number of logical Galois qudits equaling the number of rows, m , in the matrix. When run at the level of qubits, this means its spatial footprint is $s \cdot m$. As an example, the $8\text{CCZ} \rightarrow 2\text{CCZ}$ protocol is described by a matrix over $\mathbb{F}_8$ with 3 rows — see Construction 4.3 — and therefore the spatial footprint of this protocol is 9.

There is one more consideration when calculating the spatial footprint: that of compression. Certain protocols may be run on even fewer logical qubits than the number of rows in their binary matrices [Xu+26]. For example, the matrix for the Haah-Hastings $64T \rightarrow 2\text{CCZ}$ protocol has 17 rows, but by writing its X stabilizer generators in a particular way, and ordering its columns appropriately, it becomes clear that it may be run on only 10.⁵² We briefly discussed in Appendix C how the compression technique may also be applied at the level of Galois qudits to the matrices of $\mathbb{F}_q$ describing Galois qudit codes, thus allowing for some compression. Improvements in the spatial footprint appear small, however, and so we forgo this for simplicity. The only compressed footprint that will appear in our competitive protocols will be the $64T \rightarrow 2\text{CCZ}$ Haah-Hastings protocol mentioned. Now, when this Haah-Hastings protocol appears at the first level of a concatenated scheme, we will always use its compressed footprint, that is, we always imagine the factory being run in its compressed form. Accordingly, for example, the spatial footprint of the $(64T \rightarrow 2\text{CCZ}) \cdot (8\text{CCZ} \rightarrow 2\text{CCZ})$ factory is taken to be $10 + 2 \times 9 = 28$. On the other hand, if the Haah-Hastings protocol appears at any lower level of a concatenated scheme like, for example, $(14T \rightarrow 2T) \cdot (64T \rightarrow 2\text{CCZ})$, we may or may not consider each $64T \rightarrow 2\text{CCZ}$ to be compressed. We will consider multiple setups where the protocol is run in its compressed, and its uncompressed form; we will return to this subtlety when we elucidate the error calculations in Section D.2. As a final note on spatial footprints, note that the (non-distillation) synthesis conversions can be run on 3 logical qubits in the case of $3T \rightarrow \text{CS}$ [HC17], and 4 logical qubits in the case of $2\text{CS} \rightarrow \text{CCZ}$ [Bev+20].

We now turn to the time calculations. We aim to calculate the “time rate” of a concatenated scheme. As mentioned in Section 1.3.2, the time rate is defined to be the number of average retained output states per unit time for a given scheme. We also recall that the unit of time is chosen to be one $|T\rangle$ state injection, since our setup requires the injection of top-level states one-by-one, and we ignore the time taken to perform Cliffords, since these are architecture-specific considerations, and we wish to be as architecture-agnostic as possible. We also recall that the time taken to inject one $|\text{CS}\rangle$ state is taken to be two units of time. Ignoring post-selection, the time-rate of a given post-selection scheme is very easily calculated, since all output states are retained. For example, one run of the $(64T \rightarrow 2\text{CCZ}) \cdot (8\text{CCZ} \rightarrow 2\text{CCZ})$ concatenation injects $64 \times 8 = 512$ $|T\rangle$ states sequentially,⁵³ as is therefore imagined as taking 512 units of time, when ignoring post-selection. One run of this concatenated scheme produces $2 \times 2 = 4$ $|\text{CCZ}\rangle$ states and so, ignoring post-selection, the time rate of this concatenated scheme is $4/512 = 1/128 \approx 0.0078$. As another example that consider $|\text{CS}\rangle$ state injection, consider a scheme $(16\text{CS} \rightarrow 2\text{CS}) \cdot (6\text{CS} \rightarrow 2\text{CS}) \cdot (2\text{CS} \rightarrow 1\text{CCZ})$, where we recall that $2\text{CS} \rightarrow 1\text{CCZ}$ is simply a synthesis scheme, not distillation. In this case, to overcome the problem of correlated errors in the top factory, two copies of the second level $6\text{CS} \rightarrow 2\text{CS}$ distillation scheme are run in parallel. Each of the two outputs from each of the two second-level factories are fed to separate $2\text{CS} \rightarrow 1\text{CCZ}$ protocols, meaning that four of those bottom-level protocols are run in parallel.⁵⁴ The spatial footprint of this protocol is thus taken to be $10 + 2 \times 6 + 4 \times 4 = 38$. For its time rate, one run of this protocol requires the top-level $16\text{CS} \rightarrow 2\text{CS}$ protocol to be run 12 times serially, so that

⁵²It also turns out that our novel $64T \rightarrow 2\text{CCZ}$ ($d = 4$) protocol — see Construction 5.10 — with the smaller any-error coefficient can be compressed from 15 to 14 qubits. Again, it turns out that this protocol is uncompetitive relative to the corresponding Haah-Hastings protocol for the main distillation tasks that we benchmark, and so we need not consider this.

⁵³Recall that, because the output states of the top-level factory have correlated errors, we must run two of the $8\text{CCZ} \rightarrow 2\text{CCZ}$ lower-level factory. The top-level factory must therefore be run 64 times in total to fill out those factories inputs.

⁵⁴Strictly speaking, because this bottom-level protocol is just synthesis, not distillation, one does not have to worry about correlated errors, and so one could run only two of these in parallel in this case. However, for simplicity in our calculations, we treat these synthesis protocols in the same way as the distillation protocols: separating the outputs of former levels into multiple copies of them.

the two middle $6\text{CS} \rightarrow 2\text{CS}$ protocols may each be run twice, ultimately outputting 4 distilled $|\text{CCZ}\rangle$ states. Because the injection of one $|\text{CS}\rangle$ state is deemed to take two units of time, the time-rate of this protocol, ignoring post-selection, is taken to be $\frac{4}{2 \cdot (16 \cdot 12)} = \frac{1}{96} \approx 0.0104$. More generally, the time-rate of a concatenated scheme, ignoring post-selection, may be easily calculated by multiplying the number of outputs at each level, multiplying the number of inputs at each level, dividing the former by the latter, and further dividing by two if the injection is that of $|\text{CS}\rangle$ states.

Taking post-selection into account makes little difference to the time rates at the error rates we care about ($p = 10^{-3}$ and 10^{-6}), for competitive protocols. Nevertheless, let us consider how the time rate may be calculated to first order in p . Higher order terms may be found via similar reasoning; we find via extensive testing that these have no meaningful effect for our competitive protocols, and at our values of p . To find the first order correction to the time rate, we note that retries of the protocol may be necessary in a concatenated chain because of error detections in any level. However, when considering the first order corrections to the time rate, we need only consider the error detections in the first distillation protocols of the chain, since lower levels detect errors with probabilities $\Omega(p^2)$. Therefore, to obtain the first-order correction to the time rate, we may multiply the time rate ignoring post-selection by the success probability of the first distillation protocol in the chain. Going back to our example $(64T \rightarrow 2\text{CCZ}) \cdot (8\text{CCZ} \rightarrow 2\text{CCZ})$, the success probability of one run of the $64T \rightarrow 2\text{CCZ}$ protocol, to first order in p , is $1 - 64p$. As such, the time rate, to first order in p , of this concatenated scheme is $\frac{1}{128}(1 - 64p) = \frac{1}{128} - \frac{1}{2}p$.

We consider two further examples now for completeness. Suppose that we are injecting $|\text{CS}\rangle$ states, and our top-level protocol is $4\text{CS} \rightarrow 1\text{CS}$. We consider a uniform error model on injected $|\text{CS}\rangle$ states, that is, the two qubits of the injected $|\text{CS}\rangle$ states suffer Z errors independently, each with probability p . With this, one may show that the $4\text{CS} \rightarrow 1\text{CS}$ protocol has success probability $1 - 8p + O(p^2)$.⁵⁵ The first-order time rate of any concatenated scheme with $4\text{CS} \rightarrow 1\text{CS}$ at the top level may be found by multiplying its no-post-selection time rate by $(1 - 8p)$. The final example we give is the situation in which the concatenated chain begins with a synthesis protocol; consider, for example, that the chain begins with $(3T \rightarrow \text{CS}) \cdot (4\text{CS} \rightarrow 1\text{CS})$. There is no post-selection in a synthesis protocol, and finding the first-order correction to the time-rate requires us to consider the error probability of the top *distillation* protocol in the chain. Given our usual independent Z errors in injected $|T\rangle$ states, with probability p , the error distribution of $3T \rightarrow \text{CS}$ turns out to be Z_1, Z_2 , and Z_1Z_2 , each with probability p , to leading order in p . With such CS states, the success probability of $4\text{CS} \rightarrow 1\text{CS}$ turns out to be $1 - 12p$, to leading order in p . Thus, the first-order time rate of any concatenated chain beginning with $(3T \rightarrow \text{CS}) \cdot (4\text{CS} \rightarrow 1\text{CS})$ may be found by multiplying the no-post-selection time rate by $1 - 12p$.

We conclude this subsection by noting that our protocols' volumetric rate may be found simply by dividing its time rate by its spatial footprint. Returning to our running example of $(64T \rightarrow 2\text{CCZ}) \cdot (8\text{CCZ} \rightarrow 2\text{CCZ})$, its volumetric rate, to first order in p , is $\frac{1}{128 \cdot 28} - \frac{1}{2 \cdot 28}p = \frac{1}{3584} - \frac{1}{56}p$.

D.2 Error Calculations

We now make clear how we calculate the error rate of our (concatenated) protocols, when they are successful. Since one run of our schemes usually gives multiple outputs, the natural error rate to consider when benchmarking these schemes in the context of a wider quantum computational architecture is the *per-state average error, conditioned on acceptance*. The best way to explain this initially is to consider the Haah-Hastings $64T \rightarrow 2\text{CCZ}$ protocol [HH18a]. The probability of any error in the outputs (conditioned on acceptance) is $2944p^4$, to leading order in p . Breaking this down, it turns out that the probability that the probability that the “first” output state, but not the “second”, has an error, is $576p^4$, and it happens that the probability that the second output state has an error, but not the first, is the same: $576p^4$. The probability that both have an error is $1792p^4$. We therefore find that

⁵⁵It is important to note here how we binarize the protocol. Throughout, we binarize our $\mathbb{F}_4$ codes in the self-dual normal basis $\{\omega, \omega^2\}$ — see Section 4.5 — and our $\mathbb{F}_8$ codes in the self-dual normal basis given in Equation (46). Given a self-dual basis $(\alpha_i)_{i=0}^{s-1}$ for $\mathbb{F}_{2^s}/\mathbb{F}_2$, the matrix over $\mathbb{F}_{2^s}$ describing the distillation protocol may be binarized by replacing each entry of the matrix, γ , with the $s \times s$ binary matrix which has entries $(\text{tr}(\gamma\alpha_i\alpha_j))$, where i and j run from 0 to $s - 1$.

the probability that the first state has an error is $2368p^4$, which happens to also be the probability that the second state has an error. Therefore, the per-state average error, conditioned on acceptance, is $2368p^4$, to leading order in p . Note that, by multiplying the per-state average error by the number of outputs, one obtains an upper bound on the probability of any error ($2944p^4$ in this case), but this bound is typically very loose, as one can see here, because the errors on outputted states are usually highly correlated.

There is an additional important point to make on this front, in the context of how we run our concatenated schemes. We adopt what one might call a *global post-selection policy* for our concatenated protocols, which means the following. Suppose, as is often the case, that the first distillation protocol in the chain produces $k_A > 1$ outputs. Then, to overcome the problem of correlated errors, we run k_A copies of the second-level factory in parallel. If one copy of the second-level factory has n_B inputs, we run the first-level factory n_B times, such that the second-level factories may be completed. After completing the second-level factories, there can be a subtlety. Suppose that some of the second-level factories accept, but some reject. In this case, our stricter *global post-selection policy* says that we should throw away the outputs of *all* second-level factories. This is in contrast to a weaker, “local” post-selection policy, which would call for one to only reject the outputs of the second-level factories that rejected. Our choice of global post-selection has a significant effect on the error rates of accepted states (by multiple orders of magnitude in some cases).

Let us now gain some intuition for why global post-selection improves error rates over local post-selection with an example, namely, the $(64T \rightarrow 2CCZ) \cdot (8CCZ \rightarrow 2CCZ)$ concatenation, where two of the second-level factory are run in parallel. Recalling the matrix for the $8CCZ \rightarrow 2CCZ$ protocol, see Construction 4.3, we see that its only check, at the level of Galois qudits, is an all-ones row. This means that it is a distance 2 protocol, and at leading order, one run of the $8CCZ \rightarrow 2CCZ$ protocol produces errorful outputs when 2 of its 8 inputs have an error, *and they are the same error*. For example, suppose that 2 of the 8 inputs to one run of this protocol have errors $e_1, e_2 \in \mathbb{F}_2^3$: $e_1, e_2 \neq 0$. There will be an error on the outputs exactly when $e_1 \neq e_2$. Thus, when running the $(64T \rightarrow 2CCZ) \cdot (8CCZ \rightarrow 2CCZ)$ concatenated scheme, with our global post-selection policy, the leading-order way that this can produce accepted outputs with errors is that some 2 out of the 8 runs of the top-level $64T \rightarrow 2CCZ$ protocol produce outputs with errors $(e_1 | f_1) \in \mathbb{F}_2^3 \oplus \mathbb{F}_2^3$ and $(e_2 | f_2) \in \mathbb{F}_2^3 \oplus \mathbb{F}_2^3$, where $(e_1 | f_1) \neq (0 | 0)$, and $(e_2 | f_2) \neq (0 | 0)$, and moreover $e_1 = e_2$, and $f_1 = f_2$. On the other hand, with local post-selection, accepting errorful outputs states only requires $e_1 = e_2$, or separately, $f_1 = f_2$. The result of this distinction is apparent in the combinatorics. Fixing 2 out of the 8 runs of the $64T \rightarrow 2CCZ$ factory to fail, considering the $2944^2 = 8,667,136$ ways they can fail, one finds that 2,056,192 of them result in accepted errorful outputs under local post-selection, where errors need only agree on one second-level factory, whereas only 385,024 of them result in accepted errorful outputs under global post-selection, where errors must agree on both second-level factories.⁵⁶

The above simple example shows a case where global post-selection provides a moderate improvement to the error rate over local post-selection, although in other cases the improvement is much more substantial. We therefore deem this choice to be worthwhile, despite the drawback that making the calculations under the global post-selection policy will be much harder than under the local post-selection policy. Indeed, with the definitions given, it is possible in-principle to directly calculate the error rate of any of our concatenated schemes by, for example, forming one large binary matrix describing the protocol, and enumerating error paths. However, for our largest protocols, this becomes computationally intractable, especially if we want to obtain higher-order terms.

To make our error rate computations tractable, and also to obtain the strongest possible performance, we invoke several twirling/averaging procedures, to simplify the problem. In the context of

⁵⁶A few caveats are in order for the exposition of this paragraph. The first is that we are only considering leading-order behaviour. The same intuitions hold for higher-order behaviour, but the calculations become difficult to neatly expose; we also find throughout the analysis of our competitive protocols that higher-order terms provide minimal corrections to the leading-order behaviour. The second caveat is that what we have considered in this last calculation is the any-error probability (because it is easier to explain), whereas what we actually want is the per-state average error probability, although the intuition for the latter is very much the same. The final caveat is that, very shortly, we will discuss how we will be performing certain state twirling/averaging in order to make our largest calculations tractable, and the resulting performance as strong as possible; what we have shown above does not consider this.

magic state distillation, the most well-known form of twirling is the Clifford-twirling of diagonal magic states [BK05; BH12], so that we may treat our magic states as only having Z errors, without X errors. We only work with diagonal magic states in this paper, and we assume throughout that they are twirled in this way. There are further, less standard, twirling/averaging techniques employed, to make the calculations more tractable, and to ensure the strongest possible performance, as follows:

1. For every (noisy) multi-qubit magic state (like $|\text{CS}\rangle$ and $|\text{CCZ}\rangle$), at every stage, we randomly permute the qubits of the multi-qubit magic state (2 or 3 in the case of $|\text{CS}\rangle$ and $|\text{CCZ}\rangle$, respectively). This has the effect of uniformizing the probabilities of errors over their weights. For example, for a (noisy) $|\text{CCZ}\rangle$ state, we do not need to keep track of the probabilities of 7 Z errors, but only the probabilities of 3 different error weights. We may call this randomization a *permutation twirl*.
2. Between different layers in a concatenation, we randomly route different magic states into different copies of the next-level factory. For example, in the $(20T \rightarrow 3\text{CCZ} [\text{CH17}]) \cdot (8\text{CCZ} \rightarrow 2\text{CCZ})$ concatenation, since the first level has three outputs, we run three copies of the second level $8\text{CCZ} \rightarrow 2\text{CCZ}$ factory. For each of the 8 successful runs of the $20T \rightarrow 3\text{CCZ}$, the 3 outputs are randomly routed to the three copies of the second-level factory. This means we only have to keep track of per-state average error probabilities between concatenation levels. In the case of multi-qubit magic states being passed between levels, like $|\text{CS}\rangle$ and $|\text{CCZ}\rangle$, we must only keep track of per-state average error probabilities of each weight.
3. For every copy of every distillation step in a concatenated scheme, that is not at the top level, we randomly permute the columns of the matrix describing that copy of the protocol. For example, in the case of a concatenation of $(14T \rightarrow 2T) \cdot (64T \rightarrow 2\text{CCZ})$, there are two copies of the second-level factory, and we consider randomly permuting the columns of the two $64T \rightarrow 2\text{CCZ}$ factories independently, averaging the resulting output error rates over this random choice. Note that, when such a protocol arises from an $\mathbb{F}_4$ or $\mathbb{F}_8$ -code (meaning it is used for $|\text{CS}\rangle$ or $|\text{CCZ}\rangle$ distillation, respectively), we randomly permute the columns of the matrix over $\mathbb{F}_{2^s}$, before binarizing the matrix (rather than binarizing and then randomizing), since each column of the $\mathbb{F}_{2^s}$ -matrix corresponds to one input multi-qubit magic state consumption, and it would not make sense to split the consumption of individual input states.

One may wonder what it actually means, in terms of actually running a concatenated protocol, to make such randomizations. Formally, what we are doing here, is considering the above randomizations, and we are going to calculate the (per-state average) output error rate *in expectation over the above random choices*. Having calculated a value (or at least an upper bound) on this expected value, it necessarily means that there is *some* choice of these randomizations that leads to the (per-state average) output error rate being at most this value.⁵⁷

Of the above three randomizations, it will likely be clear that the first two make the error calculations more tractable, with the intuition being that they allow us to keep track of less information. The third randomization, point 3, does not always make the calculation more tractable, but can dramatically improve performance, as we describe now with an example.

Example D.1. *We consider the leading order term of the per-state average error probability of the $(14T \rightarrow 2T [\text{BH12}]) \cdot (64T \rightarrow 2\text{CCZ} [\text{HH18a}])$ concatenation. First, it turns out that, at leading order, all accepted logical errors on the $14T \rightarrow 2T$ protocol affect both outputs. Indeed, with probability $7p^2 + O(p^3)$, one run of the top-level protocol fails, and it leaves Z errors on both output $|T\rangle$ states. Because of this, and because $|T\rangle$ states have only one qubit, the first two randomizations above have no effect. First, let us suppose that we did not apply randomization 3 above. We imagine that the two copies of the Haah-Hastings protocol are run with a common ordering of their columns. Any one copy of $64T \rightarrow 2\text{CCZ}$ fails at fourth order. It turns out that out of $\binom{64}{4} = 635376$ possible*

⁵⁷This is not a deep statement; it is simply the statement that if X is a random variable, and f is a function, then $\mathbb{E}_X[f(X)] \leq V \implies$ there is *some* choice for X giving $f(X)$ at most V ; this is exactly the same idea used in [Wil+26] to make the calculation of the error rate of a concatenated scheme more tractable.

size-4 errors, 3248 cause the protocol to accept. Of those, 304 cause no logical error on the two output $|CCZ\rangle$ states, 1152 cause exactly one of them to have an error, and 1792 cause both of them to be errorful. Thus, considering the full concatenate scheme, two of the four outputs have an error with probability $1152 \cdot (7p^2)^4 = 2765952p^8$, whereas all four outputs have an error with probability $1792 \cdot (7p^2)^4 = 4302592p^8$, at leading order. The leading order per-state average output of the whole concatenated scheme is therefore

$$\frac{2765952}{2}p^8 + 4302592p^8 = 5685568p^8. \quad (186)$$

Let us now instead suppose that the randomization of point 3 above has been applied. Some 4 out of the 64 runs of the top-level factory fail with probability $M \cdot (7p^2)^4$, where $M := \binom{64}{4} = 635376$. For a leading-order failure, each copy of the Haah-Hastings protocol sees error sets of size 4, independently distributed randomly amongst their inputs of size 4. The number of such sets that cause one of the four outputs to be errorful is $2 \times 304 \times 1152 = 700416$, the number that cause two of the four outputs to be errorful is $1152^2 + 2 \times 304 \times 1792 = 2416640$, the number that cause three to be errorful is $2 \times 1152 \times 1792 = 4128768$, and $1792^2 = 3211264$ cause all four to be errorful. Given that four of the first-level factories fail, the probability of exactly one, two, three, and four bad outputs are thus $\frac{700416}{M^2}$, $\frac{2416640}{M^2}$, $\frac{4128768}{M^2}$, and $\frac{3211264}{M^2}$, respectively. The leading-order per-state average error probability is thus

$$M \cdot (7p^2)^4 \cdot \left[\frac{1}{4} \frac{700416}{M^2} + \frac{2}{4} \frac{2416640}{M^2} + \frac{3}{4} \frac{4128768}{M^2} + \frac{4}{4} \frac{3211264}{M^2} \right] \approx 29064.25p^8. \quad (187)$$

We see that by randomizing the columns of the second-level factories, we have improved the error rate by almost a factor of 200.

There is a further consideration to be aware of, considering Example D.1 above. We recall from Section D.1 above that the $64T \rightarrow 2CCZ$ protocol can be *compressed* from a spatial footprint of 17 logical qubits to 10. However, doing so requires us to run the protocol in a very particular way, indeed, it requires a particular ordering of its columns. Two possibilities thus arise for this particular concatenated scheme; one can either consider a minimal space layout, where both copies of the second protocol are run in the compressed form, but since the two protocols share a common column ordering, the leading-order error rate will be as in Equation (186). On the other hand, a different layout, with higher spatial footprint, can achieve leading-order error rate at most as in Equation (187). Note that the only thing that mattered in our calculation was the *relative* ordering of the columns of the two protocols. We can, therefore, fix the ordering of the columns of one copy of this protocol to one allowing compression, and only randomize the other. Since the spatial footprint of $14T \rightarrow 2T$ is 5, the spatial footprint of the former layout is $5 + 2 \times 10 = 25$, whereas for the latter layout, it is $5 + 10 + 17 = 32$.

The impact of these layout considerations on our broader survey is relatively minor. Indeed, the only compressible protocol that we consider for our competitive schemes is this $64T \rightarrow 2CCZ$ protocol [HH18a]. In addition, these considerations only appear when such a protocol is not the top-level distillation scheme in a concatenated chain, since permuting the columns of a top-level factory has no effect. Given that we only consider concatenated schemes with distance ≤ 8 , the only concatenated chains where these considerations have an effect is the concatenation of a distance 2 $(3k + 8)T \rightarrow kT$ protocol [BH12] with the $64T \rightarrow 2CCZ$ factory. For each of these, we consider a low-footprint layout, where all second-level factories have a fixed column ordering, and all are compressed, and we consider a high-footprint layout, where one second-level factory is compressed, and the rest have randomized columns (one fact can always have fixed column ordering and compressed because it is only relative column ordering that matters), and so are assumed to each occupy 17 logical qubits. Future work may consider achieving a “best of both worlds” situation, where well-chosen column orderings on each factory minimizes the error rate, while allowing for some compression.

Let us provide a further example now.

Example D.2. We consider the leading order error probability of the $(2CS \rightarrow CCZ) \cdot (9CCZ \rightarrow 1CCZ)$ concatenation. Note that the first step is a known synthesis conversion [HC17; Bev+20], whereas the latter step is our novel protocol, see Construction 4.5. Beginning with the synthesis step, with independent Z errors on each of the 4 qubits of the 2 $|CS\rangle$ states, one checks that, at leading order, the resulting $|CCZ\rangle$ state has Z errors on it with weight 1, 2 and 3 with probability $w_1 := \frac{5}{2}p$, $w_2 := p$, and $w_3 := \frac{p}{2}$, respectively. The random permutation of the columns of the $\mathbb{F}_8$ matrix describing

$9CCZ \rightarrow 1CCZ$, as in the randomization 3, has no effect in this case, because there is only one second-level factory. Next, since the $9CCZ \rightarrow 1CCZ$ protocol is distance 3, at leading order, it requires 3 errorful inputs to fail. As always, the columns of this protocol are randomly permuted (meaning that the columns of the $\mathbb{F}_8$ -matrix are randomly permuted before binarization). It is convenient to define a function $K(a, b, c)$, where $a, b, c \in \{1, 2, 3\}$, and $a \leq b \leq c$. $K(a, b, c)$ is the probability, given that exactly three of the nine input $|CCZ\rangle$ states had errors, and those errors had weights a, b, c , what is the probability that the $9CCZ \rightarrow 1CCZ$ protocol suffers an undetectable logical error? Formally, we have

$$K(a, b, c) = \frac{1}{9 \cdot 8 \cdot 7 \binom{3}{a} \binom{3}{b} \binom{3}{c}} \sum_{\substack{i,j,k=1,\dots,9 \\ i,j,k \text{ pairwise distinct}}} \sum_{\substack{x,y,z \in \mathbb{F}_2^3 \\ |x|=a, |y|=b, |z|=c}} \mathbb{I} \left[\begin{matrix} E(x,y,z;i,j,k) \\ \text{is an undetectable logical error} \end{matrix} \right]. \quad (188)$$

Here, we write, for convenience, $E(x, y, z; i, j, k)$ for the input error to the binarized $9CCZ \rightarrow 1CCZ$ protocol which has x, y, z in the i, j, k 'th positions, respectively, and $\mathbb{I}[\cdot]$ is the indicator function.⁵⁸ The matrix is small, and we may easily calculate the values $K(a, b, c)$, as shown in Table 5. Next, we

(a, b, c)	$K(a, b, c)$	(a, b, c)	$K(a, b, c)$
(1, 1, 1)	1/63	(1, 1, 2)	17/567
(1, 1, 3)	1/189	(1, 2, 2)	1/189
(1, 2, 3)	1/27	(1, 3, 3)	1/63
(2, 2, 2)	23/567	(2, 2, 3)	1/189
(2, 3, 3)	1/63	(3, 3, 3)	1/21

Table 5

let P_{abc} be the number of distinct permutations of the numbers $a \leq b \leq c$; for example $P_{123} = 6$, but $P_{112} = 3$. Then, we have the (per-state average) leading order error rate of the concatenated protocol:

$$\binom{9}{3} \sum_{1 \leq a \leq b \leq c \leq 3} P_{abc} w_a w_b w_c K(a, b, c) = \frac{2888}{27} p^3. \quad (189)$$

D.3 Tables of Schemes

We now tabulate the statistics relevant to some of our competitive protocols in Tables 6, 7, 8, and 9. To keep the tables of reasonable size, we only tabulate the statistics of protocols that appear in some Pareto frontier over a range that covers the x -axis for at least half an order of magnitude. For each concatenated scheme, we show the protocol chain, the leading order term for the (per-state average) output error, as well as the actual value at physical input error rates 10^{-3} and 10^{-6} , the spatial footprint, and the time and volume rates. The values of output error rate are calculated using two-term expansions (leading order and second-leading order). Because of different amounts of post-selection, the time and volume rates do depend mildly on input physical error rate. We display values for input physical error rate 10^{-3} , but the values at 10^{-6} are typically only different by a few percent.

⁵⁸It is worth emphasizing that this function, and more generally the output error rate of the concatenated protocols involving the $\mathbb{F}_4$ and $\mathbb{F}_8$ codes, do depend in general on the way that the matrices are binarized. Throughout, we consider binarizing $\mathbb{F}_4$ matrices in the self-dual normal basis $\{\omega, \omega^2\}$; see Section 4.5, and $\mathbb{F}_8$ matrices in the self-dual normal basis given in Equation (46). The procedure for binarizing a matrix in this basis is shown above and in Equation (47), and the procedure in $\mathbb{F}_4$ is entirely analogous.

Protocol chain	Leading-order output error	Output error		Spatial footprint	Resource rate	
		$p = 10^{-3}$	$p = 10^{-6}$		Time	Volume
$(3T \rightarrow CS) * (\mathbf{72CS} \rightarrow \mathbf{35CS})$	$2874p^2$	3.08×10^{-3}	2.87×10^{-9}	75	1.27×10^{-1}	1.69×10^{-3}
$(3T \rightarrow CS)$	$3p$	3.00×10^{-3}	3.00×10^{-6}	3	3.33×10^{-1}	1.11×10^{-1}
$(3T \rightarrow CS) * (\mathbf{28CS} \rightarrow \mathbf{13CS})$	$498p^2$	5.11×10^{-4}	4.98×10^{-10}	31	1.42×10^{-1}	4.57×10^{-3}
$(12T \rightarrow CS)$	$18p^2$	1.81×10^{-5}	1.80×10^{-11}	4	8.23×10^{-2}	2.06×10^{-2}
$(3T \rightarrow CS) * (\mathbf{21CS} \rightarrow \mathbf{4CS})$	$414p^3$	4.22×10^{-7}	4.14×10^{-16}	17	5.95×10^{-2}	3.50×10^{-3}
$(3T \rightarrow CS) * (\mathbf{12CS} \rightarrow \mathbf{5CS}) * (\mathbf{12CS} \rightarrow \mathbf{5CS})$	$10^{3.18}p^4$	1.55×10^{-9}	1.52×10^{-21}	75	5.58×10^{-2}	7.44×10^{-4}
$(12T \rightarrow CS) * (\mathbf{6CS} \rightarrow \mathbf{2CS})$	$1296p^4$	1.31×10^{-9}	1.30×10^{-21}	10	2.74×10^{-2}	2.74×10^{-3}
$(12T \rightarrow CS) * (\mathbf{21CS} \rightarrow \mathbf{4CS})$	$10^{4.95}p^6$	9.07×10^{-14}	8.94×10^{-32}	18	1.57×10^{-2}	8.71×10^{-4}
$(3T \rightarrow CS) * (\mathbf{21CS} \rightarrow \mathbf{4CS}) * (\mathbf{14CS} \rightarrow \mathbf{6CS})$	$10^{4.69}p^6$	5.04×10^{-14}	4.86×10^{-32}	73	2.55×10^{-2}	3.49×10^{-4}
$(3T \rightarrow CS) * (\mathbf{10CS} \rightarrow \mathbf{4CS}) * (\mathbf{21CS} \rightarrow \mathbf{4CS})$	$10^{3.57}p^6$	3.82×10^{-15}	3.74×10^{-33}	69	2.46×10^{-2}	3.57×10^{-4}
$(3T \rightarrow CS) * (\mathbf{12CS} \rightarrow \mathbf{5CS}) * (\mathbf{16CS} \rightarrow \mathbf{2CS})$	$10^{3.01}p^6$	1.05×10^{-15}	1.03×10^{-33}	65	1.67×10^{-2}	2.57×10^{-4}
$(12T \rightarrow CS) * (\mathbf{4CS} \rightarrow \mathbf{CS}) * (\mathbf{6CS} \rightarrow \mathbf{2CS})$	$10^{6.23}p^8$	1.71×10^{-18}	1.68×10^{-42}	14	6.86×10^{-3}	4.90×10^{-4}
$(18T \rightarrow 2CS) * (\mathbf{6CS} \rightarrow \mathbf{2CS}) * (\mathbf{14CS} \rightarrow \mathbf{6CS})$	$10^{5.84}p^8$	7.09×10^{-19}	6.91×10^{-43}	74	1.56×10^{-2}	2.11×10^{-4}
$(18T \rightarrow 2CS) * (\mathbf{8CS} \rightarrow \mathbf{3CS}) * (\mathbf{8CS} \rightarrow \mathbf{3CS})$	$10^{5.14}p^8$	1.41×10^{-19}	1.37×10^{-43}	70	1.53×10^{-2}	2.19×10^{-4}
$(20T \rightarrow 4T) * (18T \rightarrow 2CS) * (\mathbf{4CS} \rightarrow \mathbf{CS})$	$10^{2.96}p^8$	9.15×10^{-22}	9.08×10^{-46}	63	5.44×10^{-3}	8.64×10^{-5}

Table 6: Statistics for a selection of competitive protocols taking injected $|T\rangle$ states as input, and distilling $|CS\rangle$ states as output. We display those protocols that appear in one of our Pareto frontiers over a range at least half an order of magnitude on the x -axis.

Protocol chain	Leading-order output error	Output error		Spatial footprint	Resource rate	
		$p = 10^{-3}$	$p = 10^{-6}$		Time	Volume
$(146T \rightarrow 24CCZ)$	$4030p^2$	4.04×10^{-3}	4.03×10^{-9}	73	1.40×10^{-1}	1.92×10^{-3}
$(44T \rightarrow 7CCZ)$	$460p^2$	4.61×10^{-4}	4.60×10^{-10}	22	1.52×10^{-1}	6.91×10^{-3}
$(8T \rightarrow CCZ)$	$28p^2$	2.81×10^{-5}	2.80×10^{-11}	4	1.24×10^{-1}	3.10×10^{-2}
$(20T \rightarrow 3CCZ) * (\mathbf{20CCZ} \rightarrow \mathbf{6CCZ})$	$10^{4.07}p^4$	1.17×10^{-8}	1.17×10^{-20}	73	4.41×10^{-2}	6.04×10^{-4}
$(64T \rightarrow 2CCZ)$	$2368p^4$	2.38×10^{-9}	2.37×10^{-21}	10	2.92×10^{-2}	2.92×10^{-3}
$(38T \rightarrow 6CCZ) * (\mathbf{8CCZ} \rightarrow \mathbf{2CCZ})$	$10^{3.24}p^4$	1.75×10^{-9}	1.74×10^{-21}	73	3.80×10^{-2}	5.20×10^{-4}
$(8T \rightarrow CCZ) * (\mathbf{9CCZ} \rightarrow \mathbf{CCZ})$	$10^{4.58}p^6$	3.79×10^{-14}	3.76×10^{-32}	13	1.38×10^{-2}	1.06×10^{-3}
$(20T \rightarrow 3CCZ) * (\mathbf{21CCZ} \rightarrow \mathbf{3CCZ})$	$10^{4.08}p^6$	1.20×10^{-14}	1.19×10^{-32}	64	2.10×10^{-2}	3.28×10^{-4}
$(8T \rightarrow CCZ) * (\mathbf{64CCZ} \rightarrow \mathbf{8CCZ})$	$10^{7.36}p^8$	2.29×10^{-17}	2.27×10^{-41}	43	1.55×10^{-2}	3.60×10^{-4}
$(64T \rightarrow 2CCZ) * (\mathbf{8CCZ} \rightarrow \mathbf{2CCZ})$	$10^{6.83}p^8$	6.82×10^{-18}	6.77×10^{-42}	28	7.31×10^{-3}	2.61×10^{-4}
$(8T \rightarrow CCZ) * (\mathbf{14CCZ} \rightarrow \mathbf{4CCZ}) * (\mathbf{8CCZ} \rightarrow \mathbf{2CCZ})$	$10^{6.16}p^8$	1.44×10^{-18}	1.43×10^{-42}	55	8.86×10^{-3}	1.61×10^{-4}
$(14T \rightarrow 2T) * (64T \rightarrow 2CCZ)$	$10^{4.46}p^8$	2.93×10^{-20}	2.91×10^{-44}	32	4.40×10^{-3}	1.38×10^{-4}
$(20T \rightarrow 4T) * (64T \rightarrow 2CCZ)$	$10^{3.47}p^8$	2.97×10^{-21}	2.94×10^{-45}	68	6.12×10^{-3}	9.01×10^{-5}

Table 7: Statistics for a selection of competitive protocols taking injected $|T\rangle$ states as input, and distilling $|CCZ\rangle$ states as output. We display those protocols that appear in one of our Pareto frontiers over a range at least half an order of magnitude on the x -axis. The $((3k+8)T \rightarrow kT) * (64T \rightarrow 2CCZ)$ entries use the higher-footprint randomized column ordering layout.

Protocol chain	Leading-order output error	Output error		Spatial footprint	Resource rate	
		$p = 10^{-3}$	$p = 10^{-6}$		Time	Volume
(74CS $\rightarrow$ 36CS)	$10^{3.31}p^2$	2.02×10^{-3}	2.02×10^{-9}	74	2.07×10^{-1}	2.80×10^{-3}
(34CS $\rightarrow$ 16CS)	$472p^2$	4.73×10^{-4}	4.72×10^{-10}	34	2.19×10^{-1}	6.45×10^{-3}
(4CS $\rightarrow$ CS)	$12p^2$	1.20×10^{-5}	1.20×10^{-11}	4	1.24×10^{-1}	3.10×10^{-2}
(21CS $\rightarrow$ 4CS)	$123p^3$	1.25×10^{-7}	1.23×10^{-16}	14	9.12×10^{-2}	6.52×10^{-3}
(10CS $\rightarrow$ 4CS) * (16CS $\rightarrow$ 7CS)	$10^{3.21}p^4$	1.64×10^{-9}	1.63×10^{-21}	74	8.58×10^{-2}	1.16×10^{-3}
(16CS $\rightarrow$ 2CS)	$944p^4$	9.48×10^{-10}	9.44×10^{-22}	10	6.05×10^{-2}	6.05×10^{-3}
(4CS $\rightarrow$ CS) * (21CS $\rightarrow$ 4CS)	$10^{4.42}p^6$	2.67×10^{-14}	2.65×10^{-32}	18	2.36×10^{-2}	1.31×10^{-3}
(21CS $\rightarrow$ 4CS) * (14CS $\rightarrow$ 6CS)	$10^{3.63}p^6$	4.41×10^{-15}	4.29×10^{-33}	70	3.91×10^{-2}	5.59×10^{-4}
(16CS $\rightarrow$ 2CS) * (32CS $\rightarrow$ 15CS)	$10^{7.20}p^8$	1.58×10^{-17}	1.57×10^{-41}	74	2.84×10^{-2}	3.83×10^{-4}
(16CS $\rightarrow$ 2CS) * (6CS $\rightarrow$ 2CS)	$10^{5.95}p^8$	8.98×10^{-19}	8.91×10^{-43}	22	2.02×10^{-2}	9.17×10^{-4}
(6CS $\rightarrow$ 2CS) * (6CS $\rightarrow$ 2CS) *	$10^{5.13}p^8$	1.37×10^{-19}	1.36×10^{-43}	74	2.35×10^{-2}	3.18×10^{-4}
(14CS $\rightarrow$ 6CS)						
(6CS $\rightarrow$ 2CS) * (8CS $\rightarrow$ 3CS) *	$10^{4.43}p^8$	2.72×10^{-20}	2.70×10^{-44}	70	2.32×10^{-2}	3.31×10^{-4}
(8CS $\rightarrow$ 3CS)						

Table 8: Statistics for a selection of competitive protocols taking injected $|\text{CS}\rangle$ states as input, and distilling $|\text{CS}\rangle$ states as output. We display those protocols that appear in one of our Pareto frontiers over a range at least half an order of magnitude on the x -axis.

Protocol chain	Leading-order output error	Output error		Spatial footprint	Resource rate	
		$p = 10^{-3}$	$p = 10^{-6}$		Time	Volume
(26CS $\rightarrow$ 12CS) * (2CS $\rightarrow$ CCZ)	$10^{2.76}p^2$	5.82×10^{-4}	5.81×10^{-10}	74	1.09×10^{-1}	1.48×10^{-3}
(4CS $\rightarrow$ CS) * (2CS $\rightarrow$ CCZ)	$24p^2$	2.40×10^{-5}	2.40×10^{-11}	8	6.20×10^{-2}	7.75×10^{-3}
(21CS $\rightarrow$ 4CS) * (2CS $\rightarrow$ CCZ)	$246p^3$	2.49×10^{-7}	2.46×10^{-16}	30	4.56×10^{-2}	1.52×10^{-3}
(2CS $\rightarrow$ CCZ) * (9CCZ $\rightarrow$ CCZ)	$10^{2.03}p^3$	1.08×10^{-7}	1.07×10^{-16}	13	2.68×10^{-2}	2.06×10^{-3}
(16CS $\rightarrow$ 2CS) * (2CS $\rightarrow$ CCZ)	$1888p^4$	1.90×10^{-9}	1.89×10^{-21}	18	3.02×10^{-2}	1.68×10^{-3}
(8CS $\rightarrow$ 3CS) * (8CS $\rightarrow$ 3CS) *	$10^{3.15}p^4$	1.41×10^{-9}	1.41×10^{-21}	68	3.46×10^{-2}	5.09×10^{-4}
(2CS $\rightarrow$ CCZ)						
(4CS $\rightarrow$ CS) * (4CS $\rightarrow$ CS) * (2CS $\rightarrow$ CCZ)	$576p^4$	5.78×10^{-10}	5.76×10^{-22}	12	1.55×10^{-2}	1.29×10^{-3}
(16CS $\rightarrow$ 7CS) * (4CS $\rightarrow$ CS) *	$10^{2.15}p^4$	1.41×10^{-10}	1.41×10^{-22}	72	2.65×10^{-2}	3.68×10^{-4}
(2CS $\rightarrow$ CCZ)						
(4CS $\rightarrow$ CS) * (2CS $\rightarrow$ CCZ) *	$10^{4.37}p^6$	2.37×10^{-14}	2.36×10^{-32}	17	6.89×10^{-3}	4.05×10^{-4}
(9CCZ $\rightarrow$ CCZ)						
(6CS $\rightarrow$ 2CS) * (21CS $\rightarrow$ 4CS) *	$10^{3.99}p^6$	9.82×10^{-15}	9.76×10^{-33}	66	1.57×10^{-2}	2.38×10^{-4}
(2CS $\rightarrow$ CCZ)						
(21CS $\rightarrow$ 4CS) * (6CS $\rightarrow$ 2CS) *	$10^{3.32}p^6$	2.14×10^{-15}	2.09×10^{-33}	70	1.52×10^{-2}	2.17×10^{-4}
(2CS $\rightarrow$ CCZ)						
(16CS $\rightarrow$ 2CS) * (12CS $\rightarrow$ 5CS) *	$10^{6.75}p^8$	5.69×10^{-18}	5.65×10^{-42}	74	1.26×10^{-2}	1.70×10^{-4}
(2CS $\rightarrow$ CCZ)						
(16CS $\rightarrow$ 2CS) * (4CS $\rightarrow$ CS) *	$10^{5.95}p^8$	8.98×10^{-19}	8.91×10^{-43}	26	7.56×10^{-3}	2.91×10^{-4}
(2CS $\rightarrow$ CCZ)						
(6CS $\rightarrow$ 2CS) * (4CS $\rightarrow$ CS) *	$10^{4.92}p^8$	8.36×10^{-20}	8.29×10^{-44}	30	5.15×10^{-3}	1.72×10^{-4}
(4CS $\rightarrow$ CS) * (2CS $\rightarrow$ CCZ)						
(6CS $\rightarrow$ 2CS) * (6CS $\rightarrow$ 2CS) *	$10^{4.82}p^8$	6.69×10^{-20}	6.64×10^{-44}	74	9.15×10^{-3}	1.24×10^{-4}
(6CS $\rightarrow$ 2CS) * (2CS $\rightarrow$ CCZ)						
(6CS $\rightarrow$ 2CS) * (8CS $\rightarrow$ 3CS) *	$10^{4.22}p^8$	1.69×10^{-20}	1.68×10^{-44}	70	7.72×10^{-3}	1.10×10^{-4}
(4CS $\rightarrow$ CS) * (2CS $\rightarrow$ CCZ)						

Table 9: Statistics for a selection of competitive protocols taking injected $|\text{CS}\rangle$ states as input, and distilling $|\text{CCZ}\rangle$ states as output. We display those protocols that appear in one of our Pareto frontiers over a range at least half an order of magnitude on the x -axis.